

Document Title	Layered Software Architecture
Document Owner	AUTOSAR
Document Responsibility	AUTOSAR
Document Identification No	53
Document Status	published
Part of AUTOSAR Standard	Classic Platform
Part of Standard Release	R25-11

Document Title	Layered Software Architecture
Document Owner	AUTOSAR
Document Responsibility	AUTOSAR
Document Identification No	53
Document Status	published
Part of AUTOSAR Standard	Classic Platform
Part of Standard Release	R25-11

Document Change History			
Date	Release	Changed by	Change Description
2025-11-27	R25-11	AUTOSAR Release Management	➤ Added information about VDP ➤ Added information about Mirror ➤ Updated slides about libraries, DDS ➤ Removed TTCan ➤ Removed FIs and Eep ➤ Removed LdCom ➤ Minor changes
2024-11-27	R24-11	AUTOSAR Release Management	➤ Added L-SDU Router ➤ Incorporated J1939Fscp transformer into comm stack extensions ➤ Incorporated Partitioning examples ➤ Incorporated migration of BSWModuleList to BSWGeneral ➤ Added I2C in Comm Drivers ➤ Improve documentation about error handling and usage ➤ Cleanup remains of partition restart ➤ Added changes in memory manipulation library: Copy, Set, Move, Compare ➤ Remove E2EPW support ➤ Added changes to error handling

Document Change History			
Date	Release	Changed by	Change Description
2025-11-27	R25-11	AUTOSAR Release Management	➤ Added information about VDP ➤ Added information about Mirror ➤ Updated slides about libraries, DDS ➤ Removed TTCan ➤ Removed FIs and Eep ➤ Removed LdCom ➤ Minor changes
2024-11-27	R24-11	AUTOSAR Release Management	➤ Added L-SDU Router ➤ Incorporated J1939Fscp transformer into comm stack extensions ➤ Incorporated Partitioning examples ➤ Incorporated migration of BSWModuleList to BSWGeneral ➤ Added I2C in Comm Drivers ➤ Improve documentation about error handling and usage ➤ Cleanup remains of partition restart ➤ Added changes in memory manipulation library: Copy, Set, Move, Compare ➤ Remove E2EPW support ➤ Added changes to error handling

Document Change History			
Date	Release	Changed by	Change Description
2023-11-23	R23-11	AUTOSAR Release Management	➤ Added information about charging management (ChrgM) and firewall ➤ Editorial changes
2022-11-24	R22-11	AUTOSAR Release Management	➤ Incorporated new concepts for Vehicle-2-X Data Manager, MACsec, CAN XL, DDS, Secured Time Synchronization, Vehicle-2-X Support for China ➤ Editorial changes
2021-11-25	R21-11	AUTOSAR Release Management	➤ Incorporated draft concept for new Memory Driver and Memory Access
2020-11-30	R20-11	AUTOSAR Release Management	➤ Removed Pretended Networking ➤ Added caveats for E2E Protection Wrapper ➤ Layer Interaction Matrix: Allow Crypto Driver to access Memory Services ➤ Incorporated new concepts for Intrusion Detection System Manager, CP Software Clusters
2019-11-28	R19-11	AUTOSAR Release Management	➤ Incorporated new concepts for Atomic multicore safe operations, Signal-service-translation, NV data handling enhancement ➤ Changed Document Status from Final to published

Document Change History			
Date	Release	Changed by	Change Description
2023-11-23	R23-11	AUTOSAR Release Management	➤ Added information about charging management (ChrgM) and firewall ➤ Editorial changes
2022-11-24	R22-11	AUTOSAR Release Management	➤ Incorporated new concepts for Vehicle-2-X Data Manager, MACsec, CAN XL, DDS, Secured Time Synchronization, Vehicle-2-X Support for China ➤ Editorial changes
2021-11-25	R21-11	AUTOSAR Release Management	➤ Incorporated draft concept for new Memory Driver and Memory Access
2020-11-30	R20-11	AUTOSAR Release Management	➤ Removed Pretended Networking ➤ Added caveats for E2E Protection Wrapper ➤ Layer Interaction Matrix: Allow Crypto Driver to access Memory Services ➤ Incorporated new concepts for Intrusion Detection System Manager, CP Software Clusters
2019-11-28	R19-11	AUTOSAR Release Management	➤ Incorporated new concepts for Atomic multicore safe operations, Signal-service-translation, NV data handling enhancement ➤ Changed Document Status from Final to published

Document Change History			
Date	Release	Changed by	Change Description
2018-10-31	4.4.0	AUTOSAR Release Management	➤ Adopting LIN Slave Support, LinNm removed ➤ New Concepts: Key Management, 1st draft of MCAL Multicore Distribution ➤ Editorial changes
2017-12-08	4.3.1	AUTOSAR Release Management	➤ Editorial changes
2016-11-30	4.3.0	AUTOSAR Release Management	➤ Incorporated new 4.3 concepts for Crypto Stack, Vehicle-2-X Communication, SOME/IP Transport Protocol, DLT rework ➤ Removed obsolete Dbg module ➤ Editorial changes
2015-07-31	4.2.2	AUTOSAR Release Management	➤ Editorial changes

Document Change History			
Date	Release	Changed by	Change Description
2018-10-31	4.4.0	AUTOSAR Release Management	➤ Adopting LIN Slave Support, LinNm removed ➤ New Concepts: Key Management, 1st draft of MCAL Multicore Distribution ➤ Editorial changes
2017-12-08	4.3.1	AUTOSAR Release Management	➤ Editorial changes
2016-11-30	4.3.0	AUTOSAR Release Management	➤ Incorporated new 4.3 concepts for Crypto Stack, Vehicle-2-X Communication, SOME/IP Transport Protocol, DLT rework ➤ Removed obsolete Dbg module ➤ Editorial changes
2015-07-31	4.2.2	AUTOSAR Release Management	➤ Editorial changes

Document Change History			
Date	Release	Changed by	Change Description
2014-10-31	4.2.1	AUTOSAR Release Management	➤ Incorporated new 4.2 concepts for: Switch Configuration; Sender-Receiver-Serialization; CAN-FD; Large-Data-COM; E2E-Extension; Global Time Synchronization; Support for Post-build ECU-Configuration; Secure-Onboard-Communication; ASIL/QM-Protection ➤ Introduction of new error classification ➤ Editorial changes
2014-03-31	4.1.3	AUTOSAR Release Management	➤ Editorial changes
2013-03-15	4.1.1	AUTOSAR Administration	➤ Clarification of partial network support for CAN/LIN slave. ➤ New Ethernet stack extensions ➤ Added Crypto Service Manager to System Services ➤ Revised presentation of J1939 and added new J1939 modules ➤ Added new energy management concepts: "Pretended Networking", "ECU Degradation" ➤ Added new modules: "Output Compare Unit Driver" and "Time Service" ➤ Changed handling of Production Errors ➤ Fixed various typography and layout issues
2011-12-22	4.0.3	AUTOSAR Administration	➤ Added a note for the R3-compatibility FlexRay Transport Layer FrArTp on slide "ki890". ➤ Added an overview chapter for energy management and partial networking ➤ Corrected examples regarding DEM symbol generation ➤ Fixed minor typography issues ➤ Clarification of term AUTOSAR-ECU on slide "94jt1" ➤ Corrected CDD access description for EcuM on slide "11123"

Document Change History			
Date	Release	Changed by	Change Description
2014-10-31	4.2.1	AUTOSAR Release Management	➤ Incorporated new 4.2 concepts for: Switch Configuration; Sender-Receiver-Serialization; CAN-FD; Large-Data-COM; E2E-Extension; Global Time Synchronization; Support for Post-build ECU-Configuration; Secure-Onboard-Communication; ASIL/QM-Protection ➤ Introduction of new error classification ➤ Editorial changes
2014-03-31	4.1.3	AUTOSAR Release Management	➤ Editorial changes
2013-03-15	4.1.1	AUTOSAR Administration	➤ Clarification of partial network support for CAN/LIN slave. ➤ New Ethernet stack extensions ➤ Added Crypto Service Manager to System Services ➤ Revised presentation of J1939 and added new J1939 modules ➤ Added new energy management concepts: "Pretended Networking", "ECU Degradation" ➤ Added new modules: "Output Compare Unit Driver" and "Time Service" ➤ Changed handling of Production Errors ➤ Fixed various typography and layout issues
2011-12-22	4.0.3	AUTOSAR Administration	➤ Added a note for the R3-compatibility FlexRay Transport Layer FrArTp on slide "ki890". ➤ Added an overview chapter for energy management and partial networking ➤ Corrected examples regarding DEM symbol generation ➤ Fixed minor typography issues ➤ Clarification of term AUTOSAR-ECU on slide "94jt1" ➤ Corrected CDD access description for EcuM on slide "11123"

Document Change History			
Date	Release	Changed by	Change Description
2009-12-18	4.0.1	AUTOSAR Administration	➤ Added a note regarding support for System Basis Chips on slide "94juq" ➤ Clarification of DBG and DLT text on slide "3edfg" ➤ Corrected DBG description on slide "11231"
2010-02-02	3.1.4	AUTOSAR Administration	➤ The document has been newly structured. There are now 3 main parts: ■ Architecture ■ Configuration ■ Integration and Runtime Aspects ➤ The whole content has been updated to reflect the content of the R 4.0 specifications. ➤ Topics which have been newly introduced or heavily extended in release 4.0 have been added. E.g.,. Multi-Core Systems, Partitioning, Mode Management, Error Handling, Reporting and Diagnostic, Debugging, Measurement and Calibration, Functional Safety etc ➤ Legal disclaimer revised
2008-08-13	3.1.1	AUTOSAR Administration	➤ Legal disclaimer revised
2007-12-21	3.0.1	AUTOSAR Administration	➤ Updates based on new wakeup/startup concepts ➤ Detailed explanation for post-build time configuration ➤ "Slimming" of LIN stack description ➤ ICC2 figure ➤ Document meta information extended ➤ Small layout adaptations made

Document Change History			
Date	Release	Changed by	Change Description
2009-12-18	4.0.1	AUTOSAR Administration	➤ Added a note regarding support for System Basis Chips on slide "94juq" ➤ Clarification of DBG and DLT text on slide "3edfg" ➤ Corrected DBG description on slide "11231"
2010-02-02	3.1.4	AUTOSAR Administration	➤ The document has been newly structured. There are now 3 main parts: ■ Architecture ■ Configuration ■ Integration and Runtime Aspects ➤ The whole content has been updated to reflect the content of the R 4.0 specifications. ➤ Topics which have been newly introduced or heavily extended in release 4.0 have been added. E.g.,. Multi-Core Systems, Partitioning, Mode Management, Error Handling, Reporting and Diagnostic, Debugging, Measurement and Calibration, Functional Safety etc ➤ Legal disclaimer revised
2008-08-13	3.1.1	AUTOSAR Administration	➤ Legal disclaimer revised
2007-12-21	3.0.1	AUTOSAR Administration	➤ Updates based on new wakeup/startup concepts ➤ Detailed explanation for post-build time configuration ➤ "Slimming" of LIN stack description ➤ ICC2 figure ➤ Document meta information extended ➤ Small layout adaptations made

Document Change History			
Date	Release	Changed by	Change Description
2007-01-24	2.1.15	AUTOSAR Administration	➤ ICC clustering added. ➤ Document contents harmonized ➤ Legal disclaimer revised ➤ Release Notes added ➤ “Advice for users” revised ➤ “Revision Information” added
2006-11-28	2.1.1	AUTOSAR Administration	Rework Of: ➤ Error Handling ➤ Scheduling Mechanisms ➤ More updates according to architectural decisions in R2.0
2006-01-02	1.0.1	AUTOSAR Administration	➤ Correct version released
2005-05-31	1.0.0	AUTOSAR Administration	➤ Initial release

Document Change History			
Date	Release	Changed by	Change Description
2007-01-24	2.1.15	AUTOSAR Administration	➤ ICC clustering added. ➤ Document contents harmonized ➤ Legal disclaimer revised ➤ Release Notes added ➤ “Advice for users” revised ➤ “Revision Information” added
2006-11-28	2.1.1	AUTOSAR Administration	Rework Of: ➤ Error Handling ➤ Scheduling Mechanisms ➤ More updates according to architectural decisions in R2.0
2006-01-02	1.0.1	AUTOSAR Administration	➤ Correct version released
2005-05-31	1.0.0	AUTOSAR Administration	➤ Initial release

Disclaimer

Disclaimer

This work (specification and/or software implementation) and the material contained in it, as released by AUTOSAR, is for the purpose of information only. AUTOSAR and the companies that have contributed to it shall not be liable for any use of the work.

The material contained in this work is protected by copyright and other types of intellectual property rights. The commercial exploitation of the material contained in this work requires a license to such intellectual property rights.

This work may be utilized or reproduced without any modification, in any form or by any means, for informational purposes only. For any other purpose, no part of the work may be utilized or reproduced, in any form or by any means, without permission in writing from the publisher.

The work has been developed for automotive applications only. It has neither been developed, nor tested for non-automotive applications.

The word AUTOSAR and the AUTOSAR logo are registered trademarks.

Disclaimer

Disclaimer

This work (specification and/or software implementation) and the material contained in it, as released by AUTOSAR, is for the purpose of information only. AUTOSAR and the companies that have contributed to it shall not be liable for any use of the work.

The material contained in this work is protected by copyright and other types of intellectual property rights. The commercial exploitation of the material contained in this work requires a license to such intellectual property rights.

This work may be utilized or reproduced without any modification, in any form or by any means, for informational purposes only. For any other purpose, no part of the work may be utilized or reproduced, in any form or by any means, without permission in writing from the publisher.

The work has been developed for automotive applications only. It has neither been developed, nor tested for non-automotive applications.

The word AUTOSAR and the AUTOSAR logo are registered trademarks.

Table of contents

1. Architecture

1. Overview of Software Layers

2. Content of Software Layers
3. Content of Software Layers in Multi-Core Systems
4. Content of Software Layers in Mixed-Critical Systems
5. Overview of Modules
6. Interfaces: General Rules
7. Interfaces: Interaction of Layers
8. Overview of CP Software Clusters

2. Configuration

3. Integration and Runtime Aspects

Table of contents

1. 架构

1. 软件层概述 2. 软件层内容 3. 多核系统中的软件层内容 4. 混合关键系统中的软件层内容 5. 模块概述 6. 接口：通用规则 7. 接口：层间交互 8. CP软件集群概述

2. 配置

3. 集成与运行时方面

Introduction

Purpose and Inputs

Purpose of this document

The **Layered Software Architecture** describes the software architecture of AUTOSAR:

- it describes in an top-down approach the hierarchical structure of AUTOSAR software and
- maps the Basic Software Modules to software layers and
- shows their relationship.

This document does not contain requirements and is informative only. The examples given are not meant to be complete in all respects.

This document focuses on static views of a conceptual layered software architecture:

- it does not specify a structural software architecture (design) with detailed static and dynamic interface descriptions,
 - these information are included in the specifications of the basic software modules themselves.

Inputs

This document is based on specification and requirement documents of AUTOSAR.

Introduction

目的与输入

本文档的目的

分层软件架构描述了AUTOSAR的软件架构：

它采用自上而下的方法描述了AUTOSAR软件的分层结构，将基础软件模块映射到软件层，并展示了它们之间的关系。

本文档不包含要求，仅供参考。所举示例并非在所有方面都力求完备。

本文档侧重于概念分层软件架构的静态视图：它并未规定包含详细静态与动态接口描述的结构化软件架构（设计），■ 这些信息已包含在基础软件模块自身的规范中。

输入

本文档基于AUTOSAR的规范和需求文档。

Introduction

Scope and Extensibility

Application scope of AUTOSAR

AUTOSAR is dedicated for Automotive ECUs. Such ECUs have the following properties:

- strong interaction with hardware (sensors and actuators),
- connection to vehicle networks like CAN, LIN, FlexRay or Ethernet,
- microcontrollers (typically 16 or 32 bit) with limited resources of computing power and memory (compared with enterprise solutions),
- Real Time System and
- program execution from internal or external flash memory.

NOTE: In the AUTOSAR sense an ECU means one microcontroller plus peripherals and the according software/configuration. The mechanical design is not in the scope of AUTOSAR. This means that if more than one microcontroller is arranged in a housing, then each microcontroller requires its own description of an AUTOSAR-ECU instance.

AUTOSAR extensibility

The AUTOSAR Software Architecture is a generic approach:

- standard modules can be extended in functionality, while still being compliant,
 - still, their configuration has to be considered in the automatic Basic SW configuration process!
- non-standard modules can be integrated into AUTOSAR-based systems as Complex Drivers and
- further layers cannot be added.

Introduction

范围与可扩展性

AUTOSAR的应用范围

AUTOSAR专用于汽车电子控制单元（ECU）。此类ECU具有以下特性：

与硬件（传感器和执行器）的强交互作用，
连接到车辆网络，如CAN、LIN、FlexRay或以太网，
微控制器（通常为16或32位）在计算能力和内存资源方面有限（与企业级解决方案相比），

实时系统与

程序执行从内部或外部闪存。

注意：在AUTOSAR的定义中，一个ECU指一个微控制器加上外设及相应的软件/配置。机械设计不在AUTOSAR的范围内。这意味着，如果一个外壳内布置了多个微控制器，那么每个微控制器都需要单独描述为一个AUTOSAR-ECU实例。

AUTOSAR可扩展性

TAUTOSAR软件架构是一种通用方法：

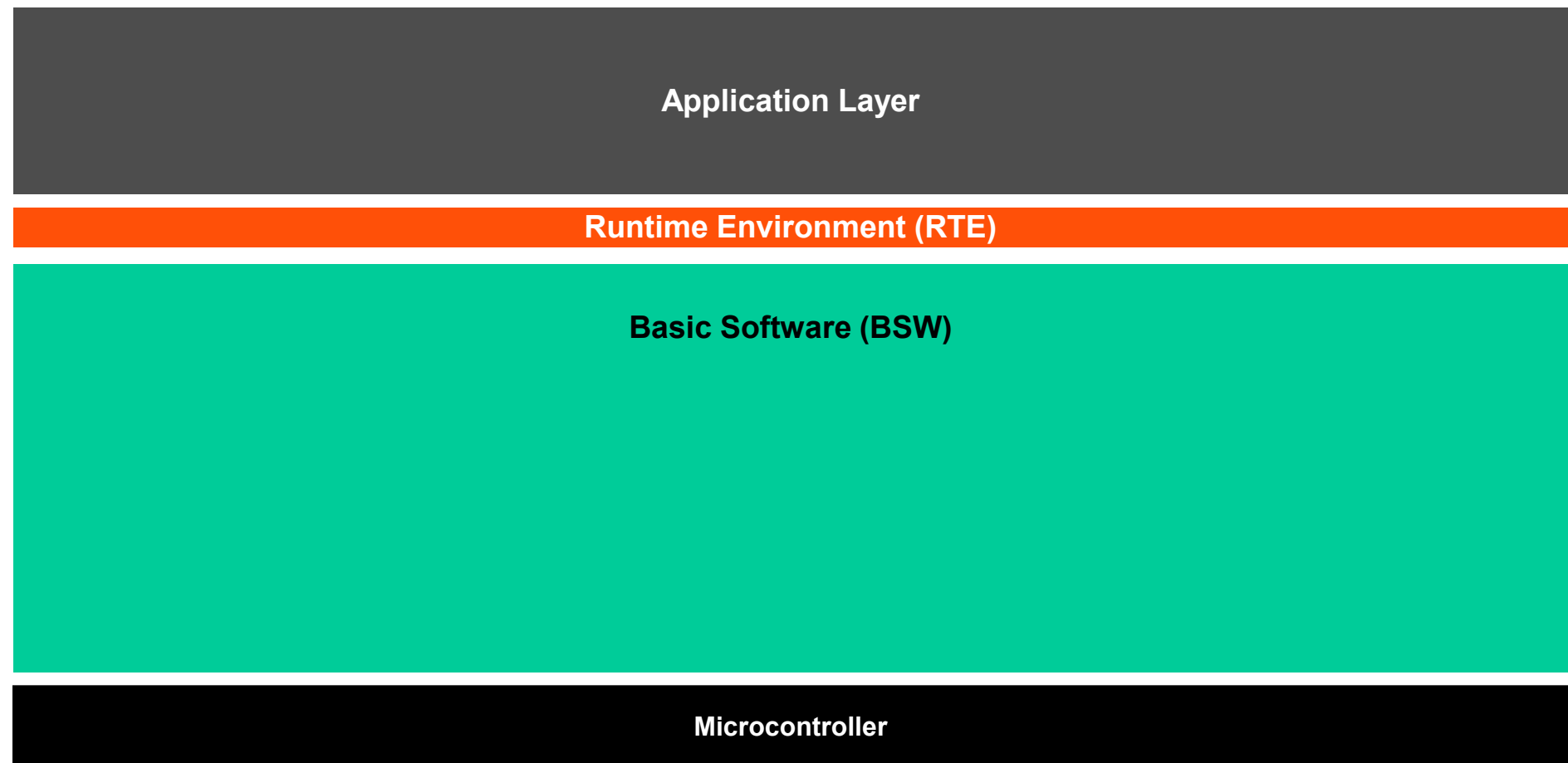
标准模块可在保持合规性的同时扩展功能，■但其配置仍需在自动基础软件配置流程中予以考虑！
非标准模块可作为复杂驱动程序集成至基于AUTOSAR的系统中。

无法添加更多层。

Architecture – Overview of Software Layers

Top view

The AUTOSAR Architecture distinguishes on the highest abstraction level between three software layers: Application, Runtime Environment and Basic Software which run on a Microcontroller.



Architecture – Overview of Software Layers

俯视图

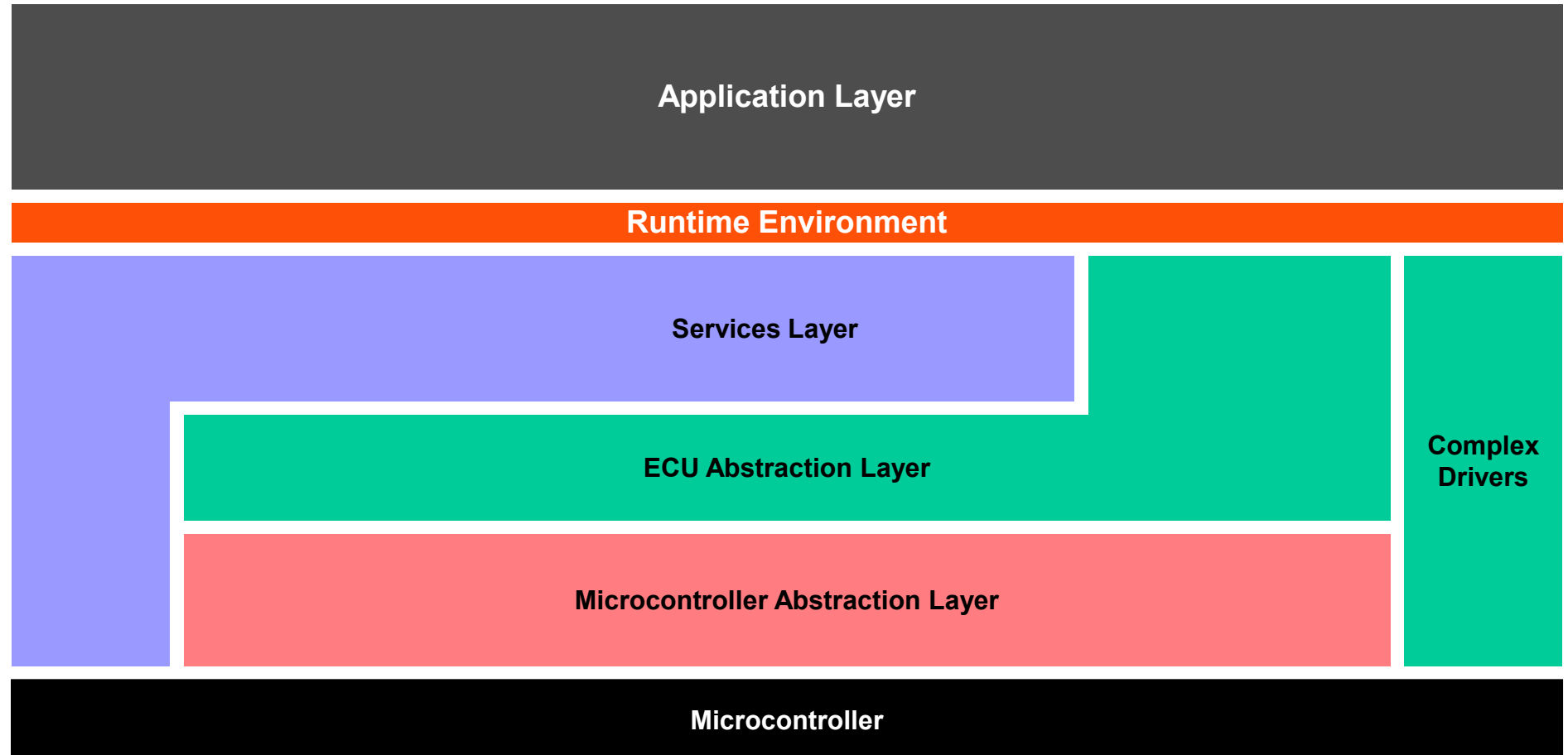
AUTOSAR架构在最高抽象层次上区分了三个软件层：运行在微控制器上的应用层、运行时环境层和基础软件层。



Architecture – Overview of Software Layers

Coarse view

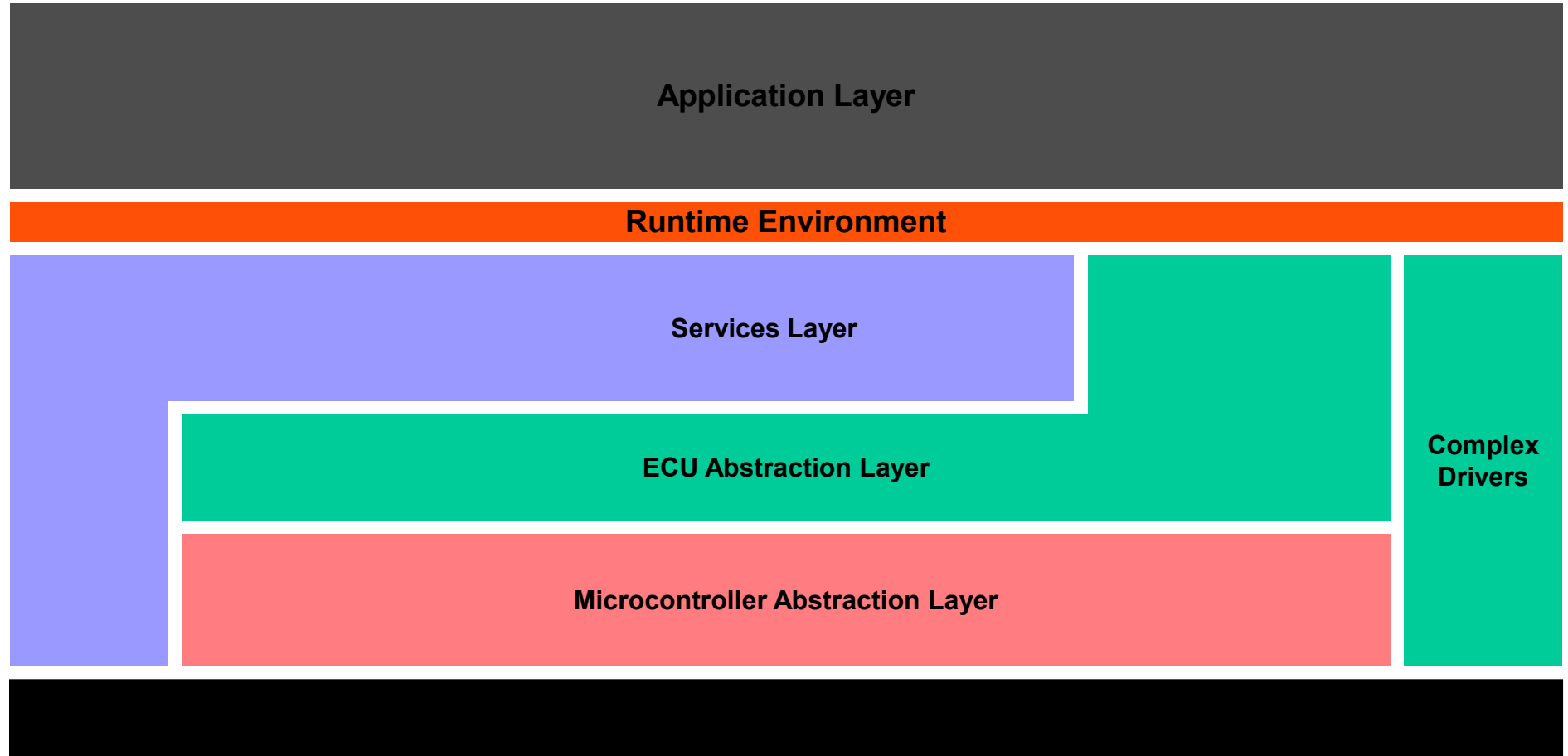
The AUTOSAR Basic Software is further divided in the layers: Services, ECU Abstraction, Microcontroller Abstraction and Complex Drivers.



Architecture – Overview of Software Layers

粗略视图

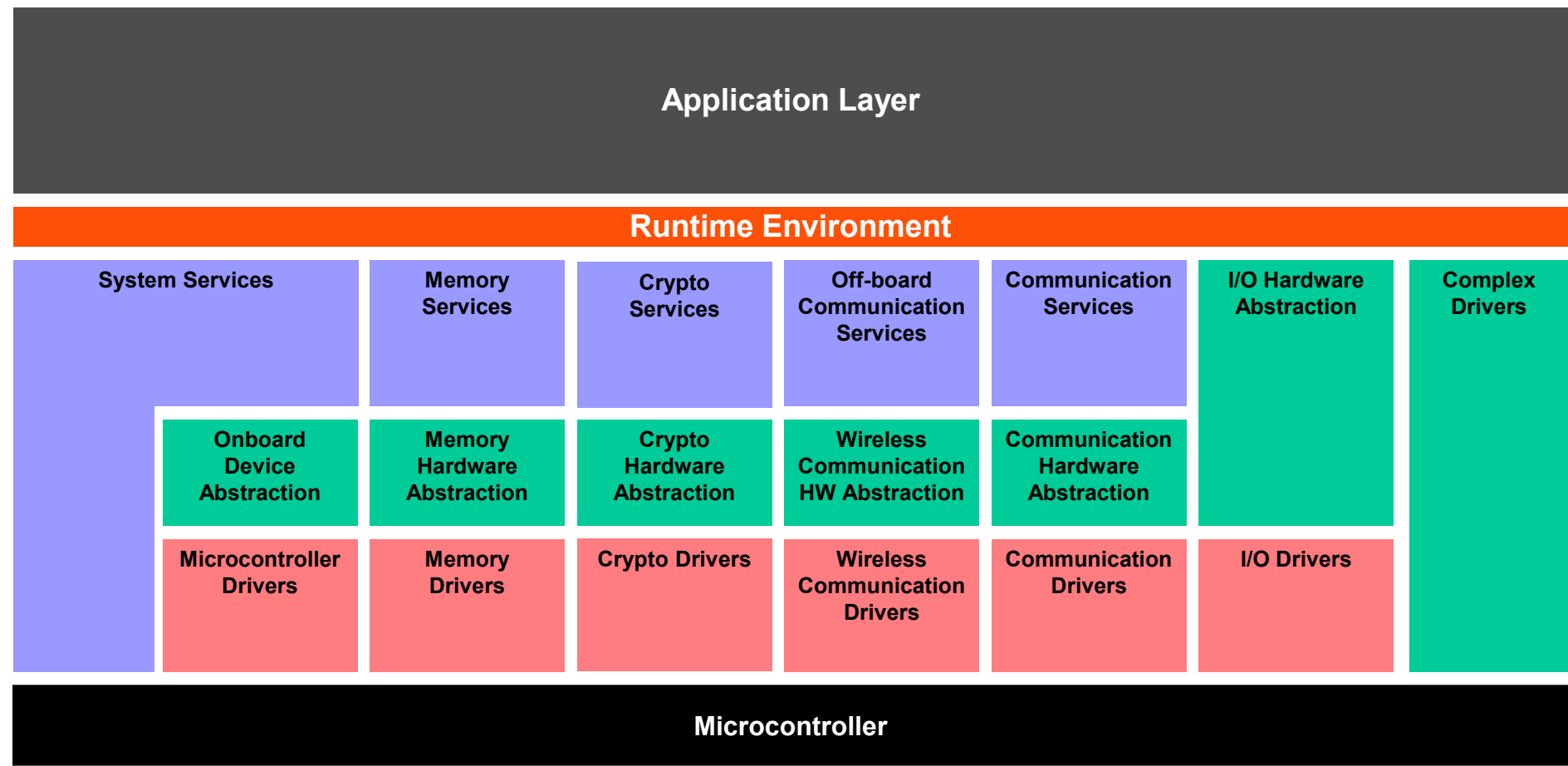
AUTOSAR基础软件进一步划分为以下层次：服务层、ECU抽象层、微控制器抽象层以及复杂驱动层。



Architecture – Overview of Software Layers

Detailed view

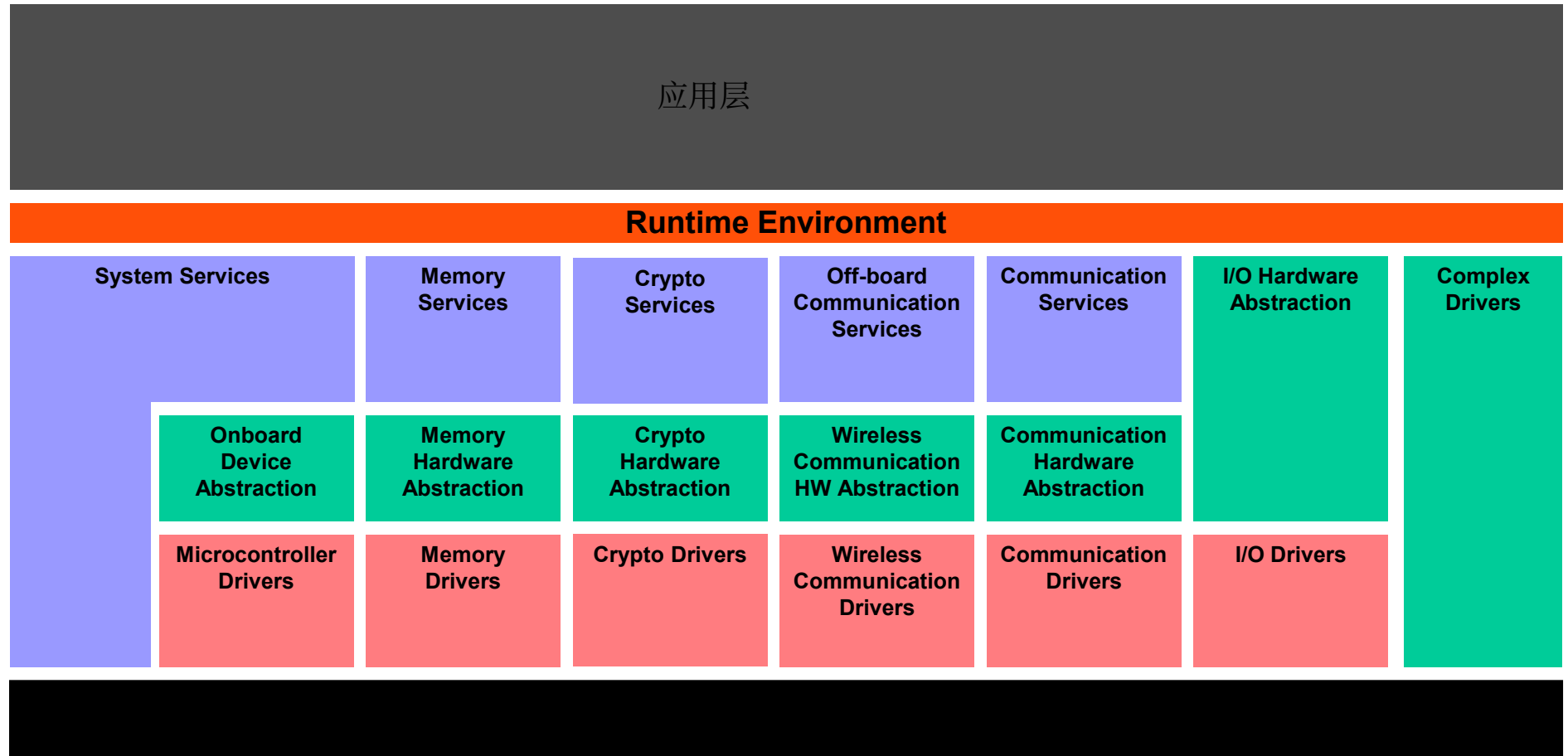
The Basic Software Layers are further divided into functional groups. Examples of Services are System, Memory and Communication Services.



Architecture – Overview of Software Layers

详细视图

基础软件层进一步划分为功能组。服务的示例包括系统、内存和通信服务。



Architecture – Overview of Software Layers

Microcontroller Abstraction Layer

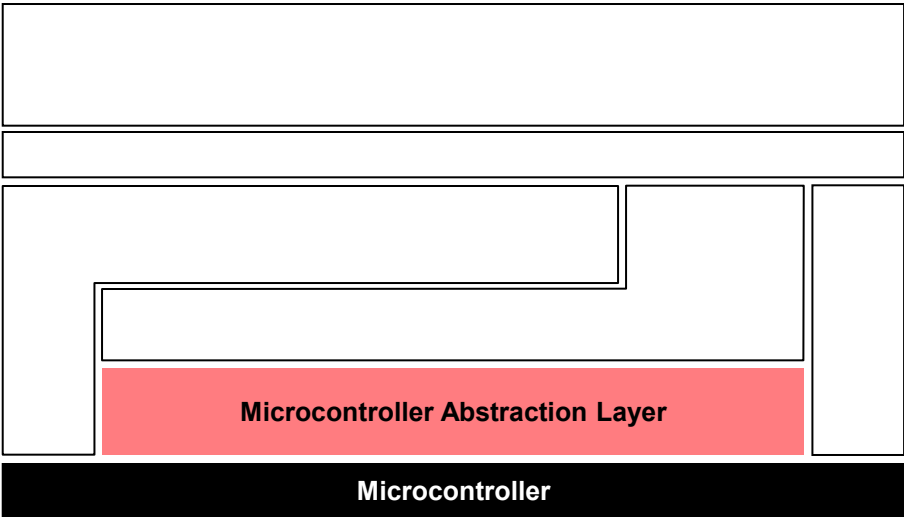
The **Microcontroller Abstraction Layer** is the lowest software layer of the Basic Software. It contains internal drivers, which are software modules with direct access to the μ C and internal peripherals.

Task

Make higher software layers independent of μ C

Properties

- Implementation: μ C dependent
- Upper Interface: standardized and μ C independent



Architecture – Overview of Software Layers

微控制器抽象层

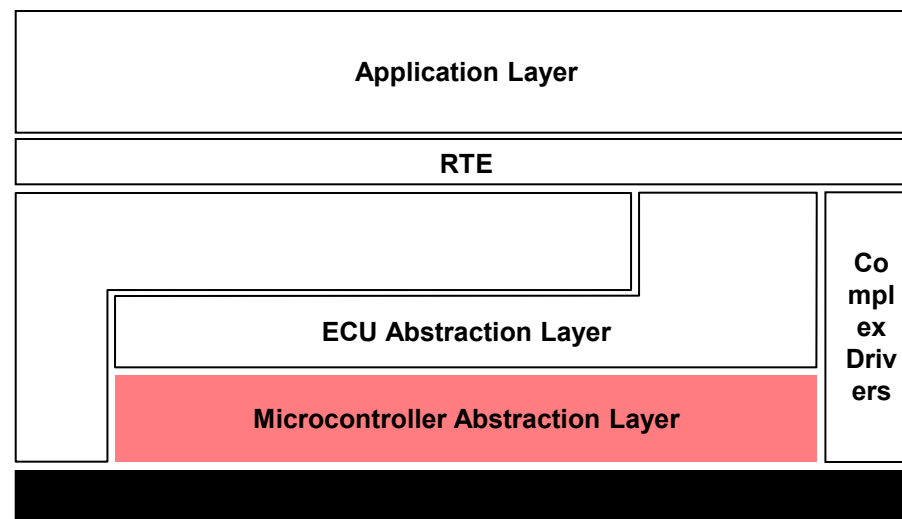
微控制器抽象层是基础软件的最低软件层。它包含内部驱动程序，这些是可直接访问微控制器及内部外设的软件模块。

任务

使高层软件独立于 μ C

属性

实现： μ C 依赖的上层接口：标准化且
 μ C 独立



Architecture – Overview of Software Layers

ECU Abstraction Layer

The **ECU Abstraction Layer** interfaces the drivers of the Microcontroller Abstraction Layer. It also contains drivers for external devices.

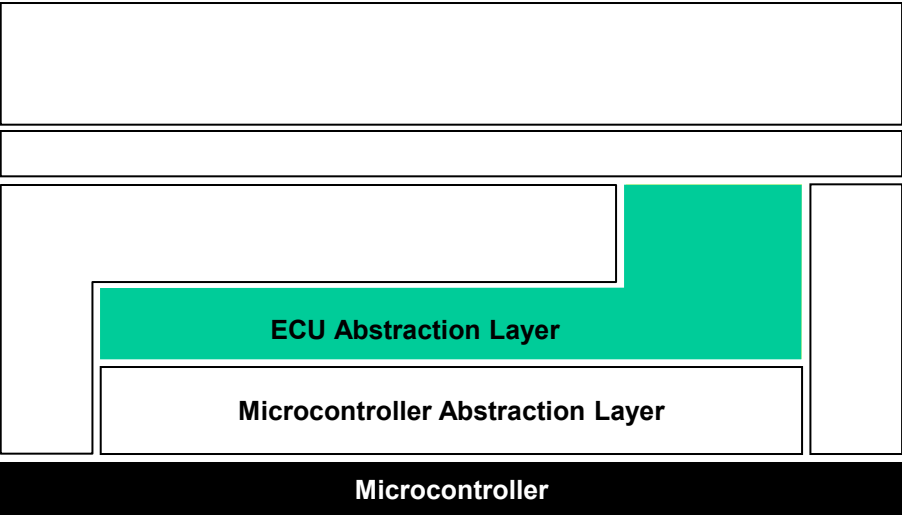
It offers an API for access to peripherals and devices regardless of their location (μ C internal/external) and their connection to the μ C (port pins, type of interface)

Task

Make higher software layers independent of ECU hardware layout

Properties

- Implementation: μ C independent, ECU hardware dependent
- Upper Interface: μ C and ECU hardware independent



Architecture – Overview of Software Layers

ECU抽象层

ECU抽象层与微控制器抽象层的驱动程序接口。
它还包含外部设备的驱动程序。

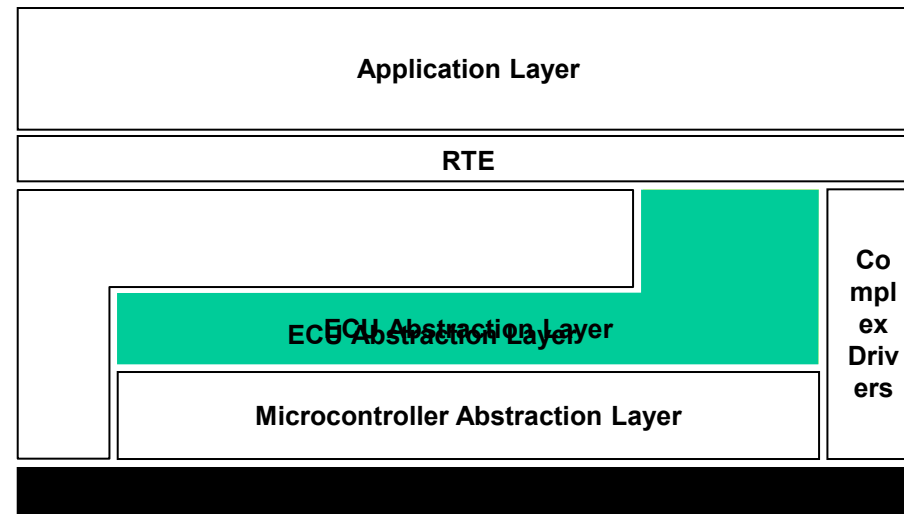
它提供了一个API，用于访问外围设备和器件，
无论其位置（微控制器内部/外部）以及它们与
微控制器的连接方式（端口引脚、接口类型）如
何。

任务

使高层软件独立于ECU硬件布局

属性

实现：μC独立，ECU硬件依赖 上层接口μC与ECU
U硬件独立



Architecture – Overview of Software Layers

Complex Drivers

The **Complex Drivers Layer** spans from the hardware to the RTE.

Task

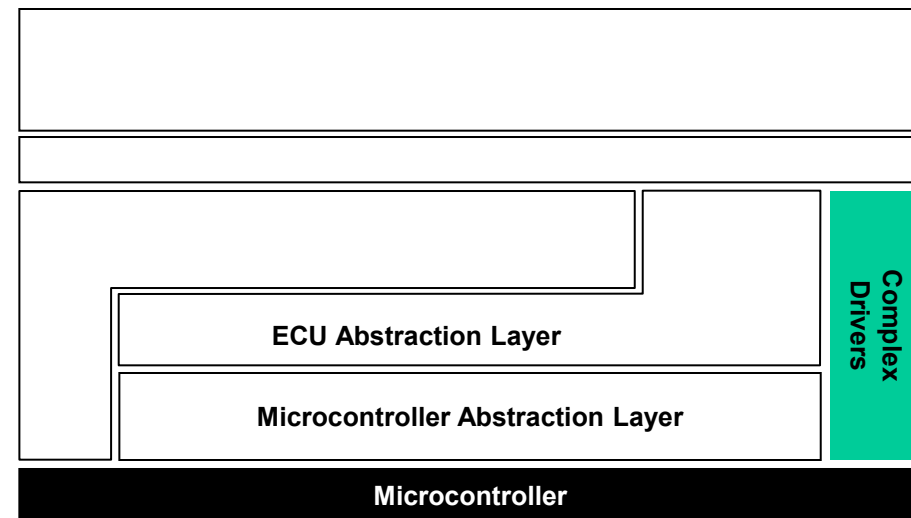
Provide the possibility to integrate special purpose functionality, e.g. drivers for devices:

- which are not specified within AUTOSAR,
- with very high timing constraints or
- for migration purposes etc.

Properties

Implementation: might be application, μ C and ECU hardware dependent

Upper Interface: might be application, μ C and ECU hardware dependent



Architecture – Overview of Software Layers

复杂驱动因素

复杂驱动层从硬件延伸至运行时环境（RTE）

。

任务

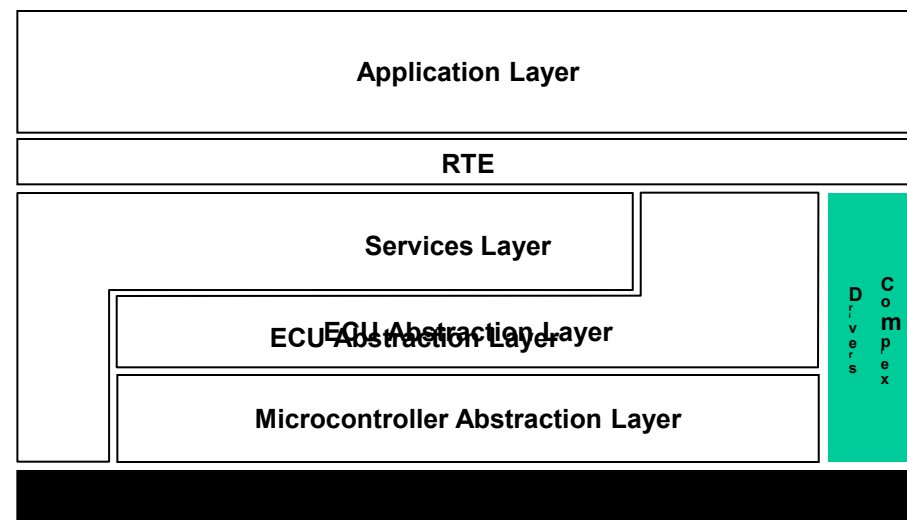
提供集成特殊功能的可能性，例如设备驱动程序：

未在AUTOSAR中明确规定的部分，具有极高时序约束的情况，或用于迁移目的等。

属性

实现：可能依赖于应用、 μ C和ECU硬件

上层接口：可能依赖于应用、 μ C和ECU硬件



Architecture – Overview of Software Layers

Services Layer

The **Services Layer** is the highest layer of the Basic Software which also applies for its relevance for the application software: while access to I/O signals is covered by the ECU Abstraction Layer, the Services Layer offers:

- Operating system functionality
- Vehicle network communication and management services
- Memory services (NVRAM management)
- Diagnostic Services (including UDS communication, error memory and fault treatment)
- ECU state management, mode management
- Logical and temporal program flow monitoring (WdgManager)

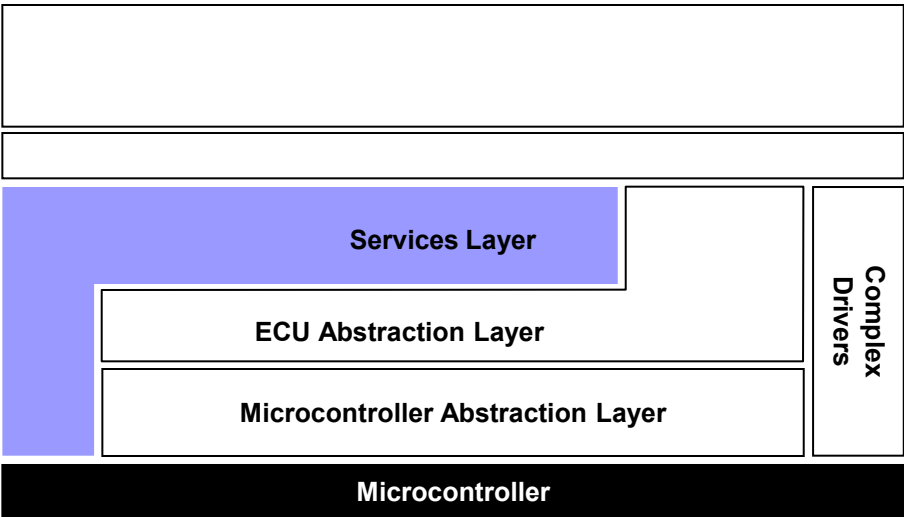
Task

Provide basic services for applications, RTE and basic software modules.

Properties

Implementation: mostly μ C and ECU hardware independent

Upper Interface: μ C and ECU hardware independent

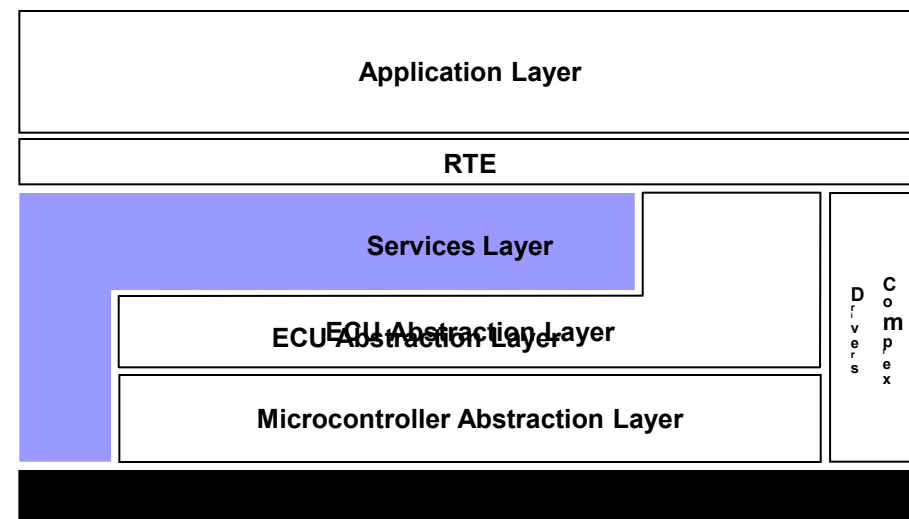


Architecture – Overview of Software Layers

服务层

服务层是基础软件的最高层，其重要性同样适用于应用软件：ECU抽象层负责处理对I/O信号的访问，而服务层则提供：

操作系统功能 车辆网络通信与管理服务 存储服务（
NVRAM管理） 诊断服务（包括UDS通信、错误存储与故障处理）
ECU状态管理、模式管理 逻辑与时间程序流
监控（WdgManager）



任务

为应用程序、RTE和基础软件模块提供基本服务

。

属性

实现：主要与 μ C和ECU硬件无关 上层接口：与 μ C和ECU硬件无关

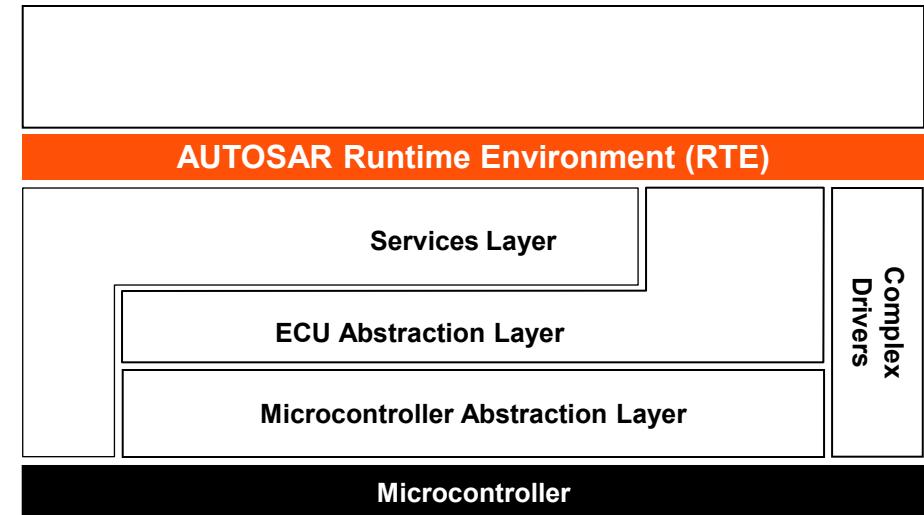
Architecture – Overview of Software Layers

AUTOSAR Runtime Environment (RTE)

The **RTE** is a layer providing communication services to the application software (AUTOSAR Software Components and/or AUTOSAR Sensor/Actuator components).

Above the RTE the software architecture style changes from “layered” to “component style”.

The AUTOSAR Software Components communicate with other components (inter and/or intra ECU) and/or services via the RTE.



Task

Make AUTOSAR Software Components independent from the mapping to a specific ECU.

Properties

Implementation: ECU and application specific (generated individually for each ECU)

Upper Interface: completely ECU independent

Architecture – Overview of Software Layers

AUTOSAR运行时环境 (RTE)

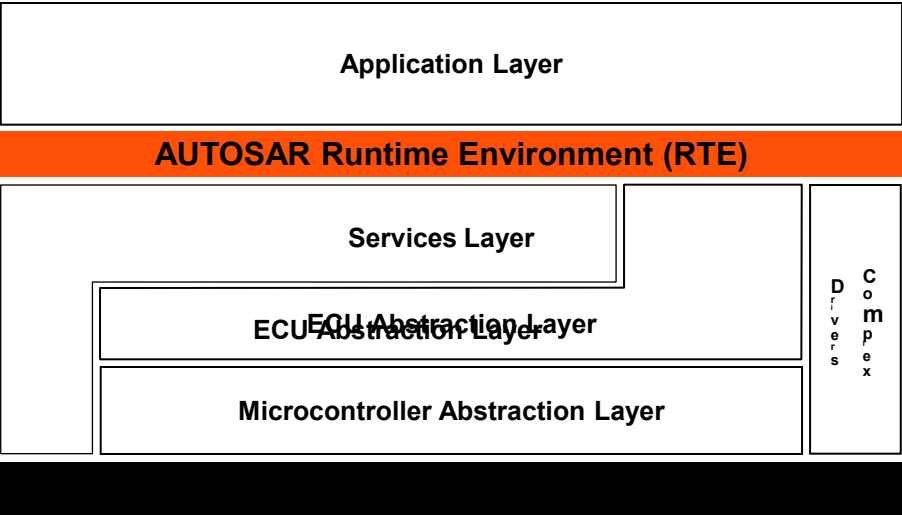
RTE是为应用软件（AUTOSAR软件组件和/或AUTOSAR传感器/执行器组件）提供通信服务的层。

在RTE之上，软件架构风格从“分层式”转变为“组件式”。

AUTOSAR软件组件通过RTE与其他组件（ECU间和/或ECU内）和/或服务进行通信。

任务
使AUTOSAR软件组件独立于特定ECU的映射。

属性
实现方式：ECU与应用特定（为每个ECU单独生成） 上层
接口：完全独立于ECU



Architecture – Overview of Software Layers

Introduction to types of services

The **Basic Software** can be subdivided into the following types of services:

- **Input/Output (I/O)**
Standardized access to sensors, actuators and ECU onboard peripherals
- **Memory**
Standardized access to internal/external memory (non volatile memory)
- **Crypto**
Standardized access to cryptographic primitives including internal/external hardware accelerators
- **Communication**
Standardized access to: vehicle network systems, ECU onboard communication systems and ECU internal SW
- **Off-board Communication**
Standardized access to: Vehicle-to-X communication, in vehicle wireless network systems, ECU off-board communication systems
- **System**
Provision of standardizable (operating system, timers, error memory) and ECU specific (ECU state management, watchdog manager) services and library functions

基本如**基础软件**可以细分为以下几种类型

服务：

输入/输出（I/O）对传感器、执行器和ECU车载外设的标准化访问

内存 对内部/外部存储器（非易失性存储器）的标准化访问

加密标准化访问加密原语，包括内部/外部硬件加速器

沟通

标准化访问：车辆网络系统、ECU车载通信系统及ECU内部软件

板外通信

标准化接入：车联万物通信、车载无线网络系统、ECU车外通信系统

系统

提供可标准化（操作系统、定时器、错误存储器）和ECU特定（ECU状态管理、看门狗管理器）的服务及库函数

Architecture – Introduction to Basic Software Module Types

Driver (internal)

A **driver** contains the functionality to control and access an internal or an external device.

Internal devices are located inside the microcontroller. Examples for internal devices are:

- Internal EEPROM
- Internal CAN controller
- Internal ADC

A driver for an internal device is called **internal driver** and is located in the Microcontroller Abstraction Layer.

Architecture – Introduction to Basic Software Module Types

驱动程序（内部）

驱动程序包含控制和访问内部或外部设备的功能。

内部设备位于微控制器内部。内部设备的例子有：

- 内部EEPROM

- 内部CAN控制器

- 内部ADC

内部设备的驱动程序称为内部驱动程序，位于微控制器抽象层。

Architecture – Introduction to Basic Software Module Types

Driver (external)

External devices are located on the ECU hardware outside the microcontroller. Examples for external devices are:

- External EEPROM
- External watchdog
- External flash

A driver for an external device is called **external driver** and is located in the ECU Abstraction Layer. It accesses the external device via drivers of the Microcontroller Abstraction Layer.

This way also components integrated in System Basis Chips (SBCs) like transceivers and watchdogs are supported by AUTOSAR.

- Example: a driver for an external EEPROM with SPI interface accesses the external EEPROM via the handler/driver for the SPI bus.

Exception:

The drivers for memory mapped external devices (e.g. external flash memory) may access the microcontroller directly. Those external drivers are located in the Microcontroller Abstraction Layer because they are microcontroller dependent.

外部设备位于微控制器之外的ECU硬件上。外部设备的例子有：

外部EEPROM

外部看门狗

外部闪存

外部设备的驱动程序称为外部驱动程序，**位于ECU抽象层。它通过微控制器抽象层的驱动程序访问外部设备。**

这种方式也支持集成在系统基础芯片（SBCs）中的组件，如收发器和看门狗，由AUTOSAR提供支持。

示例：一个带有SPI接口的外部EEPROM驱动程序通过SPI总线处理程序/驱动程序访问外部EEPROM。

例外：

用于内存映射外部设备（例如外部闪存）的驱动程序可以直接访问微控制器。这些外部驱动程序位于微控制器抽象层中，因为它们依赖于特定的微控制器。

Architecture – Introduction to Basic Software Module Types

Interface

An **Interface (interface module)** contains the functionality to abstract from modules which are architecturally placed below them. E.g., an interface module which abstracts from the hardware realization of a specific device. It provides a generic API to access a specific type of device independent on the number of existing devices of that type and independent on the hardware realization of the different devices.

The interface does not change the content of the data.

In general, interfaces are located in the **ECU Abstraction Layer**.

Example: an interface for a CAN communication system provides a generic API to access CAN communication networks independent on the number of CAN Controllers within an ECU and independent of the hardware realization (on chip, off chip).

接口

接口（接口模块）包含对架构上位于其下方的模块进行抽象的功能。例如，一个接口模块可以对特定设备的硬件实现进行抽象。它提供一个通用API来访问特定类型的设备，该API独立于该类型现有设备的数量，也独立于不同设备的硬件实现。

接口不会改变数据的内容。

通常，接口位于ECU抽象层中。

示例：CAN通信系统的接口提供了一个通用API，用于访问CAN通信网络，该接口独立于ECU内CAN控制器的数量，也独立于硬件实现方式（片上或片外）。

Architecture – Introduction to Basic Software Module Types

Handler

A **handler** is a specific interface which controls the concurrent, multiple and asynchronous access of one or multiple clients to one or more drivers. I.e. it performs buffering, queuing, arbitration, multiplexing.

The handler does not change the content of the data.

Handler functionality is often incorporated in the driver or interface (e.g. SPIHandlerDriver, ADC Driver).

Architecture – Introduction to Basic Software Module Types

处理器 handler

处理器是一种特定接口，用于控制一个或多个客户端对一个或多个驱动器的并发、多路和异步访问。即它执行缓冲、排队、仲裁和多路复用功能。

handler

处理程序不会改变数据的内容。

handler

处理程序功能通常集成在驱动程序或接口中（例如SPIHandlerDriver、ADC Driver）。

Architecture – Introduction to Basic Software Module Types

Manager

A **manager** offers specific services for multiple clients. It is needed in all cases where pure handler functionality is not enough to abstract from multiple clients.

Besides handler functionality, a manager can evaluate and change or adapt the content of the data.

In general, managers are located in the **Services Layer**

Example: The NVRAM manager manages the concurrent access to internal and/or external memory devices like flash and EEPROM memory. It also performs distributed and reliable data storage, data checking, provision of default values etc.

Manager

经理为多个客户提供特定服务。在所有纯粹的处理程序功能不足以从多个客户中抽象出来的情况下，都需要它。

除了处理程序功能外，管理器还可以评估、更改或调整数据的内容。

Manager

通常，管理者位于服务层

示例：NVRAM管理器管理对内部和/或外部存储设备（如闪存和EEPROM）的并发访问。它还执行分布式可靠数据存储、数据检查、提供默认值等功能。

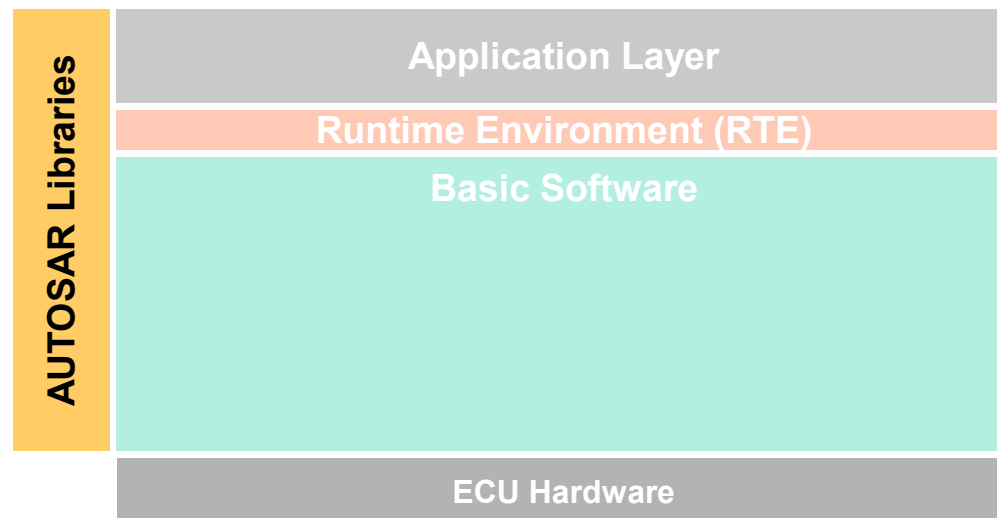
Architecture – Overview of Software Layers

Introduction to Libraries (1)

Libraries are a collection of functions for related purposes

Libraries:

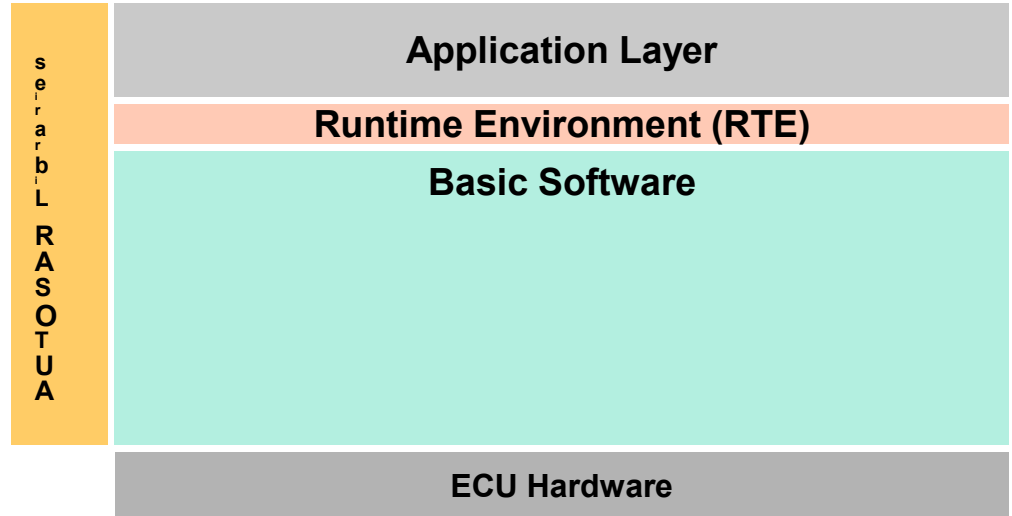
- can be called by BSW modules (that including the RTE), SW-Cs, libraries or integration code
- run in the context of the caller in the same protection environment
- can only call libraries, no calls to BSW modules are allowed
- are re-entrant
- do not have internal states
- do not require any initialization
- are synchronous, i.e. they do not have wait points
- may have a configuration, but this is discouraged



库是用于相关目的函数集合

库:

可由BSW模块（包括RTE）、SW-C、库或集成代码调用 在调用者的上下文中运行，处于相同的保护环境
仅能调用库，不允许调用BSW模块 可重入 无内部状态 无需任何初始化 同步执行，即无等待点
可配置，但不建议使用配置



Architecture – Overview of Software Layers

Introduction to Libraries (2)

The following libraries are specified within AUTOSAR:

- Fixed point mathematical (Mfx),
- Floating point mathematical (Mfl),
- Interpolation for fixed point data (Ifx),
- Interpolation for floating point data (IfI),
- Extended functions (e.g. 64bits calculation, filtering, etc.) (Efx),
- Bit handling (Bfx),
- E2E communication (E2E),
- CRC calculation (Crc),
- Atomic multicore safe operations (Bmc),
- Memory standard functions (Msf)

AUTOSAR中规定了以下库：

定点数学运算 (Mfx)， 浮点数学运算 (Mfl)， 定点数据插值 (Ifx)， 浮点数据插值 (IfI)， 扩展功能（例如64位计算、滤波等） (Efx)， 位处理 (Bfx)， E2E通信 (E2E)， CRC计算 (Crc)， 原子多核安全操作 (Bmc)， 内存标准函数 (Msf)

Table of contents

1. Architecture

1. Overview of Software Layers
2. **Content of Software Layers**
3. Content of Software Layers in Multi-Core Systems
4. Content of Software Layers in Mixed-Critical Systems
5. Overview of Modules
6. Interfaces: General Rules
7. Interfaces: Interaction of Layers
8. Overview of CP Software Clusters

2. Configuration

3. Integration and Runtime Aspects

Table of contents

1. 架构

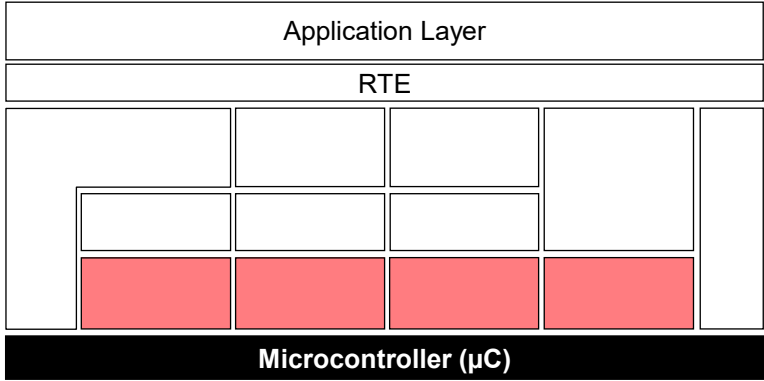
1. 软件层概述 2. 软件层内容 3. 多核系统中的软件层内容 4
. 混合关键系统中的软件层内容 5. 模块概述 6. 接口：通用
规则 7. 接口：层间交互 8. CP软件集群概述

2. 配置

3. 集成与运行时方面

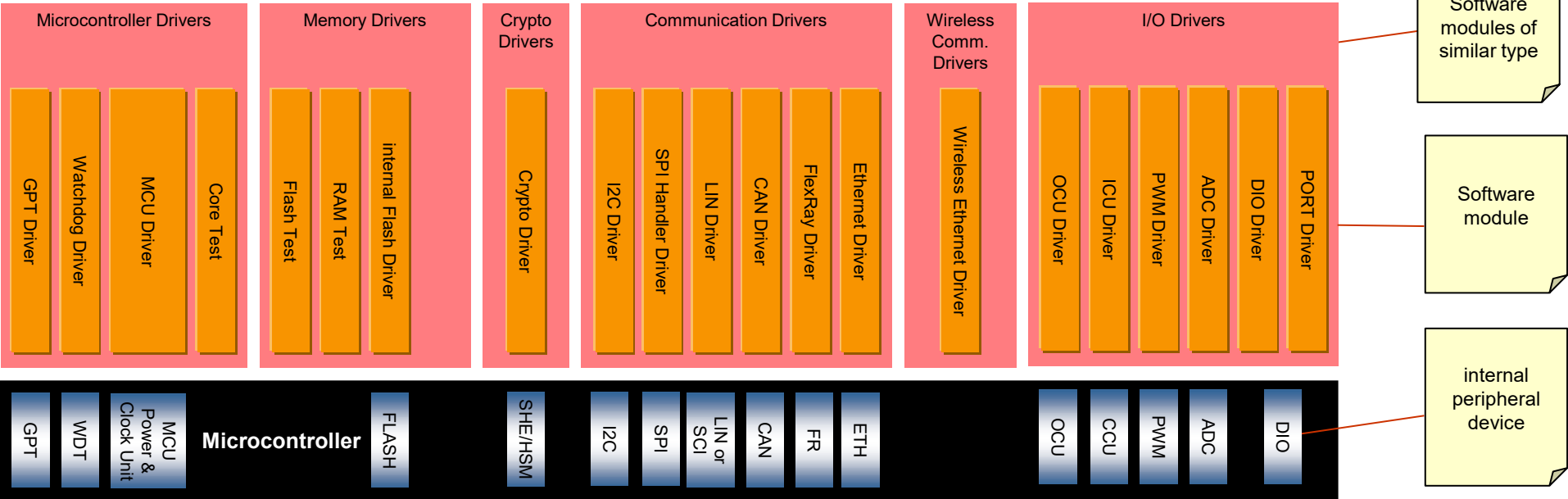
Architecture – Content of Software Layers

Microcontroller Abstraction Layer



The **µC Abstraction Layer** consists of the following module groups:

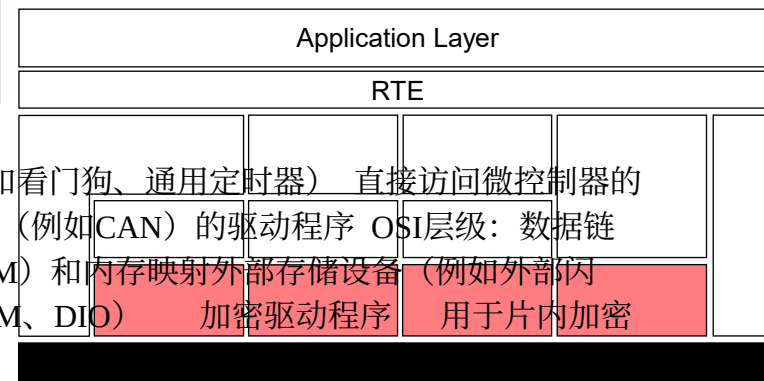
- **Microcontroller Drivers**
Drivers for internal peripherals (e.g. Watchdog, General Purpose Timer)
Functions with direct µC access (e.g. Core test)
- **Communication Drivers**
Drivers for ECU onboard (e.g. SPI) and vehicle communication (e.g. CAN).
OSI-Layer: Part of Data Link Layer
- **Memory Drivers**
Drivers for on-chip memory devices (e.g. internal Flash, internal EEPROM) and memory mapped external memory devices (e.g. external Flash)
- **I/O Drivers:**
Drivers for analog and digital I/O (e.g. ADC, PWM, DIO)
- **Crypto Drivers** Drivers for on-chip crypto devices like SHE or HSM
- **Wireless Communication Drivers:** Drivers for wireless network systems (in-vehicle or off-board communication)



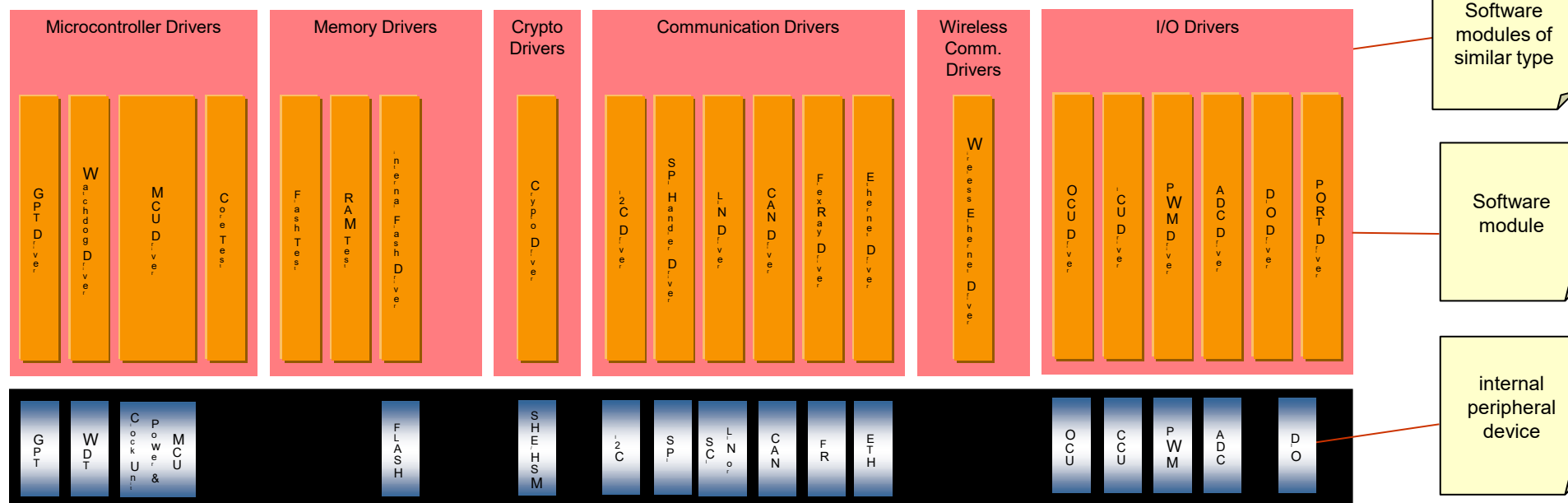
Architecture – Content of Software Layers

微控制器抽象层

微控制器抽象层包含以下模块组： 微控制器驱动程序 内部外设驱动程序（例如看门狗、通用定时器） 直接访问微控制器的功能（例如核心测试） 通信驱动程序 用于ECU板载通信（例如SPI）和车辆通信（例如CAN）的驱动程序 OSI层级：数据链路层的一部分 存储器驱动程序 用于片内存储设备（例如内部闪存、内部EEPROM）和内存映射外部存储设备（例如外部闪存）的驱动程序 I/O驱动程序： 用于模拟和数字I/O的驱动程序（例如ADC、PWM、DIO） 加密驱动程序 用于片内加密设备（如SHE或HSM）的驱动程序



Wireless Communication Drivers: Drivers for wireless network systems (in-vehicle or off-board communication)

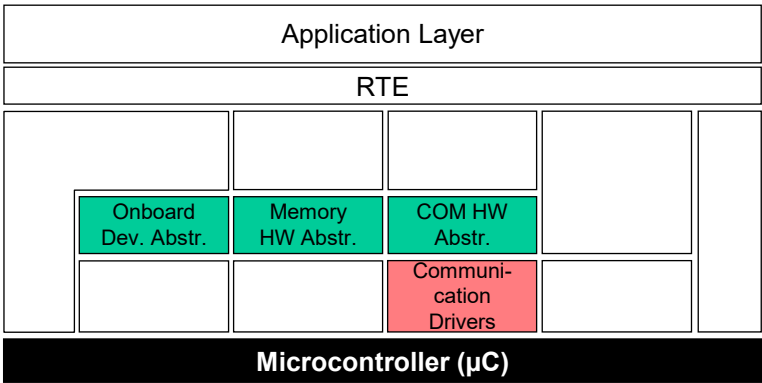


Architecture – Content of Software Layers

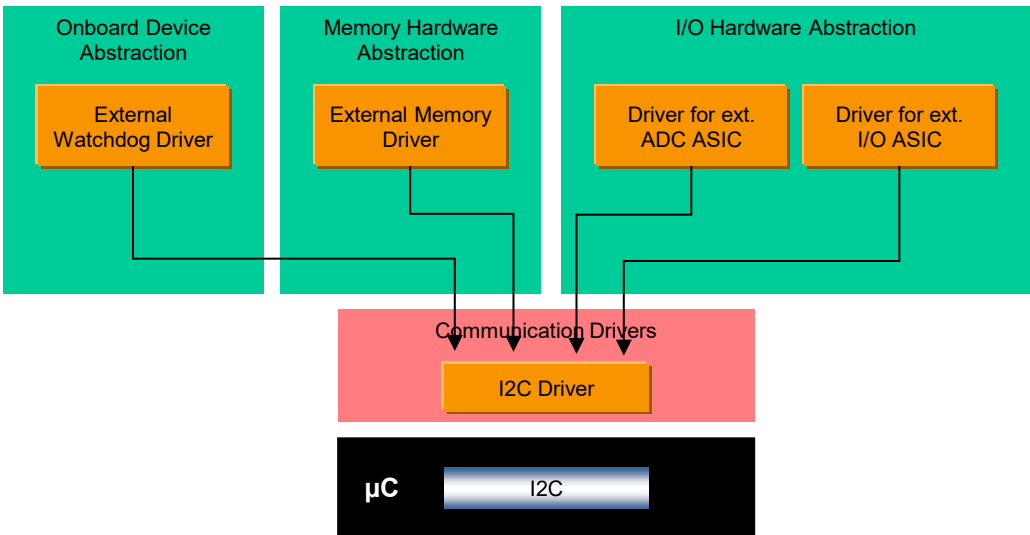
Microcontroller Abstraction Layer: I2C Driver

The **I2C Driver** allows concurrent access of several nodes to one or more I2C busses.

- I²C (Inter-Integrated Circuit) is a 2-wire serial data bus widely used in automotive sensors or actuators.
- Examples:
 - Temperature Sensor
 - 3-axis Accelerometer Sensor
 - Air Pressure Sensor
 - EEPROM
 - ...



Example:



Architecture – Content of Software Layers

微控制器抽象层：I2C 驱动程序

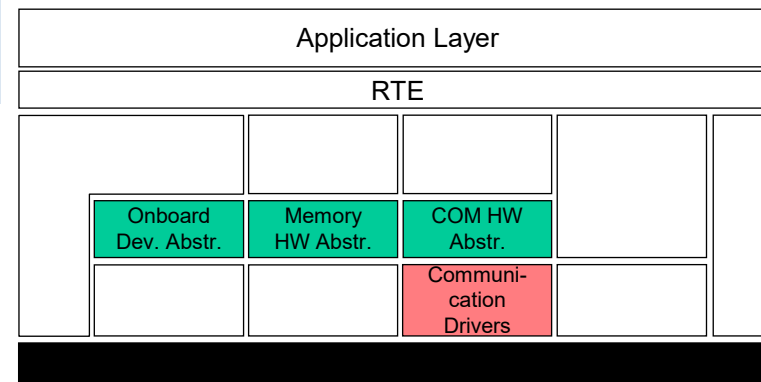
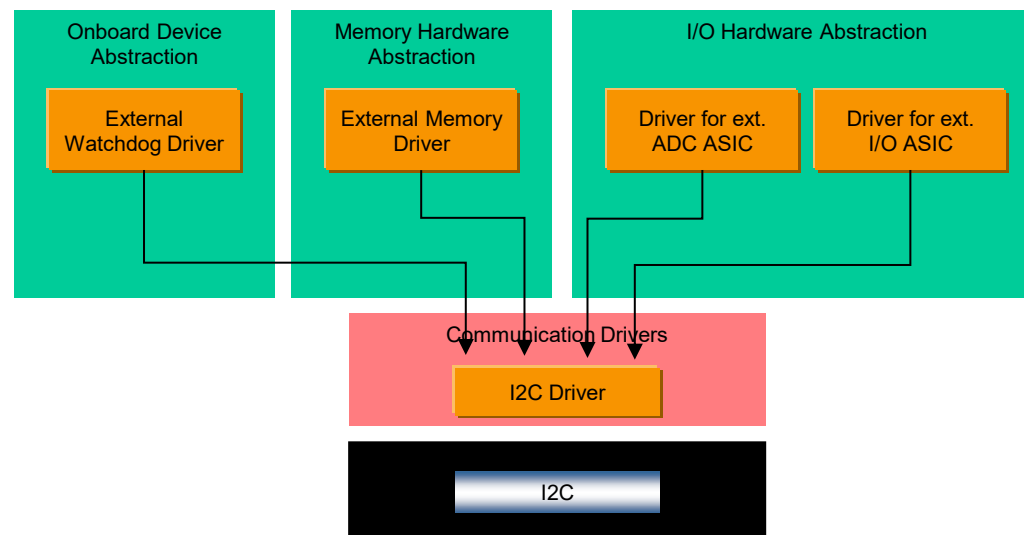
I2C驱动程序允许多个节点同时访问一个或多个I2C总线。

I²C（Inter-Integrated Circuit）是一种广泛应用于汽车传感器或执行器的2线串行数据总线。

示例：

- 温度传感器
- 三轴加速度传感器
- 气压传感器
- 电可擦可编程只读存储器
- ...

示例：

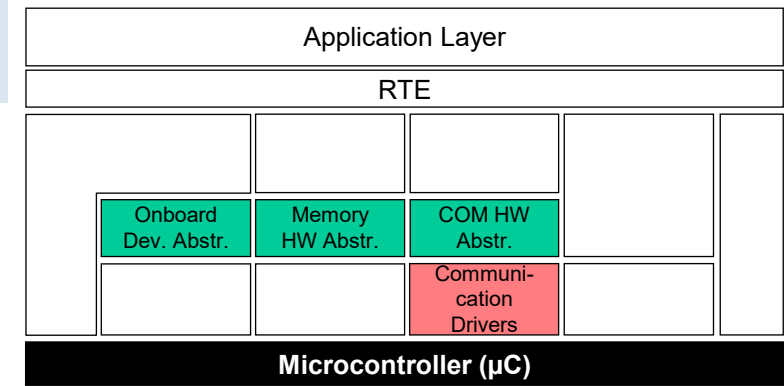


Architecture – Content of Software Layers

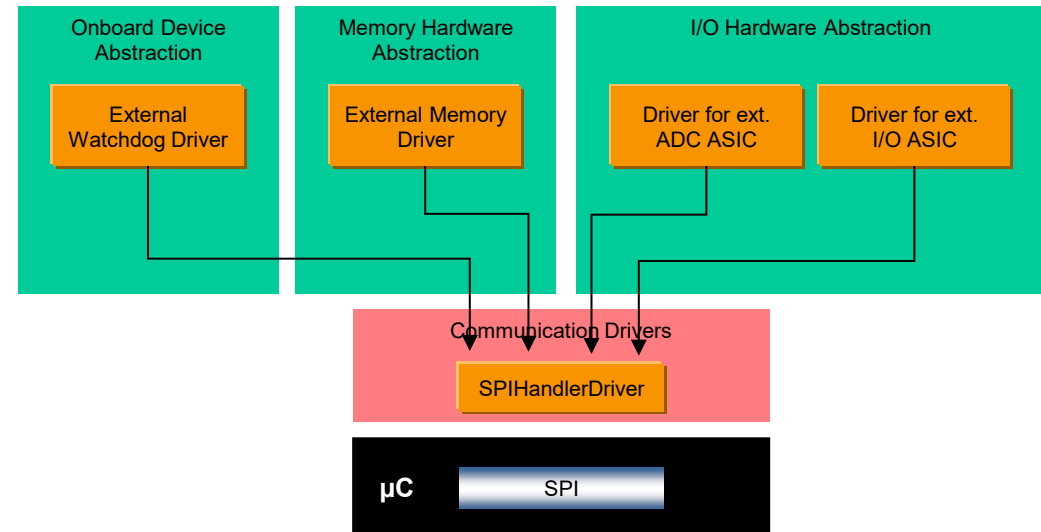
Microcontroller Abstraction Layer: SPIHandlerDriver

The **SPIHandlerDriver** allows concurrent access of several clients to one or more SPI busses.

To abstract all features of a SPI microcontroller pins dedicated to Chip Select, those shall directly be handled by the SPIHandlerDriver. That means those pins shall not be available in DIO Driver.



Example:

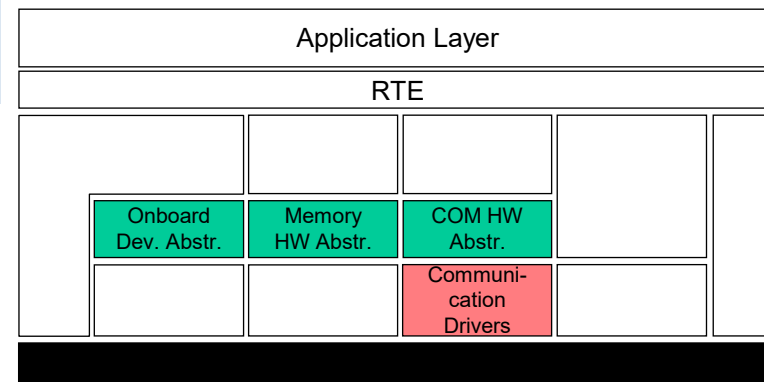


Architecture – Content of Software Layers

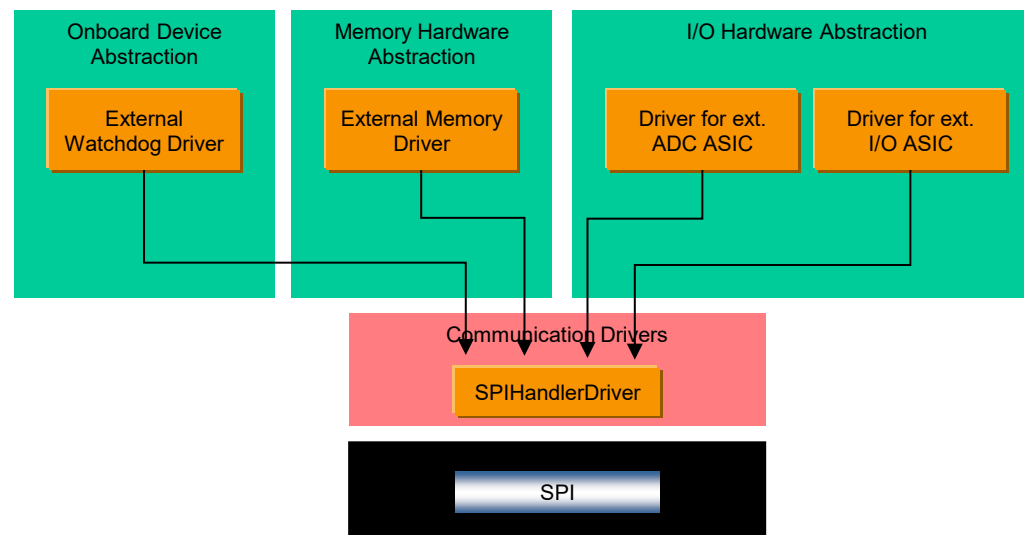
微控制器抽象层：SPIHandlerDriver

SPIHandlerDriver允许多个客户端并发访问一个或多个SPI总线。

为了抽象SPI微控制器中专门用于片选引脚的所有特性，这些引脚应直接由SPIHandlerDriver处理。这意味着这些引脚不应在DIO驱动器中可用。



示例：



Architecture – Content of Software Layers

Complex Drivers

A **Complex Driver** is a module which implements non-standardized functionality within the basic software stack.

An **example is to** implement complex sensor evaluation and actuator control with direct access to the μC using specific interrupts and/or complex μC peripherals (like PCP, TPU), e.g.

- Injection control
- Electric valve control
- Incremental position detection

Task:

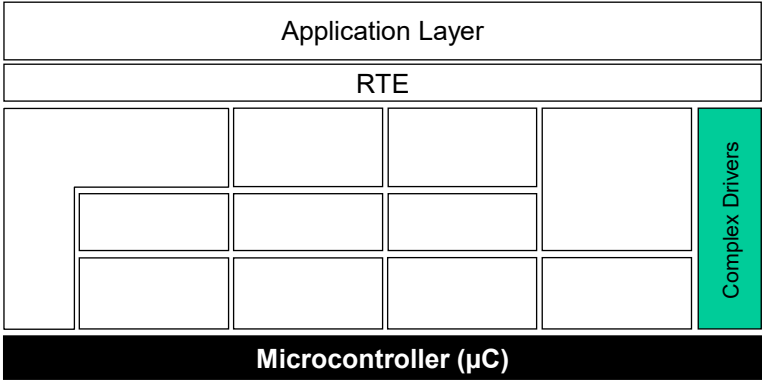
Fulfill the special functional and timing requirements for handling complex sensors and actuators

Properties:

Implementation: highly μC , ECU and application dependent

Upper Interface to SW-Cs: specified and implemented according to AUTOSAR (AUTOSAR interface)

Lower interface: restricted access to Standardized Interfaces



Example:



Architecture – Content of Software Layers

复杂驱动因素

复杂驱动程序是在基础软件栈内实现非标准化功能的模块。

一个例子是使用特定的中断和/或复杂的微控制器外设（如PCP、TPU）直接访问μC，来实现复杂的传感器评估和执行器控制。

注入控制

电动阀门控制

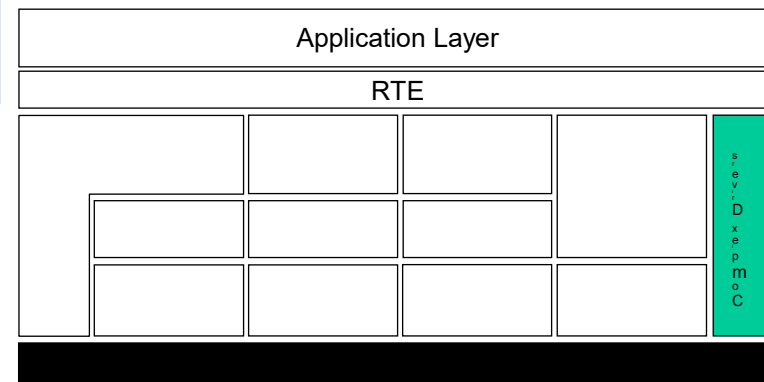
增量位置检测

任务：

满足处理复杂传感器和执行器的特殊功能和时序要求

属性：

实现：高度依赖于微控制器、ECU和应用程序 上层接口至软件组件：根据AUTOSAR规范指定与实现（AUTOSAR接口） 下层接口：对标准化接口的受限访问



示例：



Architecture – Content of Software Layers

ECU Abstraction: I/O Hardware Abstraction

The **I/O Hardware Abstraction** is a group of modules which abstracts from the **location** of peripheral I/O devices (on-chip or on-board) and the **ECU hardware layout** (e.g. μ C pin connections and signal level inversions). The I/O Hardware Abstraction does not abstract from the sensors/actuators!

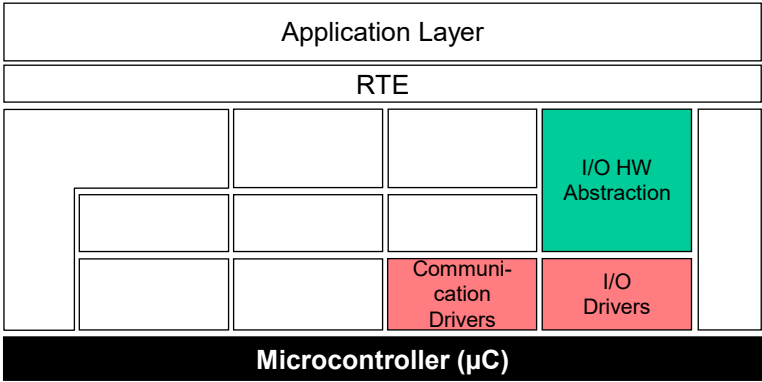
The different I/O devices might be accessed via an I/O signal interface.

Task:

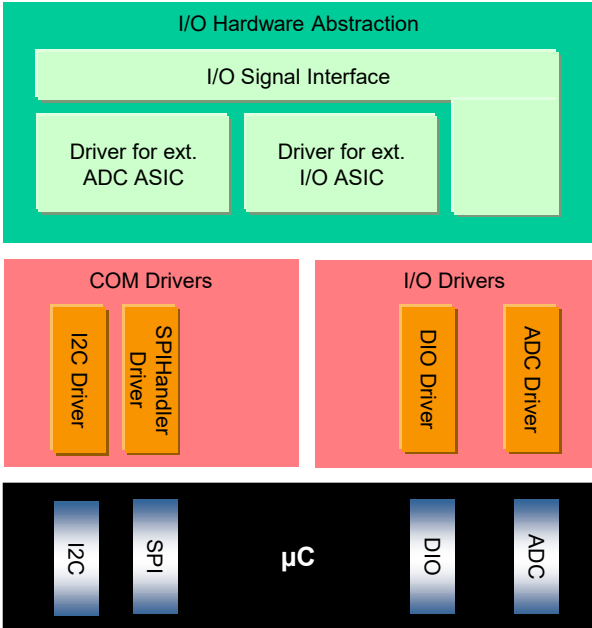
Represent I/O signals as they are connected to the ECU hardware (e.g. current, voltage, frequency).
Hide ECU hardware and layout properties from higher software layers.

Properties:

Implementation: μ C independent, ECU hardware dependent
Upper Interface: μ C and ECU hardware independent, dependent on signal type specified and implemented according to AUTOSAR (AUTOSAR interface)



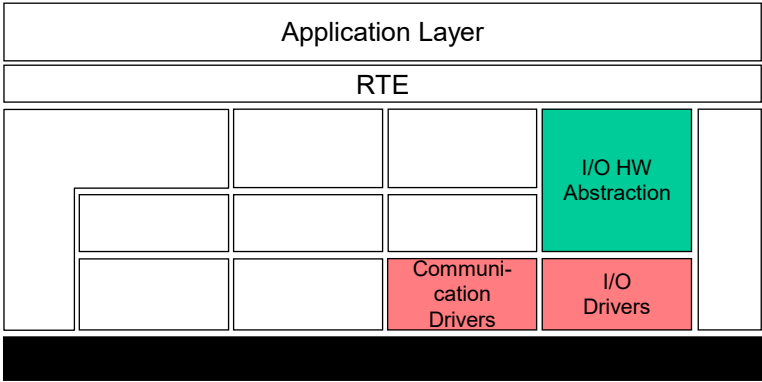
Example:



Architecture – Content of Software Layers

ECU抽象化: I/O Hardware Abstraction

I/O硬件抽象是一组模块，它从外围I/O设备（片上或板上）的**location**和**ECU hardware layout**（例如微控制器引脚连接和信号电平反转）中抽象出来。I/O硬件抽象不对传感器/执行器进行抽象！



不同的I/O设备可能通过I/O信号接口进行访问。

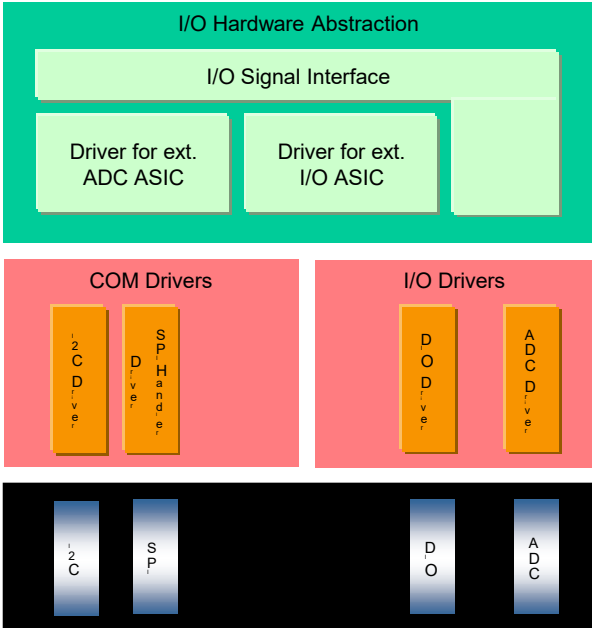
任务：

将I/O信号表示为它们连接到ECU硬件的方式（例如电流、电压、频率）。向更高层的软件层隐藏ECU硬件和布局属性。

属性：

实现：μC独立，ECU硬件依赖 上层接口μC与ECU硬件独立，依赖于指定信号类型，并根据AUTOSAR（AUTOSAR接口）实现

示例：



Architecture – Content of Software Layers

ECU Abstraction: Communication Hardware Abstraction

The **Communication Hardware Abstraction** is a group of modules which abstracts from the **location** of communication controllers and the **ECU hardware layout**. For all communication systems a **specific** Communication Hardware Abstraction is required (e.g. for LIN, CAN, FlexRay).

Example: An ECU has a microcontroller with 2 internal CAN channels and an additional on-board ASIC with 4 CAN controllers. The CAN-ASIC is connected to the microcontroller via SPI.

The communication drivers are accessed via bus specific interfaces (e.g. CAN Interface).

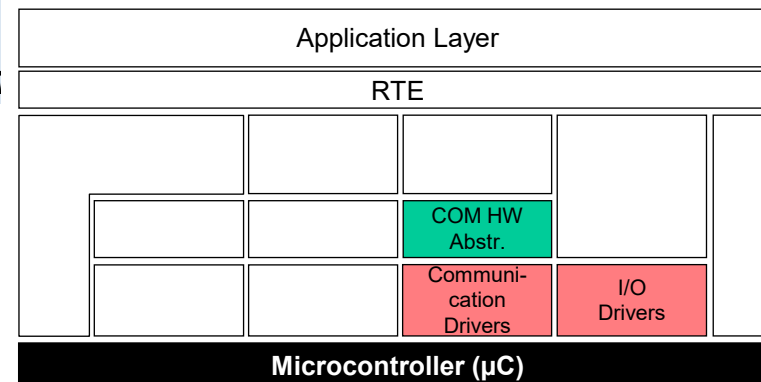
Task:

Provide equal mechanisms to access a bus channel regardless of it's location (on-chip / on-board)

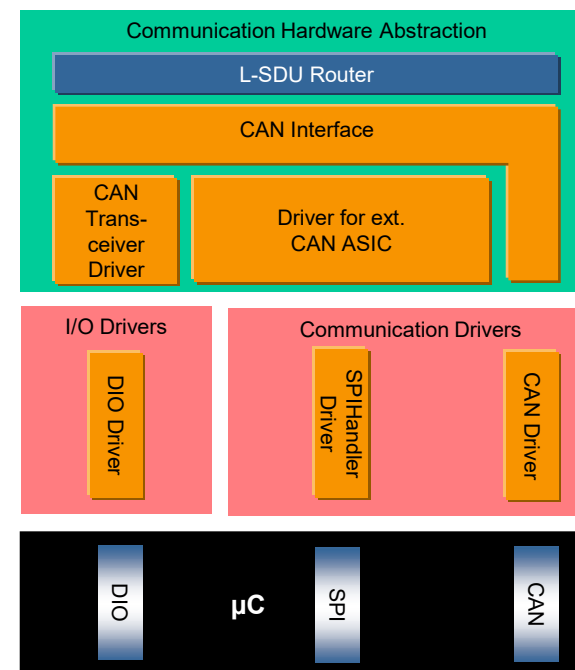
Properties:

Implementation: μ C independent, ECU hardware dependent and external device dependent

Upper Interface: bus dependent, μ C and ECU hardware independent



Example:



Architecture – Content of Software Layers

ECU抽象化: *Communication Hardware Abstraction*

通信硬件抽象是一组模块，它从通信控制器的**location**和**ECU hardware layout**中抽象出来。对于所有通信系统，都需要一个**specific**通信硬件抽象（例如LIN、CAN、FlexRay）。

示例：一个ECU拥有一个带有2个内部CAN通道的微控制器，以及一个带有4个CAN控制器的额外板载ASIC。该CAN-ASIC通过SPI与微控制器连接。

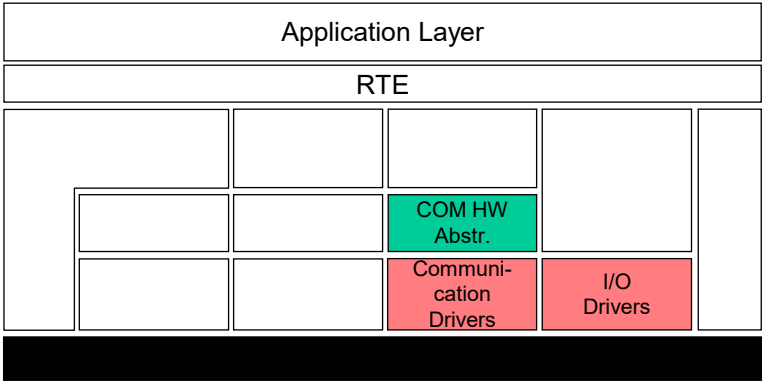
通信驱动程序通过总线特定接口（例如CAN接口）进行访问。

任务：

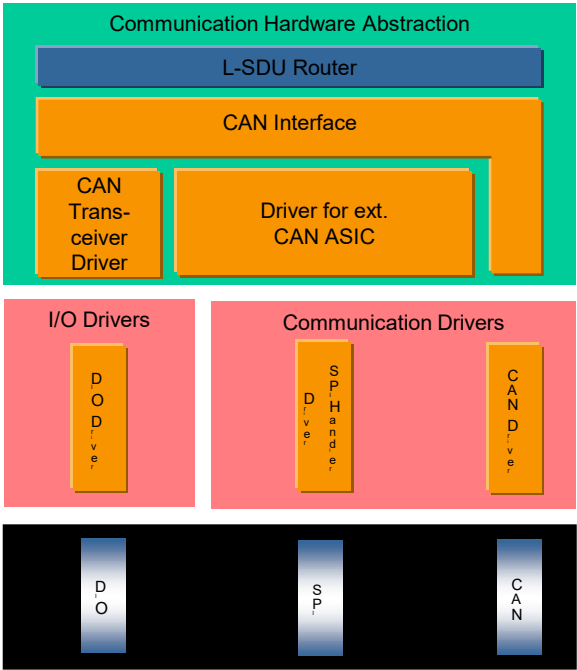
无论其位置（片上/板上）如何，都提供平等的机制来访问总线通道。

属性：

实现：μC独立，ECU硬件依赖及外部设备依赖
 上层接口：总线依赖，μC与ECU硬件独立



示例：



Architecture – Content of Software Layers

Scope: Memory Hardware Abstraction

The **Memory Hardware Abstraction** is a group of modules which abstracts from the **location** of peripheral memory devices (on-chip or on-board) and the **ECU hardware layout**.

Example: on-chip EEPROM and external EEPROM devices are accessible via the same mechanism.

The memory drivers are accessed via memory specific abstraction/emulation modules (e.g. EEPROM Abstraction).

By emulating an EEPROM abstraction on top of Flash hardware units a common access via Memory Abstraction Interface to both types of hardware is enabled.

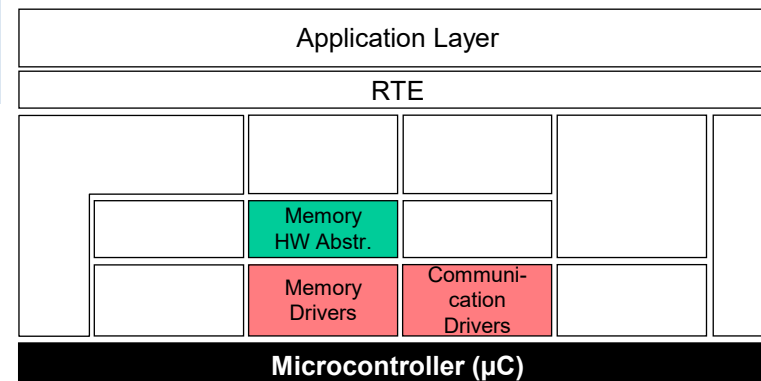
Task:

Provide equal mechanisms to access internal (on-chip) and external (on-board) memory devices and type of memory hardware (EEPROM, Flash).

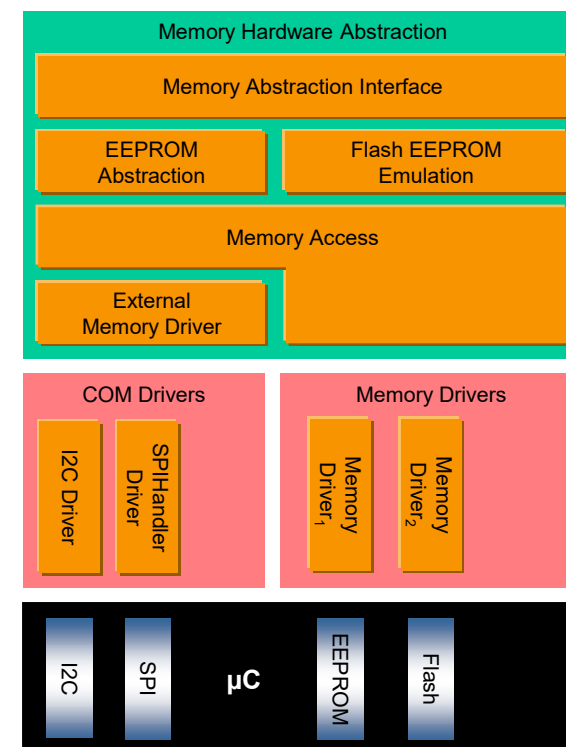
Properties:

Implementation: μ C independent, external device dependent

Upper Interface: μ C, ECU hardware and memory device independent



Example:



Architecture – Content of Software Layers

范围：内存硬件抽象

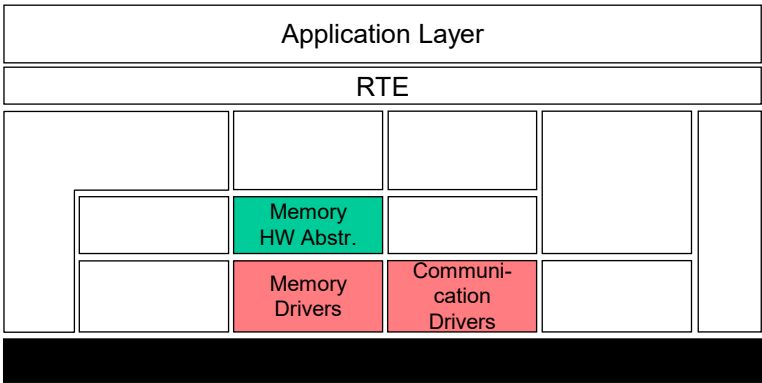
内存硬件抽象是一组模块，它从外围存储设备（片上或板上）的 **location** 和 **ECU hardware layout** 中抽象出来。

示例：片上EEPROM和外部EEPROM设备可通过同一机制访问。

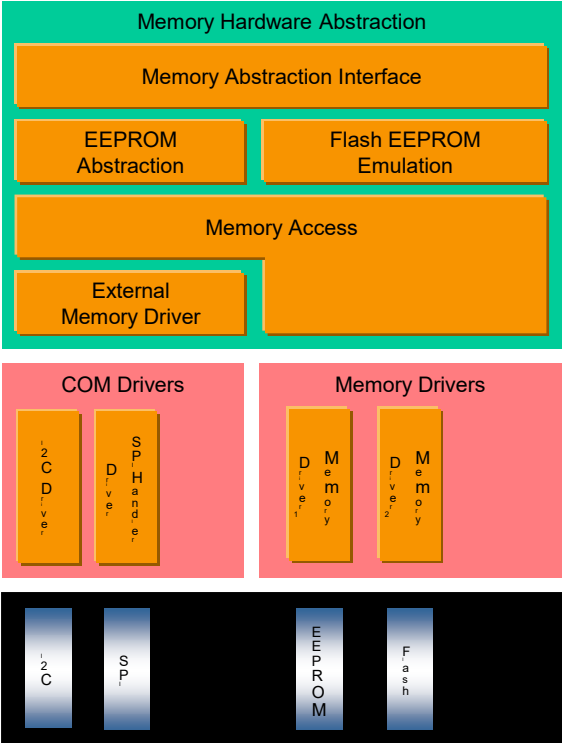
内存驱动程序通过内存特定的抽象/仿真模块（例如EEPROM抽象）进行访问。通过在Flash硬件单元之上仿真EEPROM抽象，实现了通过内存抽象接口对两种硬件类型的统一访问。

任务：
提供访问内部（片上）和外部（板上）存储设备以及存储器硬件类型（EEPROM、Flash）的平等机制。

属性：
实现：μC独立，外部设备依赖 上层接口μC、ECU硬件和存储设备独立



示例：



Architecture – Content of Software Layers

Onboard Device Abstraction

The **Onboard Device Abstraction** contains drivers for ECU onboard devices which cannot be seen as sensors or actuators like internal or external watchdogs. Those drivers access the ECU onboard devices via the μ C Abstraction Layer.

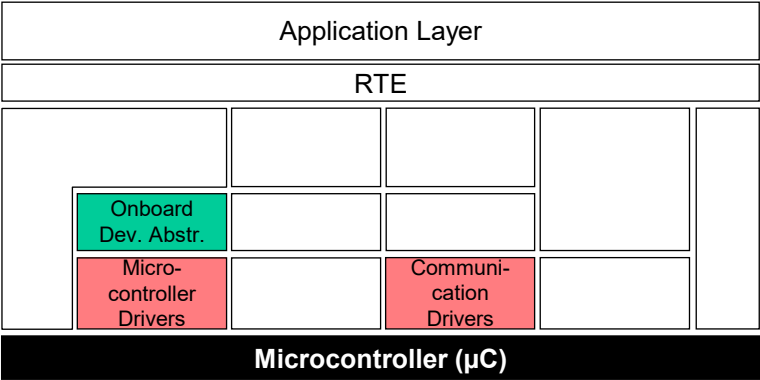
Task:

Abstract from ECU specific onboard devices.

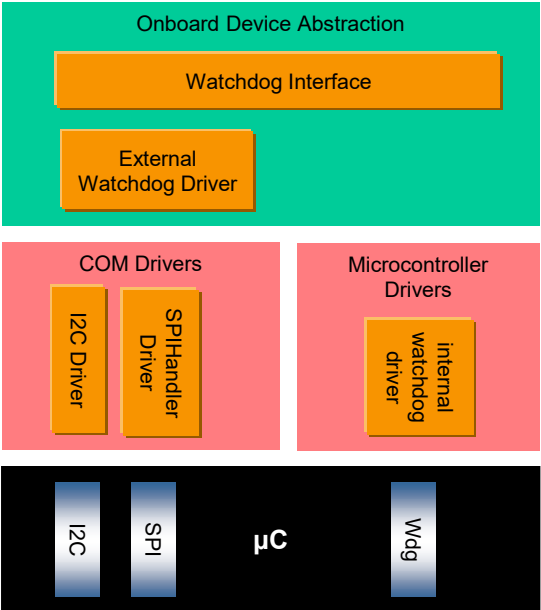
Properties:

Implementation: μ C independent, external device dependent

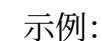
Upper Interface: μ C independent, partly ECU hardware dependent



Example:



属性：
实现：μC独立，外部设备依赖 上层接口：μC
独立，部分依赖ECU硬件



Architecture – Content of Software Layers

Scope: Crypto Hardware Abstraction

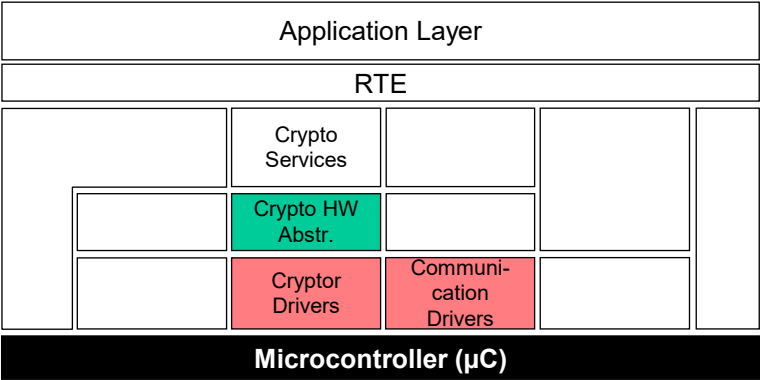
The **Crypto Hardware Abstraction** is a group of modules which abstracts from the **location** of cryptographic primitives (internal- or external hardware or software-based).

Example: AES primitive is realized in SHE or provided as software library

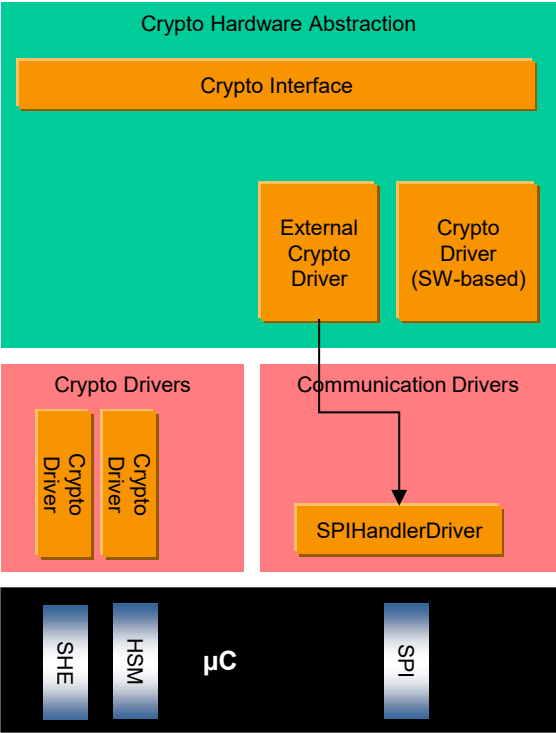
Task:
Provide equal mechanisms to access internal (on-chip) and software cryptographic devices.

Properties:
Implementation: μ C independent

Upper Interface: μ C, ECU hardware and crypto device independent



Example:



Architecture – Content of Software Layers

范围：加密硬件抽象

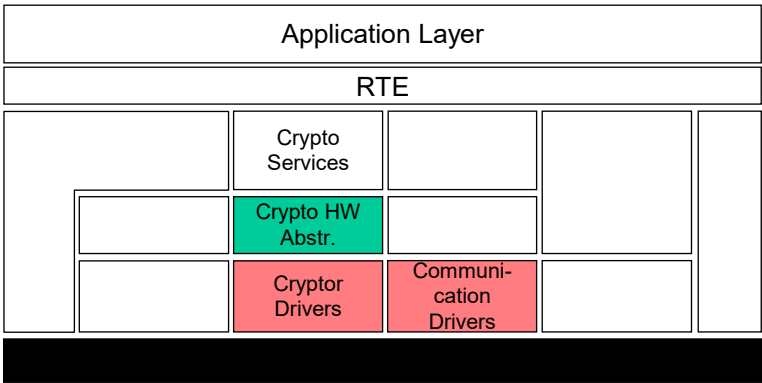
加密硬件抽象是一组模块，它从密码原语（基于内部或外部硬件或软件）的 **location** 中抽象出来。

示例：AES原语在SHE中实现或作为软件库提供

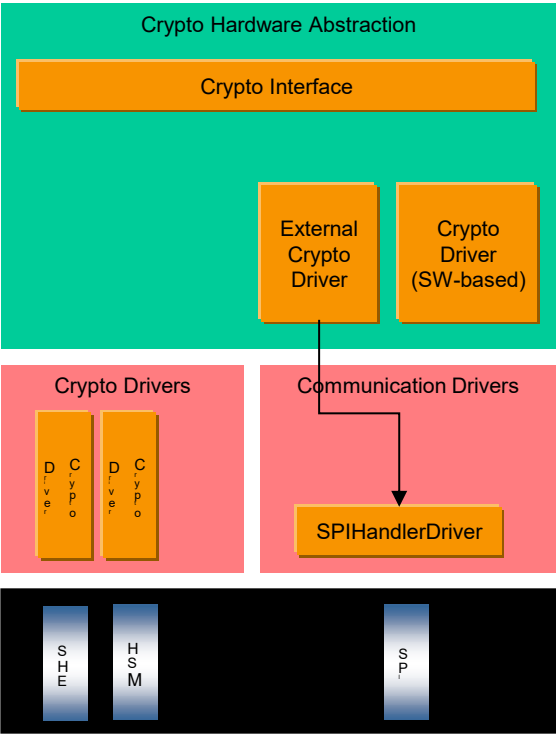
任务：
为访问内部（片上）和软件加密设备提供平等的机制。

属性：
实现：μC 独立

上层接口：独立于μC、ECU硬件和加密设备



示例：



Architecture – Content of Software Layers

Services: Crypto Services

The **Crypto Services** consist of three modules

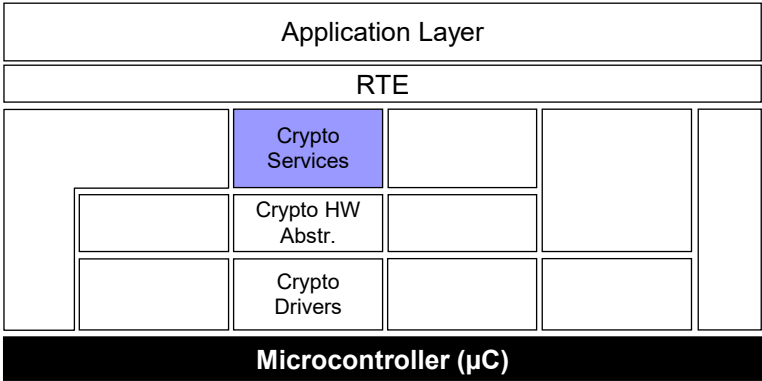
- the Crypto Service Manager is responsible for the management of cryptographic jobs
- the Key Manager interacts with the key provisioning master (either in NVM or Crypto Driver) and manages the storage and verification of certificate chains
- The Intrusion Detection System Manager is responsible for handling security events reported by BSW modules or SW-C

Task:

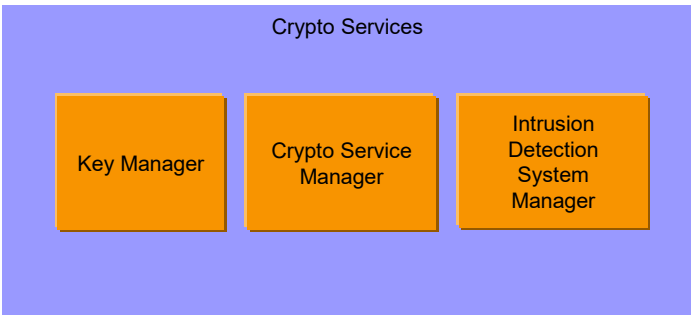
Provide cryptographic primitives, IDS services and key storage to the application in a uniform way.
Abstract from hardware devices and properties.

Properties:

Implementation: μ C and ECU hardware independent, highly configurable
Upper Interface: μ C and ECU hardware independent specified and implemented according to AUTOSAR (AUTOSAR interface)



Example:



Architecture – Content of Software Layers

服务：加密服务

加密服务由三个模块组成

加密服务管理器负责管理加密作业 密钥管理器与
密钥配置主控（位于NVM或加密驱动中）交互，并管理
证书链的存储与验证 入侵检测系统管理器负责处理
由BSW模块或SW-C报告的安全事件

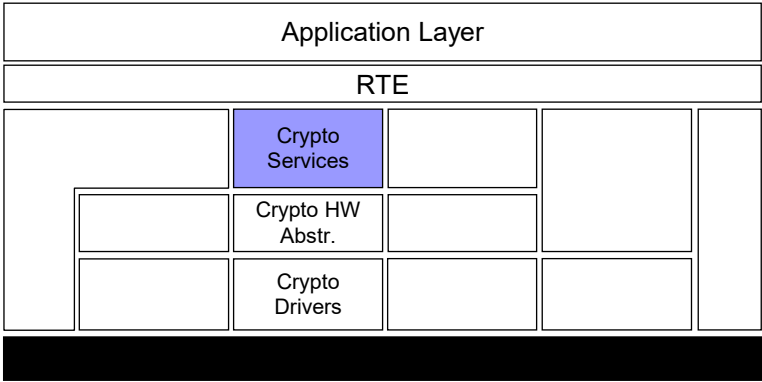
任务：

以统一的方式为应用程序提供加密原语、IDS服务和密
钥存储。抽象于硬件设备和属性之外。

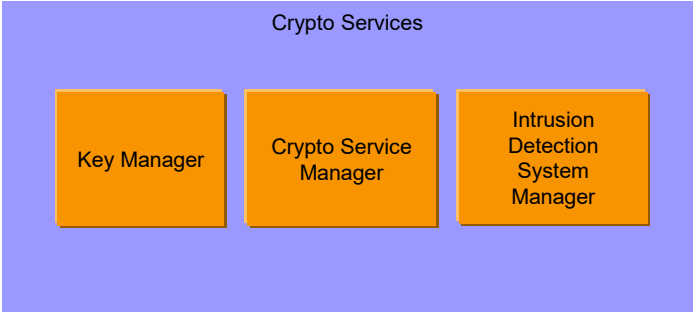
属性：

实现：μC与ECU硬件无关，高度可配置 上层接口μC与
ECU硬件无关，依据AUTOSAR规范制定与实现

(AUTOSAR接口)



示例：



Architecture – Content of Software Layers

Communication Services – General

The **Communication Services** are a group of modules for vehicle network communication (CAN, LIN, FlexRay and Ethernet). They interface with the communication drivers via the communication hardware abstraction.

Task:

Provide a uniform interface to the vehicle network for communication.

Provide uniform services for network management

Provide uniform interface to the vehicle network for diagnostic communication

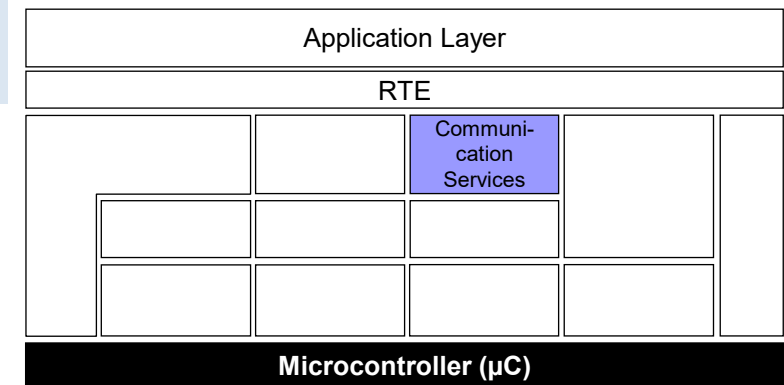
Hide protocol and message properties from the application.

Properties:

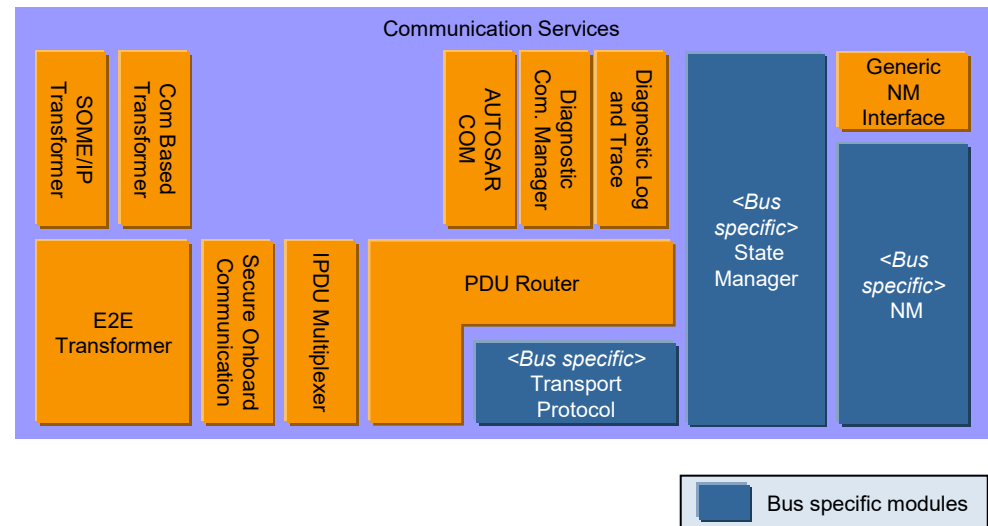
Implementation: μ C and ECU HW independent, partly dependent on bus type

Upper Interface: μ C, ECU hardware and bus type independent

The communication services will be detailed for each relevant vehicle network system on the following pages.



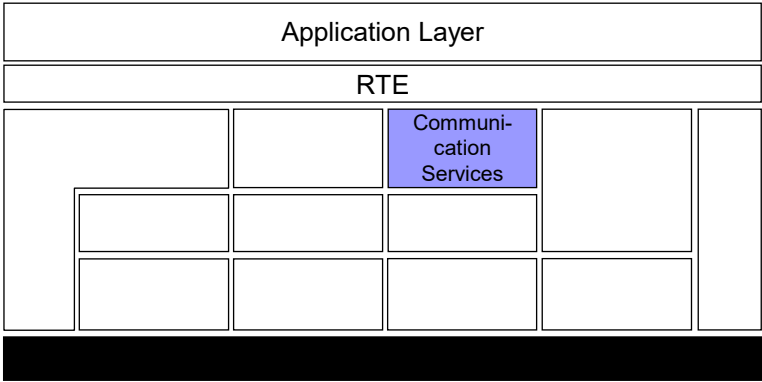
Example:



Architecture – Content of Software Layers

通信服务 - 通用

通信服务是一组用于车辆网络通信（CAN、LIN、FlexRay和以太网）的模块。它们通过通信硬件抽象层与通信驱动程序进行交互。



任务：
为车辆网络通信提供统一接口。 为网络管理提供统一服务。
为车辆网络提供统一接口。

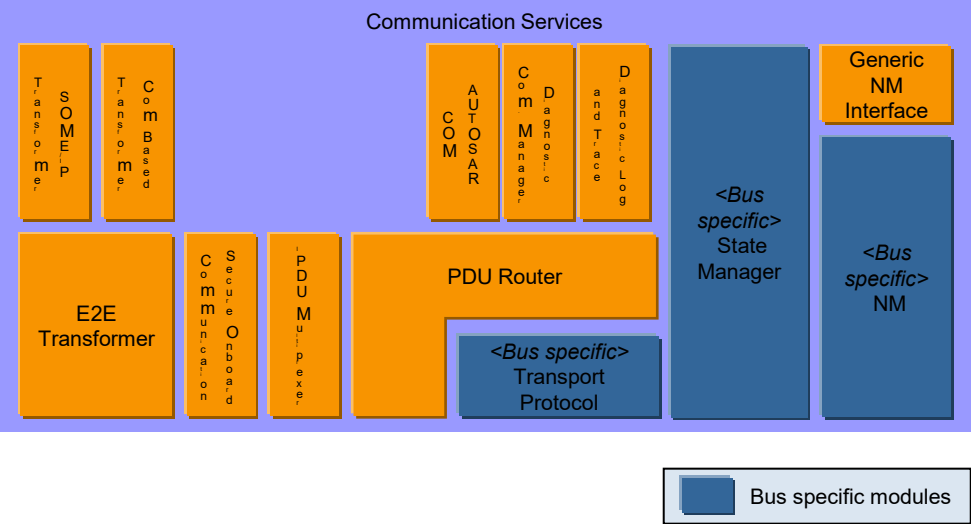
诊断通信
对应用程序隐藏协议和消息属性。

属性：
实现：μC和ECU硬件独立，部分依赖于总线类型

上层接口：独立于μC、ECU硬件和总线类型

相关车辆网络系统的通信服务将在以下页面中详细说明。

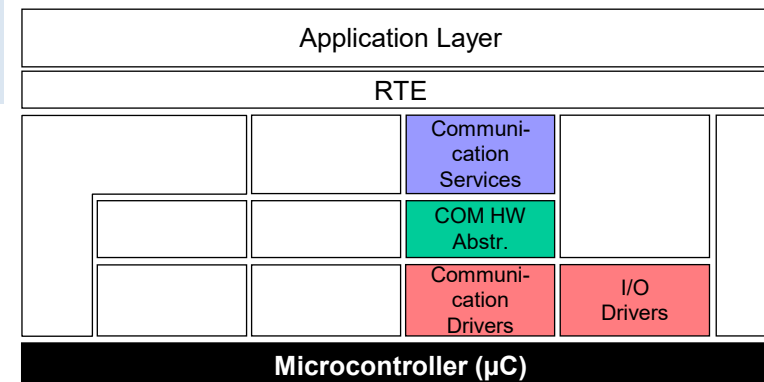
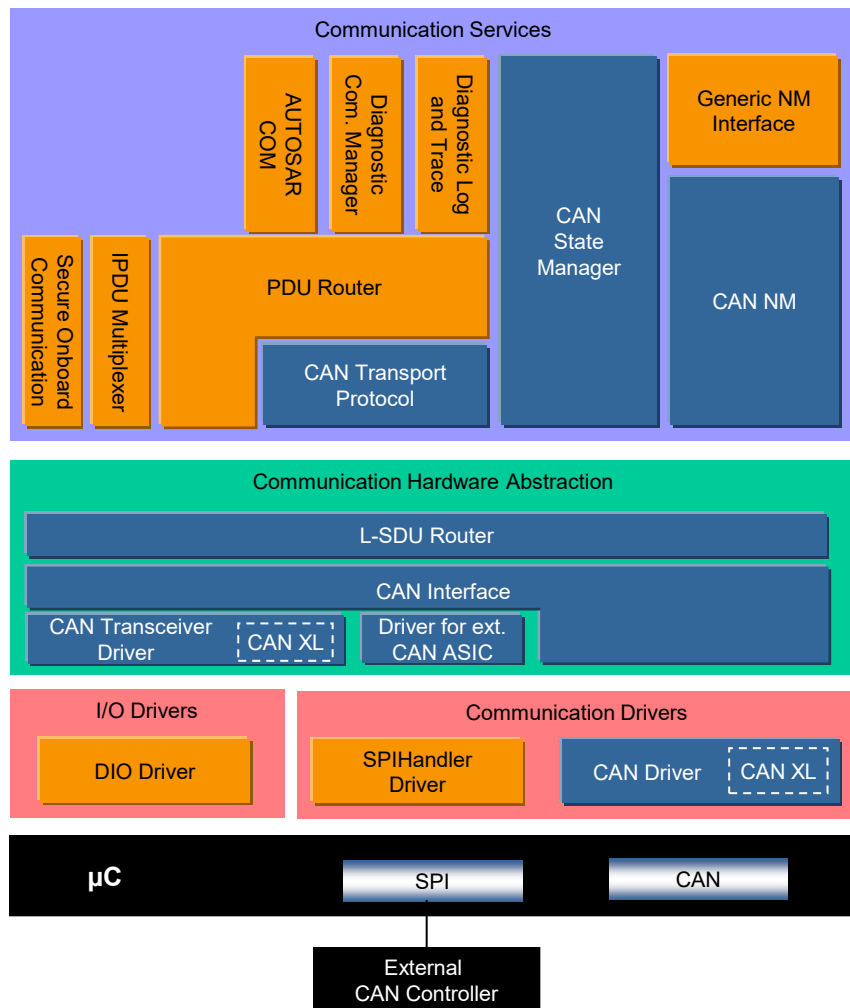
示例：



Architecture – Content of Software Layers

Communication Stack – CAN

Example:



The **CAN Communication Services** are a group of modules for vehicle network communication with the communication system CAN.

Task:

- Provide a uniform interface to the CAN network. Hide protocol and message properties from the application.

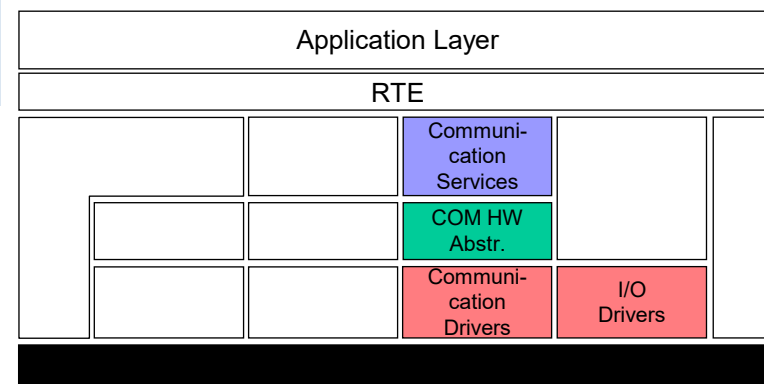
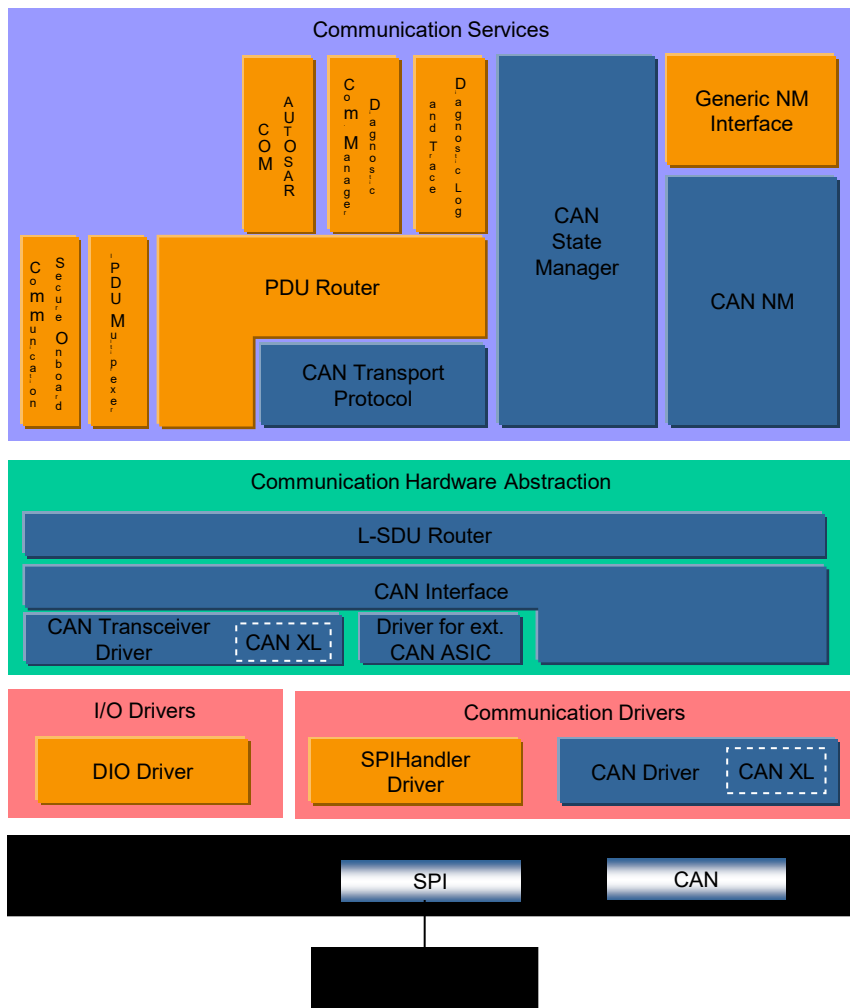
The **CAN Communication Stack** supports:

- Classic CAN communication (CAN 2.0)
- CAN FD communication, if supported by hardware
- CAN XL communication, if supported by hardware

Architecture – Content of Software Layers

通信栈 – CAN

示例:



CAN通信服务是一组用于车辆网络通信的模块，与通信系统CAN配合使用。

任务:

为CAN网络提供统一的接口。向应用程序隐藏协议和消息属性。

CAN通信栈支持:

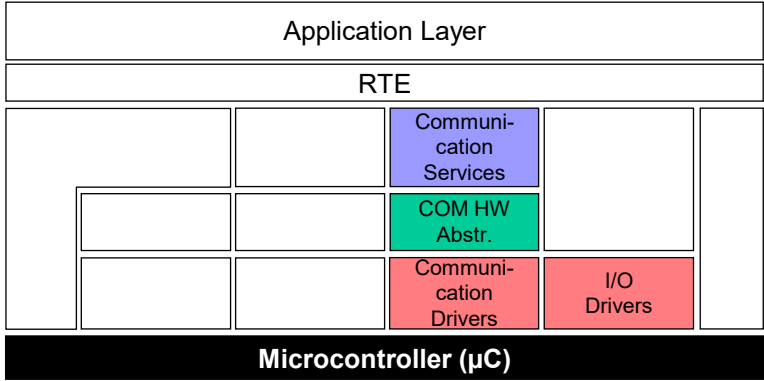
经典CAN通信 (CAN 2.0)

CAN FD通信, 如果硬件支持

CAN XL通信, 如果硬件支持

Architecture – Content of Software Layers

Communication Stack – CAN



Properties:

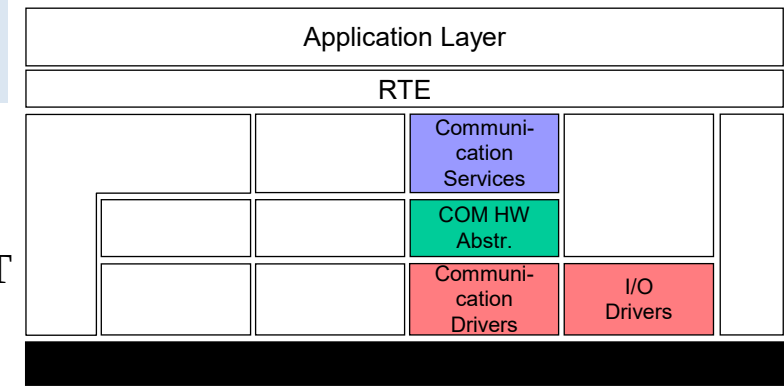
- Implementation: µC and ECU HW independent, partly dependent on CAN.
- AUTOSAR COM, Generic NM (Network Management) Interface and Diagnostic Communication Manager are the same for all vehicle network systems and exist as one instance per ECU.
- Generic NM Interface contains only a dispatcher. No further functionality is included. In case of gateway ECUs it can also include the NM coordinator functionality which allows to synchronize multiple different networks (of the same or different types) to synchronously wake them up or shut them down.
- CAN NM is specific for CAN networks and will be instantiated per CAN vehicle network system.
- The communication system specific Can State Manager handles the communication system dependent Start-up and Shutdown features. Furthermore it controls the different options of COM to send PDUs and to monitor signal timeouts.

Architecture – Content of Software Layers

通信栈 – CAN

属性：

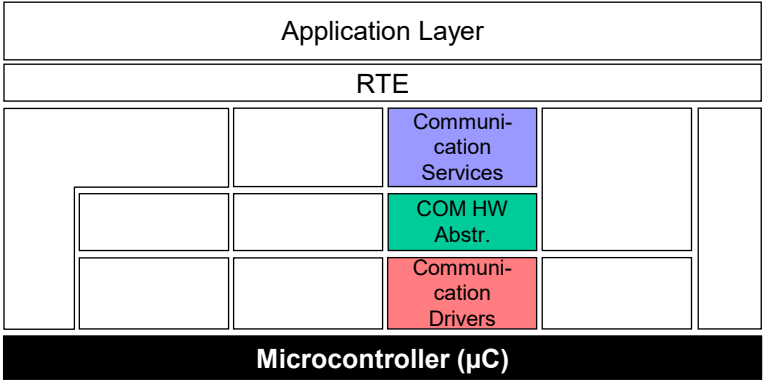
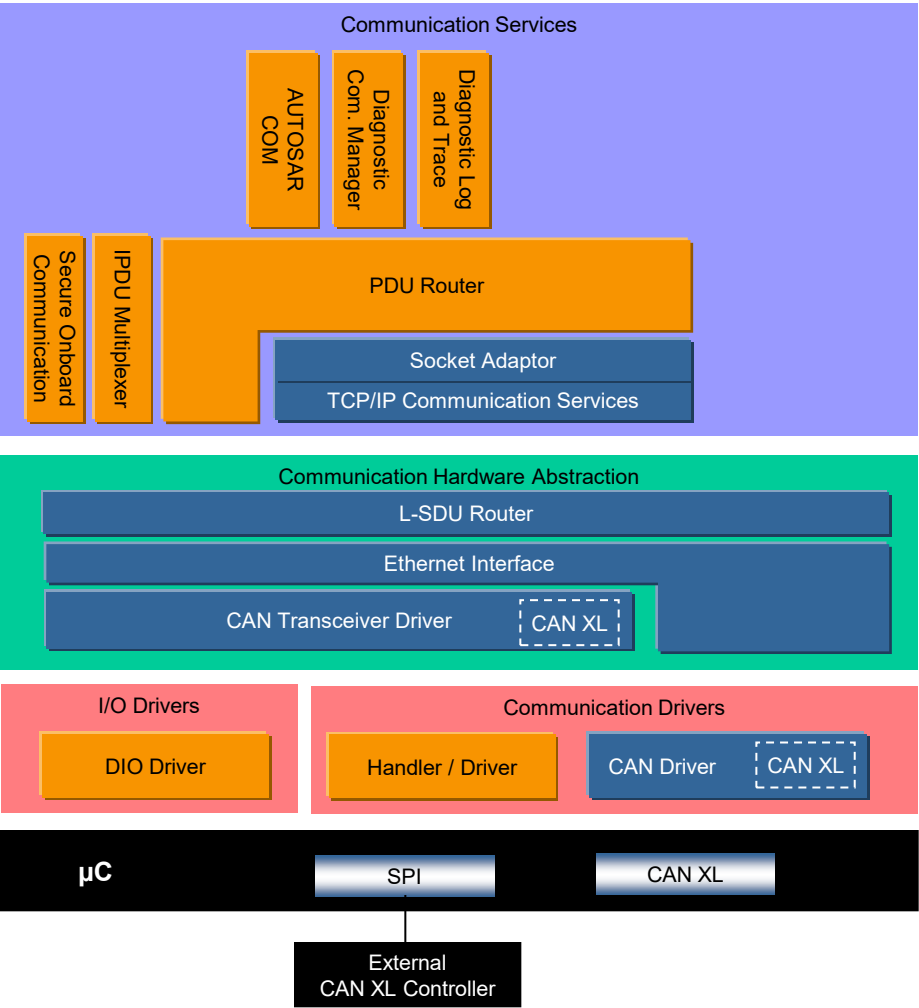
实现：微控制器与ECU硬件无关，部分依赖于CAN。 AUTOSAR COM、通用网络管理接口及诊断通信管理器在所有车辆网络系统中保持一致，每个ECU仅存在一个实例。 通用网络管理接口仅包含调度器，不集成其他功能。在网关ECU中，该接口还可集成网络管理协调器功能，用于同步多个不同网络（同类型或不同类型）的唤醒与关闭操作。 CAN网络管理专用于CAN网络，每个CAN车辆网络系统将独立实例化。 通信系统特定的**CAN状态管理器**负责处理与通信系统相关的启动和关闭功能，同时控制COM发送协议数据单元及监控信号超时的不同选项。



Architecture – Content of Software Layers

Communication Stack – Ethernet/CAN XL

Example:



CAN XL supports to directly tunnel **IEEE 802.3 Ethernet frames** for participation of IP communication.

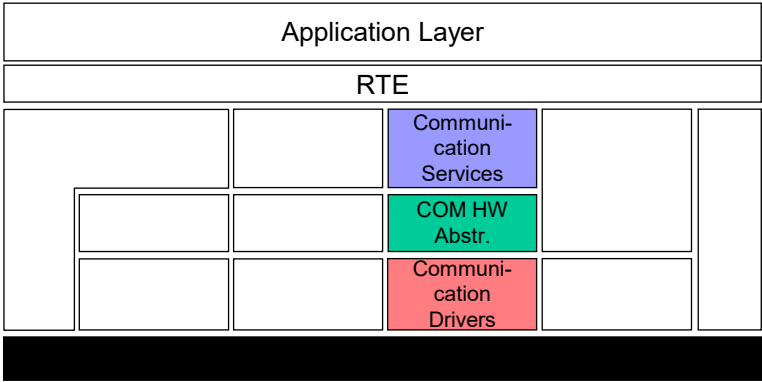
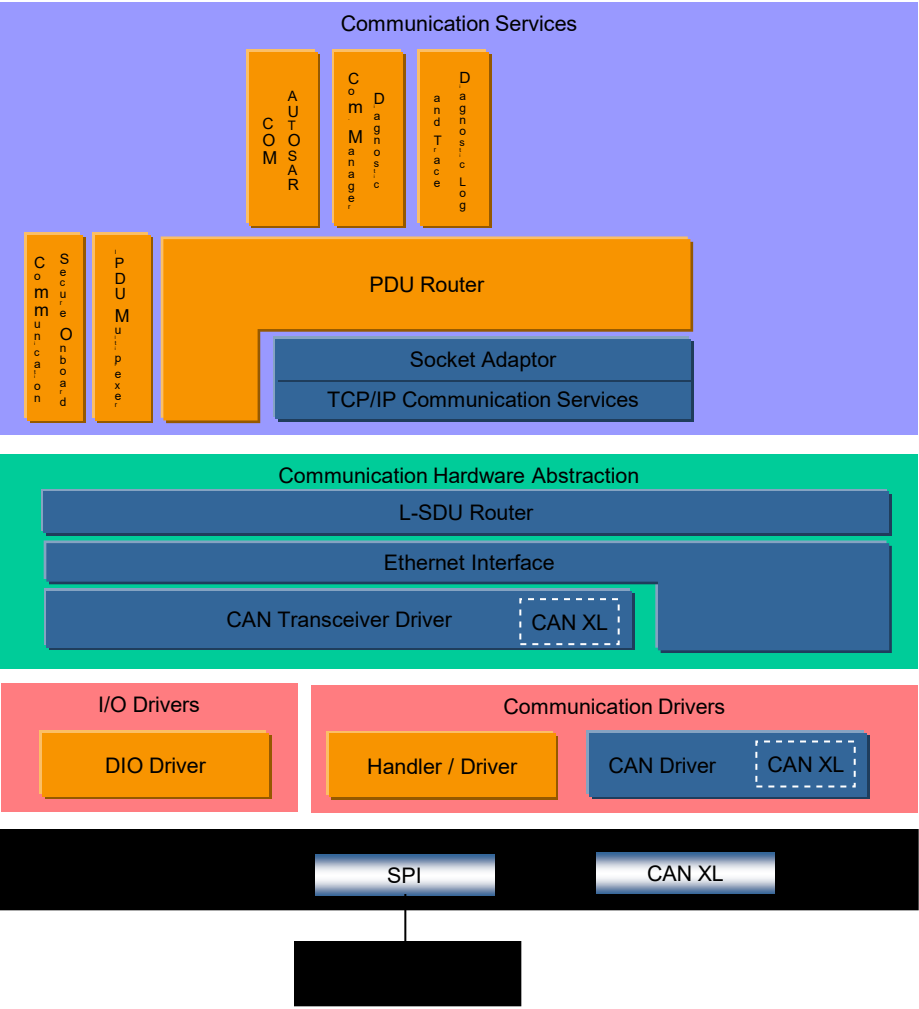
Task:

- Provide vehicle wide communication with same semantic used everywhere regardless physical connection (CAN XL / Ethernet) or communication paradigm (Signal- and Service-based communication).

Architecture – Content of Software Layers

通信栈 – 以太网/CAN XL

示例:



CAN XL支持直接隧道传输IEEE 802.3以太网帧，以参与IP通信。

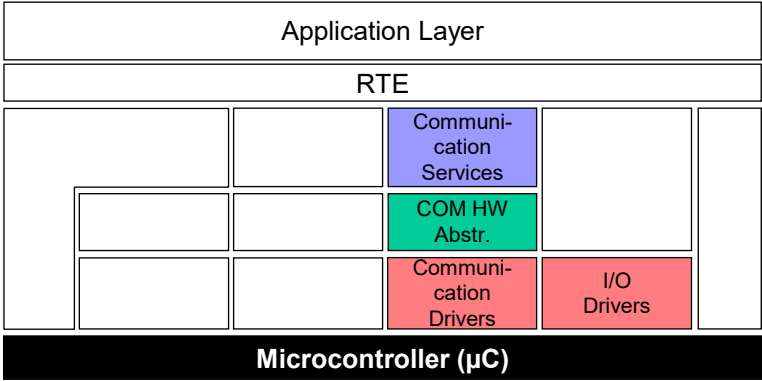
任务：
提供车辆范围内的通信，无论物理连接（CAN XL / 以太网）或通信范式（基于信号和服务的通信）如何，均使用统一的语义。

Architecture – Content of Software Layers

Communication Stack Extension – CAN XL

Properties:

- CAN XL is an absolute superset to CAN, i.e. a CAN stack which supports CAN XL can serve both a CAN and a CAN XL bus.
- CanIf, CanTrcvDrv and CanDrv are the only modules which need extensions to serve CAN XL communication.
- The properties of the communication stack CAN are also true for CAN with CAN XL functionality.



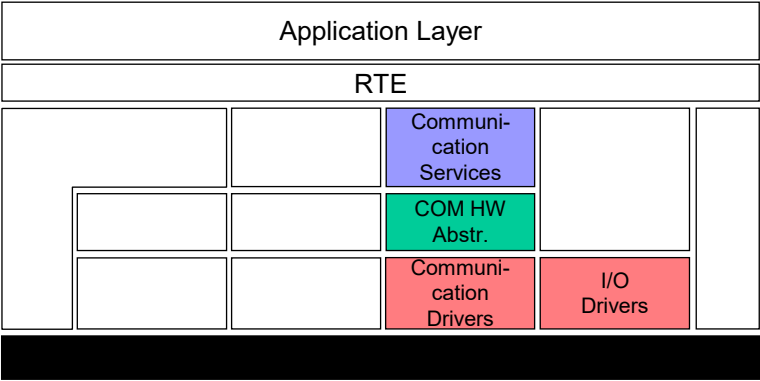
Architecture – Content of Software Layers

通信栈扩展 – CAN XL

属性:

CAN XL 是 CAN 的绝对超集，即支持 CAN XL 的 CAN 协议栈可同时服务于 CAN 和 CAN XL。

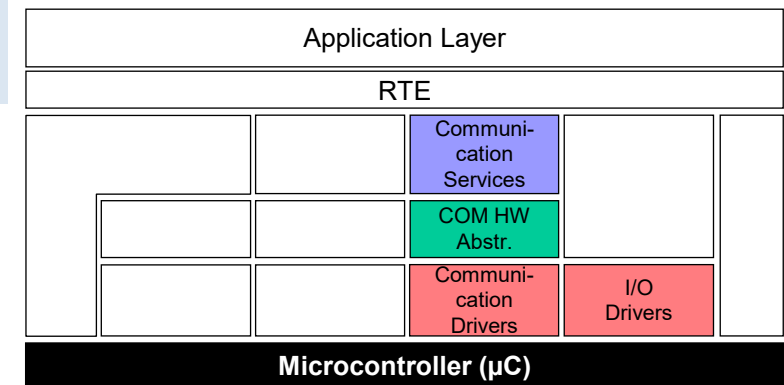
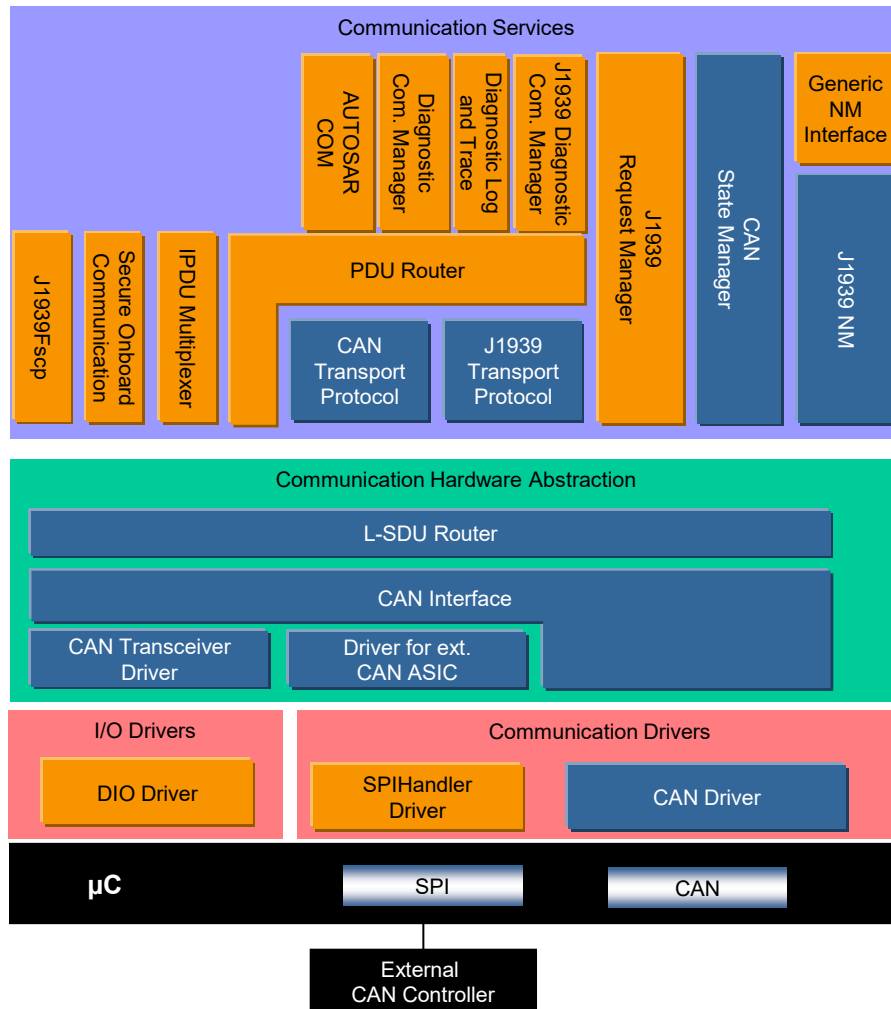
XL总线。 CanIf、CanTrcvDrv和CanDrv是唯一需要扩展以支持CAN XL通信的模块。 通信栈CAN的特性同样适用于具备CAN XL功能的CAN。



Architecture – Content of Software Layers

Communication Stack Extension – J1939

Example:



The **J1939 Communication Services** extend the plain CAN communication stack for vehicle network communication in heavy duty vehicles.

Task:

- Provide the protocol services required by J1939. Hide protocol and message properties from the application where not required.

Please Note:

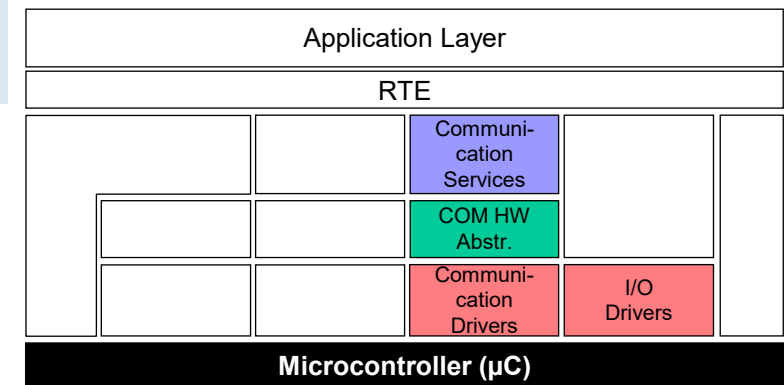
- There are two transport protocol modules in the CAN stack (CanTp and J1939Tp) which can be used alternatively or in parallel on different channels:. They are used as follows:
 - CanTp: ISO Diagnostics (DCM), large PDU transport on standard CAN bus
 - J1939Tp: J1939 Diagnostics, large PDU transport on J1939 driven CAN bus

Architecture – Content of Software Layers

Communication Stack Extension – J1939

Properties:

- Implementation: μ C and ECU HW independent, based on CAN.
- AUTOSAR COM, Generic NM (Network Management) Interface and Diagnostic Communication Manager are the same for all vehicle network systems and exist as one instance per ECU.
- Supports dynamic frame identifiers that are not known at configuration time.
- J1939 network management handles assignment of unique addresses to each ECU but does not support sleep/wakeup handling and related concepts like partial networking.
- Provides J1939 diagnostics and request handling.
- Provides support for Functional Safety per SAE J1939-76, including initialization, Safety Data Group (SDG) Operations, Safety Header/Data Message processing using PROFILE_J1939_76 E2E Transformer, and Error Management between producer and consumer Application SWCs.



Architecture – Content of Software Layers

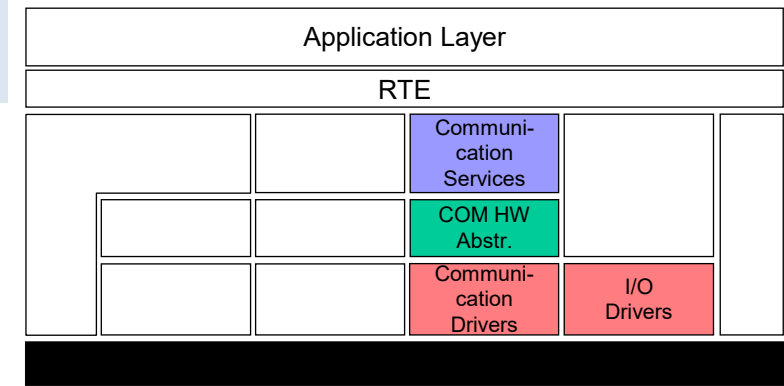
通信栈扩展 – J1939

属性:

实现: 基于CAN, 与 μ C和ECU硬件无关。

AUTOSAR COM、通用NM（网络管理）接口以及诊断通信管理器对所有车辆网络系统均相同, 且每个ECU中仅存在一个实例。支持配置时未知的动态帧标识符。J1939网络管理负责为每个ECU分配唯一地址, 但不支持休眠/唤醒处理及相关概念（如部分网络）。提供J1939诊断与请求处理功能。

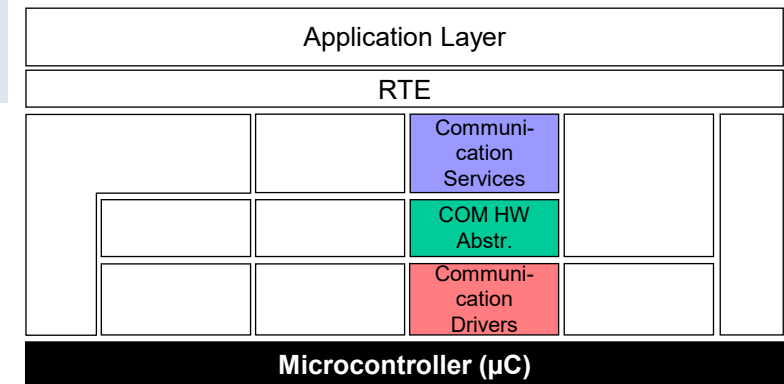
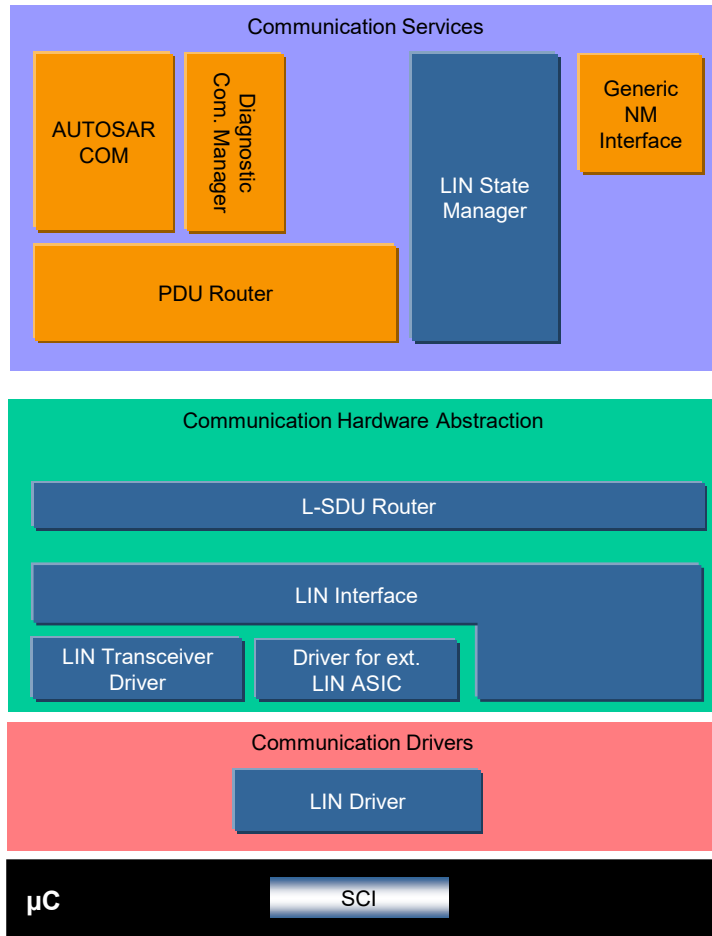
支持符合SAE J1939-76标准的功能安全, 包括初始化、安全数据组（SDG）操作、使用PROFILE_J1939_76 E2E转换器处理安全报头/数据消息, 以及生产者与消费者应用软件组件之间的错误管理。



Architecture – Content of Software Layers

Communication Stack – LIN

Example:



The **LIN Communication Services** are a group of modules for vehicle network communication with the communication system LIN.

Task:

Provide a uniform interface to the LIN network. Hide protocol and message properties from the application.

Properties:

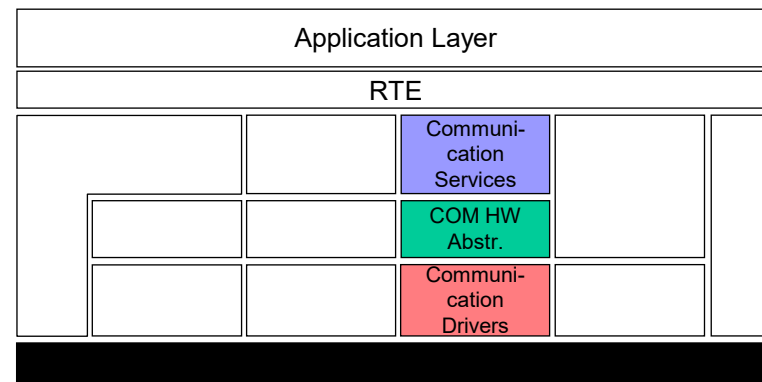
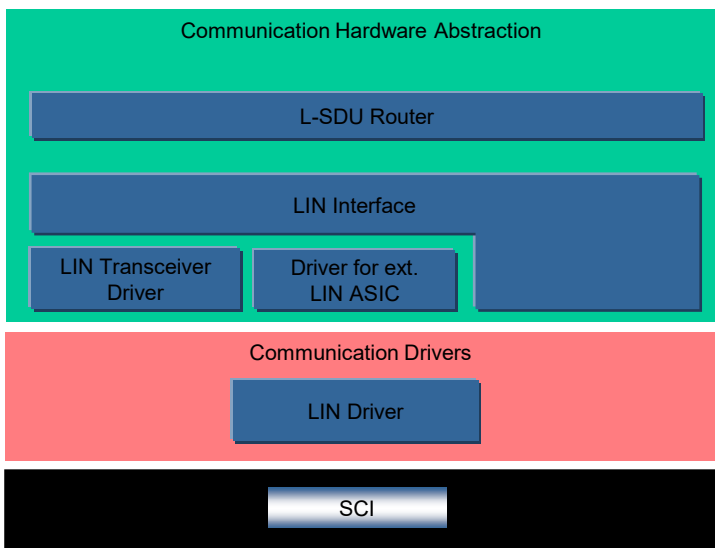
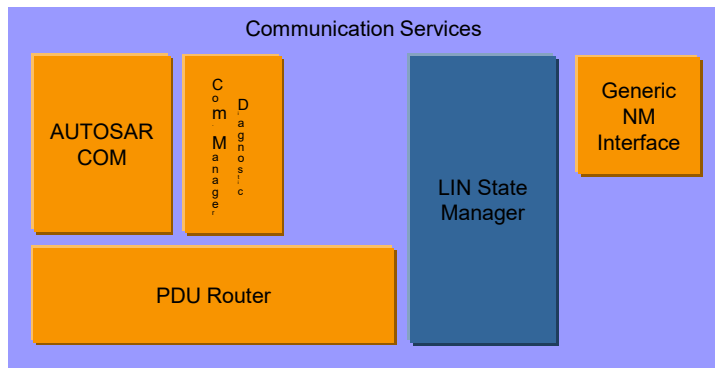
The **LIN Communication Services** contain:

- An ISO 17987 compliant communication stack with
 - Schedule table manager to handle requests to switch to other schedule tables (for LIN master nodes)
 - Communication handling of different LIN frame types
 - Transport protocol, used for diagnostics
 - A WakeUp and Sleep Interface
- An underlying LIN Driver:
 - Implementing LIN protocol and accessing the specific hardware
 - Supporting both simple UART and complex frame based LIN hardware

Architecture – Content of Software Layers

通信栈 – LIN

示例:



LIN通信服务是一组用于车辆网络与LIN通信系统进行通信的模块。

任务:

为LIN网络提供统一的接口。向应用程序隐藏协议和消息属性。

属性:

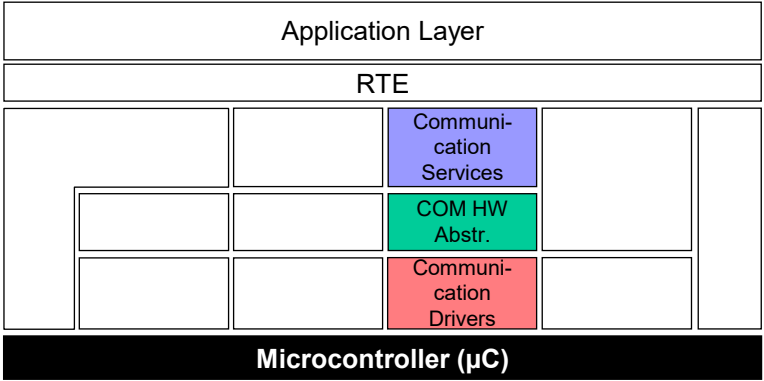
LIN通信服务包含:

符合ISO 17987标准的通信协议栈，具备

- 调度表管理器，用于处理切换到其他调度表的请求（适用于LIN主节点）
- 不同LIN帧类型的通信处理
- 传输协议，用于诊断
- 唤醒与休眠接口

底层LIN驱动程序： ■ 实现LIN协议并访问特定硬件 ■ 同时支持基于简单UART和复杂帧的LIN硬件

Architecture – Content of Software Layers
Communication Stack – LIN



Note: Integration of LIN into AUTOSAR:

- LIN Interface controls the WakeUp/Sleep API and allows the slaves to keep the bus awake (decentralized approach).
- The communication system specific LIN State Manager handles the communication dependent Start-up and Shutdown features. Furthermore it controls the communication mode requests from the Communication Manager. The LIN State Manager also controls the I-PDU groups by interfacing COM.
- When sending a LIN frame, the LIN Interface requests the data for the frame (I-PDU) from the PDU Router at the point in time when it requires the data (i.e. right before sending the LIN frame).

Architecture – Content of Software Layers

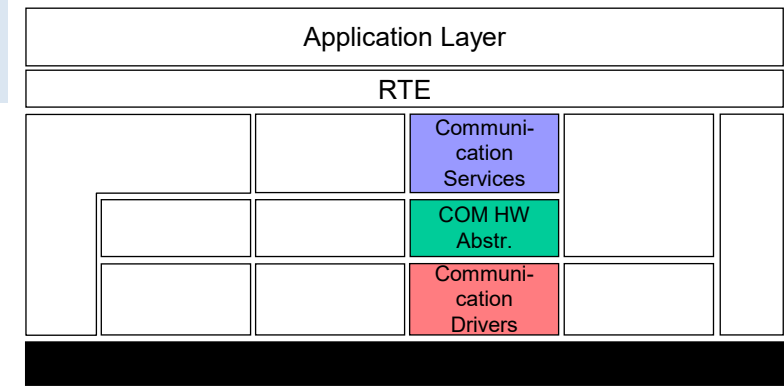
通信栈 – LIN

注意：LIN集成到AUTOSAR中：

LIN接口控制着WakeUp/Sleep API，并允许从节点保持总线唤醒状态（分布式方案）。

通信系统特定的**LIN状态管理器**负责处理与通信相关的启动和关闭功能。此外，它还控制来自**通信管理器**的通信模式请求。LIN状态管理器还通过接口COM控制I-PDU组。

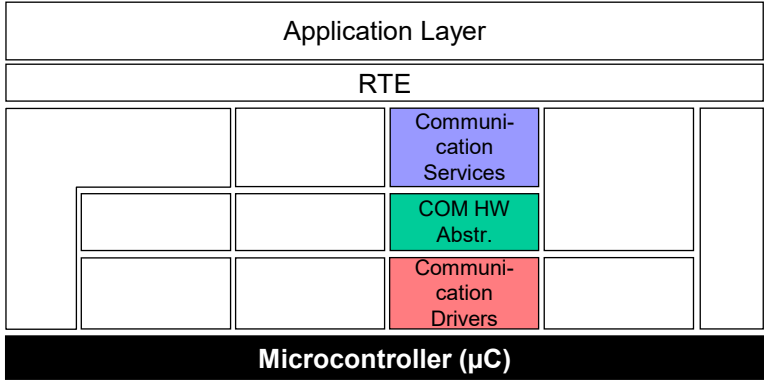
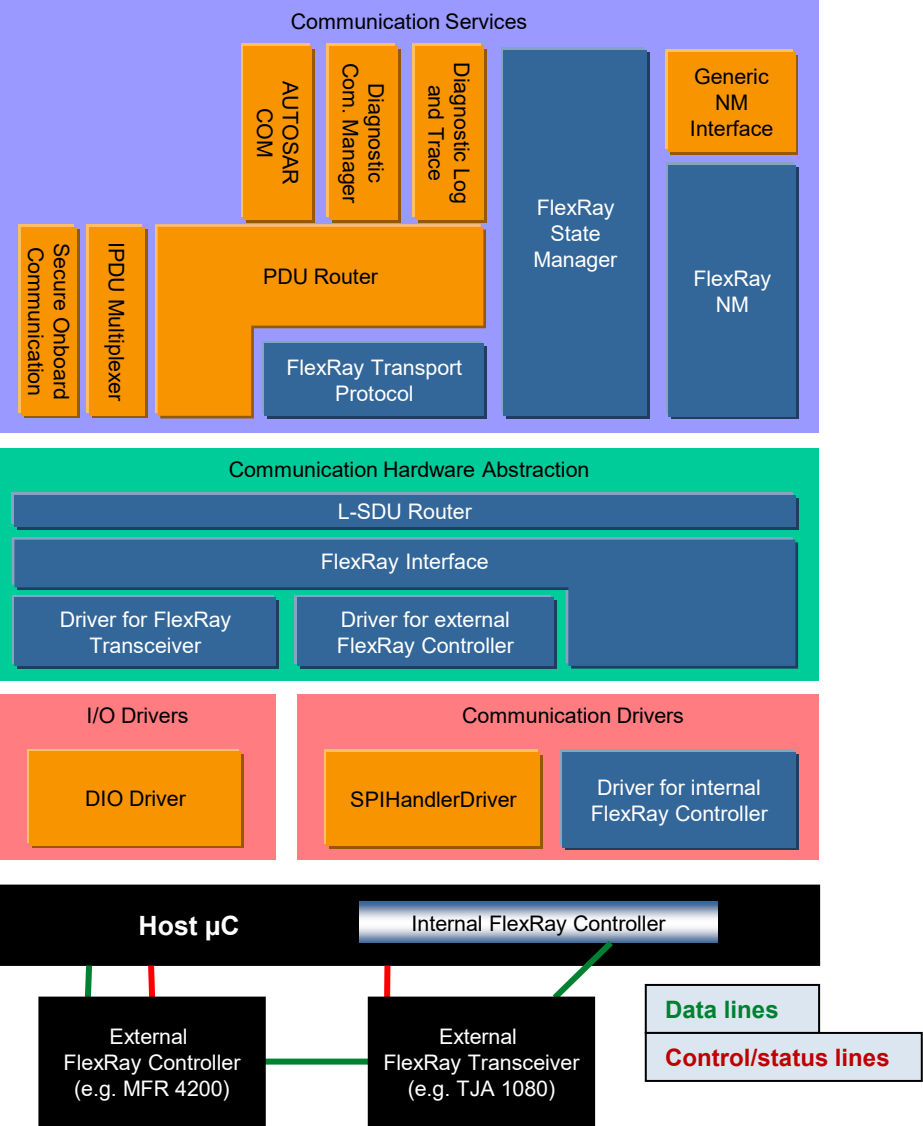
发送LIN帧时，LIN接口在需要数据（即即将发送LIN帧之前）的时刻向PDU路由器请求该帧（I-PDU）的数据。



Architecture – Content of Software Layers

Communication Stack – FlexRay

Example:



The **FlexRay Communication Services** are a group of modules for vehicle network communication with the communication system FlexRay.

Task:

- Provide a uniform interface to the FlexRay network. Hide protocol and message properties from the application.

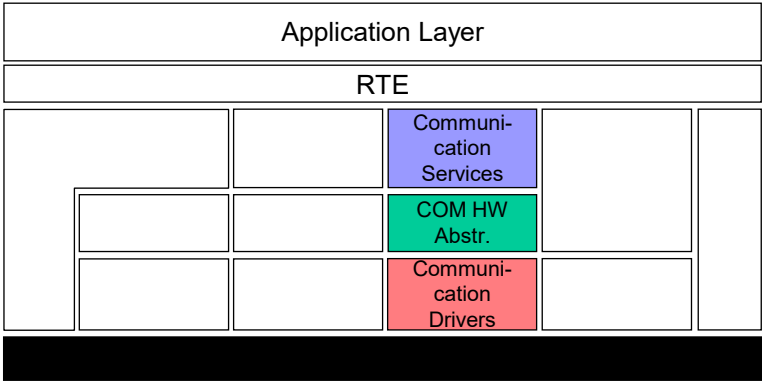
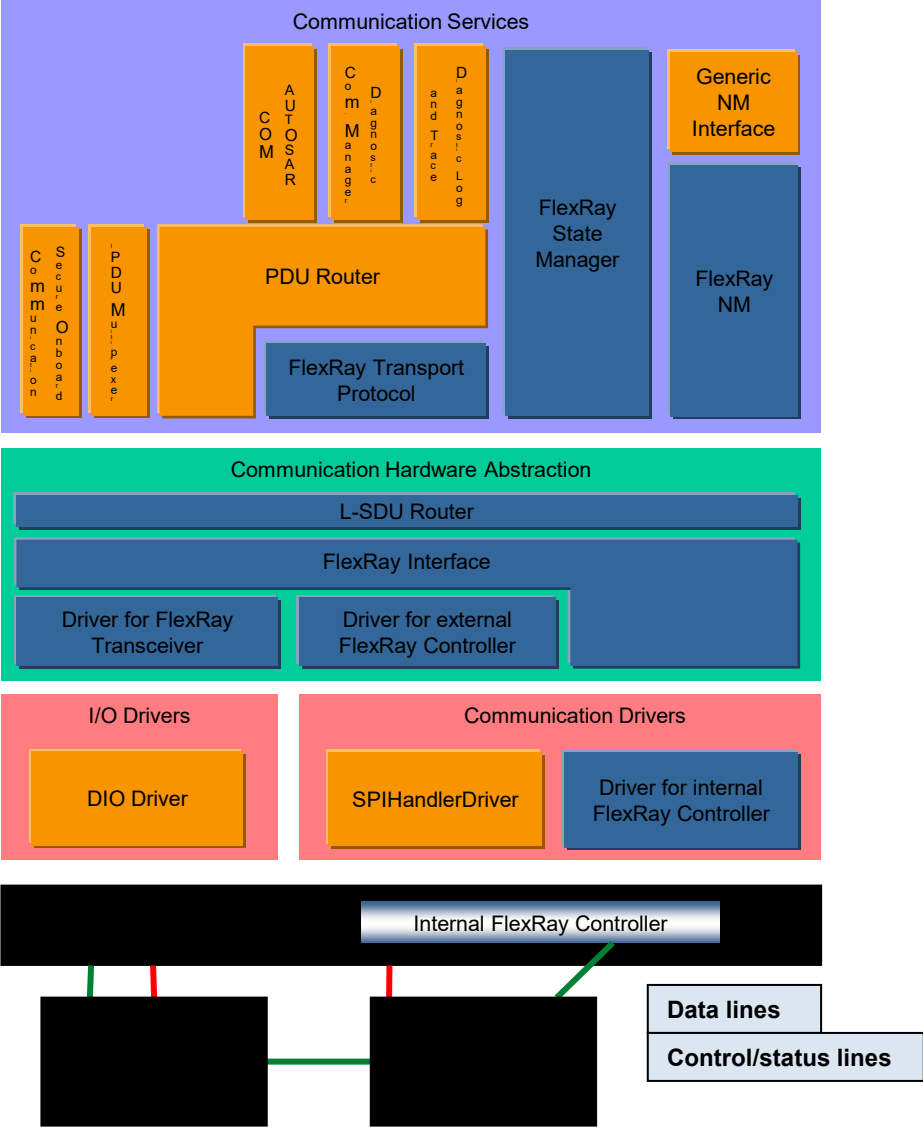
Please Note:

- There are two transport protocol modules in the FlexRay stack which can be used alternatively
 - FrTp: FlexRay ISO Transport Layer
 - FrArTp: FlexRay AUTOSAR Transport Layer, provides bus compatibility to AUTOSAR R3.x

Architecture – Content of Software Layers

通信栈 – FlexRay

示例:



FlexRay通信服务是一组用于车辆网络通信的模块，与FlexRay通信系统协同工作。

任务:

为FlexRay网络提供统一接口，向应用程序隐藏协议与消息属性。

请注意:

FlexRay堆栈中有两个传输协议模块，可以交替使用。

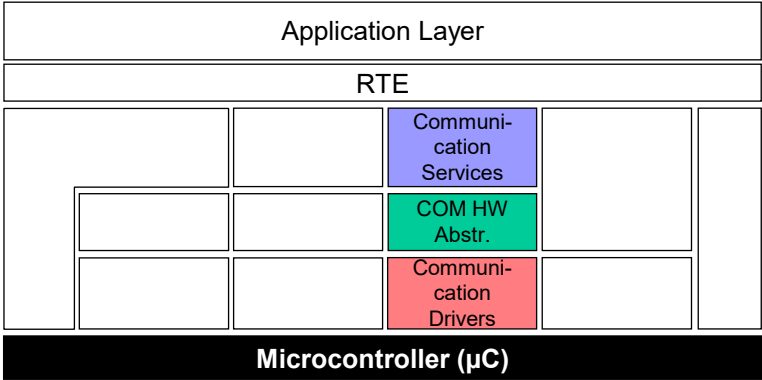
- FrTp: FlexRay ISO传输层
- FrArTp: FlexRay AUTOSAR传输层，为AUTOSAR R3.x提供总线兼容性

Architecture – Content of Software Layers

Communication Stack – FlexRay

Properties:

- Implementation: μ C and ECU HW independent, partly dependent on FlexRay.
- AUTOSAR COM, Generic NM Interface and Diagnostic Communication Manager are the same for all vehicle network systems and exist as one instance per ECU.
- Generic NM Interface contains only a dispatcher. No further functionality is included. In case of gateway ECUs, it is replaced by the NM Coordinator which in addition provides the functionality to synchronize multiple different networks (of the same or different types) to synchronously wake them up or shut them down.
- FlexRay NM is specific for FlexRay networks and is instantiated per FlexRay vehicle network system.
- The communication system specific FlexRay State Manager handles the communication system dependent Start-up and Shutdown features. Furthermore it controls the different options of COM to send PDUs and to monitor signal timeouts.



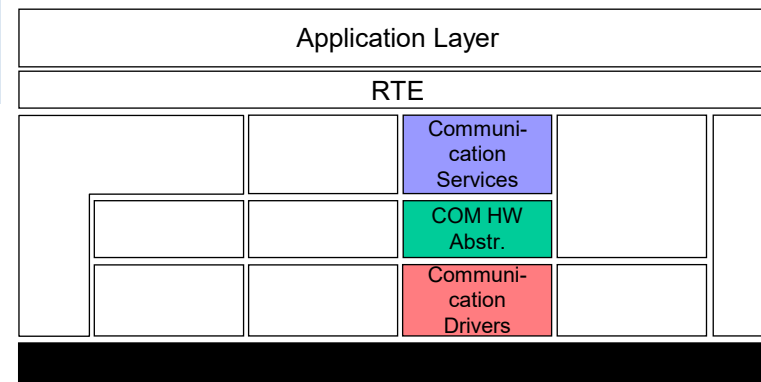
Architecture – Content of Software Layers

通信栈 – FlexRay

属性：

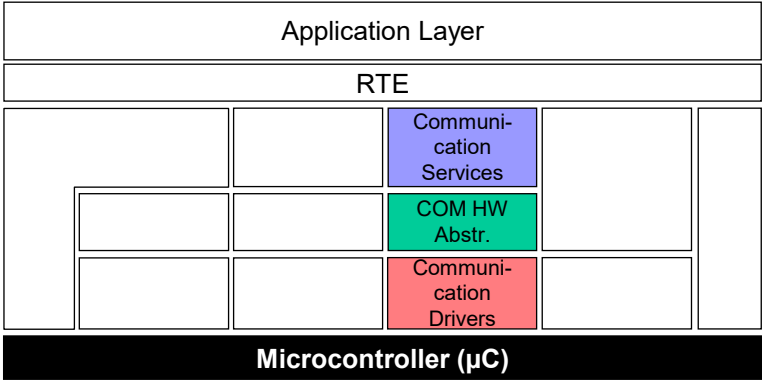
实现：微控制器与ECU硬件无关，部分依赖于FlexRay。AUTOSAR COM、通用网络管理接口及诊断通信管理器对所有车载网络系统均相同，且每个ECU仅存在一个实例。通用网络管理接口仅包含调度器，不包含其他功能。在网关ECU中，该接口由网络管理协调器替代，后者额外提供同步多个不同网络（相同或不同类型）以唤醒或关闭的功能。FlexRay网络管理专用于FlexRay网络，每个FlexRay车载网络系统均会实例化该模块。

通信系统特定的FlexRay状态管理器负责处理依赖于通信系统的启动与关闭功能，同时控制COM发送协议数据单元及监控信号超时的不同选项。

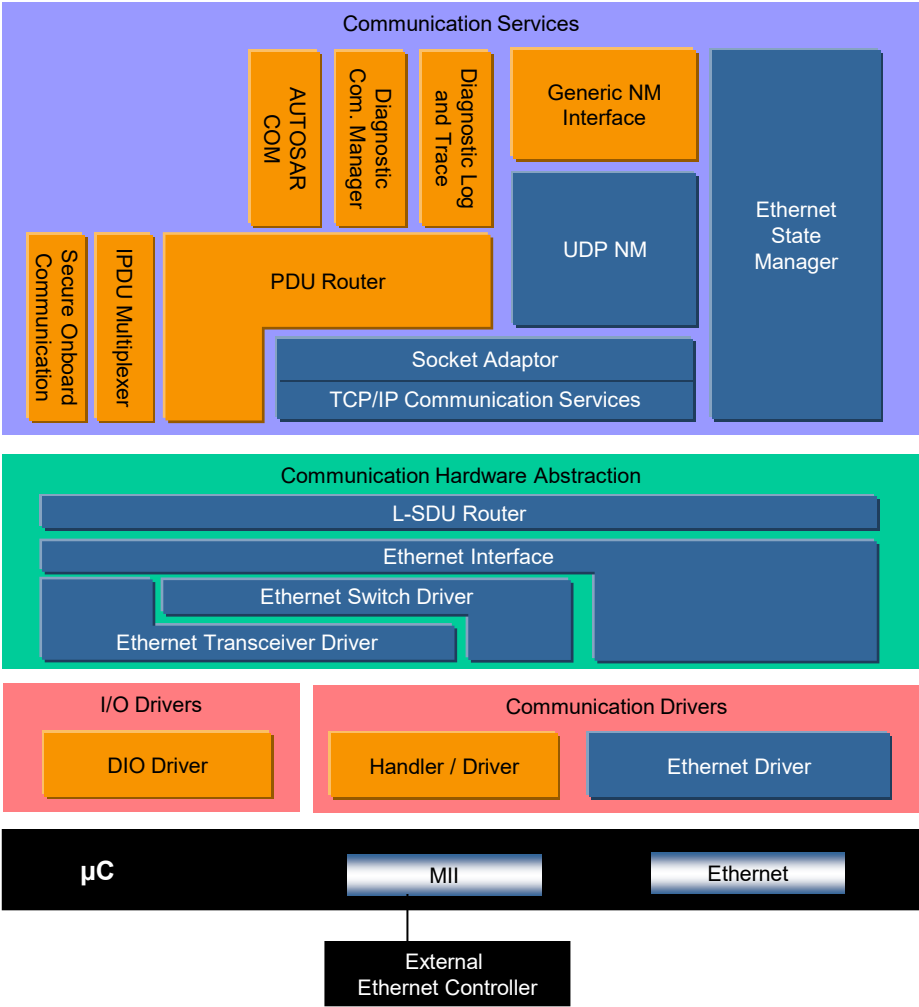


Architecture – Content of Software Layers

Communication Stack – TCP/IP



Example:



The **TCP/IP Communication Services** are a group of modules for vehicle network communication with the communication system TCP/IP.

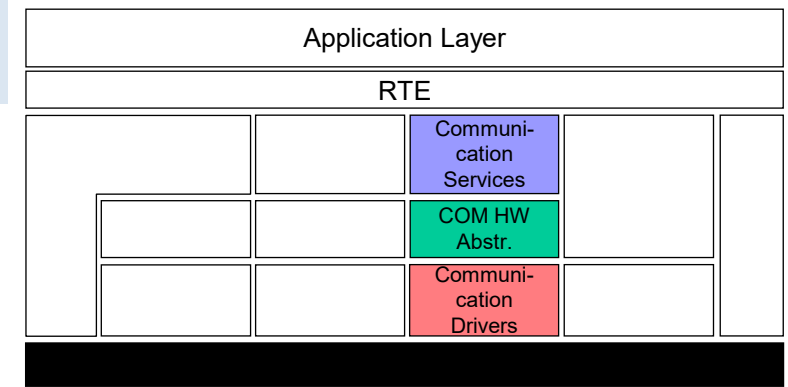
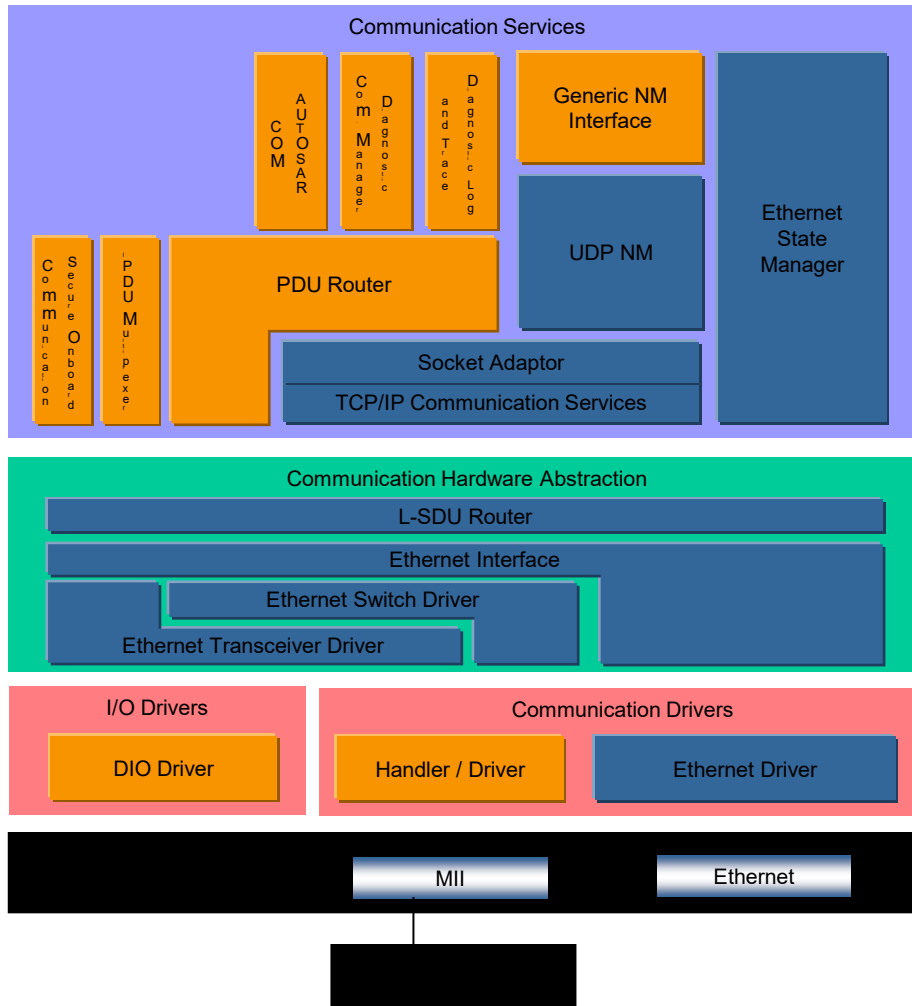
Task:

- Provide a uniform interface to the TCP/IP network. Hide protocol and message properties from the application.

Architecture – Content of Software Layers

通信栈 – TCP/IP

示例：



TCP/IP通信服务是一组用于车辆网络与TCP/IP通信系统进行通信的模块。

任务：

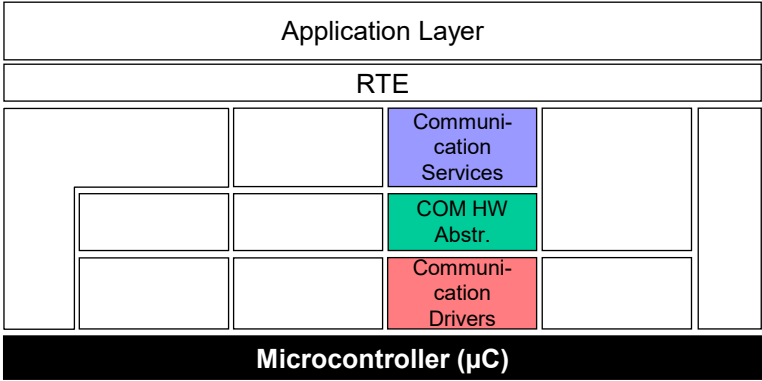
为TCP/IP网络提供统一的接口。向应用程序隐藏协议和消息属性。

Architecture – Content of Software Layers

Communication Stack – TCP/IP

Properties:

- The Tcplp module implements the main protocols of the TCP/IP protocol family (TCP, UDP, IPv4, IPv6, ARP, ICMP, DHCP) and provides dynamic, socket based communication via Ethernet.
- The Socket Adaptor module (SoAd) is the sole upper layer module of the Tcplp module.

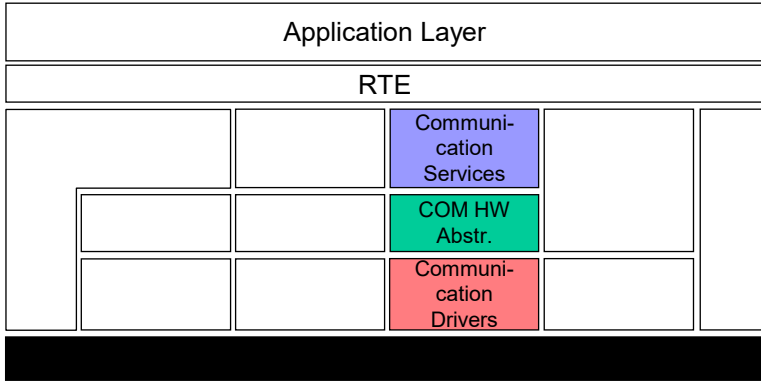


Architecture – Content of Software Layers

通信栈 – TCP/IP

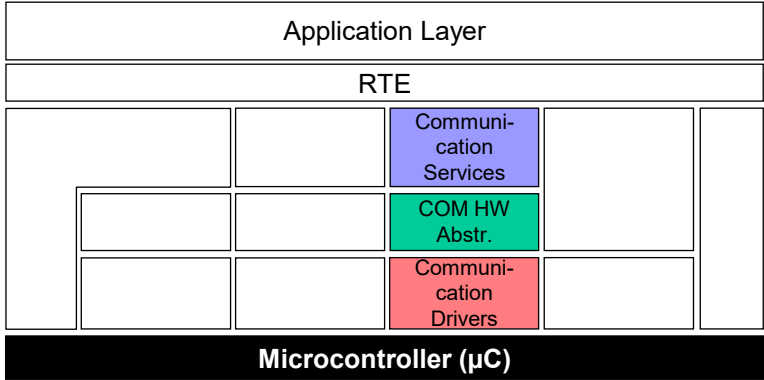
属性：

TcpIp模块实现了TCP/IP协议族（TCP、UDP、IPv4、IPv6、ARP、ICMP、DHCP）的主要协议，并通过以太网提供基于套接字的动态通信。套接字适配器模块（SoAd）是TcpIp模块唯一的上层模块。



Architecture – Content of Software Layers

Communication Stack – Firewall



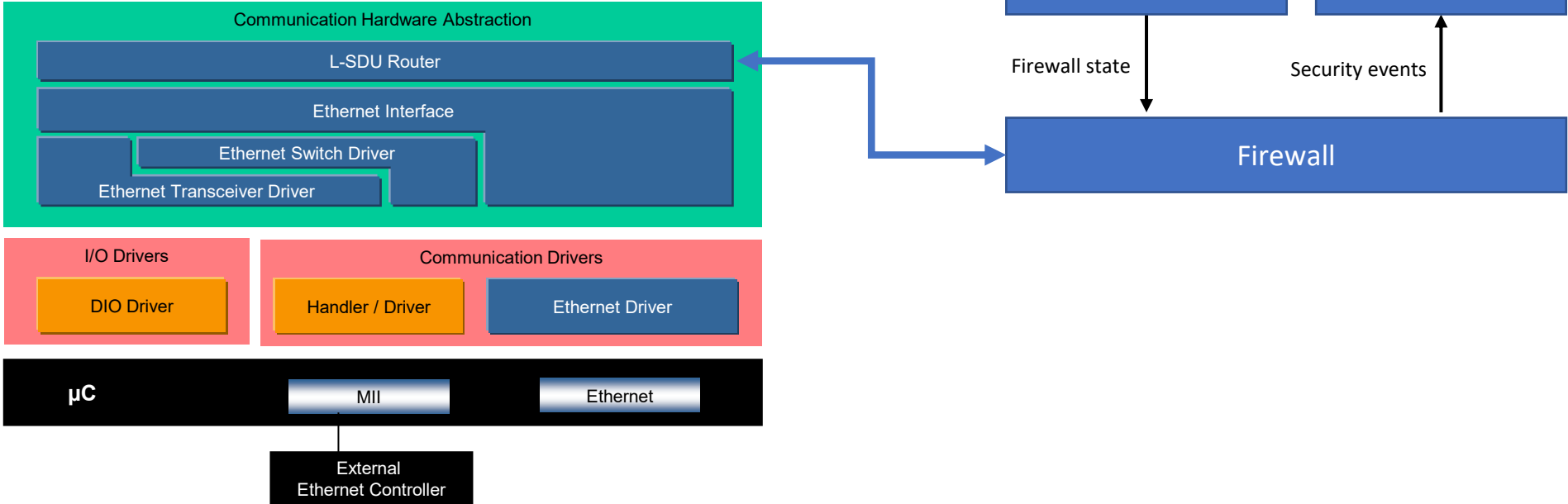
The firewall module protects the AUTOSAR stack from malicious messages by inspecting network packets and filtering them based on a pre-defined ruleset.

The firewall supports network packet inspection on 3 different levels

- Stateless packet inspection
- Stateful packet inspection
- Deep packet inspection

The firewall is connected to the IdsM module to raise security events in the case of unexpected network packets

Example:



Architecture – Content of Software Layers

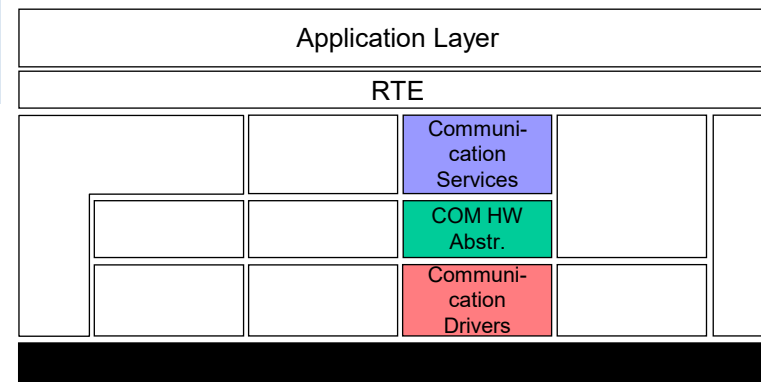
通信栈 – 防火墙

防火墙模块通过检查网络数据包并根据预定义的规则集进行过滤，保护AUTOSAR堆栈免受恶意消息的侵害。

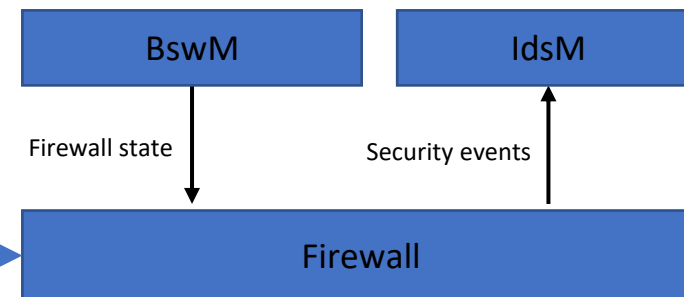
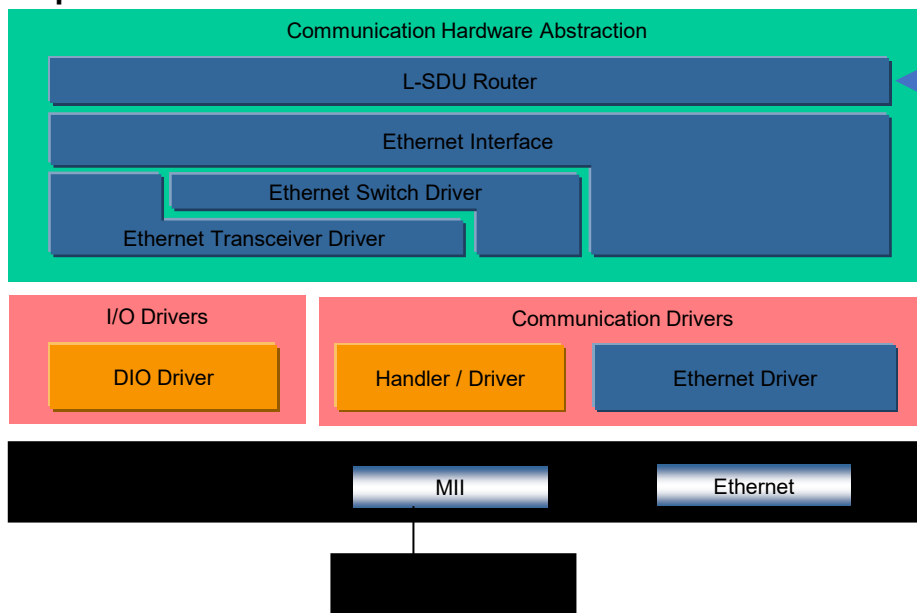
防火墙支持在三个不同级别进行网络数据包检查。

无状态数据包检查 有状态数据包检查 深度数据包检查

防火墙与IdsM模块相连，以便在出现意外网络数据包时提升安全事件。



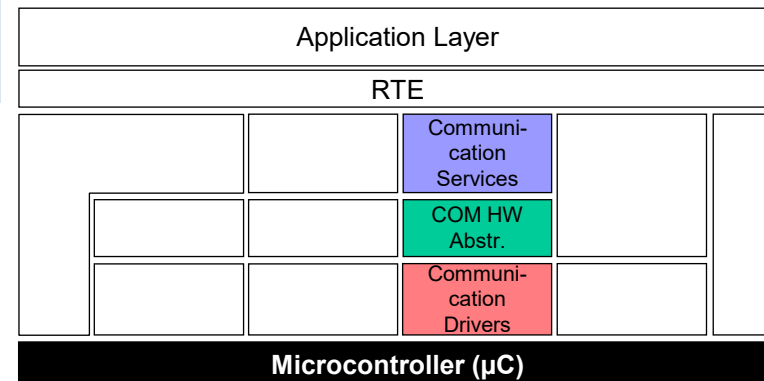
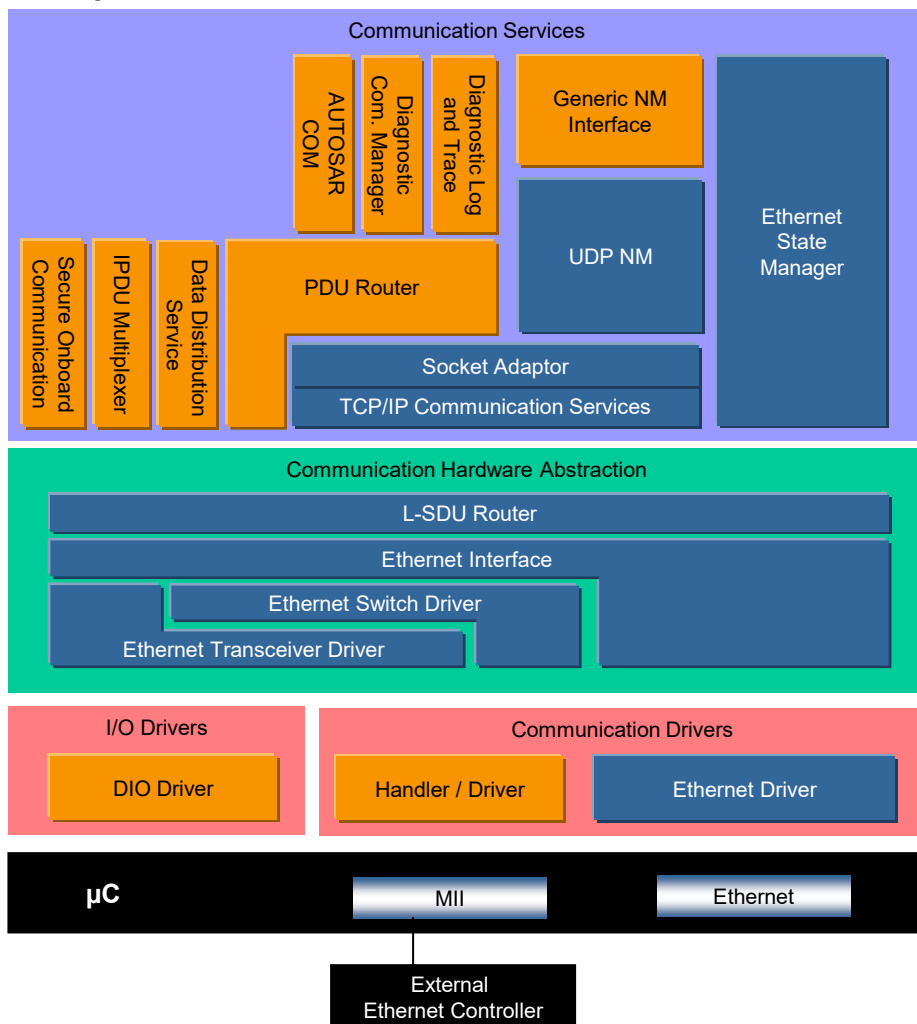
Example:



Architecture – Content of Software Layers

Communication Stack – DDS

Example:



The **Data Distribution Services** is a module for data-oriented and service-oriented vehicle network communication.

Task:

- Provide the DDS standard interfaces.

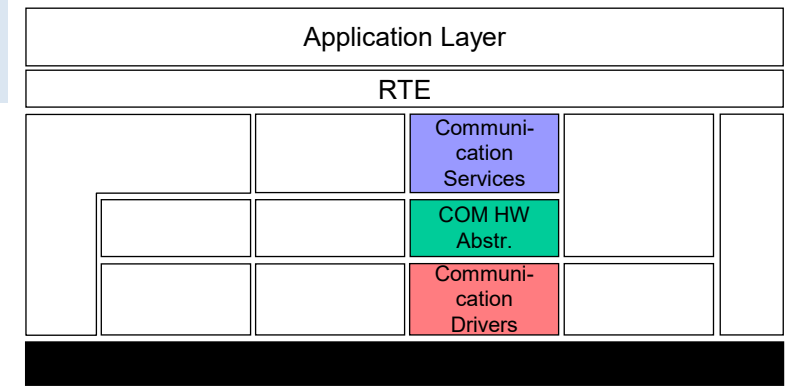
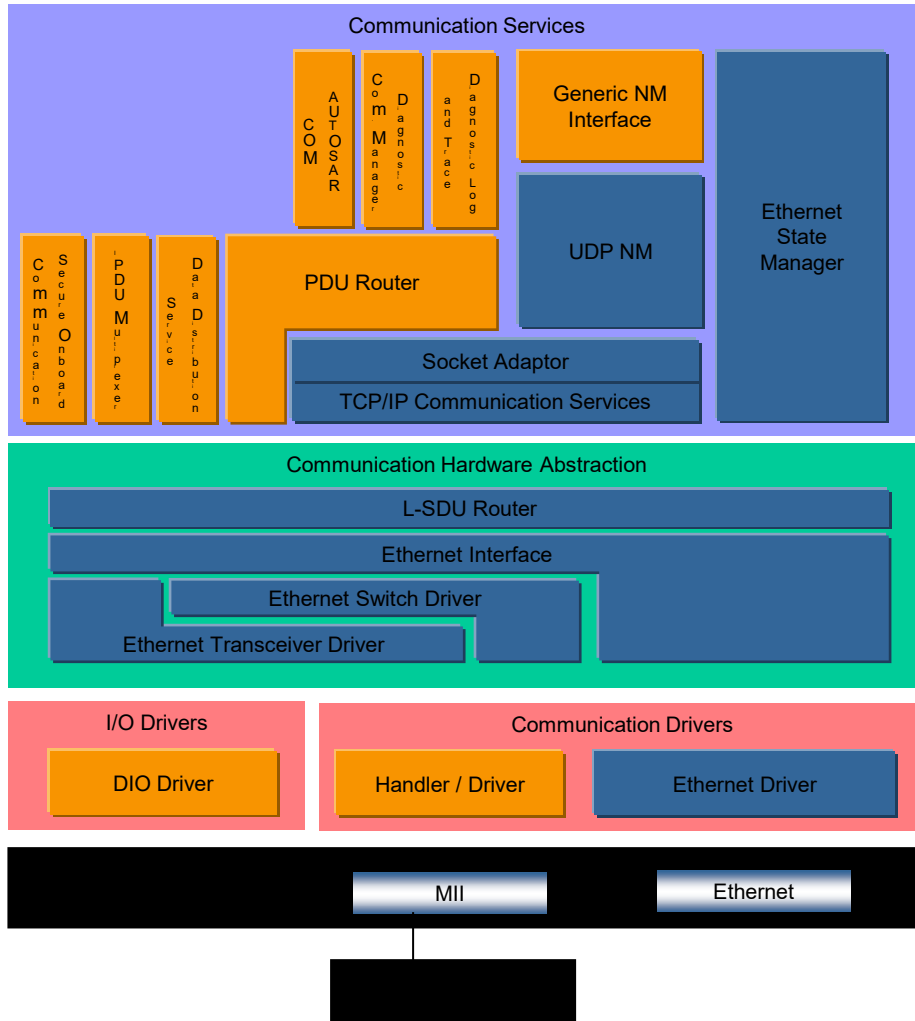
The DDS module supports:

- Signal Base Publisher/Subscriber communication path
- QoS handling
- Full static configuration

Architecture – Content of Software Layers

通信栈 – DDS

示例:



数据分发服务是一个面向数据和服务导向的车联网通信模块。

任务:

提供DDS标准接口。

DDS模块支持:

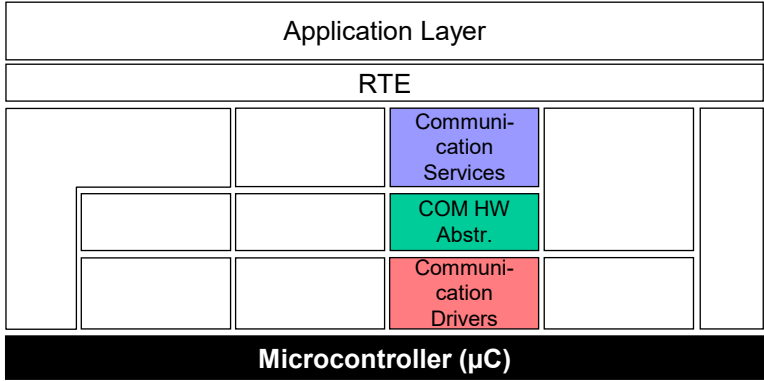
信号基础发布者/订阅者通信路径
服务质量处理 全静态配置

Architecture – Content of Software Layers

Communication Stack – DDS

Properties:

- The DDS module supports the key features of the Object Management Group (OMG) DDS standard, including SPDP and SEDP discovery protocols, the extended SOA relying on DDS-RPC and Service Discovery compliant with AUTOSAR AP.
- The Socket Adaptor module (SoAd) is the sole module able to handle the DDS-PDUs by means of the PDU Router (PduR).
- The DDS module provides E2E features and security services itself.



Architecture – Content of Software Layers

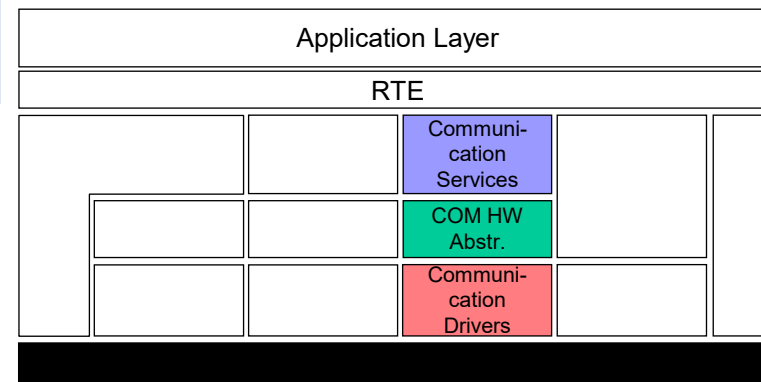
通信栈 – DDS

属性：

DDS模块支持对象管理组（OMG）DDS标准的核心特性，包括SPDP与SEDP发现协议、基于DDS-RPC的扩展型SOA架构，以及符合AUTOSAR AP规范的服务发现机制。

Socket Adaptor模块（SoAd）是唯一能够通过PDU路由器（PduR）处理DDS-PDU的模块。

DDS模块本身提供端到端特性和安全服务。



Architecture – Content of Software Layers Communication Stack - ChrgM

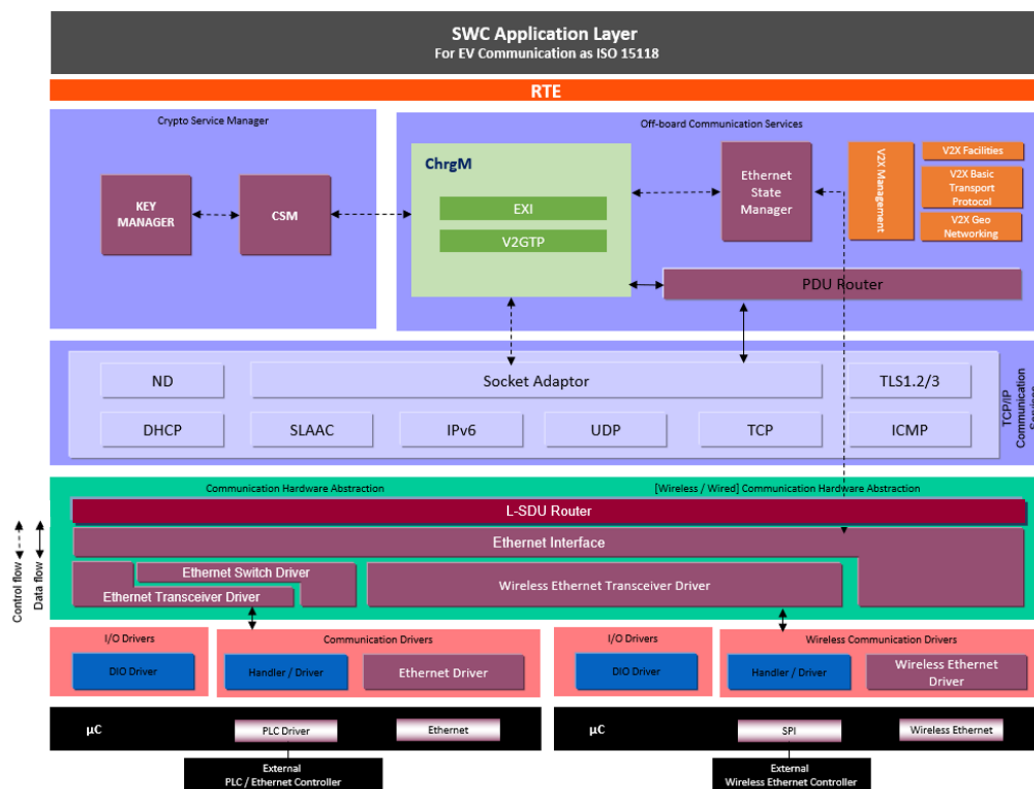
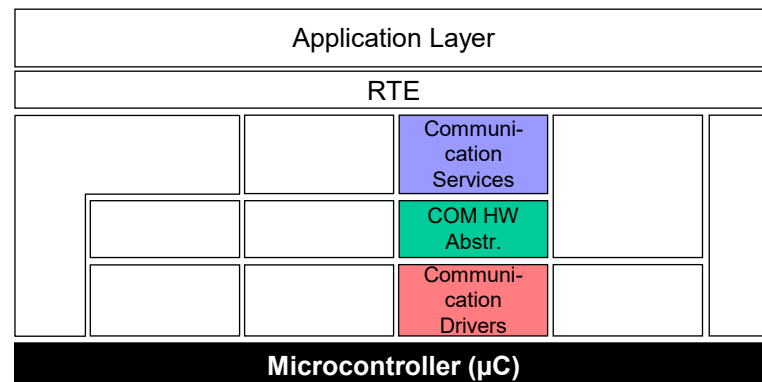
The Charging Manager (ChrgM) belongs to Communication Services of the AUTOSAR Layered Architecture.

Task:

- ChrgM controls the charging process between the EV and the EVSE as per ISO 15118-2.
- ChrgM communicates with different BSW modules such as the PduR, SoAd, Csm, KeyM, BswM, to enable the charging process.
- Provides ports which can be used by the SWCs which implement the application part of charging process.
- Provides V2GTP communication protocol
- Provides EXI encoding & decoding of messages
- ChrgM provides error handling mechanism and timers for managing communication between EV and EVSE.

ChrgM consists of two submodules: V2GTP and EXI.

- V2GTP (vehicle to grid standard protocol): this submodule formats data into a PDU.
- EXI (efficient XML interchange): this submodule converts data into byte streams as per the W3C1.0 recommendation.



Architecture – Content of Software Layers

Communication Stack - ChrgM

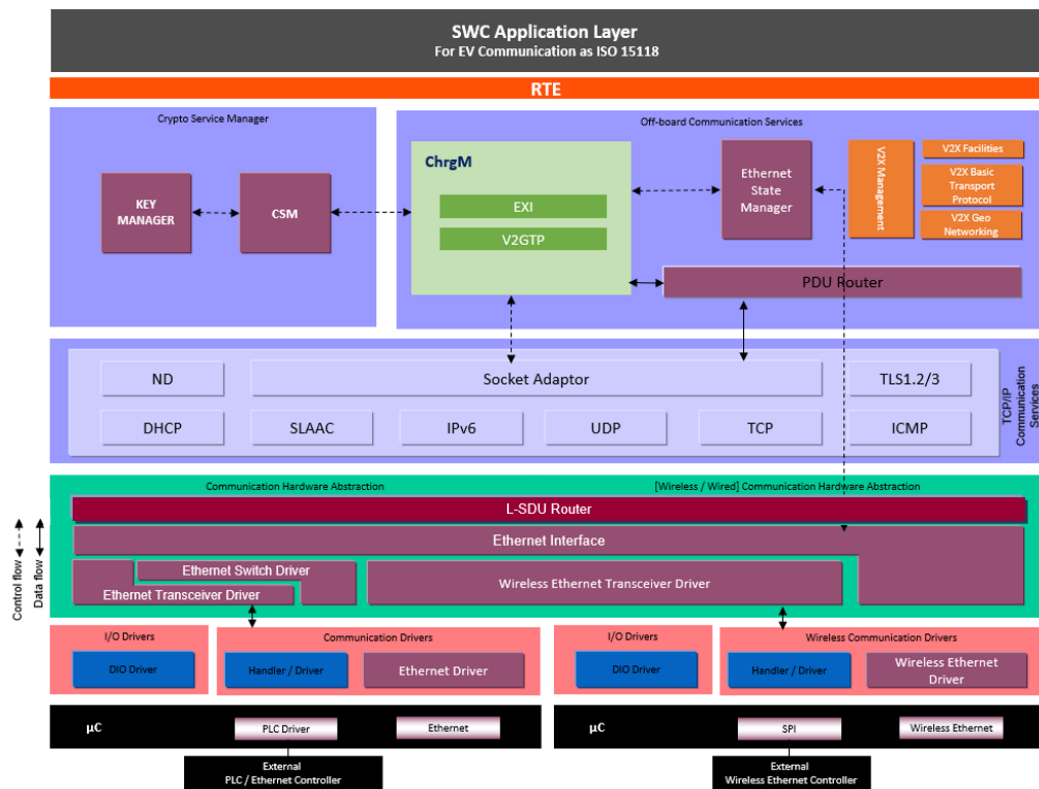
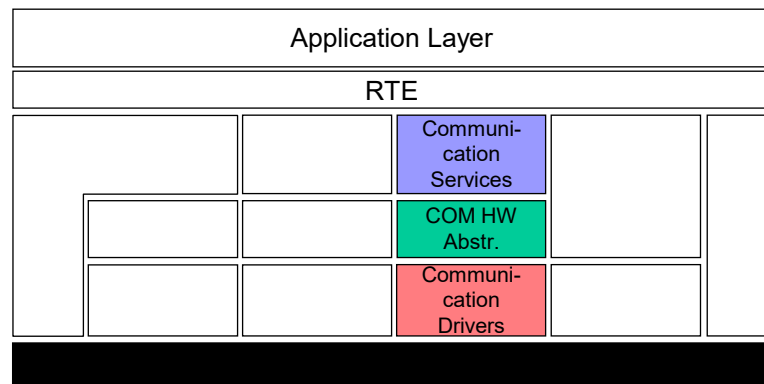
充电管理器（ChrgM）属于AUTOSAR分层架构的通信服务。

任务：

- ChrgM 根据 ISO 15118-2 标准控制电动汽车（EV）与电动汽车供电设备（EVSE）之间的充电过程。
- ChrgM 与不同的 BSW 模块（如 PduR、SoAd、Csm、KeyM、BswM）进行通信，以启用充电过程。
- 提供端口，可供实现充电过程应用部分的软件组件（SW-Cs）使用。
- 提供V2GTP通信协议
- 提供消息的EXI编码与解码
- ChrgM 为管理电动汽车（EV）与电动汽车供电设备（EVSE）之间的通信提供了错误处理机制和定时器。

ChrgM由两个子模块组成：V2GTP和EXI。

- V2GTP（车辆到电网标准协议）：此子模块将数据格式化为PDU。
- EXI（高效 XML 交换）：该子模块根据 W3C 1.0 建议将数据转换为字节流。

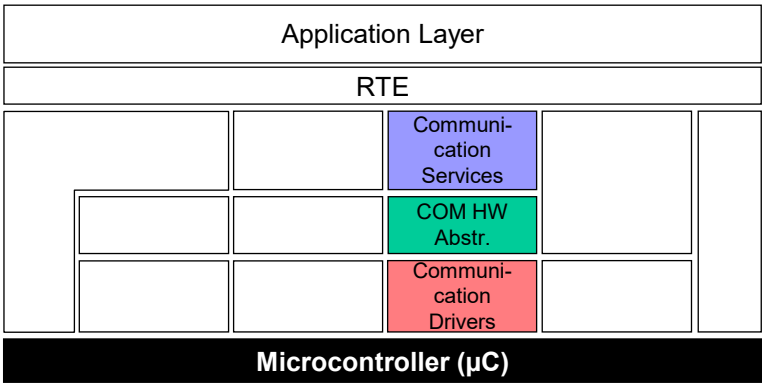


Architecture – Content of Software Layers

Communication Stack – General

General communication stack properties:

- A signal gateway is part of AUTOSAR COM to route signals.
- PDU based Gateway is part of PDU router.
- IPDU multiplexing provides the possibility to add information to enable the multiplexing of I-PDUs (different contents but same IDs on the bus).
- Multi I-PDU to container mapping provides the possibility to combine several I-PDUs into one larger (container-)I-PDU to be transmitted in one (bus specific) frame.
- Upper Interface: μ C, ECU hardware and network type independent.
- For refinement of GW architecture please refer to “Example Communication”

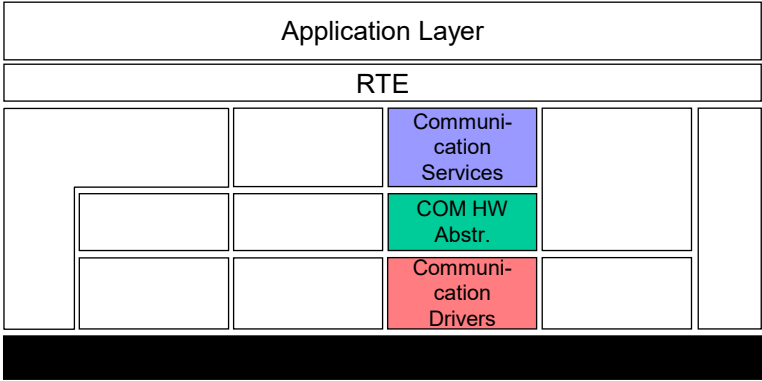


Architecture – Content of Software Layers

通信栈 – 概述

通用通信栈属性：

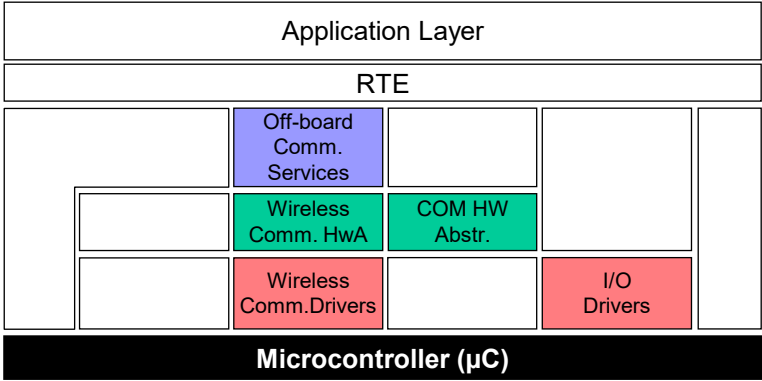
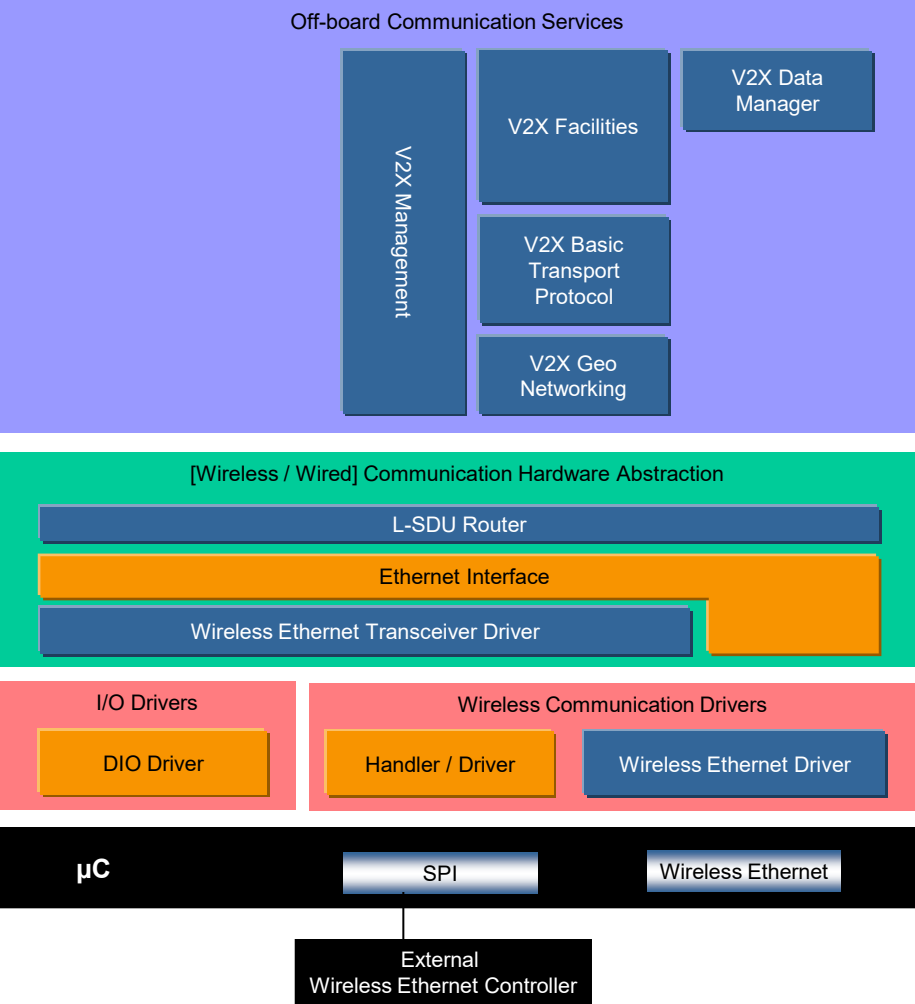
信号网关是AUTOSAR COM中用于路由信号的部分。
 基于PDU的网关是PDU路由器的一部分。 IPDU复用提供了添加信息以实现I-PDU复用的可能性（不同内容但总线ID相同）。
 多I-PDU到容器的映射提供了将多个I-PDU组合成一个更大的（容器）I-PDU的可能性，以便在单个（总线特定）帧中传输。
 上层接口：独立于μC、ECU硬件和网络类型。
 关于网关架构的细化，请参考“示例通信”。



Architecture – Content of Software Layers

Off-board Communication Stack – European Vehicle-2-X

Example:



The **European Vehicle-2-X Communication Services** are a group of modules for Vehicle-to-X communication via an ad-hoc wireless network.

- Facilities: implement the functionality for reception and transmission of standardized V2X messages, build the interface for vehicle specific SW-Cs
- Basic Transport Protocol = Layer 4
- Geo-Networking = Layer 3 (Addressing based on geographic areas, the respective Ethernet frames have their own Ether-Type)
- V2X Management: manages cross-layer functionality (like dynamic congestion control, security, position and time)
- V2X Data Manager: manages the receiving and transformation of V2X messages and sends them through RTE to SW-Cs or via SOME/IP

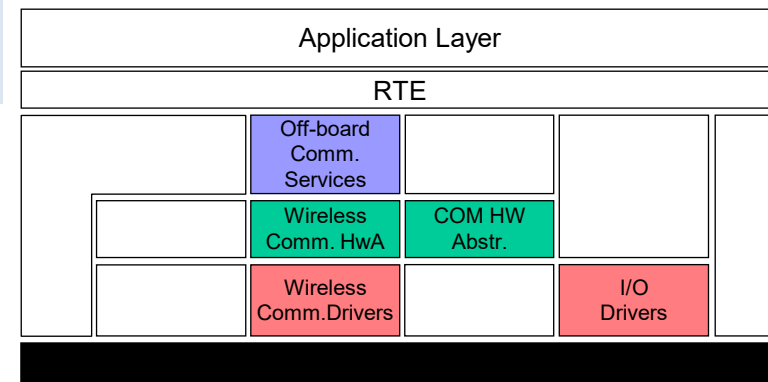
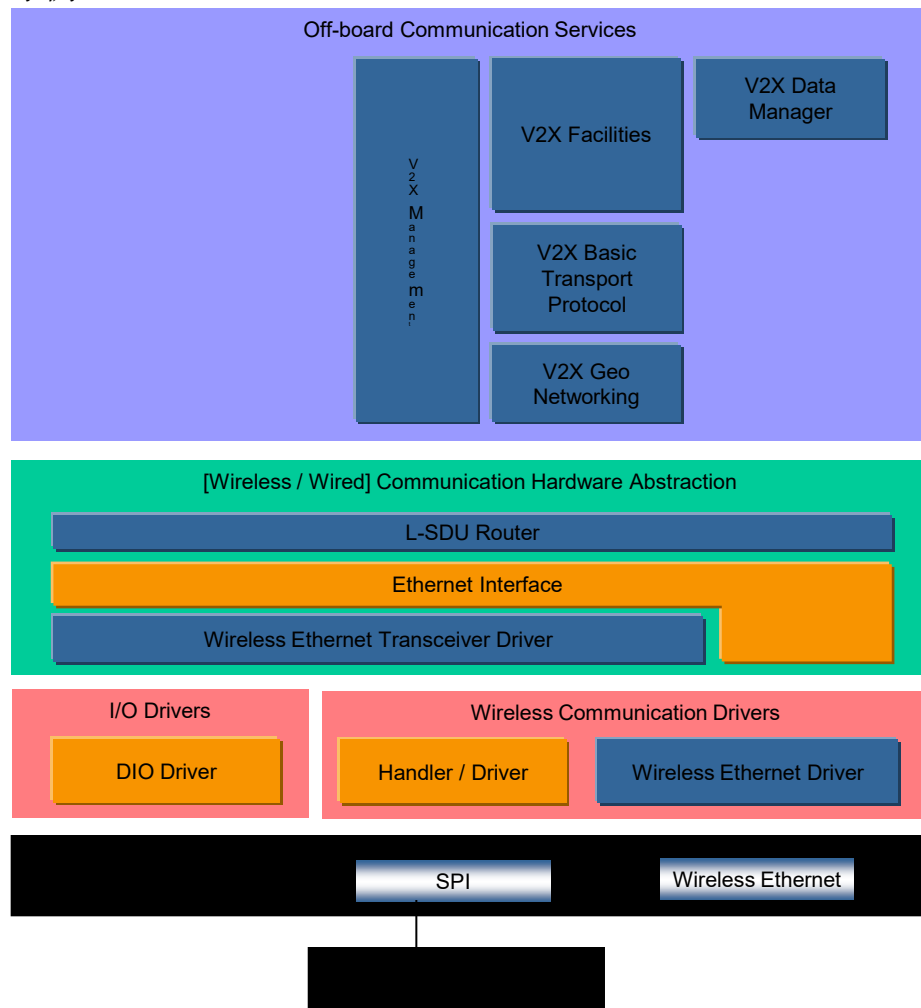
Task:

- Provide a uniform interface to the Wireless Ethernet network. Hide protocol and message properties from the application.

Architecture – Content of Software Layers

车外通信堆栈 – 欧洲车辆对X通信

示例:



欧洲车辆对X通信服务

是一组用于通过自组织无线网络进行车联网通信的模块。

设施层：实现标准化V2X消息的接收与发送功能，构建面向车辆特定软件组件的接口
基础传输协议 = 第4层
地理网络 = 第3层（基于地理区域寻址，对应以太网帧具有独立以太类型标识）
V2X管理层：管理跨层功能（如动态拥塞控制、安全机制、定位与时间同步）
V2X数据管理器：管理V2X消息的接收与格式转换，并通过运行时环境发送至软件组件或经由SOME/IP协议传输

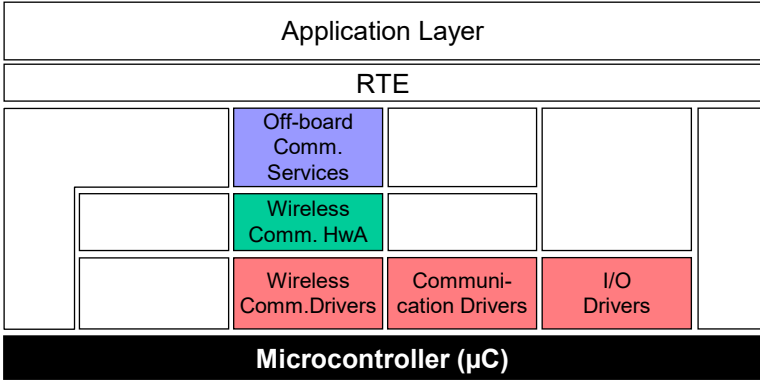
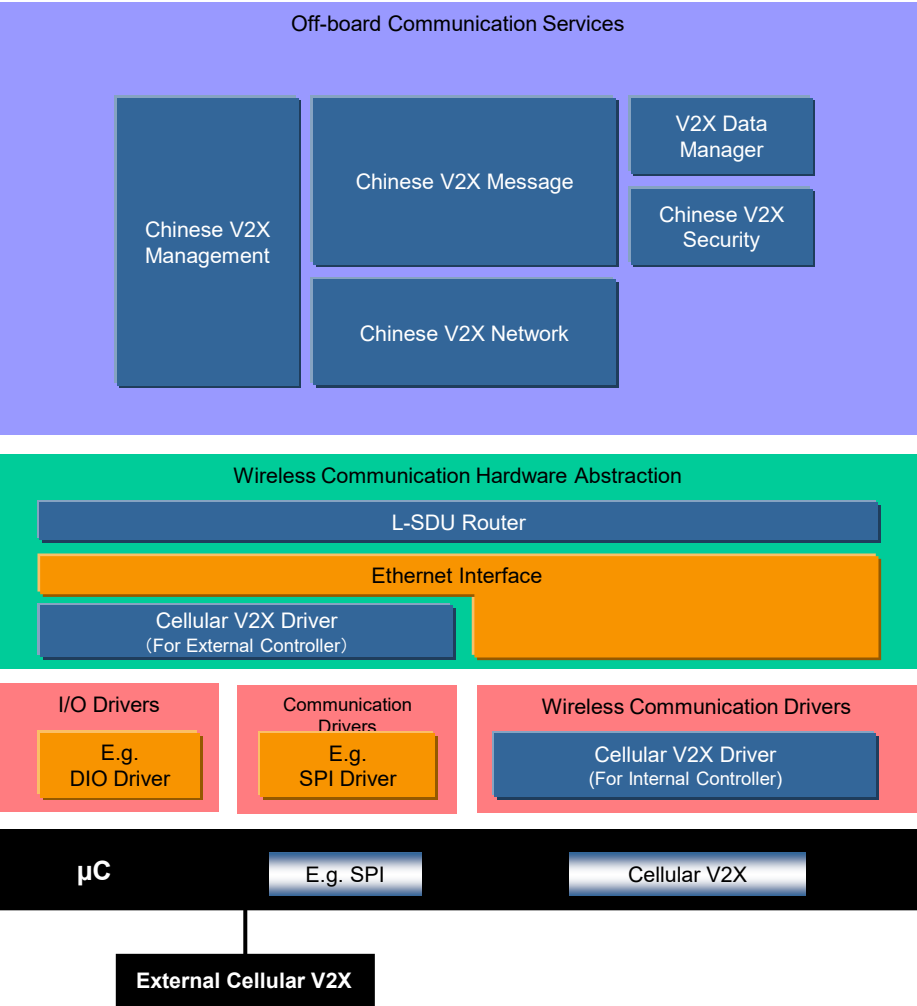
任务：

为无线以太网网络提供统一的接口。向应用程序隐藏协议和消息属性。

Architecture – Content of Software Layers

Off-board Communication Stack – Chinese Vehicle-2-X

Example:



The **Chinese Vehicle-2-X Communication Services** are a group of modules based on cellular based V2X technology following Chinese V2X standards.

- **Message:** implement the functionality for reception and transmission of standardized Chinese V2X message, build the interface for vehicle specific SW-Cs; implement management functionalities related to Message Layer(sending frequency, Position and Time, message Identifiers)
- **Security:** implement the functionality of message encapsulation, decapsulation and pseudonym management
- **Network:** message reception and transmission, Layer-2 IDs settings, etc.
- **Management:** manage cross-Layer functionality(such as Dedicated Service Advertisement, etc.)

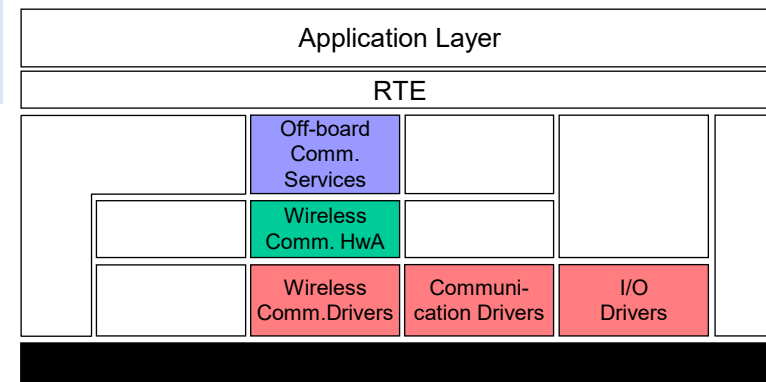
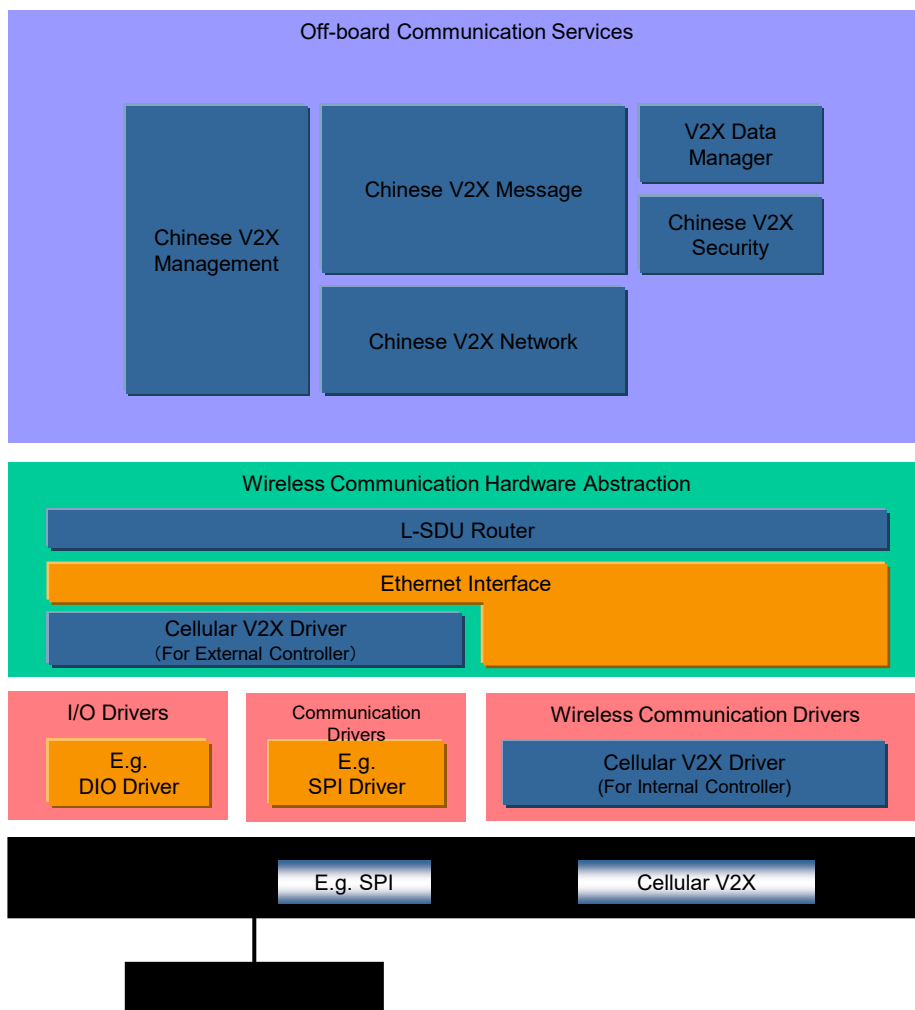
Task:

- Provide a uniform interface to the cellular based V2X network. Hide protocol and message properties from the application.

Architecture – Content of Software Layers

车外通信堆栈 – 中国车辆对万物通信

示例：



中国车辆-2-X通信服务是一组基于蜂窝V2X技术、遵循中国V2X标准的模块。

消息：实现标准化中国V2X消息的接收与发送功能，构建车辆专用软件组件（SW-C）的接口；实现与消息层相关的管理功能（发送频率、位置与时间、消息标识符） 安全：实现消息封装、解封装及假名管理功能 网络：消息接收与发送、二层ID设置等 管理：管理跨层功能（如专用服务通告等）

任务：

为基于蜂窝的V2X网络提供统一接口，向应用程序隐藏协议与消息属性。

Architecture – Content of Software Layers

Services: Memory Services

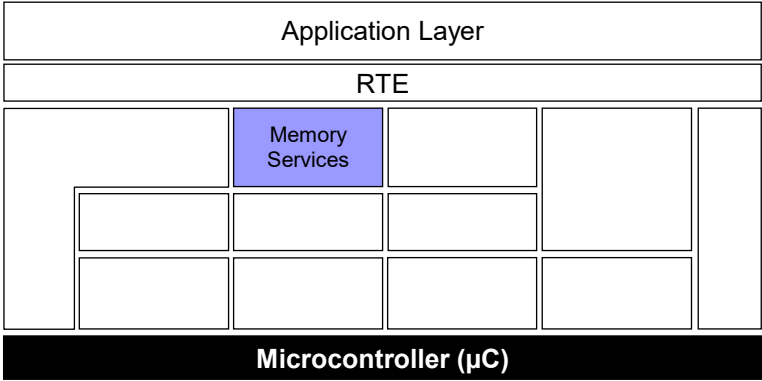
The **Memory Services** consist of one module, the NVRAM Manager. It is responsible for the management of non volatile data (read/write from different memory drivers).

Task: Provide non volatile data to the application in a uniform way. Abstract from memory locations and properties. Provide mechanisms for non volatile data management like saving, loading, checksum protection and verification, reliable storage etc.

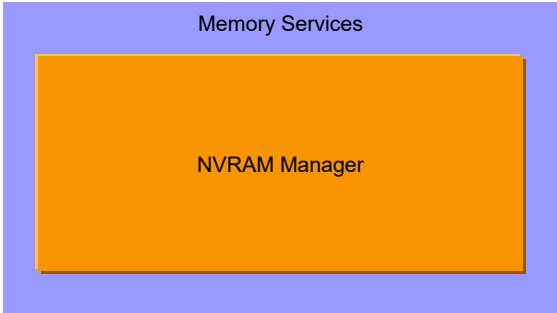
Properties:

Implementation: μ C and ECU hardware independent, highly configurable

Upper Interface: μ C and ECU hardware independent specified and implemented according to AUTOSAR (AUTOSAR interface)



Example:



Architecture – Content of Software Layers

服务：内存服务

内存服务由一个模块组成，即NVRAM管理器。它负责管理非易失性数据（从不同的内存驱动程序进行读写）。

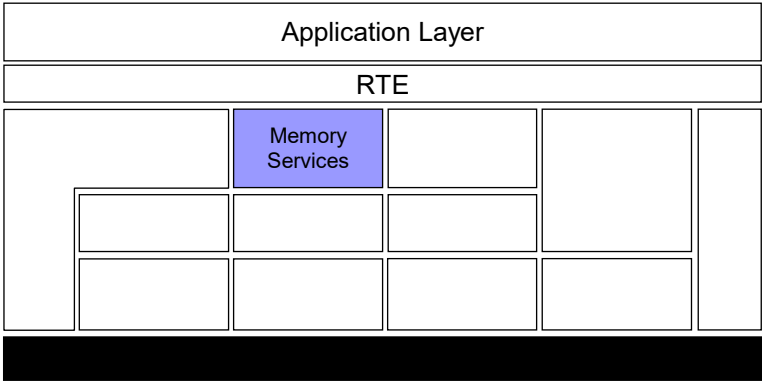
任务：以统一的方式向应用程序提供非易失性数据。抽象化内存位置和属性。提供非易失性数据管理机制，如保存、加载、校验和保护、验证、可靠存储等。

属性：

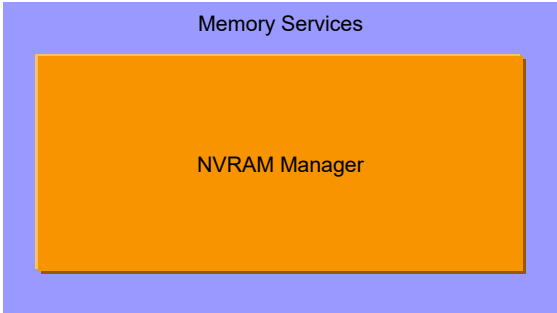
实现：μC与ECU硬件独立，高度可配置

上层接口：根据AUTOSAR规范独立指定和实现，与μC及ECU硬件无关

(AUTOSAR接口)



示例：



Architecture – Content of Software Layers

Services: System Services

The **System Services** are a group of modules and functions which can be used by modules of all layers. Examples are Real Time Operating System (which includes timer services) and Error Manager.

Some of these services are:

- μC dependent (like OS), and may support special μC capabilities (like Time Service),
- partly ECU hardware and application dependent (like ECU State Manager) or
- hardware and μC independent.

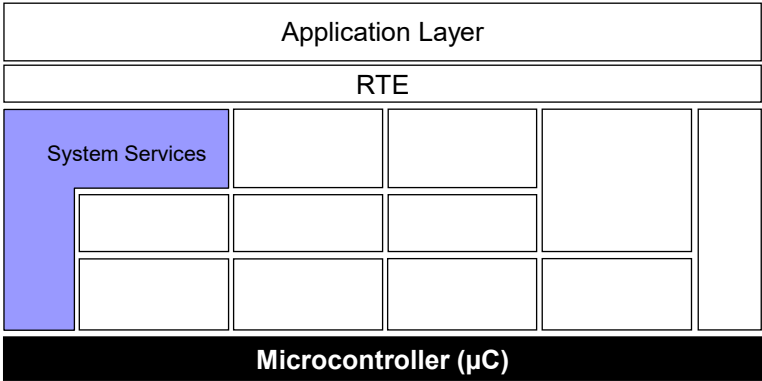
Task:

Provide basic services for application and basic software modules.

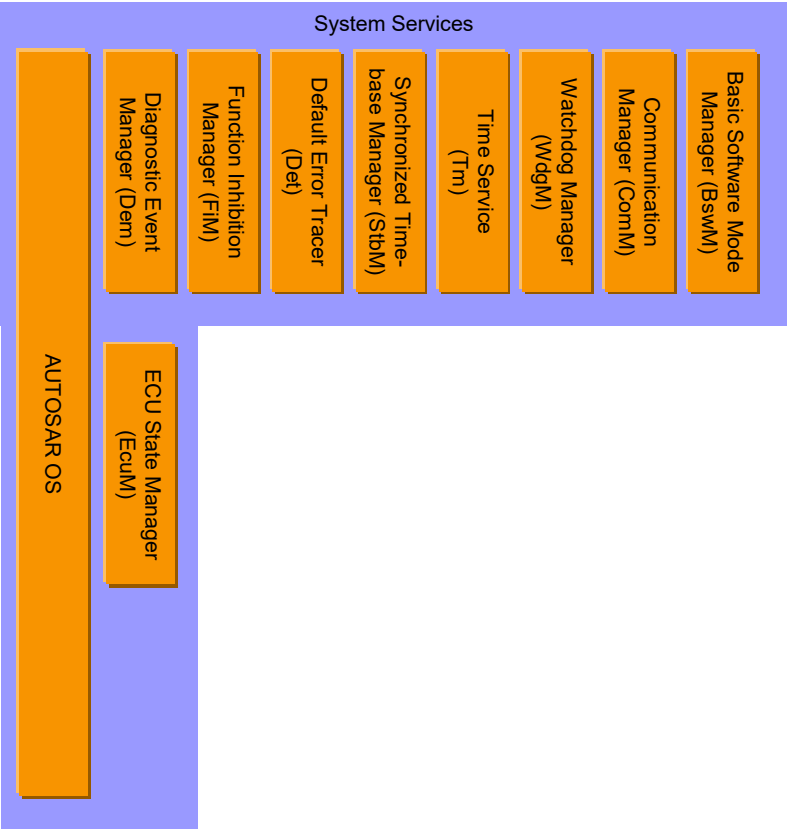
Properties:

Implementation: partly μC, ECU hardware and application specific

Upper Interface: μC and ECU hardware independent



Example:



Architecture – Content of Software Layers

服务：系统服务

系统服务是一组模块和功能，可供所有层次的模块使用。
例如实时操作系统（包含计时器服务）和错误管理器。

。

其中一些服务包括：

依赖于微控制器（如操作系统），并可能支持特殊的微控制器功能（如时间服务），部分依赖于ECU硬件和应用程序（如ECU状态管理器），或独立于硬件和微控制器。

任务：

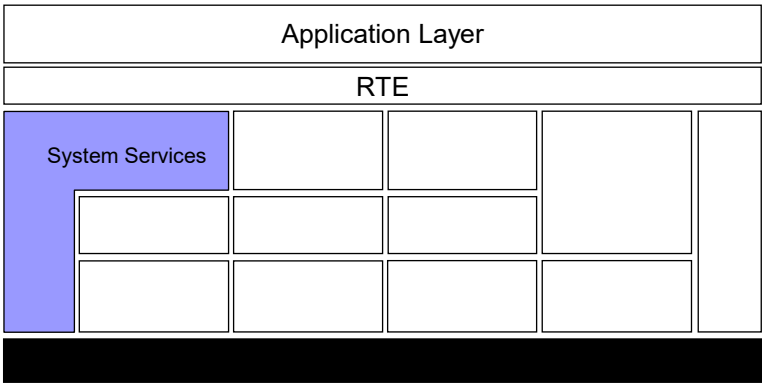
为应用程序和基础软件模块提供基本服务

。

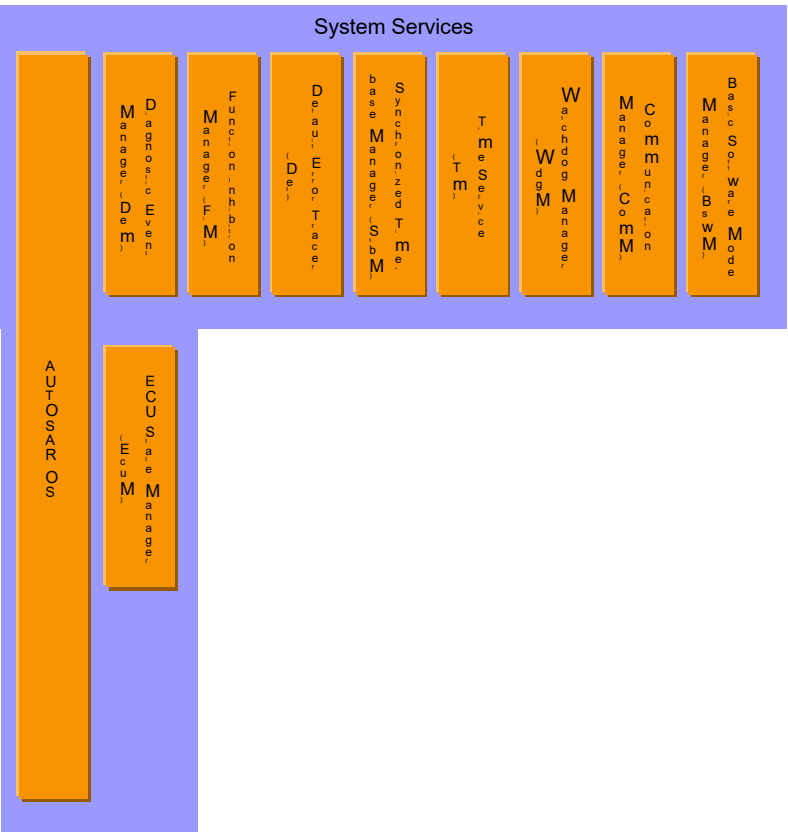
属性：

实现：部分采用微控制器，ECU硬件及应用特定的上层接口：

独立于微控制器和ECU硬件

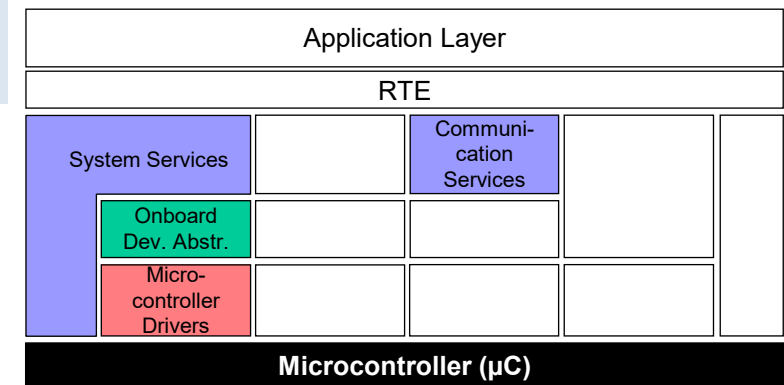
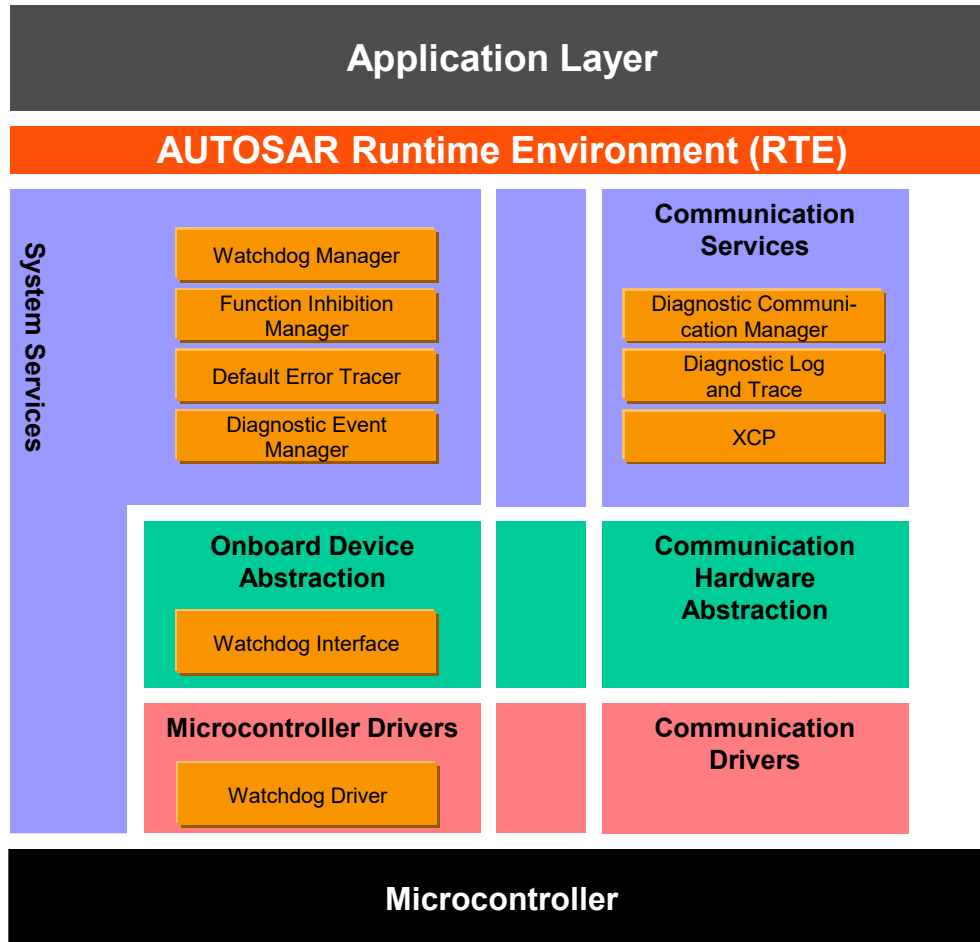


示例：



Architecture – Content of Software Layers

Error Handling, Reporting and Diagnostic



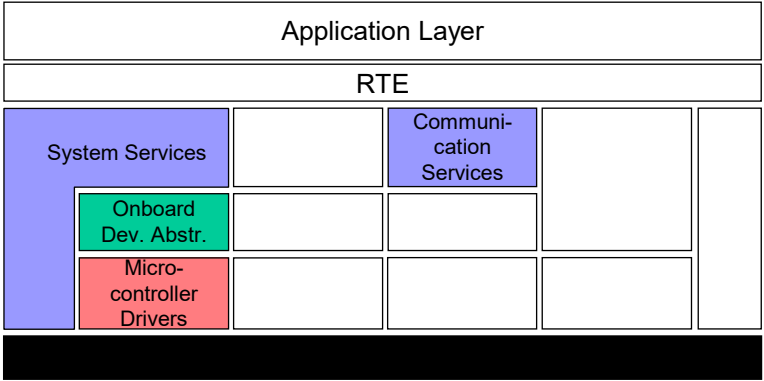
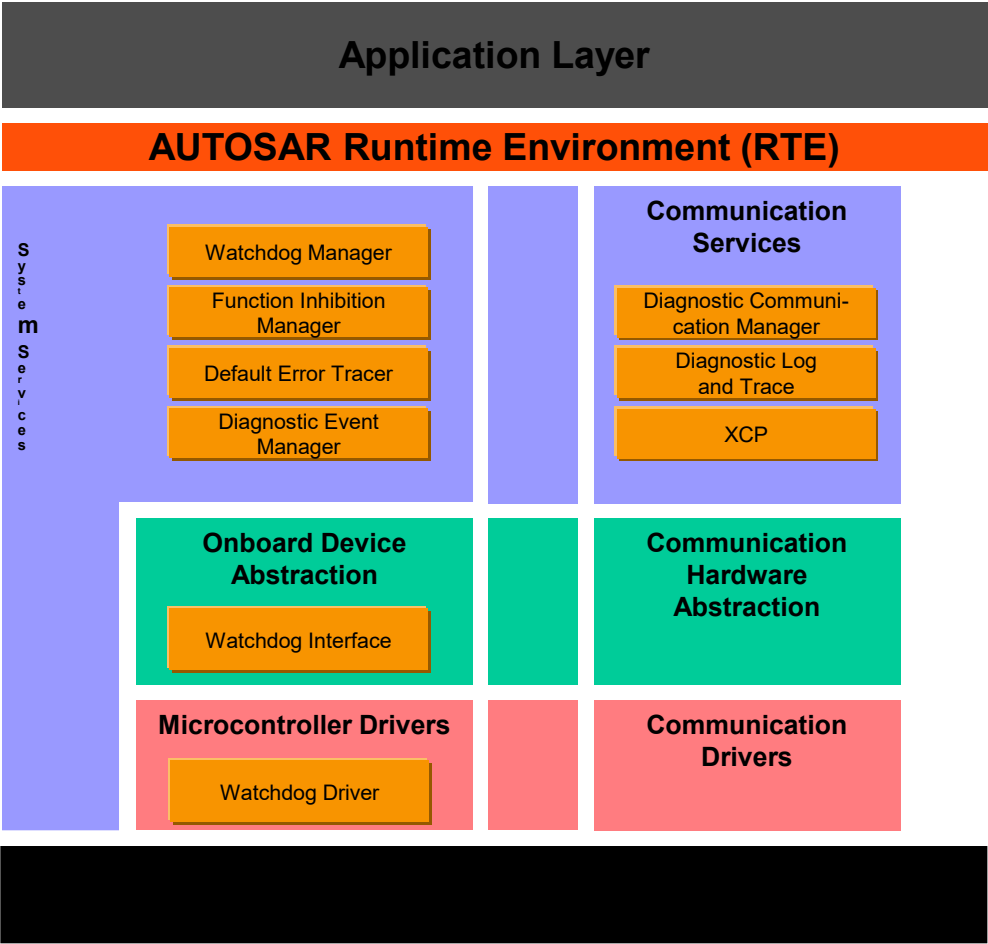
There are dedicated modules for different aspects of error handling in AUTOSAR. E.g.:

- The **Diagnostic Event Manager** is responsible for processing and storing diagnostic events (errors) and associated FreezeFrame data.
- The module **Diagnostic Log and Trace** supports logging and tracing of applications. It collects user defined log messages and converts them into a standardized format.

- All detected development errors in the Basic Software are reported to **Default Error Tracer**.
- The **Diagnostic Communication Manager** provides a common API for diagnostic services
- etc.

Architecture – Content of Software Layers

错误处理、报告与诊断



AUTOSAR中针对错误处理的不同方面设有专用模块。例如：**诊断事件管理器**负责处理和存储诊断事件（错误）及相关的冻结帧数据。**诊断日志与跟踪模块**支持应用程序的日志记录与跟踪功能，可收集用户定义的日志消息并进行转换。

将它们转换为标准化格式。

所有在基础软件中检测到的开发错误都会报告给默认错误跟踪器。
诊断通信管理器为诊断服务提供了一个通用API。
等等。

Architecture – Content of Software Layers

Application Layer: Sensor/Actuator Software Components

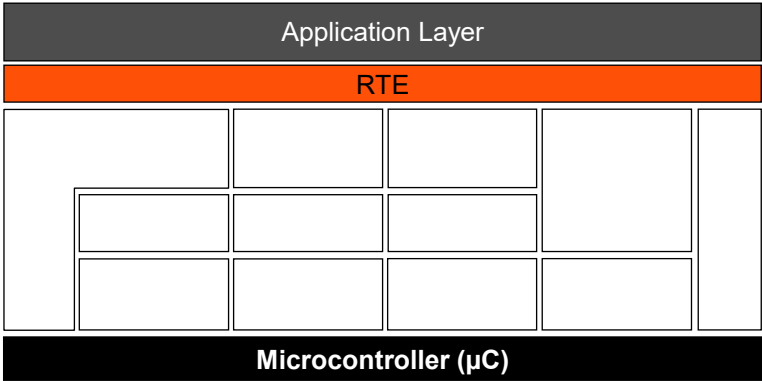
The **Sensor/Actuator AUTOSAR Software Component** is a specific type of AUTOSAR Software Component for sensor evaluation and actuator control. Though not belonging to the AUTOSAR Basic Software, it is described here due to its strong relationship to local signals. It has been decided to locate the Sensor/Actuator SW Components above the RTE for integration reasons (standardized interface implementation and interface description). Because of their strong interaction with raw local signals, relocatability is restricted.

Task:

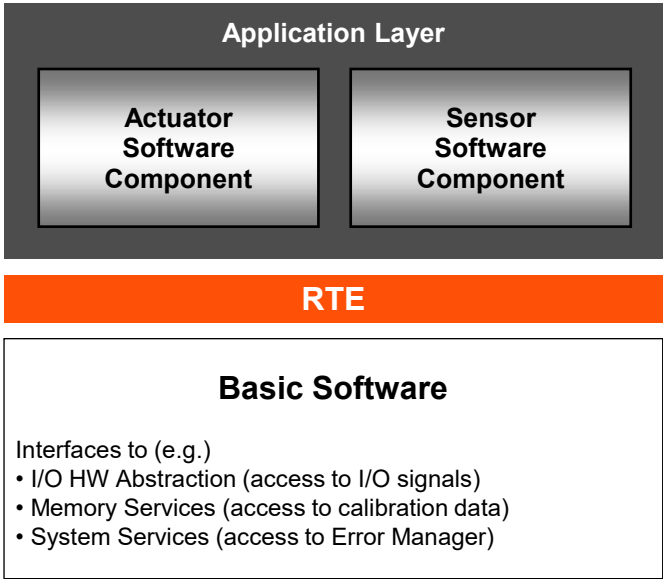
Provide an abstraction from the specific physical properties of hardware sensors and actuators, which are connected to an ECU.

Properties:

Implementation: μ C and ECU HW independent, sensor and actuator dependent



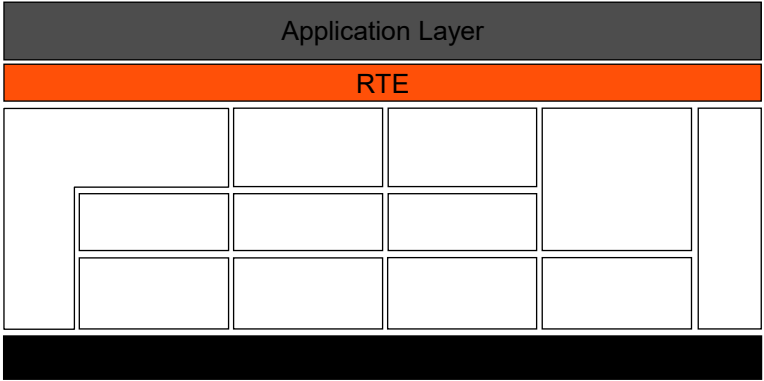
Example:



T传感器/执行器 AUTOSAR 软件组件是一种用于传感器评估和执行器控制的特定类型 AUTOSAR 软件组件。尽管不属于 AUTOSAR 基础软件，但由于其与本地信号的紧密关联，此处仍对其进行说明。出于集成原因（标准化接口实现与接口描述），已决定将传感器/执行器软件组件置于 RTE 之上。因其与原始本地信号的强交互性，可重定位性受到限制。

任务：
提供对连接到ECU的硬件传感器和执行器具体物理属性的抽象。

属性：
实现：μC和ECU硬件独立，传感器和执行器相关



示例：

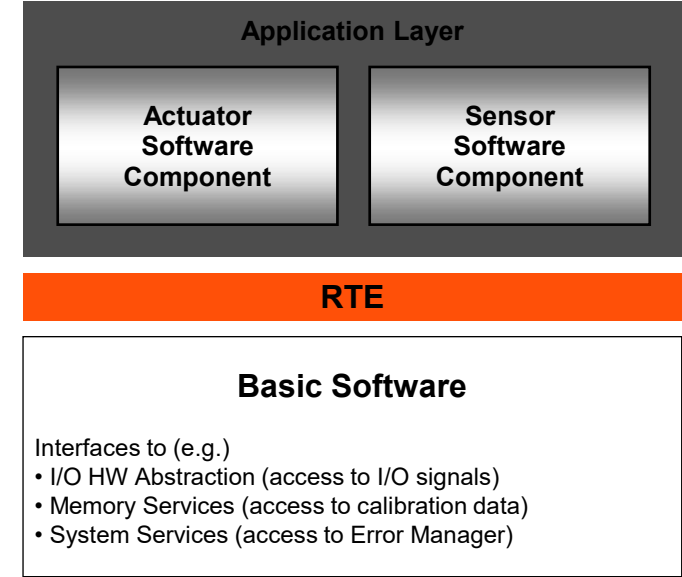


Table of contents

1. Architecture

1. Overview of Software Layers
2. Content of Software Layers
3. **Content of Software Layers in Multi-Core Systems**
4. Content of Software Layers in Mixed-Critical Systems
5. Overview of Modules
6. Interfaces: General Rules
7. Interfaces: Interaction of Layers
8. Overview of CP Software Clusters

2. Configuration

3. Integration and Runtime Aspects

Table of contents

1. 架构

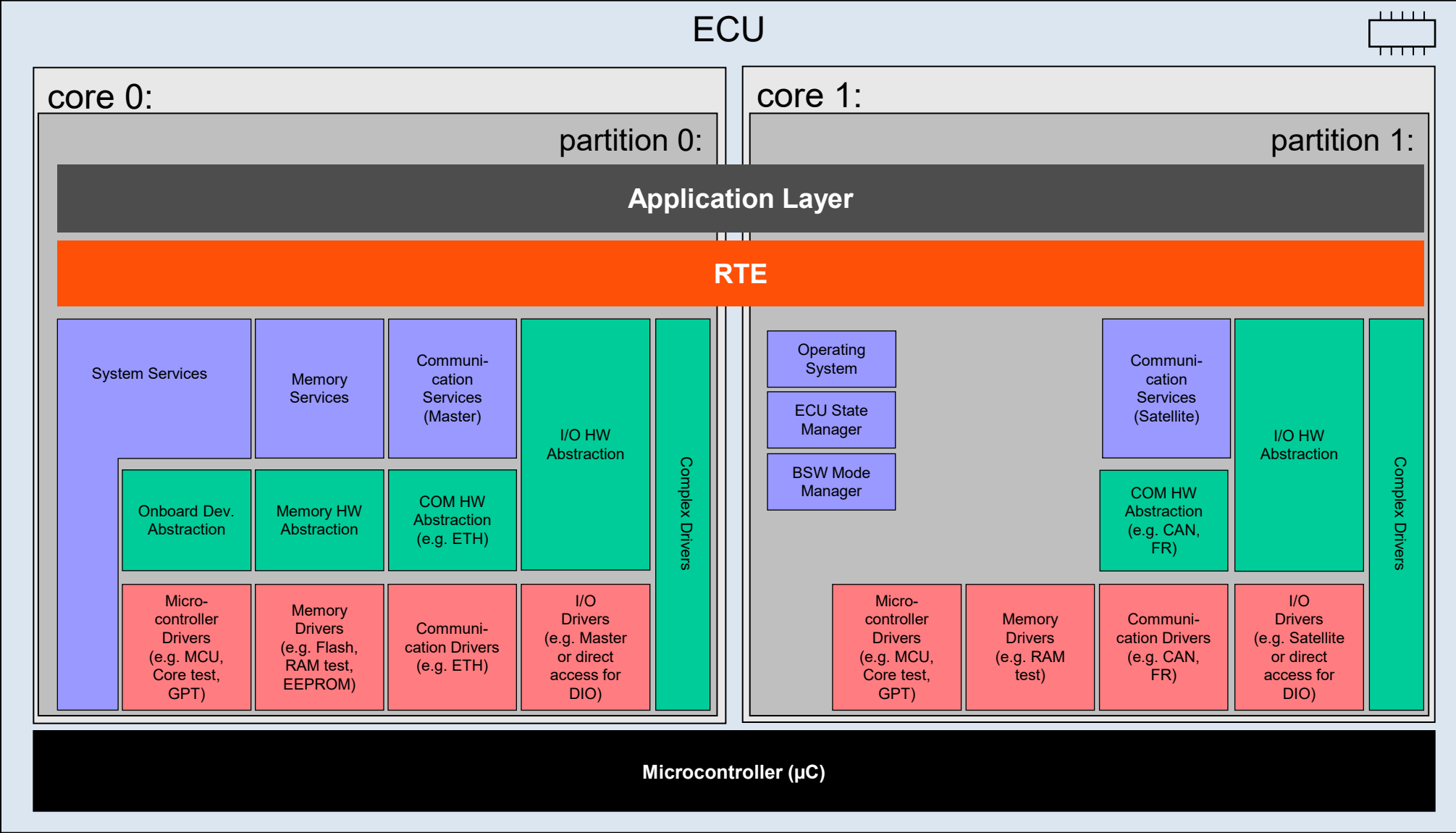
1. 软件层概述 2. 软件层内容 3. 多核系统中的软件层内容 4
. 混合关键系统中的软件层内容 5. 模块概述 6. 接口：通用
规则 7. 接口：层间交互 8. CP软件集群概述

2. 配置

3. 集成与运行时方面

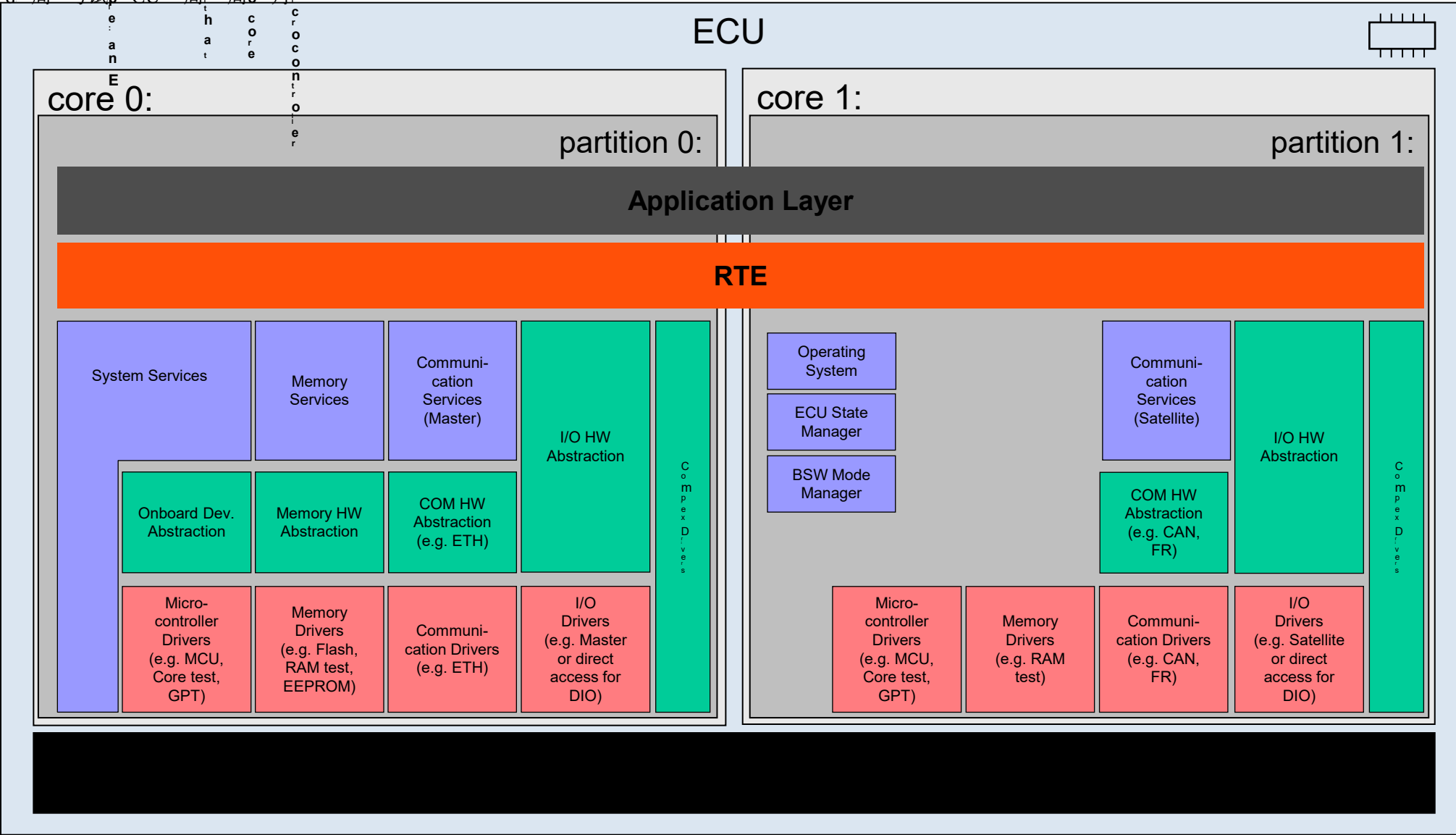
Architecture – Content of Software Layers

Example of a Layered Software Architecture for Multi-Core Microcontroller



Architecture – Content of Software Layers

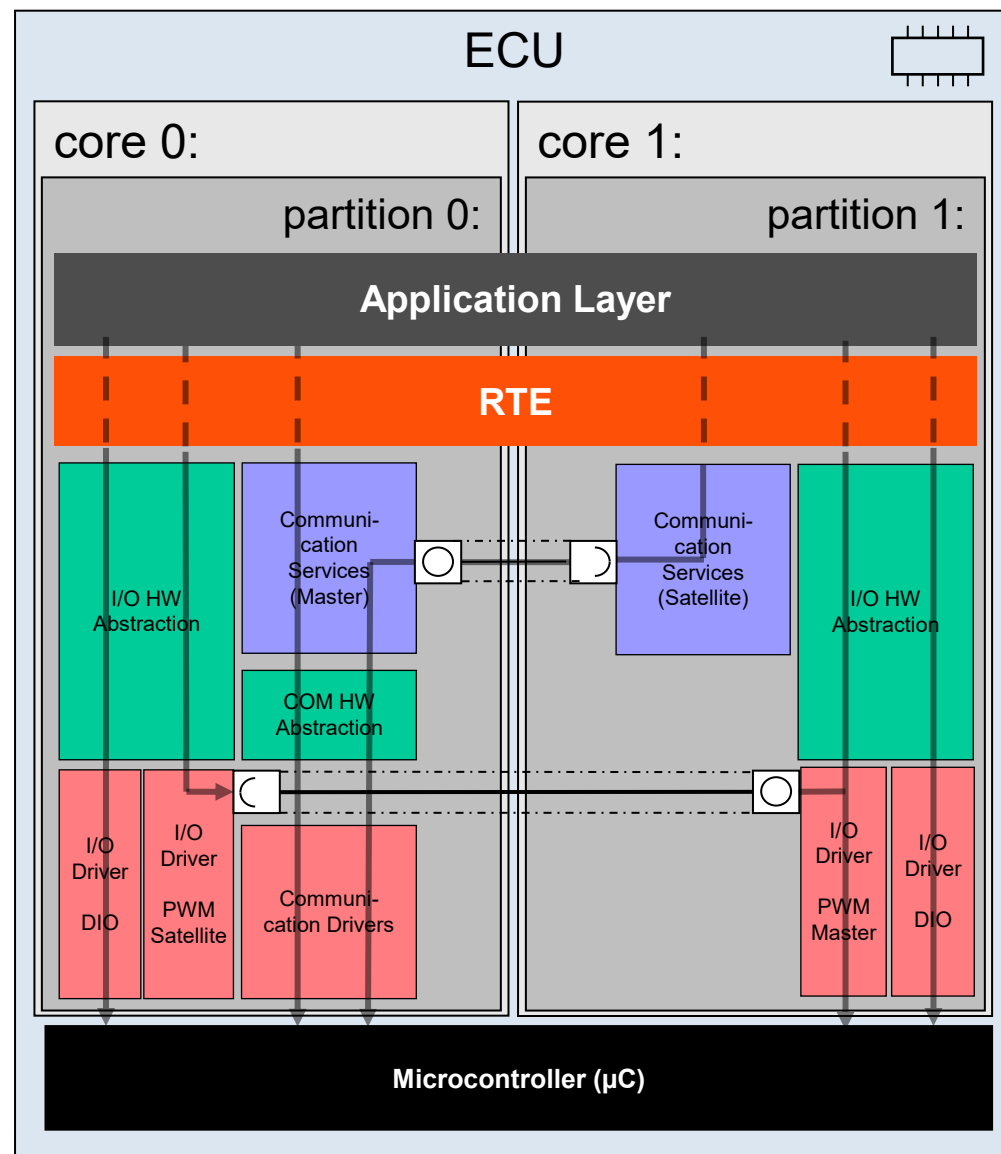
Example of a Layered Software Architecture for Multi-Core Microcontroller



Architecture – Content of Software Layers

Detailed View of Distributed BSW Modules

- BSW modules can be distributed across several partitions and cores. All partitions share the same code.
- Modules can either be completely identical on each partition, as shown for the DIO driver out of I/O stack in the figure.
- As an alternative, they can use core-dependent branching to realize different behavior. Com service and PWM driver use master-satellite communication for processing a call to the master from the according satellites.
 - The communication between master and satellite is not standardized. For example, it can be based on functions provided by the BSW scheduler or on shared memory.
- The arrows indicate which components are involved in the handling of a service call, depending on the approach to distribution and on the origin of the call.



Architecture – Content of Software Layers

分布式BSW模块的详细视图

页 d w 考试, CU w, w. m

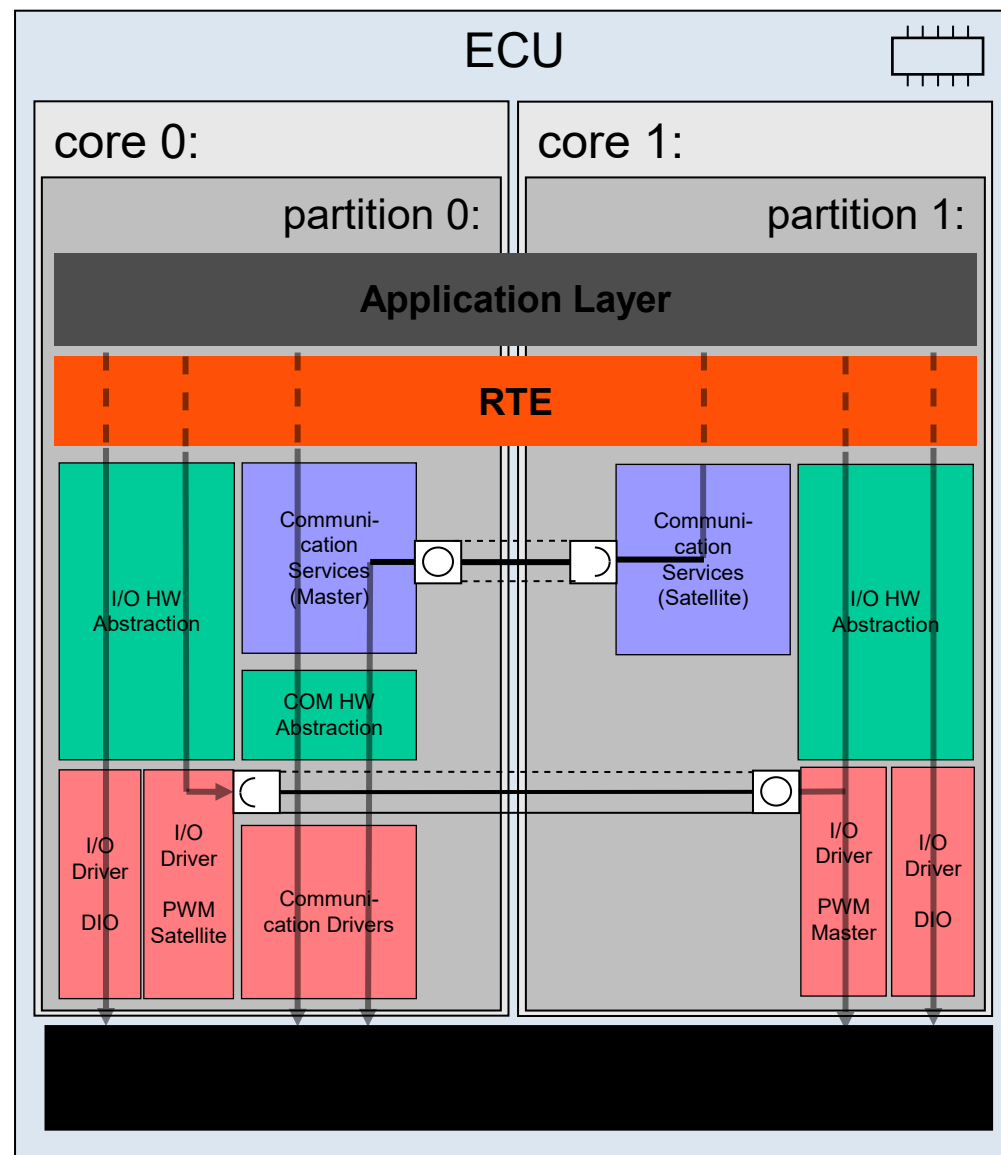
BSW模块可以分布在多个分区和核心上。
所有分区共享相同的代码。

模块可以在每个分区上完全一致，如图中I/O堆栈外的DIO驱动程序所示。

作为替代方案，他们可以使用核心依赖分支来实现不同的行为。Com服务和PWM驱动器采用主从通信方式，以处理来自相应从属设备对主设备的调用。

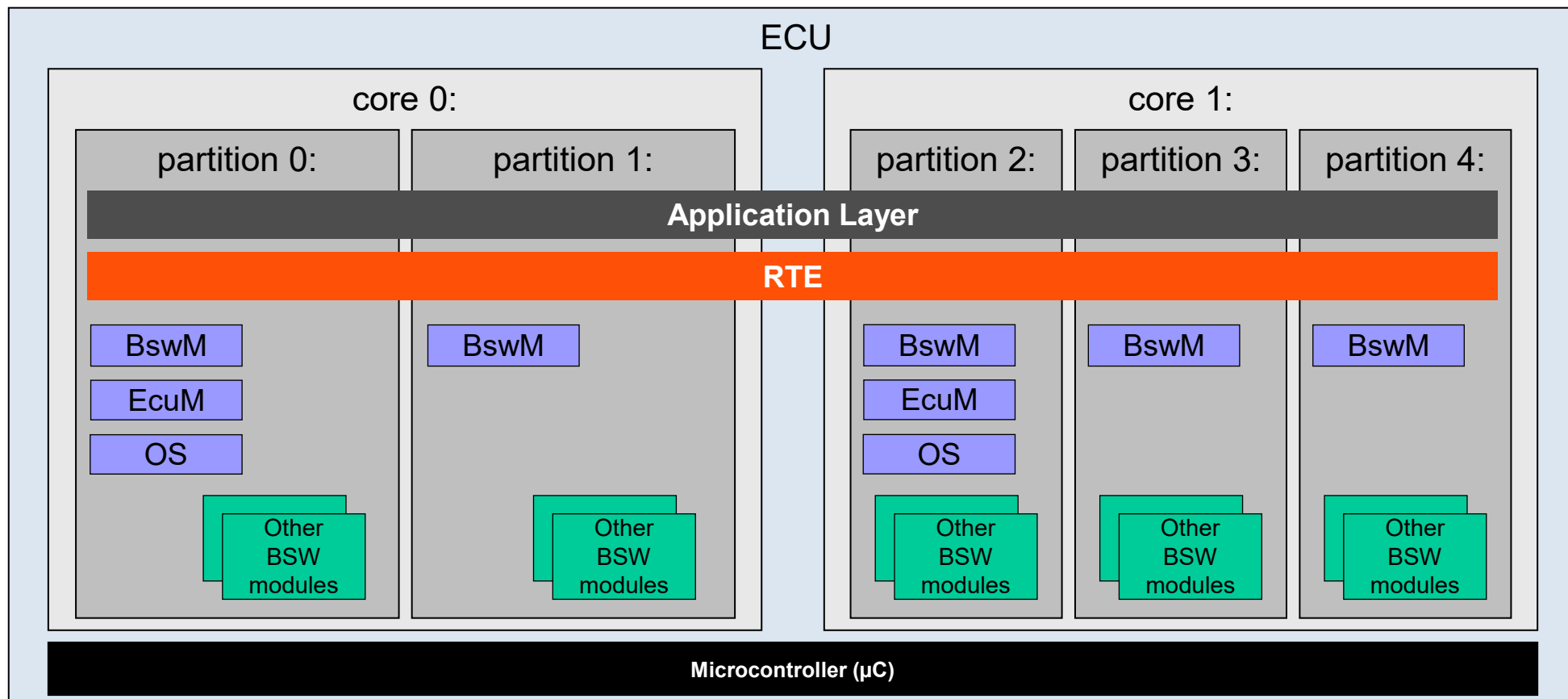
- 主节点与卫星节点之间的通信并未标准化。例如，它可以基于BSW调度器提供的功能，也可以基于共享内存实现。

箭头指示了哪些组件参与处理服务调用，这取决于分布方法以及调用的来源。



Architecture – Content of Software Layers

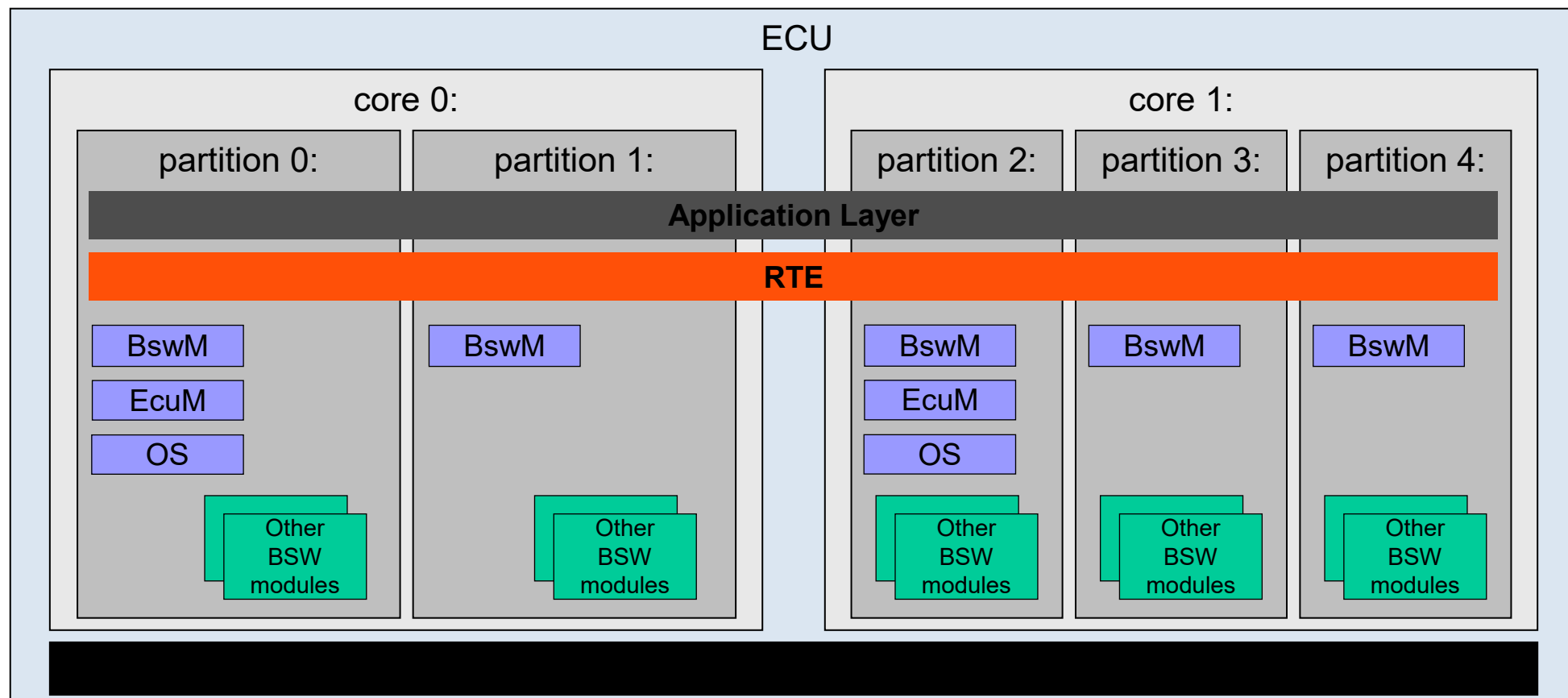
Overview of BswM, OS, EcuM and EcuM on Multiple Partitions



- Basic Software Mode Manager (BswM) in every partition that runs BSW modules
 - all these partitions are trusted
- One EcuM per core (each in a trusted partition)
- EcuM on that core that gets started via the boot-loader is the master EcuM
 - Master EcuM starts all Satellite EcuMs

Architecture – Content of Software Layers

多分区BSW模块、OS、BswM和EcuM概述



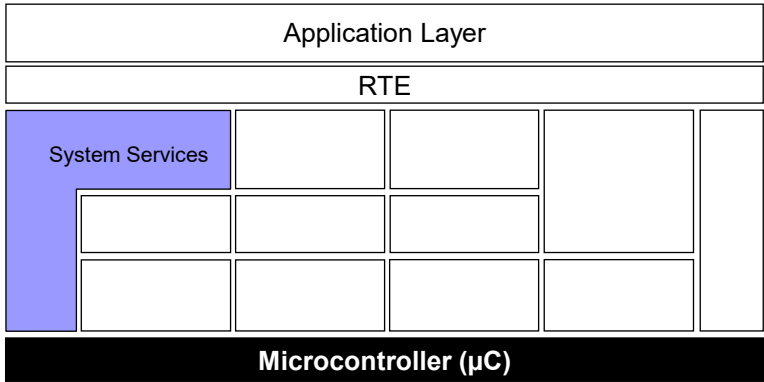
运行BSW模块的每个分区中的基础软件模式管理器（BswM） ■ 所有这些分区都是可信的

每个核心一个EcuM（各自位于可信分区中）

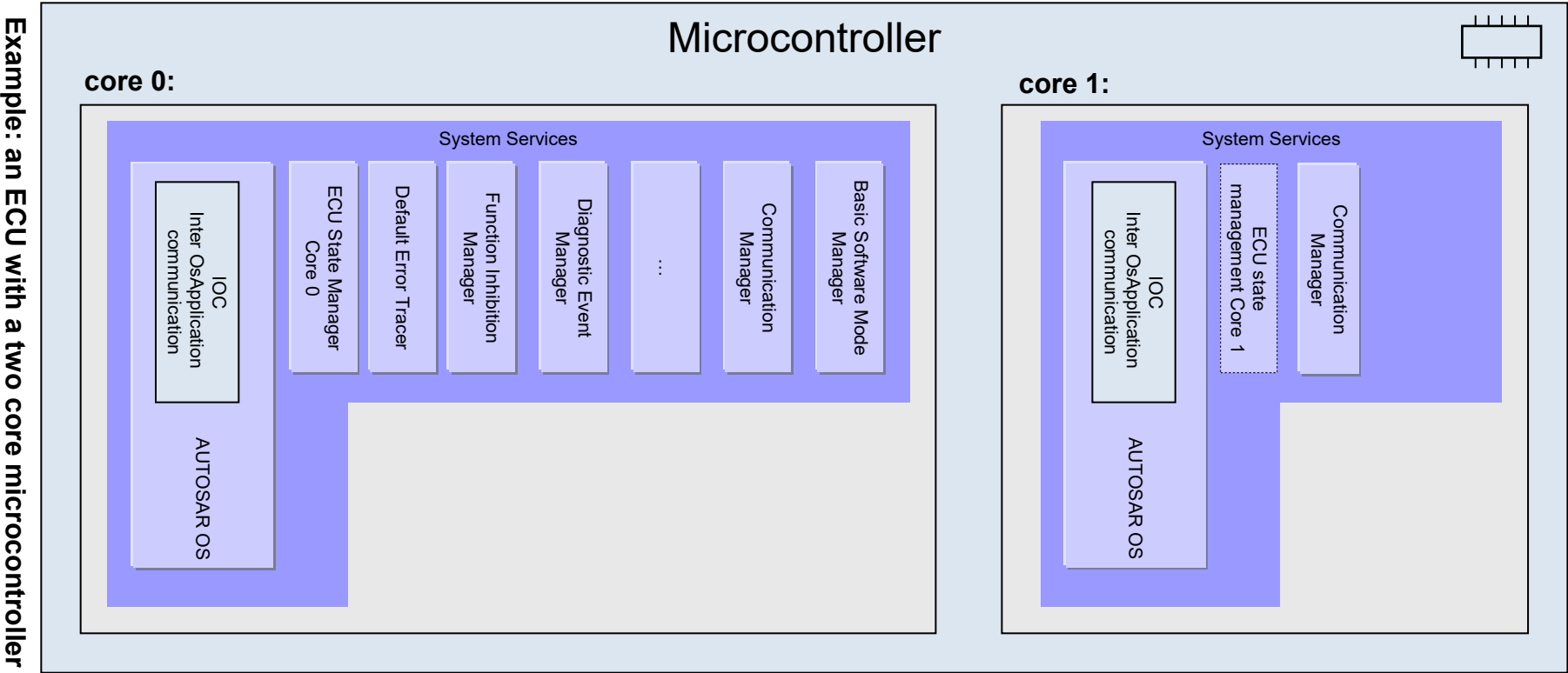
通过引导加载程序启动的那个核心上的 EcuM 是主 EcuM ■，主 EcuM 会启动所有卫星 EcuM。

Architecture – Content of Software Layers

Scope: Multi-Core System Services



- The IOC, as shown in the figure, provides communication services which can be accessed by clients which need to communicate across OS-Application boundaries on the same ECU. The IOC is part of the OS.
- BSW modules can be executable on several cores, such as the ComM in the figure. The core responsible for executing a service is determined at runtime.
- Every core runs a kind of ECU state management.



Example: an ECU with a two core microcontroller

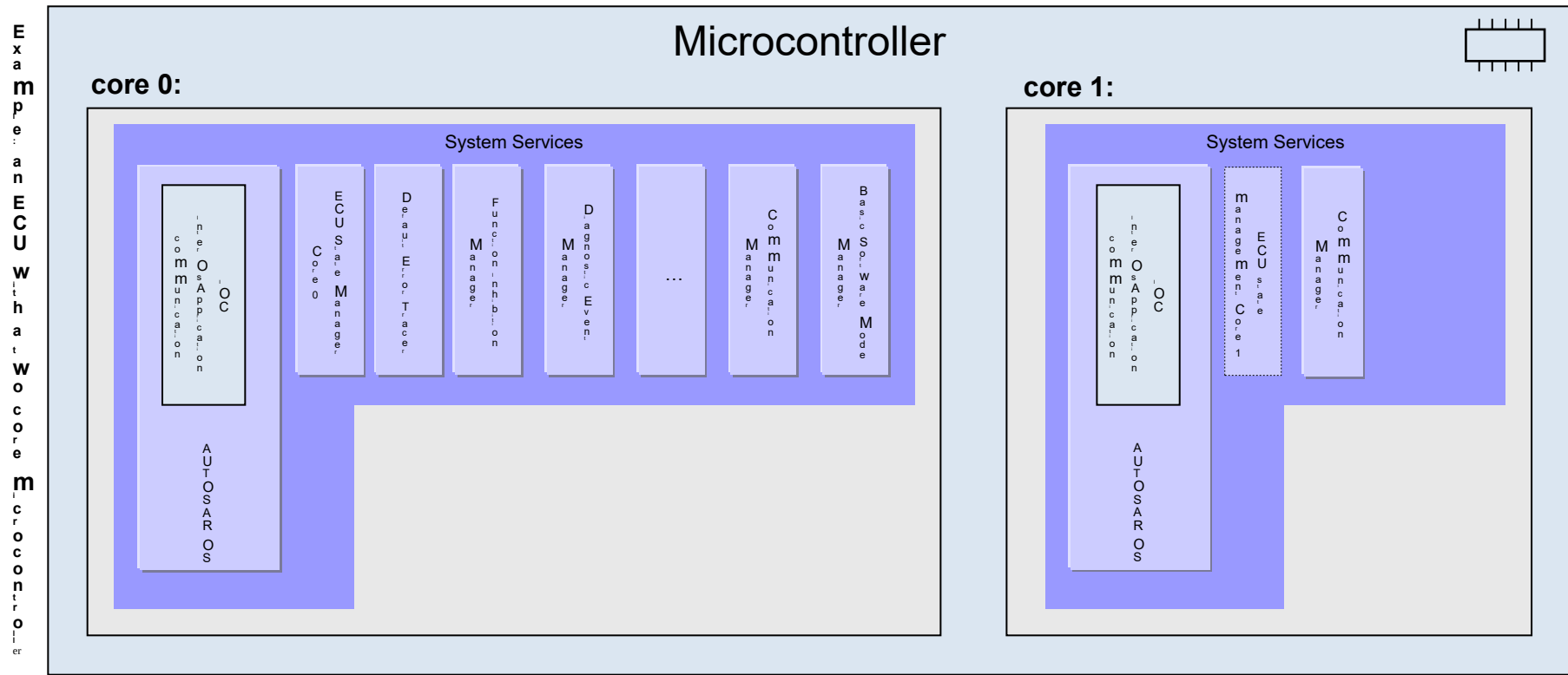
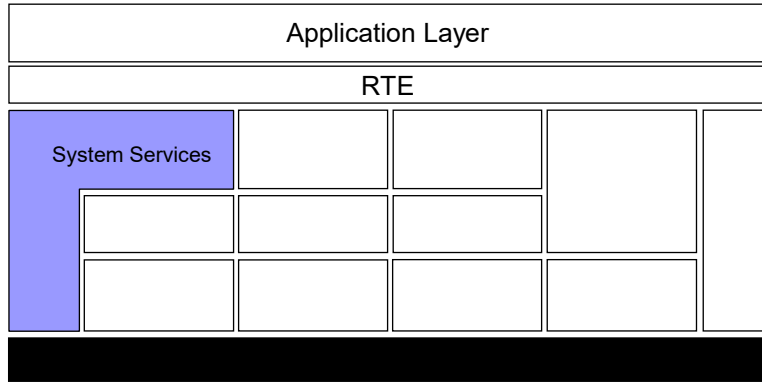
Architecture – Content of Software Layers

范围：多核系统服务

如图所示，IOC提供通信服务，可供需要在同一ECU上跨O S-Application边界进行通信的客户端访问。IOC是操作系统的一部分。

BSW模块可以在多个核心上执行，例如图中的ComM。负责执行服务的核心在运行时确定。

每个核心都运行一种ECU状态管理。



Example of an ECU with a two-core microcontroller

Table of contents

1. Architecture

1. Overview of Software Layers
2. Content of Software Layers
3. Content of Software Layers in Multi-Core Systems
4. **Content of Software Layers in Mixed-Critical Systems**
5. Overview of Modules
6. Interfaces: General Rules
7. Interfaces: Interaction of Layers
8. Overview of CP Software Clusters

2. Configuration

3. Integration and Runtime Aspects

Table of contents

1. 架构

1. 软件层概述 2. 软件层内容 3. 多核系统中的软件层内容 4. 混合关键系统中的软件层内容 5. 模块概述 6. 接口：通用规则 7. 接口：层间交互 8. CP软件集群概述

2. 配置

3. 集成与运行时方面

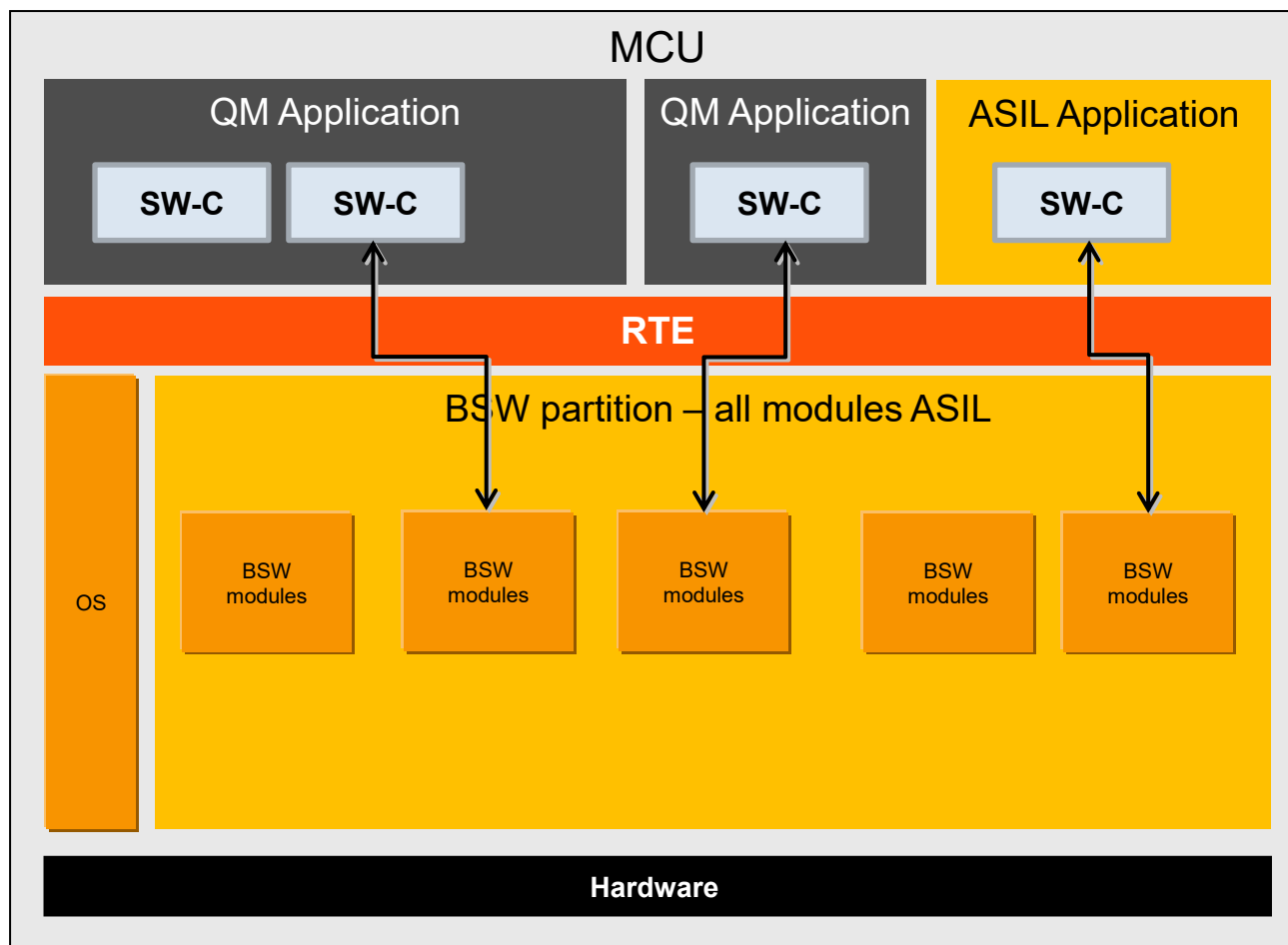
Architecture – Content of Software Layers

Overview of AUTOSAR safety handling

➤ AUTOSAR offers a flexible approach to support safety relevant ECUs. Two methods can be used:

1. All BSW modules are developed according to the required ASIL
2. Selected modules are developed according to ASIL. ASIL and non-ASIL modules are separated into different partitions (BSW distribution)

Note: The partitions are based on OS-Applications. The TRUSTED attribute of the OS-Application is not related to ASIL/non-ASIL.



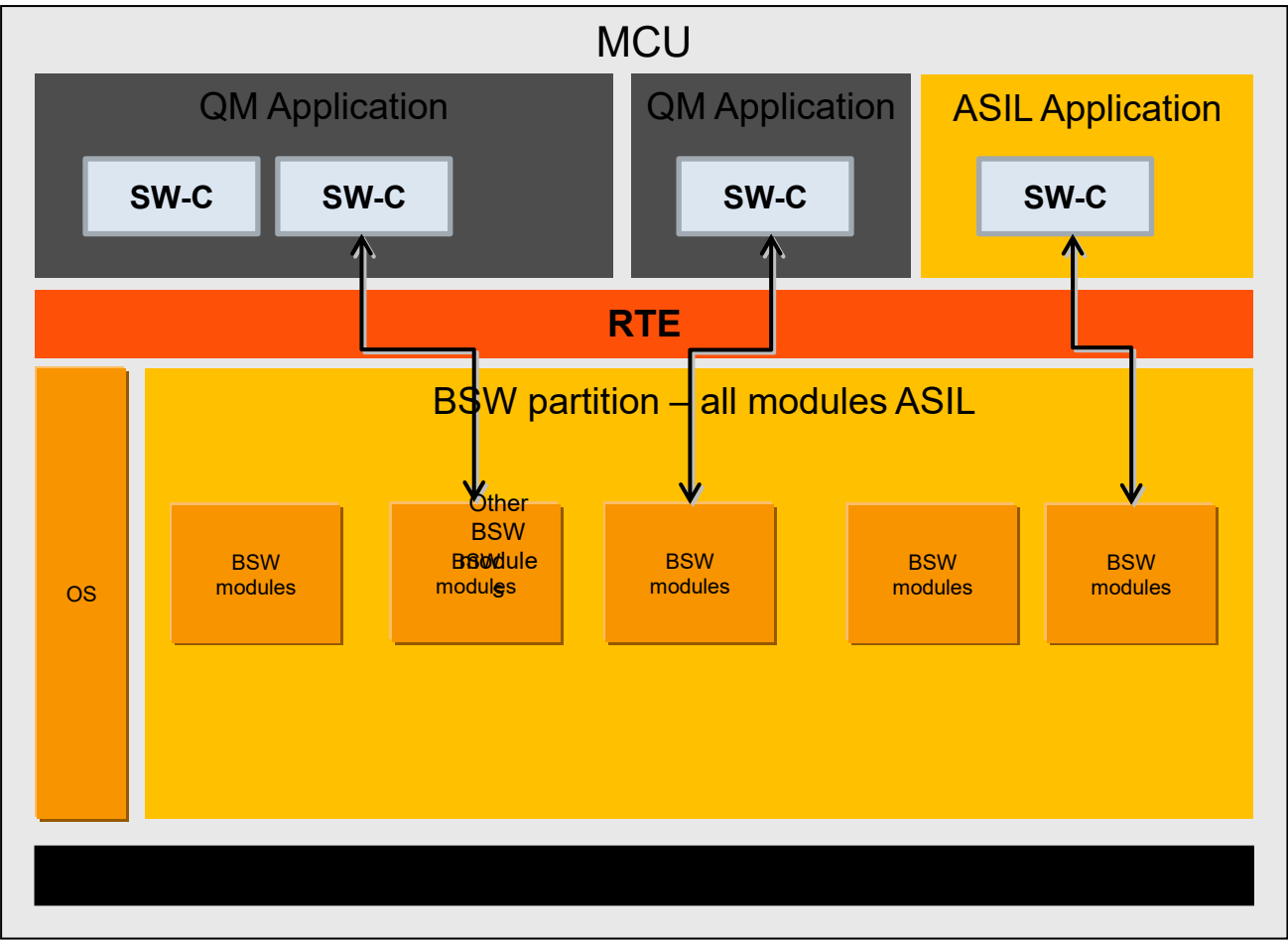
Example for usage of method (1)

Architecture – Content of Software Layers

AUTOSAR安全处理概述

AUTOSAR提供了一种灵活的方法来支持安全相关的ECU。可采用两种方法：

1. 所有BSW模块均按照要求的ASIL等级进行开发。
2. 选定的模块按照ASIL等级进行开发。ASIL与非ASIL模块被分隔到不同的分区中（BSW分布）。



方法（1）的使用示例

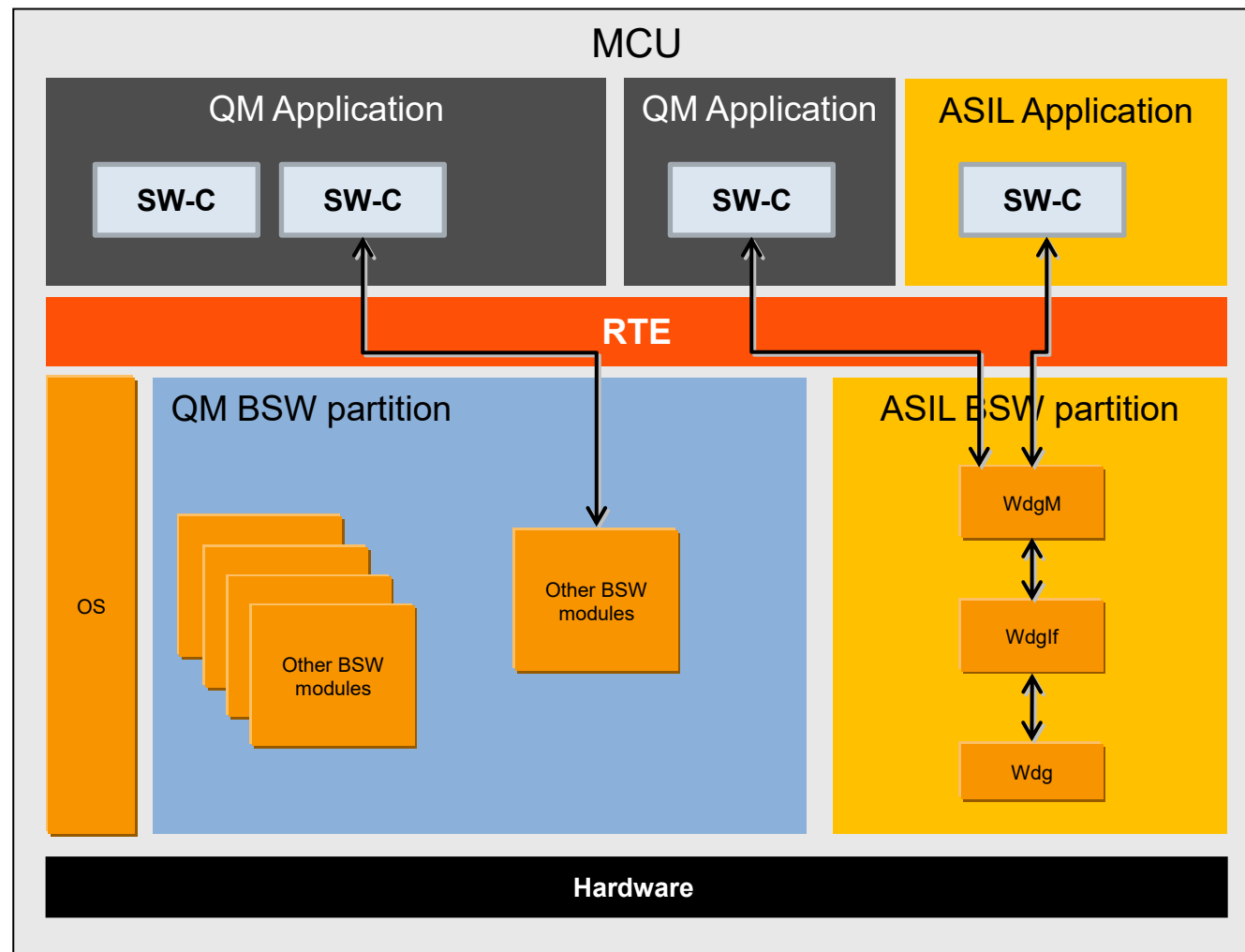
注意：分区基于操作系统应用程序。操作系统应用程序的TRUSTED属性与ASIL/非ASIL无关。

Architecture – Content of Software Layers

AUTOSAR BSW distribution for safety systems

➤ Example of using different BSW partitions

- Watchdog stack is placed in a own partition
- ASIL and non-ASIL SW-Cs can access WdgM via RTE
- Rest of BSW is placed in own partition



Architecture – Content of Software Layers

面向安全系统的AUTOSAR基础软件层分布式架构

使用不同BSW分区的示例

- 看门狗栈被放置在一个独立的分区■中，ASIL和非ASIL的SW-C可以通过RTE■访问WdgM，其余BSW则放置在各自的分区中。

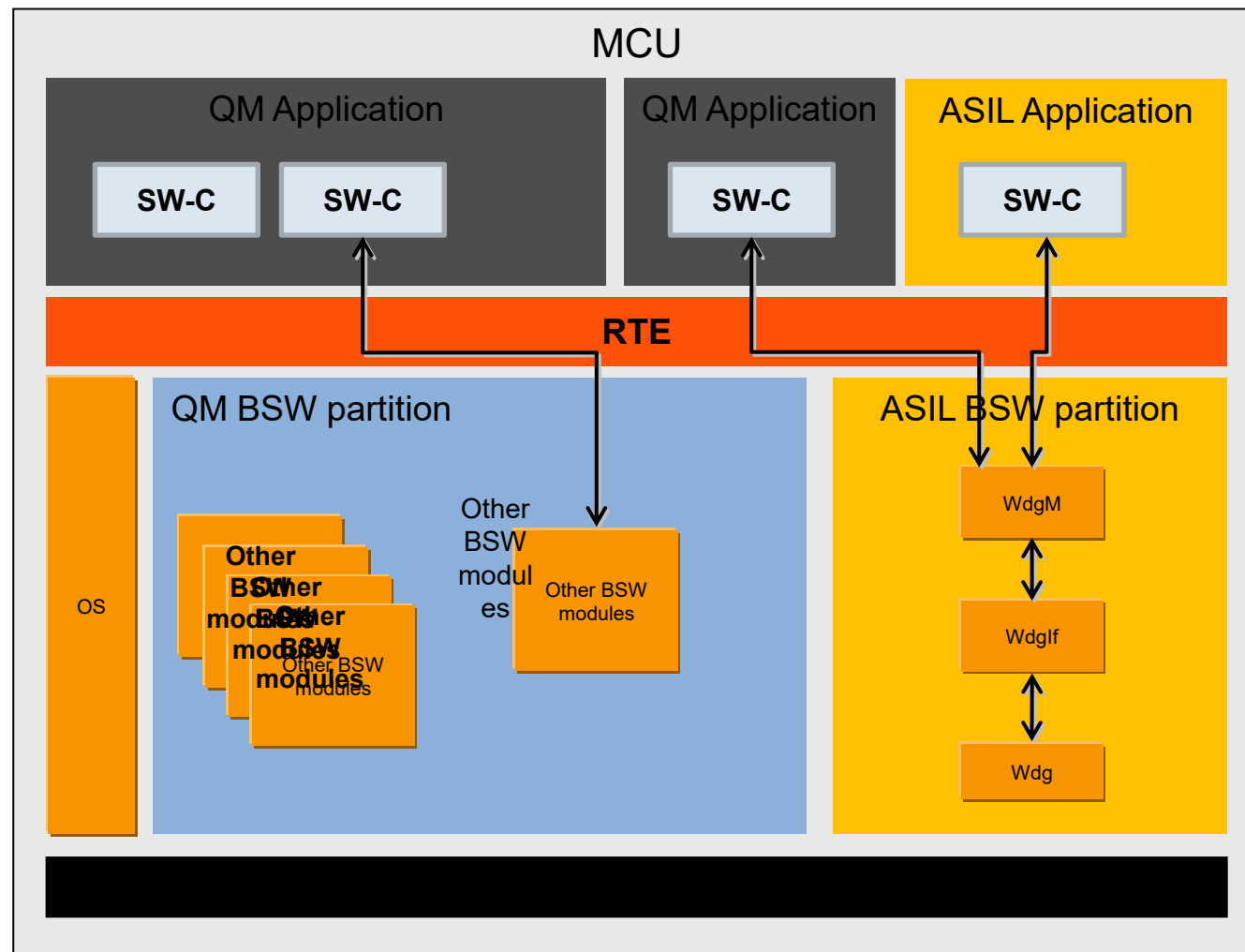


Table of contents

1. Architecture

1. Overview of Software Layers
2. Content of Software Layers
3. Content of Software Layers in Multi-Core Systems
4. Content of Software Layers in Mixed-Critical Systems

5. Overview of Modules

6. Interfaces: General Rules
7. Interfaces: Interaction of Layers
8. Overview of CP Software Clusters

2. Configuration

3. Integration and Runtime Aspects

Table of contents

1. 架构

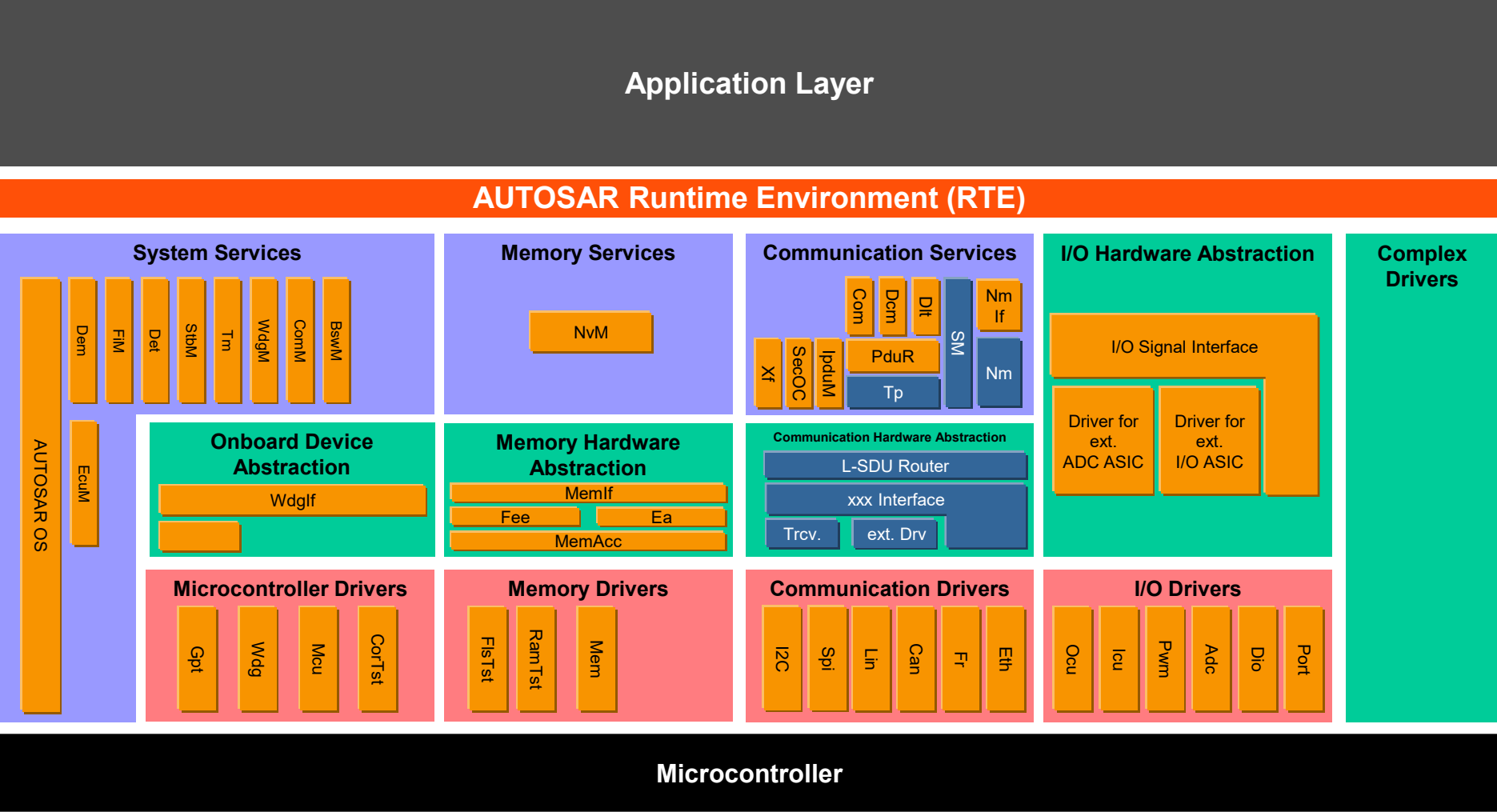
1. 软件层概述 2. 软件层内容 3. 多核系统中的软件层内容 4
. 混合关键系统中的软件层内容 5. 模块概述 6. 接口：通用
规则 7. 接口：层间交互 8. CP软件集群概述

2. 配置 3. 集成与运行时方面

Architecture

Overview of Modules – Implementation Conformance Class 3 - ICC3

This figure shows the mapping of basic software modules to AUTOSAR layers



Not all modules are shown here

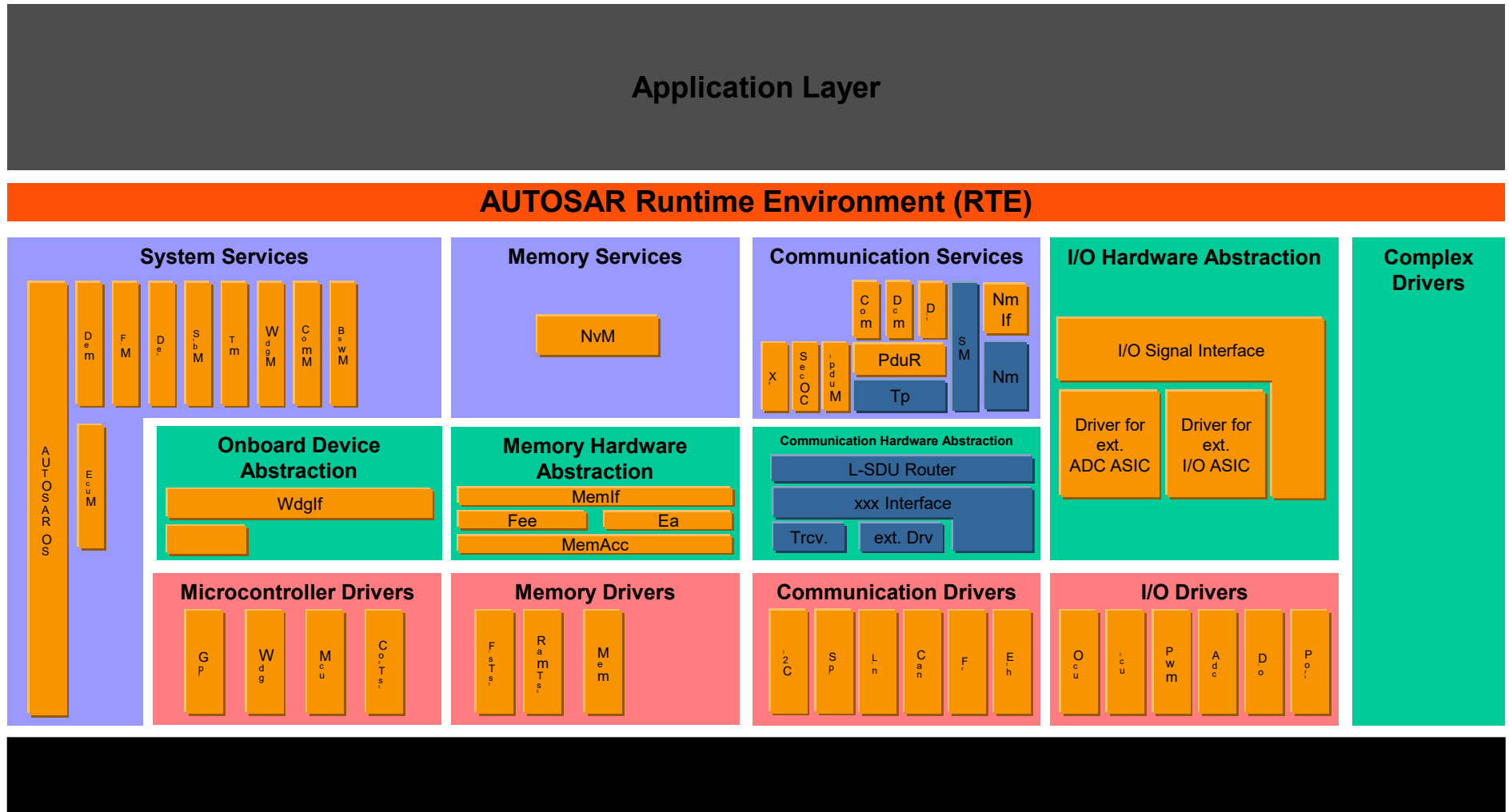
Architecture

概述

f 模块 – 实现一致性类 3

- ICC3

该图展示了基础软件模块到AUTOSAR层的映射关系

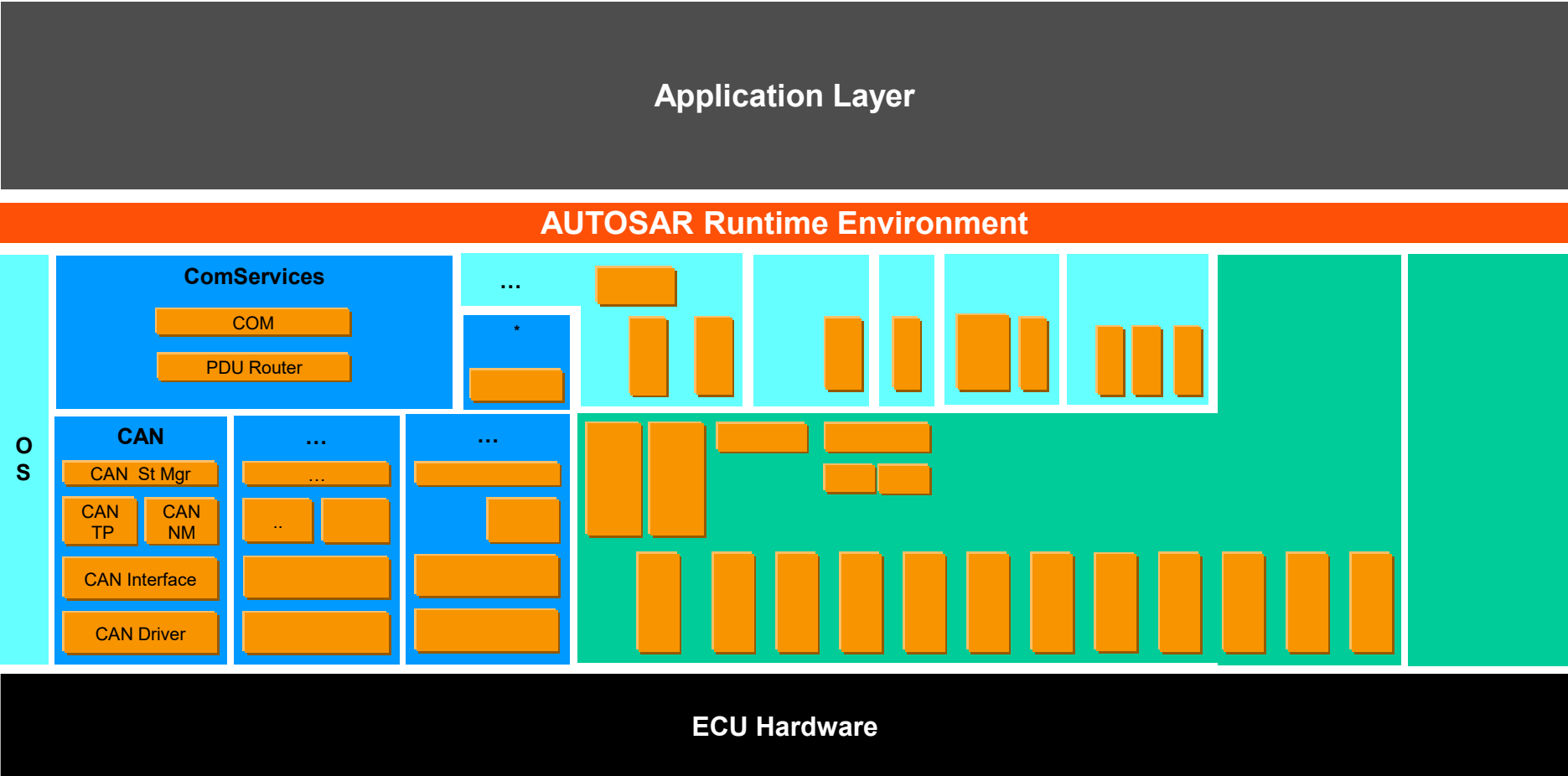


Not all modules are shown here

Architecture

Overview of Modules – Implementation Conformance Classes – ICC2

The clustering shown in this document is the one defined by the project so far. AUTOSAR is currently not restricting the clustering on ICC2 level to dedicated clusters as many different constraint and optimization criteria might lead to different ICC2 clusterings. There might be different AUTOSAR ICC2 clusterings against which compliancy can be stated based on a to be defined approach for ICC2 compliance.



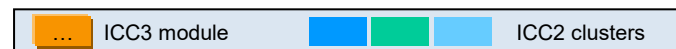
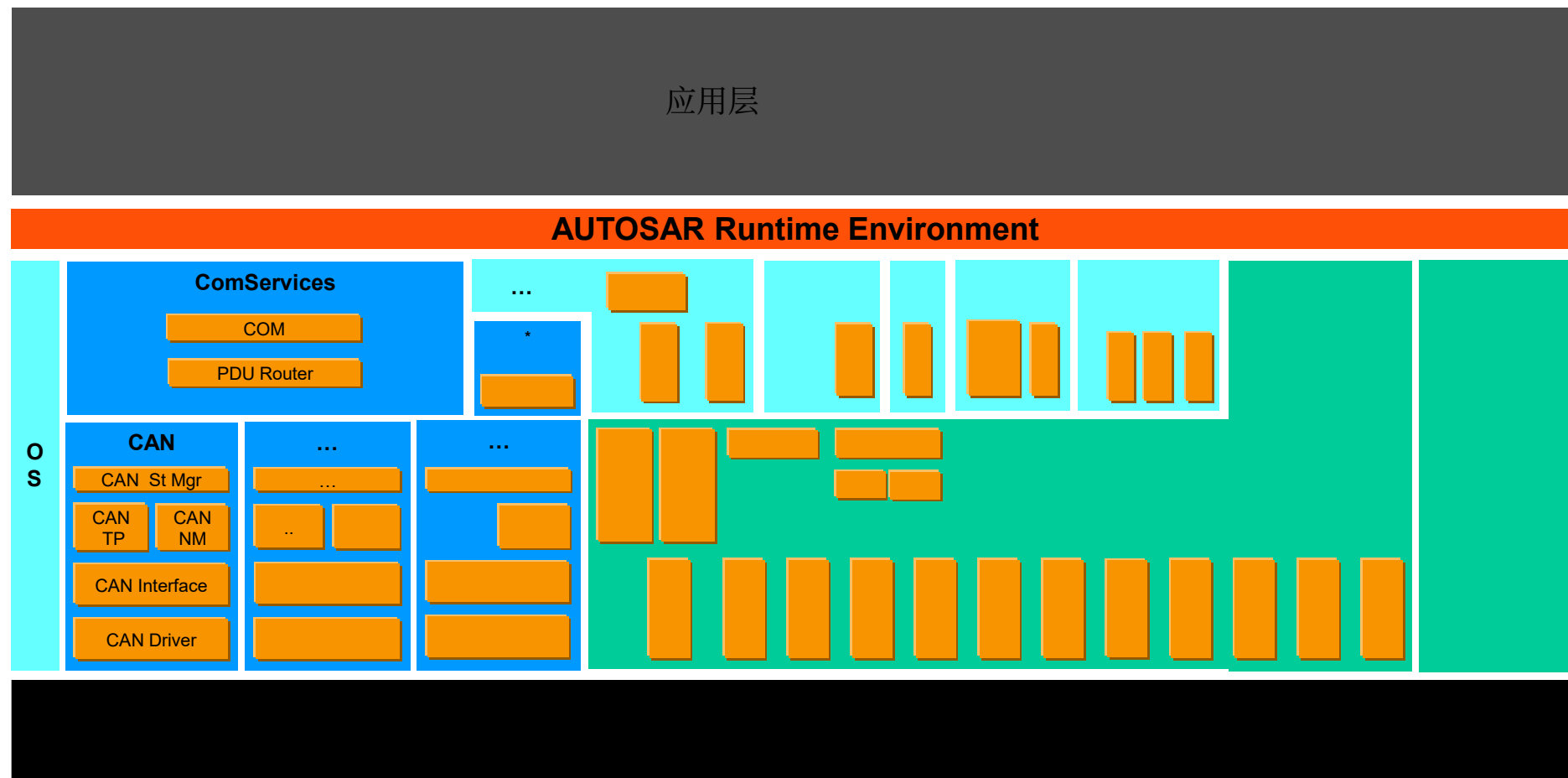
Architecture

概述

模块 – 实现一致性类别 –

ICC2

本文档中展示的聚类是项目迄今为止所定义的聚类。AUTOSAR目前并未在ICC2级别将聚类限制为特定集群，因为许多不同的约束和优化标准可能导致不同的ICC2聚类。可能存在多种不同的AUTOSAR ICC2聚类方案，针对这些方案，可以基于待定义的ICC2合规性方法声明合规性。

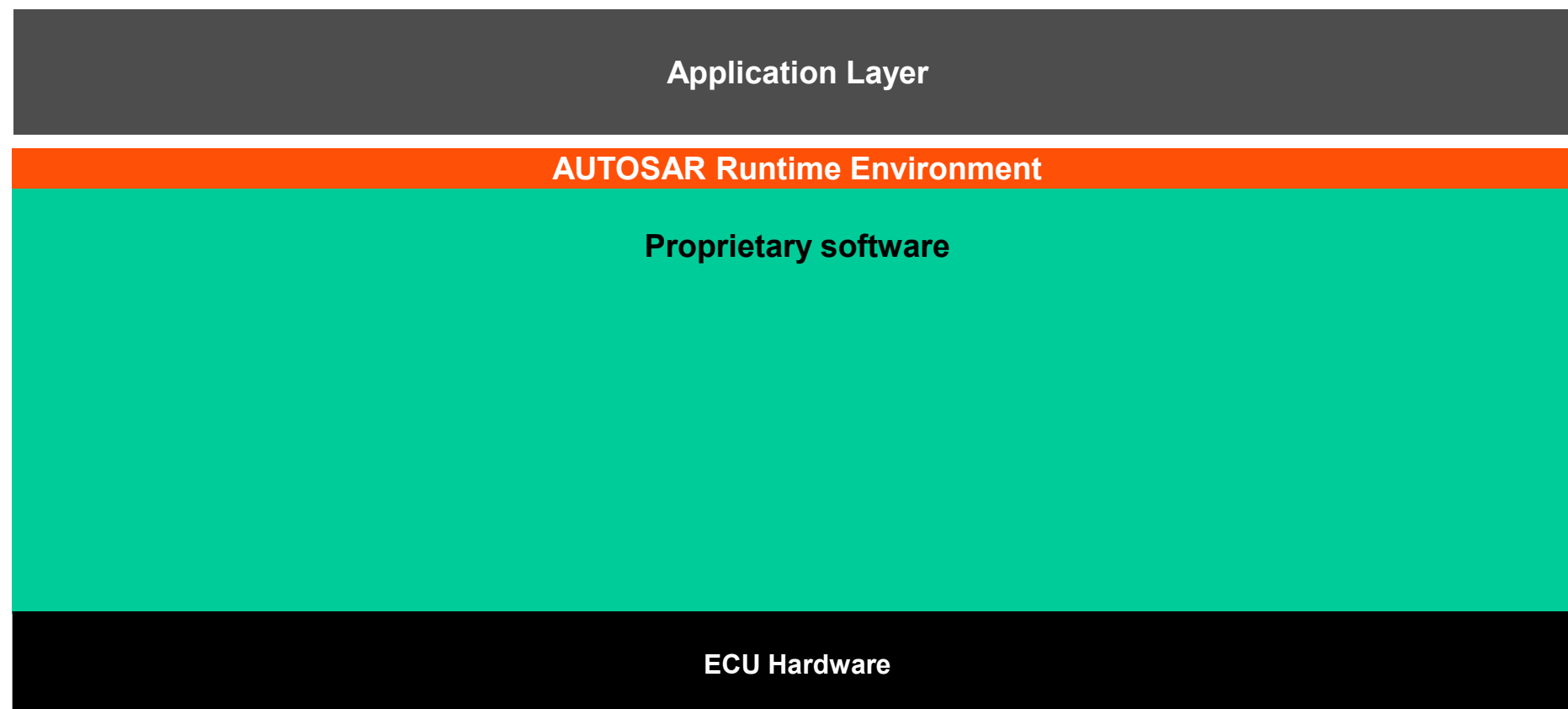


Architecture

Overview of Modules – Implementation Conformance Classes – ICC1

In a basic software which is compliant to ICC1 no modules or clusters are required.

The inner structure of this proprietary basic software is not specified.



Architecture

概述

模块 – 实现一致性类别 –

ICC1

在符合ICC1标准的基础软件中，不需要模块或集群。该专有基础软件的内部结构未作具体规定。

Application Layer

AUTOSAR Runtime Environment

Proprietary software

Architecture

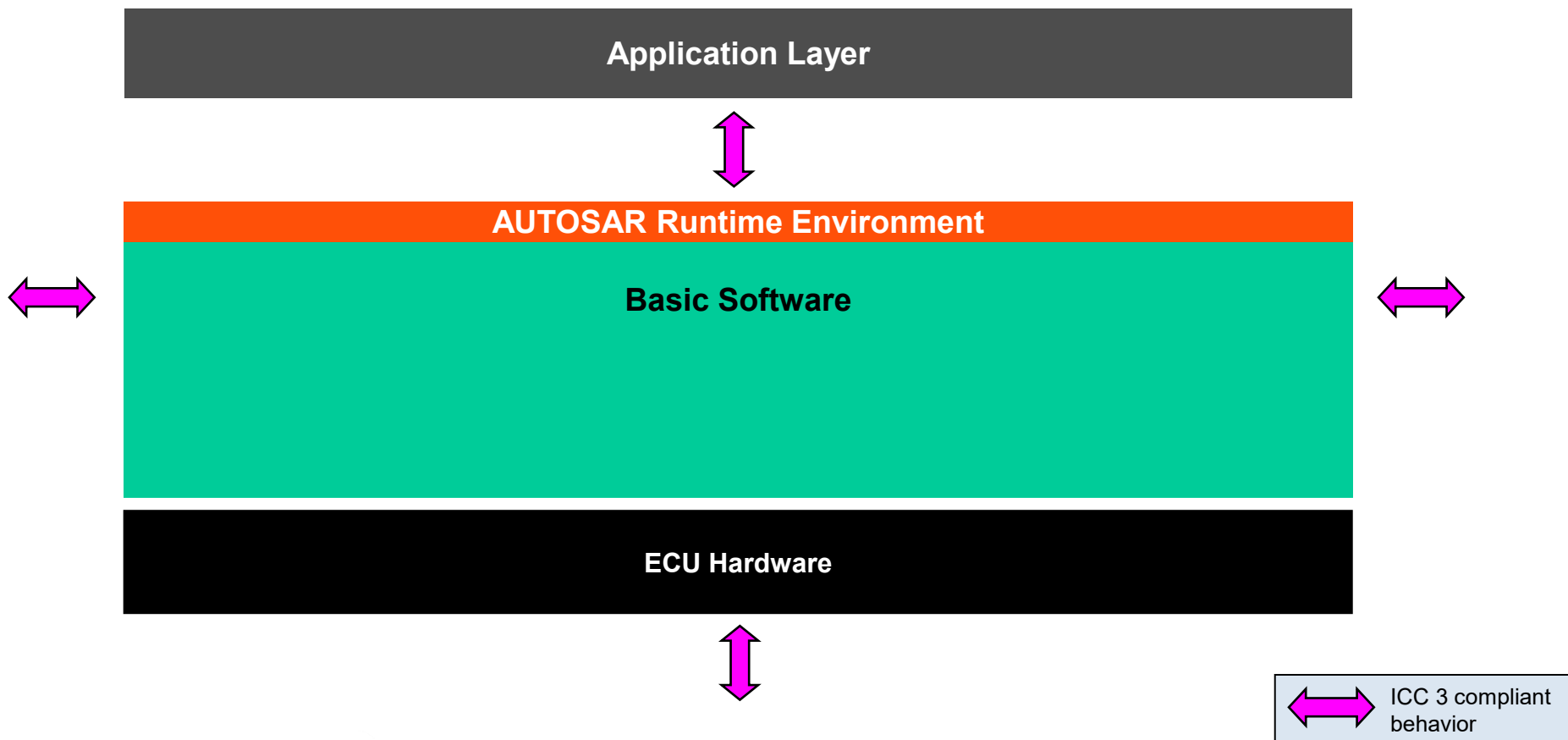
Overview of Modules – Implementation Conformance Classes – behavior to the outside

Basic software (including the RTE) which is AUTOSAR compliant (ICC1-3) has to behave to the outside as specified by the ICC3 module specification.

For example the behavior towards:

- buses,
- boot loaders and
- Applications

Additionally, the ICC1/2 configuration shall be compatible regarding the system description as in ICC3.



Architecture

模块概述 – 实现一致性类别 – 对外行为

基础软件（包括RTE）若符合AUTOSAR标准（ICC1-3），其外部行为必须遵循ICC3模块规范的规定。

例如，对于以下方面的行为：总线，引导加载程序和应用程序。此外，ICC1/2配置在系统描述方面应与ICC3兼容。符合ICC3规范的行为

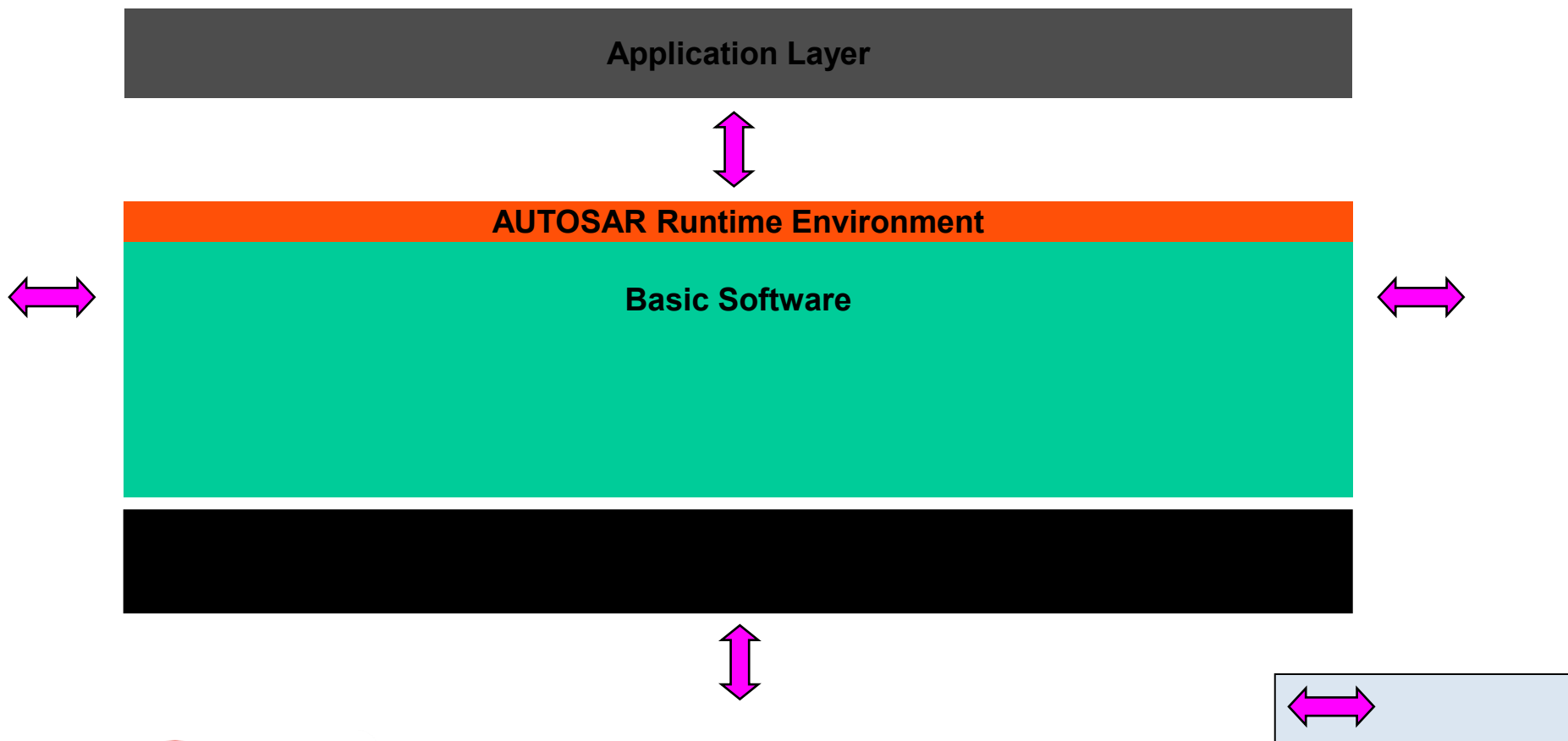


Table of contents

1. **Architecture**

1. Overview of Software Layers
2. Content of Software Layers
3. Content of Software Layers in Multi-Core Systems
4. Content of Software Layers in Mixed-Critical Systems
5. Overview of Modules
6. **Interfaces: General Rules**
7. Interfaces: Interaction of Layers
8. Overview of CP Software Clusters

2. Configuration

3. Integration and Runtime Aspects

Table of contents

1. 架构

1. 软件层概述 2. 软件层内容 3. 多核系统中的软件层内容 4
. 混合关键系统中的软件层内容 5. 模块概述 6. 接口：通用
规则 7. 接口：层间交互 8. CP软件集群概述

2. 配置 3. 集成与运行时方面

Interfaces

Type of Interfaces in AUTOSAR

AUTOSAR Interface	An "AUTOSAR Interface" defines the information exchanged between software components and/or BSW modules. This description is independent of a specific programming language, ECU or network technology. AUTOSAR Interfaces are used in defining the ports of software-components and/or BSW modules. Through these ports software-components and/or BSW modules can communicate with each other (send or receive information or invoke services). AUTOSAR makes it possible to implement this communication between Software-Components and/or BSW modules either locally or via a network.
Standardized AUTOSAR Interface	A "Standardized AUTOSAR Interface" is an "AUTOSAR Interface" whose syntax and semantics are standardized in AUTOSAR. The "Standardized AUTOSAR Interfaces" are typically used to define AUTOSAR Services, which are standardized services provided by the AUTOSAR Basic Software to the application Software-Components.
Standardized Interface	A "Standardized Interface" is an API which is standardized within AUTOSAR without using the "AUTOSAR Interface" technique. These "Standardized Interfaces" are typically defined for a specific programming language (like "C"). Because of this, "standardized interfaces" are typically used between software-modules which are always on the same ECU. When software modules communicate through a "standardized interface", it is NOT possible any more to route the communication between the software-modules through a network.

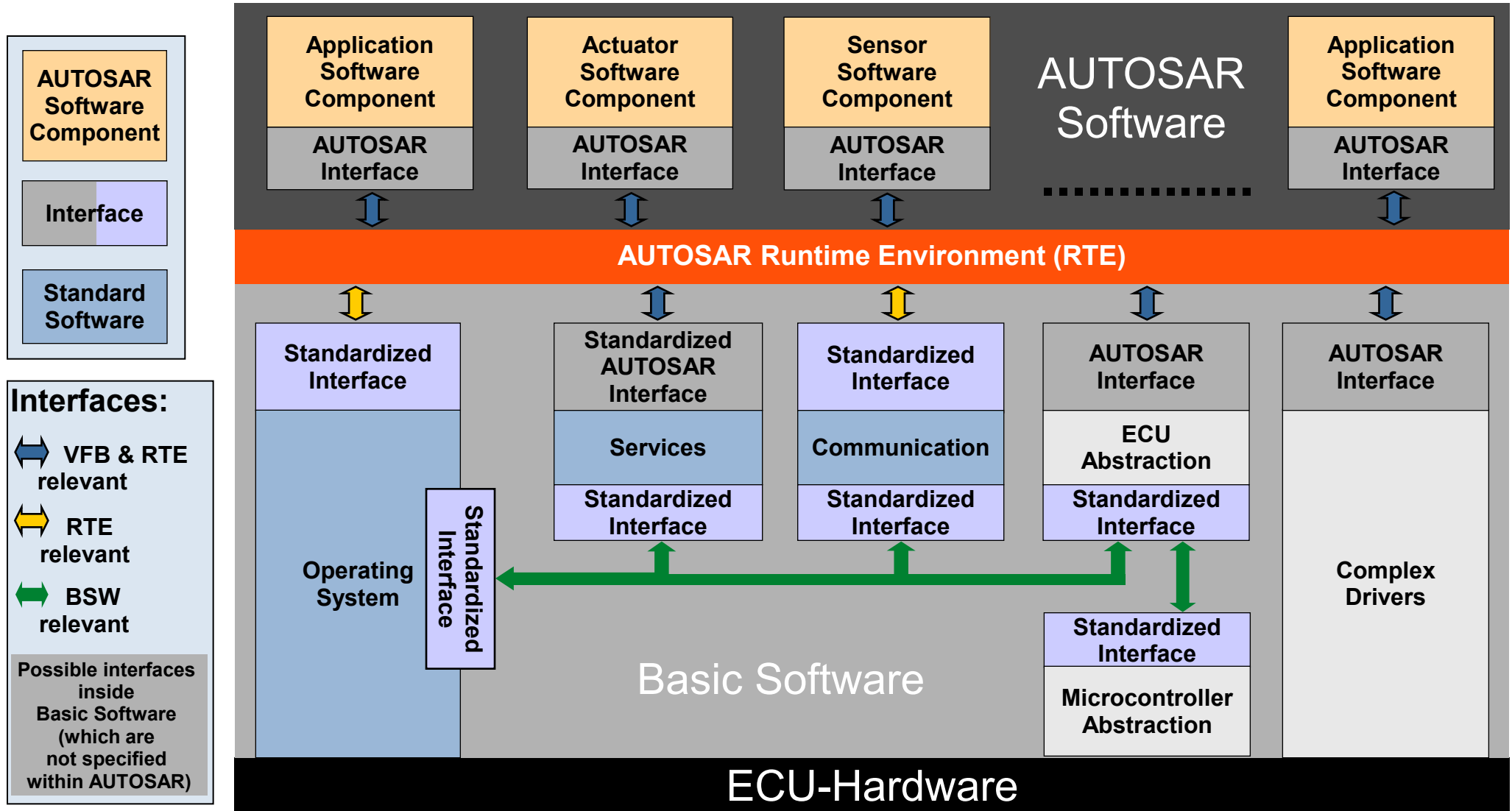
Interfaces

AUTOSAR中的接口类型

AUTOSAR Interface	An "AUTOSAR Interface" defines the information exchanged between software components and/or BSW modules. This description is independent of a specific programming language, ECU or network technology. AUTOSAR Interfaces are used in defining the ports of software-components and/or BSW modules. Through these ports software-components and/or BSW modules can communicate with each other (send or receive information or invoke services). AUTOSAR makes it possible to implement this communication between Software-Components and/or BSW modules either locally or via a network.
Standardized AUTOSAR Interface	A "Standardized AUTOSAR Interface" is an "AUTOSAR Interface" whose syntax and semantics are standardized in AUTOSAR. The "Standardized AUTOSAR Interfaces" are typically used to define AUTOSAR Services, which are standardized services provided by the AUTOSAR Basic Software to the application Software-Components.
Standardized Interface	A "Standardized Interface" is an API which is standardized within AUTOSAR without using the "AUTOSAR Interface" technique. These "Standardized Interfaces" are typically defined for a specific programming language (like "C"). Because of this, "standardized interfaces" are typically used between software-modules which are always on the same ECU. When software modules communicate through a "standardized interface", it is NOT possible any more to route the communication between the software-modules through a network.

Interfaces

Components and interfaces view (simplified)

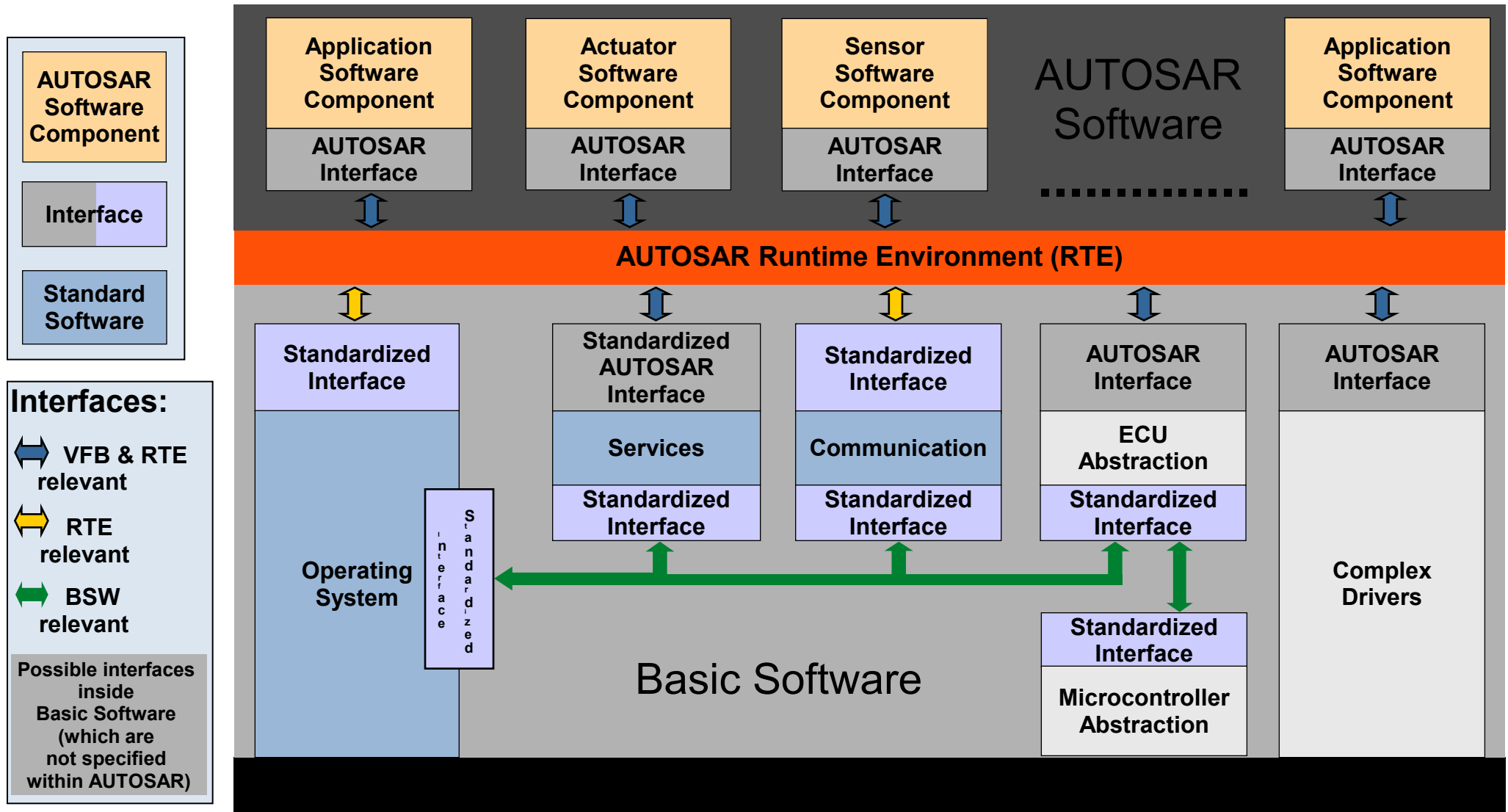


Note: This figure is incomplete with respect to the possible interactions between the layers.

5
u
4
9
d
e
g
a
p

Interfaces

组件与接口视图（简化版）



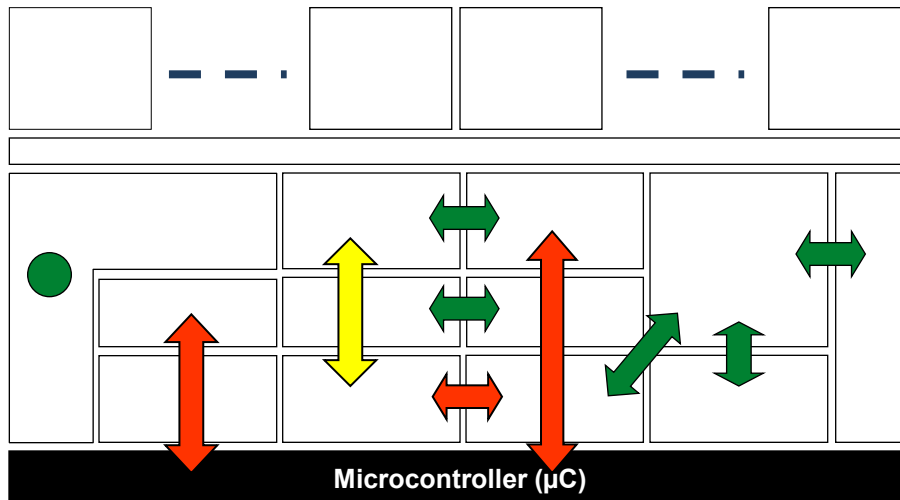
注意：该图在表示各层之间可能的交互方面并不完整。

Interfaces: General Rules

General Interfacing Rules

Horizontal Interfaces

-  Services Layer: horizontal interfaces are allowed
Example: Error Manager saves fault data using the NVRAM manager
-  ECU Abstraction Layer: horizontal interfaces are allowed
-  A complex driver may use selected other BSW modules
-  μ C Abstraction Layer: horizontal interfaces are not allowed. Exception: configurable notifications are allowed due to performance reasons.



Vertical Interfaces

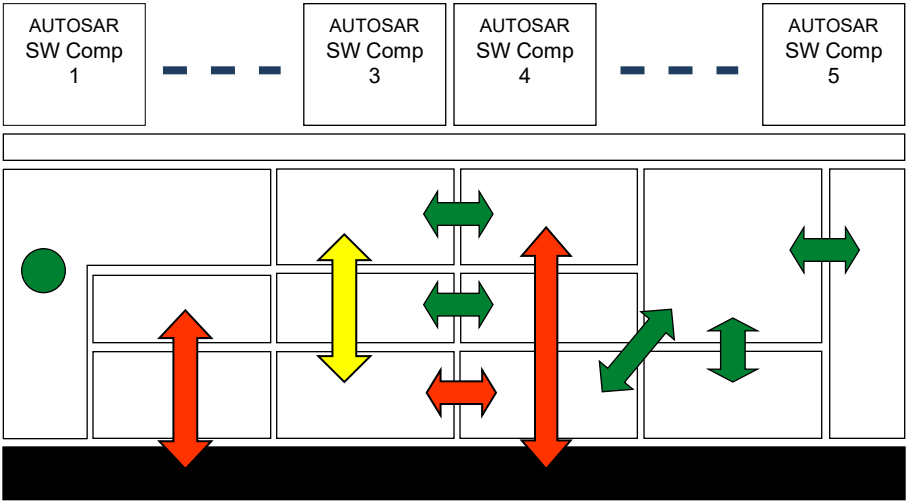
-  One Layer may access all interfaces of the SW layer below
-  Bypassing of one software layer should be avoided
-  Bypassing of two or more software layers is not allowed
-  Bypassing of the μ C Abstraction Layer is not allowed
-  A module may access a lower layer module of another layer group (e.g. SPI for external hardware)
-  All layers may interact with system services.

Interfaces: General Rules

通用接口规则

水平接口

-  服务层：允许水平接口 示例：错误管理器使用NVR
AM管理器保存故障数据
-  ECU抽象层：允许水平接口
-  一个复杂的驱动程序可能会使用选定的其他BS
W模块
-  μ C抽象层：不允许水平接口。例外：出于性能原因
，允许可配置的通知。



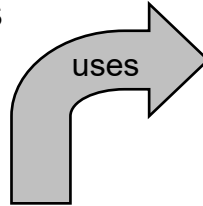
垂直接口

-  一个层可以访问其下方SW层的所有接口。
-  应避免绕过任何软件层
-  不允许绕过两个或更多软件层
-  绕过 μ C抽象层是不允许的
-  一个模块可以访问另一层组的较低层模块（例如用于外部硬件的SPI）
-  所有层都可能与系统服务交互。

Interfaces: General Rules

Layer Interaction Matrix

This normative matrix shows the allowed interactions between AUTOSAR Basic Software layers



- ✓ allowed to use
- ✗ not allowed to use
- △ restricted use (callback only)

The matrix is read **row-wise**:
Example: “I/O Drivers are allowed to use System Services and Hardware, but no other layers”.

(gray background indicates “non-Basic Software” layers)

	System Services / OS	Memory Services	Crypto Services	Communication Services	Off-board Comm. Services	Complex Drivers	I/O Hardware Abstraction	Onboard Device Abstr.	Memory HW Abstraction	Crypto HW Abstraction	Comm. HW Abstraction*	Microcontroller Drivers	Memory Drivers	Crypto Drivers	Communication Drivers*	I/O Drivers	Microcontroller Hardware
SW Components / RTE	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
System Services / OS	✓	✓	✓	✓	✓	△	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Memory Services	✓	✓	✓	✗	✗	△	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗
Crypto Services	✓	✓	✓	✗	✗	△	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗
Communication Services	✓	✓	✓	✓	✓	△	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗
Off-board Comm. Services	✓	✓	✓	✓	✓	△	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗
Complex Drivers	restricted access -> see the following two slides																
I/O Hardware Abstraction	✓	✗	✓	✗	✗	✗	✓	✓	✗	✗	✓	✓	✗	✗	✓	✓	✗
Onboard Device Abstr.	✓	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✓	✗	✗	✓	✓	✗
Memory HW Abstraction	✓	✓	✗	✗	✗	✗	✗	✓	✓	✗	✓	✗	✓	✗	✓	✗	✗
Crypto HW Abstraction	✓	✓	✓	✗	✗	✗	✗	✓	✗	✓	✗	✗	✗	✗	✗	✓	✗
Comm. HW Abstraction*	✓	✗	✗	✓	✓	✗	✗	✓	✗	✗	✓	✗	✗	✗	✓	✓	✗
Microcontroller Drivers	✓	✗	✗	✗	✗	✗	△	△	✗	△	✗	△	✗	✗	✗	△	✓
Memory Drivers	✓	✗	✗	✗	✗	✗	✗	✗	△	✗	✗	✗	✗	✗	✗	✗	✓
Crypto Drivers	✓	✓	✗	✗	✗	✗	✗	✗	✗	△	✗	✗	✗	✗	✗	✗	✓
Communication Drivers*	✓	✗	✗	✗	✗	✗	✗	△	✗	✗	△	✗	✗	✗	✗	✓	✓
I/O Drivers	✓	✗	✗	✗	✗	✗	△	△	✗	△	✗	△	✗	✗	✗	△	✓

*: includes wired and wireless communication



- (gray background indicates “non-Basic Software” layers)

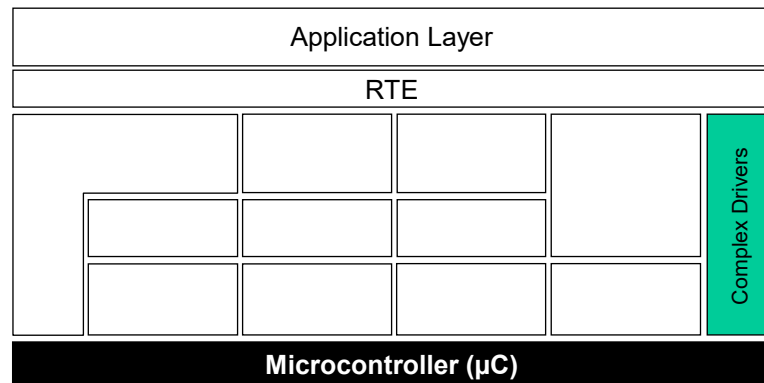
The diagram illustrates a 2D grid with 10 rows and 20 columns. The rows are color-coded: the first 4 rows are purple, the next 4 rows are green, and the last 2 rows are red. The columns are color-coded: the first 5 columns are purple, the next 5 columns are green, the next 5 columns are red, and the last 5 columns are grey. A legend in the top-left corner shows a grey arrow pointing right with the word "uses" inside it, indicating that the grey columns are used for the grey rows.

*: includes wired and wireless communication

Interfaces

Interfacing with Complex Drivers (1)

Complex Drivers may need to interface to other modules in the layered software architecture, or modules in the layered software architecture may need to interface to a Complex Driver. If this is the case, the following rules apply:



1. Interfacing from modules of the layered software architecture to Complex Drivers

This is only allowed if the Complex Driver offers an interface which can be generically configured by the accessing AUTOSAR module.

A typical example is the PDU Router: a Complex Driver may implement the interface module of a new bus system. This is already taken care of within the configuration of the PDU Router.

2. Interfacing from a Complex Driver to modules of the layered software architecture

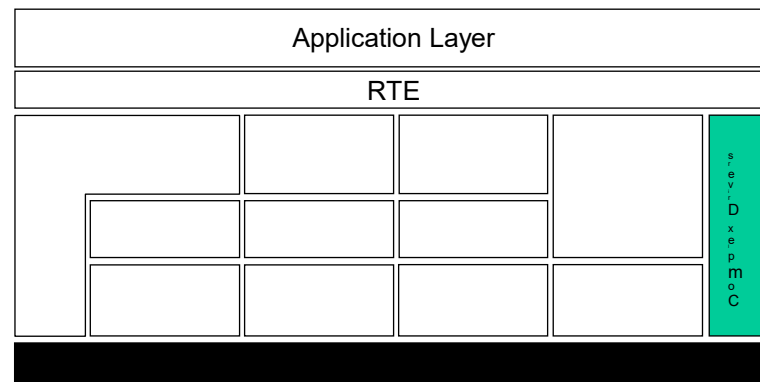
Again, this is only allowed if the respective modules of the layered software architecture offer the interfaces, and are prepared to be accessed by a Complex Driver. Usually this means that

- The respective interfaces are defined to be re-entrant.
- If call back routines are used, the names are configurable
- No upper module exists which does a management of states of the module (parallel access would change states without being noticed by the upper module)

Interfaces

与复杂驱动程序接口（1）

复杂驱动程序可能需要与分层软件架构中的其他模块对接，或者分层软件架构中的模块可能需要与复杂驱动程序对接。若存在此类情况，应遵循以下规则：



1. 接口 从分层软件架构的模块到Com

plex 驱动程序

这仅在复杂驱动程序提供一个可由访问的AUTOSAR模块通用配置的接口时才被允许。一个典型例子是PDU路由器：复杂驱动程序可以实现新总线系统的接口模块。这一点已在PDU路由器的配置中得到处理。

2. 从复杂驱动程序到分层软件架构模块的接口对接

同样，这只有在分层软件架构的相应模块提供了接口，并且准备好被复杂驱动程序访问时才被允许。通常这意味着

各接口被定义为可重入式。 若使用回调例程，其名称可配置。 不存在对模块状态进行管理的上层模块（并行访问可能改变状态而未被上层模块察觉）。

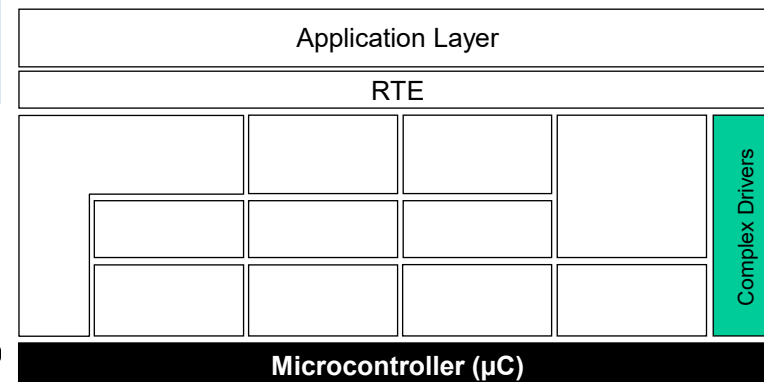
Interfaces

Interfacing with Complex Drivers (2)

In general, it is possible to access the following modules:

- The SPI driver
- The GPT driver
- The I/O drivers with the restriction that reentrancy often only exists for separate groups/channels/etc. Parallel access to the same group/channel/etc. is mostly not allowed. This has to be taken care of during configuration.
- The NVRAM Manager as exclusive access point to the memory stack
- The Watchdog Manager as exclusive access point to the watchdog stack
- The PDU Router as exclusive bus and protocol independent access point to the communication stack
- The bus specific interface modules as exclusive bus specific access point to the communication stack
- The NM Interface Module as exclusive access point to the network management stack
- The Communication Manager (only from upper layer) and the Basic Software Mode Manager as exclusive access points to state management
- Det, Dem and Dlt
- The OS as long as the used OS objects are not used by a module of the layered software architecture

Still, for each module it is necessary to check if the respective function is marked as being re-entrant. For example, 'init' functions are usually not re-entrant and should only be called by the ECU State Manager.



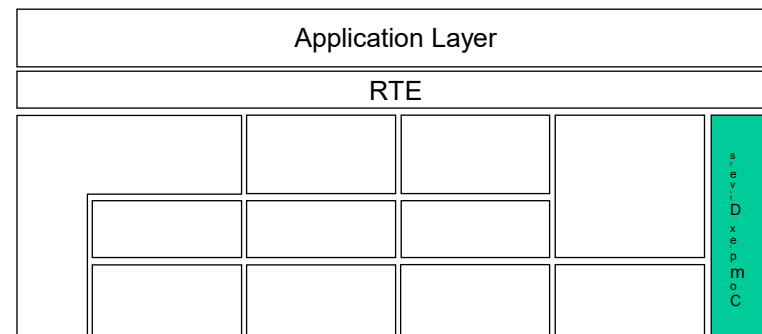
Interfaces

与复杂驱动程序接口 (2)

通常可以访问以下模块：

SPI驱动程序 GPT驱动程序 I/O驱动程序，但需注意可重入性通常仅存在于{v*}

分离组/通道等。通常不允许并行访问同一组/通道等。这需要在配置过程中加以注意。NVRAM管理器作为内存栈的独占访问点 看门狗管理器作为看门狗栈的独占访问点 PDU路由器作为通信栈中独立于总线与协议的独占访问点 总线专用接口模块作为通信栈中总线专用的独占访问点 NM接口模块作为网络管理栈的独占访问点 通信管理器（仅来自上层）与基础软件模式管理器作为状态管理的独占访问点 Det、Dem和Dlt 操作系统（前提是所使用的OS对象未被分层软件架构中的模块占用） 尽管如此，每个模块仍需检查其相应功能是否标记为可重入。例如，“初始化”函数通常不可重入，应仅由ECU状态管理器调用。

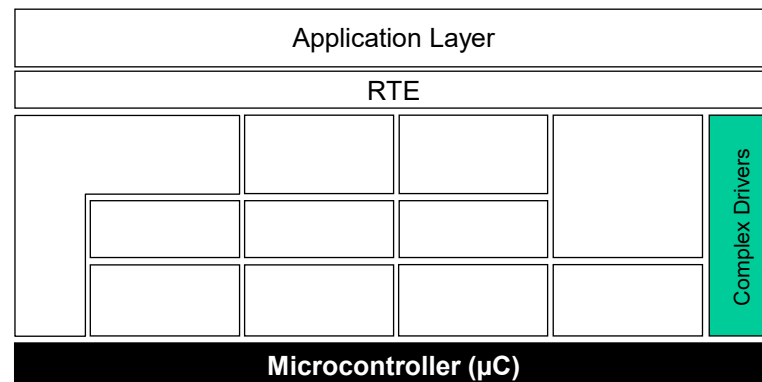


Interfaces

Interfacing with Complex Drivers (3)

In case of multi-core architectures, there are additional rules:

- The BSW can be distributed across several cores. The core responsible for executing a call to a BSW service is determined by the task mapping of its BswOperationInvokedEvent.
- Crossing partition and core boundaries is permitted for module internal communication only, using a master/satellite implementation.
- Consequently, if the CDD needs to access standardized interfaces of the BSW, it needs to reside on the same core.
- In case a CDD resides on a different core, it can use the normal port mechanism to access AUTOSAR interfaces and standardized AUTOSAR interfaces. This invokes the RTE, which uses the IOC mechanism of the operating system to transfer requests to the other core.
- However, if the CDD needs to access standardized interfaces of the BSW and does not reside on the same core,
 - either a satellite providing the standardized interface can run on the core where the CDD resides and forward the call to the other core
 - or a stub part of the CDD needs to be implemented on the other core, and communication needs to be organized CDD-local using the IOC mechanism of the operating system similar to what the RTE does.
- Additionally, in the latter case the initialization part of the CDD also needs to reside in the stub part on the different core.



Interfaces

与复杂驱动程序接口 (3)

在多核架构的情况下，存在额外的规则：

BSW可以分布在多个核心上。负责执行对BSW服务调用的核心由其BswOperationInvokedEvent的任务映射决定。

仅允许模块内部通信跨越分区和核心边界，采用主/从实现方式。

因此，如果CDD需要访问BSW的标准化接口，它必须驻留在同一核心上。

如果CDD位于不同的核心上，它可以使用正常的端口机制来访问AUTOSAR接口和标准化的AUTOSAR接口。这会调用RTE，RTE利用操作系统的IOC机制将请求传输到另一个核心。

然而，如果CDD需要访问BSW的标准化接口且不驻留在同一核心上，

- 要么一个提供标准化接口的卫星可以在CDD所在的核心上运行，并将调用转发到另一个核心■；要么需要在另一个核心上实现CDD的存根部分，并且需要使用操作系统的IOC机制（类似于RTE所做的那样）来组织CDD本地的通信。

此外，在后一种情况下，CDD的初始化部分也需要驻留在不同核心的存根部分。

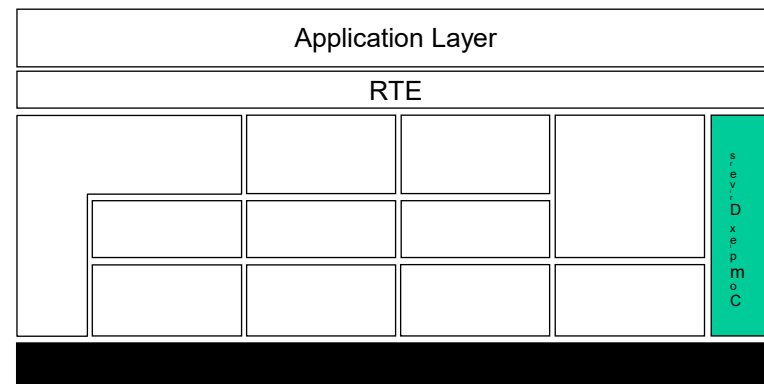


Table of contents

1. Architecture

1. Overview of Software Layers
2. Content of Software Layers
3. Content of Software Layers in Multi-Core Systems
4. Content of Software Layers in Mixed-Critical Systems
5. Overview of Modules
6. Interfaces: General Rules
7. Interfaces: Interaction of Layers
8. Overview of CP Software Clusters

2. Configuration

3. Integration and Runtime Aspects

Table of contents

1. 架构

1. 软件层概述 2. 软件层内容 3. 多核系统中的软件层内容 4
. 混合关键系统中的软件层内容 5. 模块概述 6. 接口：通用
规则 7. 接口：层间交互 8. CP软件集群概述

2. 配置 3. 集成与运行时方面

Interfaces: Interaction of Layers – Example “Memory”

Introduction

The following pages explain using the example „memory“:

- What are the features / difference of the available memory service modules?
- How do the software layers interact?
- How do the software interfaces look like?
- What is inside the ECU Abstraction Layer?
- How can abstraction layers be implemented efficiently?

Interfaces: Interaction of Layers – Example “Memory”

引言

以下页面以“内存”为例进行说明：

可用的内存服务模块有哪些特性/区别？

软件各层之间如何交互？

软件界面看起来是怎样的？

ECU抽象层内部包含什么？

抽象层如何高效实现？

Background: Comparison between memory service modules and memory types

- The different service modules (memory managers) abstract from the used non-volatile (NV) memory, but the properties of the hardware impact their design and how access is realized.
- There are constraints on the use of the different listed modules depending on the properties of the used NV hardware.
- The following table lists the properties of the modules and related NV memory.

Module	Use cases, features	Supported NV memory properties	Example hardware
NvM	- Storage of module data (e.g. Error information, special configuration info, status information, diagnostic data, ...) - Supports many reader/writer (BSW and SW-C) in parallel. - Mostly read during start-up and written in shutdown, but intermediate reads/writes during normal operation are also supported - Typical data size per user is bytes to some KiB	- Direct (memory mapped) and indirect (e.g. via SPI) NV access - Serialized access (read-while-write-in-same-HW-segment may not work → NvM always buffer the data)	- Internal data flash (via Flash EEPROM emulation) - External EEPROM / data flash
BndM	- Storage of car specific data (Very rare) Writes via diagnostics, only in „controlled environment“ (e.g. repair shop) - Supports many readers (SW-C) in parallel - Users have direct access via pointer - Typical size many KiB	- Direct access of NV data (via pointer) is required - Parallel read of NV data is required	- Internal data flash - Internal code flash
FOTA (manager)	- Storage of model specific car data/code - Very few users, typically only one - Typical size in MiB - Write new data in the background e.g. over several driving cycles (interruptible and preemptable update procedure)	Read-While-Write (e.g. via memory abstraction/partitioning)	Internal and external code flash

Background: Comparison between memory service modules and memory types

不同的服务模块（内存管理器）从所使用的非易失性（NV）内存中抽象出来，但硬件的特性会影响它们的设计以及访问的实现方式。

不同列出的模块的使用受到所用NV硬件属性的限制。

下表列出了模块及相关NV存储器的属性。

Module	Use cases, features	Supported NV memory properties	Example hardware
NvM	- Storage of module data (e.g. Error information, special configuration info, status information, diagnostic data, ...) - Supports many reader/writer (BSW and SW-C) in parallel. - Mostly read during start-up and written in shutdown, but intermediate reads/writes during normal operation are also supported - Typical data size per user is bytes to some KiB	- Direct (memory mapped) and indirect (e.g. via SPI) NV access - Serialized access (read-while-write-in-same-HW-segment may not work → NvM always buffer the data)	- Internal data flash (via Flash EEPROM emulation) - External EEPROM / data flash
BndM	- Storage of car specific data (Very rare) Writes via diagnostics, only in „controlled environment“ (e.g. repair shop) - Supports many readers (SW-C) in parallel - Users have direct access via pointer - Typical size many KiB	- Direct access of NV data (via pointer) is required - Parallel read of NV data is required	- Internal data flash - Internal code flash
FOTA (manager)	- Storage of model specific car data/code - Very few users, typically only one - Typical size in MiB - Write new data in the background e.g. over several driving cycles (interruptible and preemptable update procedure)	Read-While-Write (e.g. via memory abstraction/partitioning)	Internal and external code flash

Interfaces: Interaction of Layers – Example “Memory”

This example shows how the NVRAM Manager and the Watchdog Manager interact with drivers on an assumed hardware configuration:

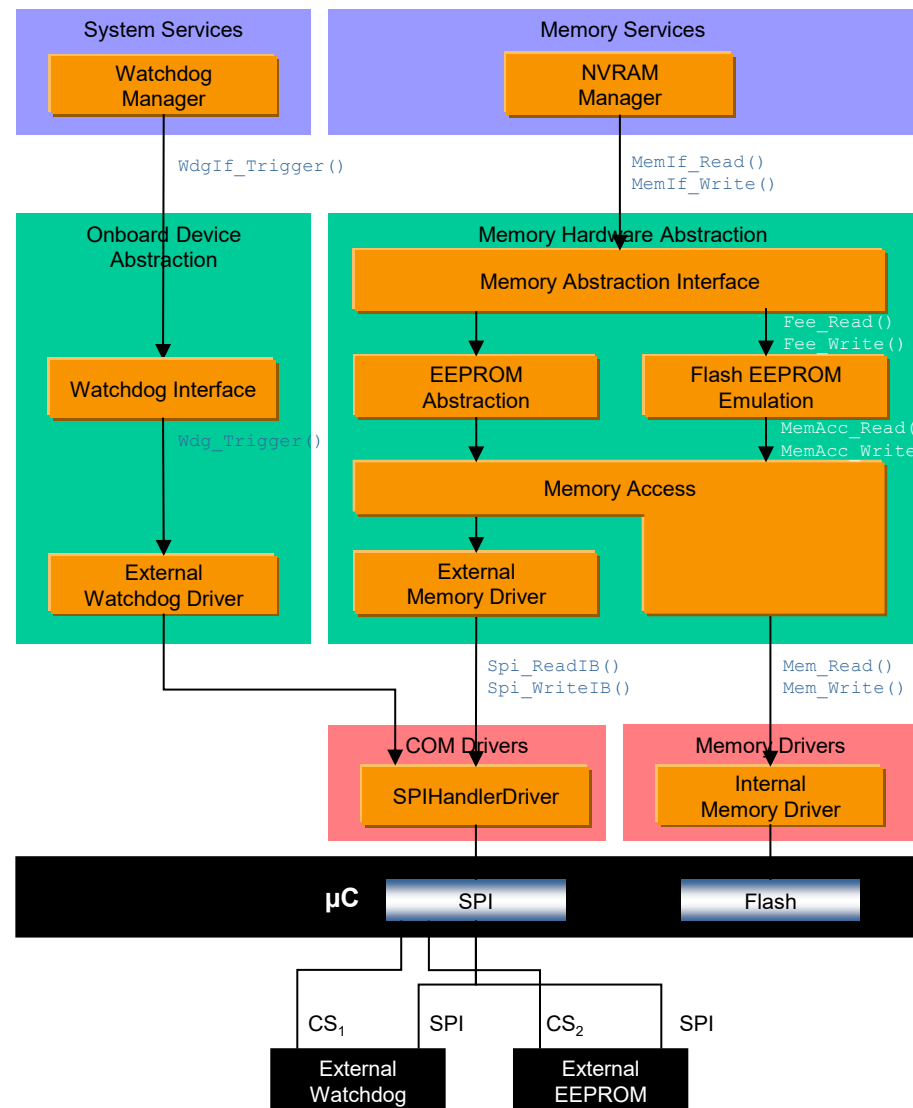
The ECU hardware includes an external EEPROM and an external watchdog connected to the microcontroller via the same SPI.

The SPIHandlerDriver controls the concurrent access to the SPI hardware and has to give the watchdog access a higher priority than the EEPROM access.

The microcontroller includes also an internal flash which is used in parallel to the external EEPROM. The EEPROM Abstraction and the Flash EEPROM Emulation have an API that is semantically identical.

The Memory Abstraction Interface can be realized in the following ways:

- routing during runtime based on device index (int/ext)
- routing during runtime based on the block index (e.g. > 0x01FF = external EEPROM)
- routing during configuration time via ROM tables with function pointers inside the NVRAM Manager (in this case the Memory Abstraction Interface only exists „virtually“)



Interfaces: Interaction of Layers – Example “Memory”

示例与初览

此示例展示了NVRAM管理器与看门狗管理器如何在假定的硬件配置下与驱动程序交互：

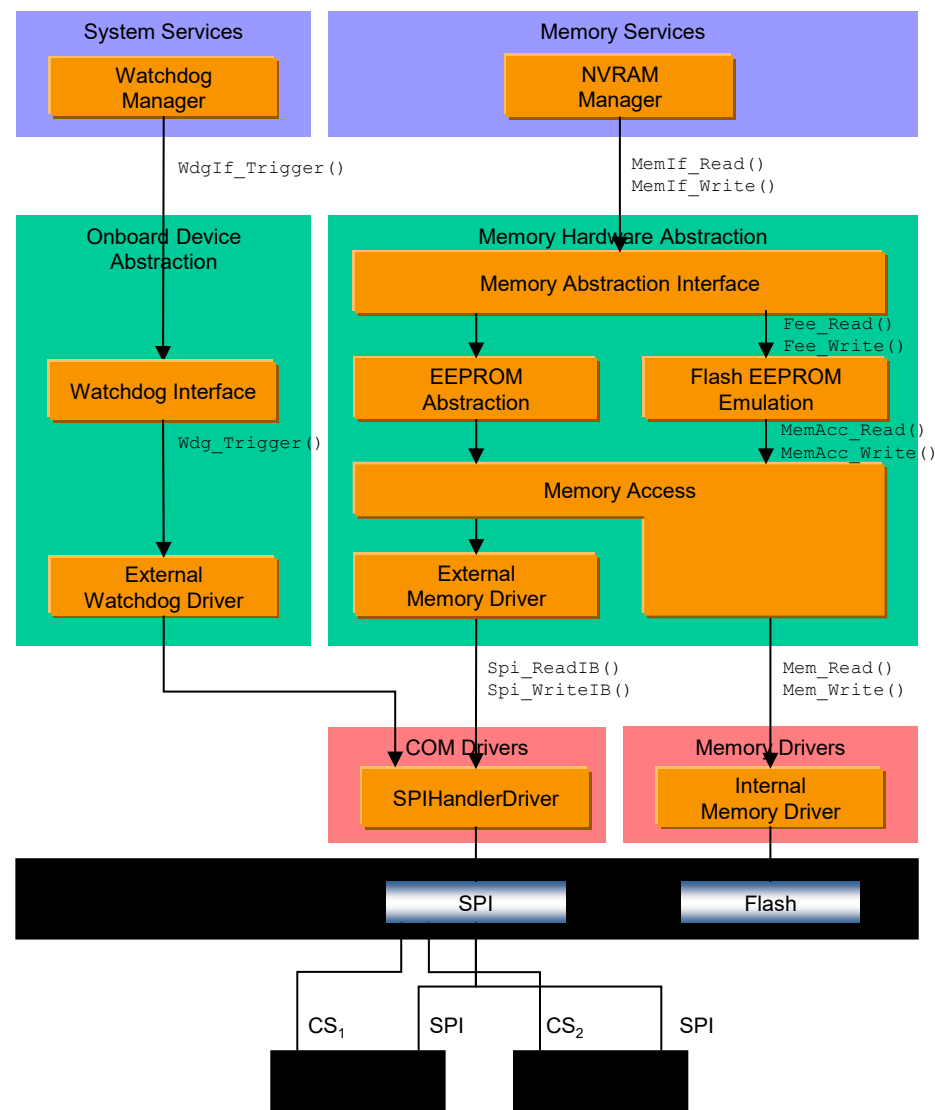
ECU硬件包括一个外部EEPROM和一个外部看门狗，它们通过相同的SPI接口连接到微控制器。

SPIHandlerDriver控制对SPI硬件的并发访问，必须赋予看门狗访问比EEPROM访问更高的优先级。

该微控制器还包含一个内部闪存，它与外部EEPROM并行使用。EEPROM抽象层和闪存EEPROM仿真提供的API在语义上是完全相同的。

内存抽象接口可以通过以下方式实现：

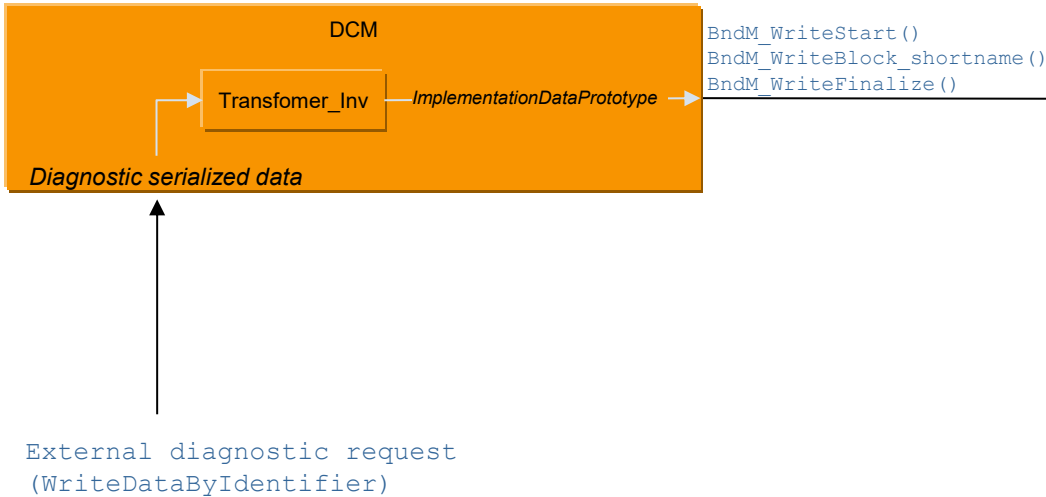
在运行时基于设备索引（内部/外部）进行路由 在运行时基于块索引进行路由（例如 > 0x01FF = 外部EEPROM） 在配置期间通过NVRAM管理器内带函数指针的ROM表进行路由（此时内存抽象接口仅“虚拟”存在）



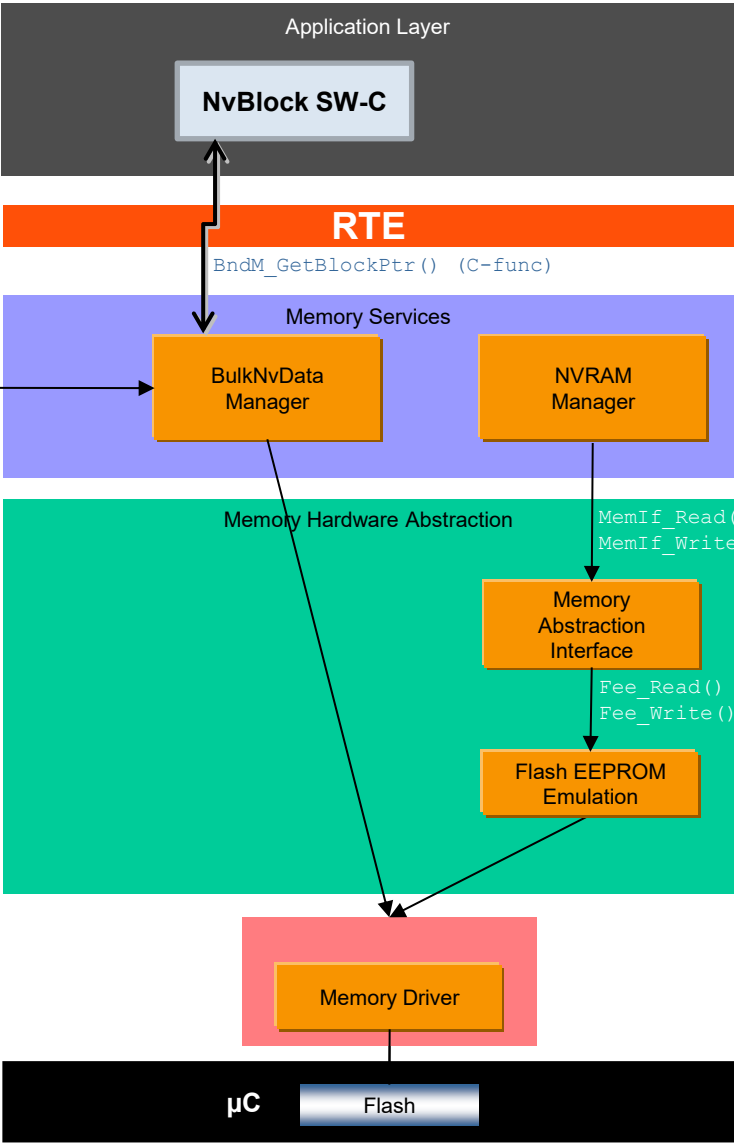
Interfaces: Interaction of Layers – Example “Memory”

Bulk NV Data Manager

Use-case Bulk NV Data Manager (BndM):
Persistent data which is very infrequently written
and additionally huge in size.



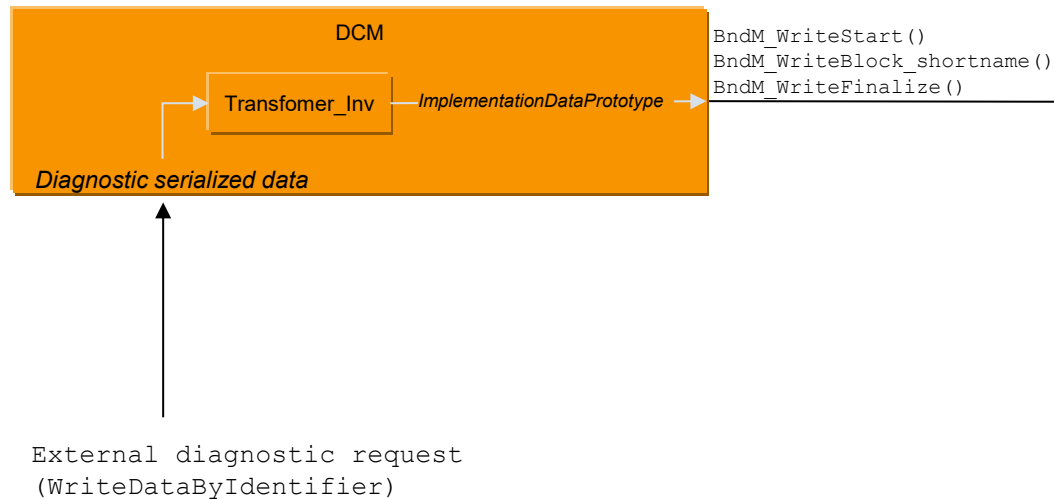
Use-case NVRAM Manager (NvM):
Persistent data which is high frequently updated
or small in its size



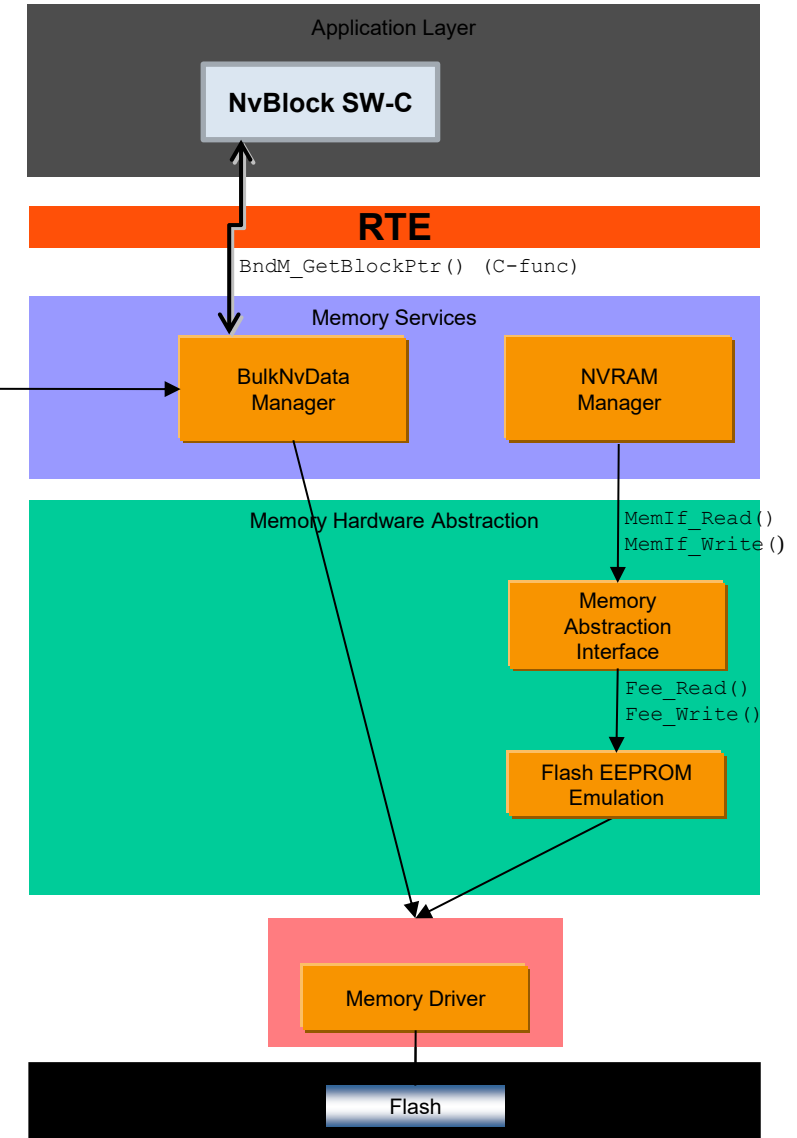
Interfaces: Interaction of Layers – Example “Memory”

批量NV数据管理器

Use-case Bulk NV Data Manager (BndM):
Persistent data which is very infrequently written
and additionally huge in size.



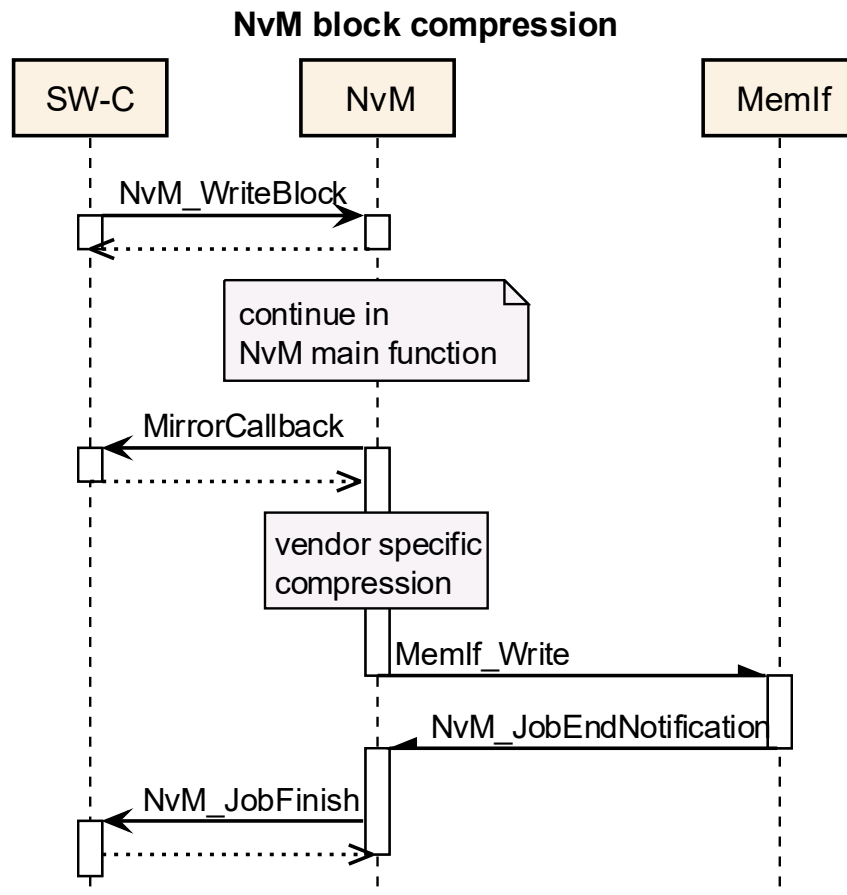
Use-case NVRAM Manager (NvM):
Persistent data which is high frequently updated
or small in its size



Interfaces: Interaction of Layers – Example “Memory”

NvM Block Compression

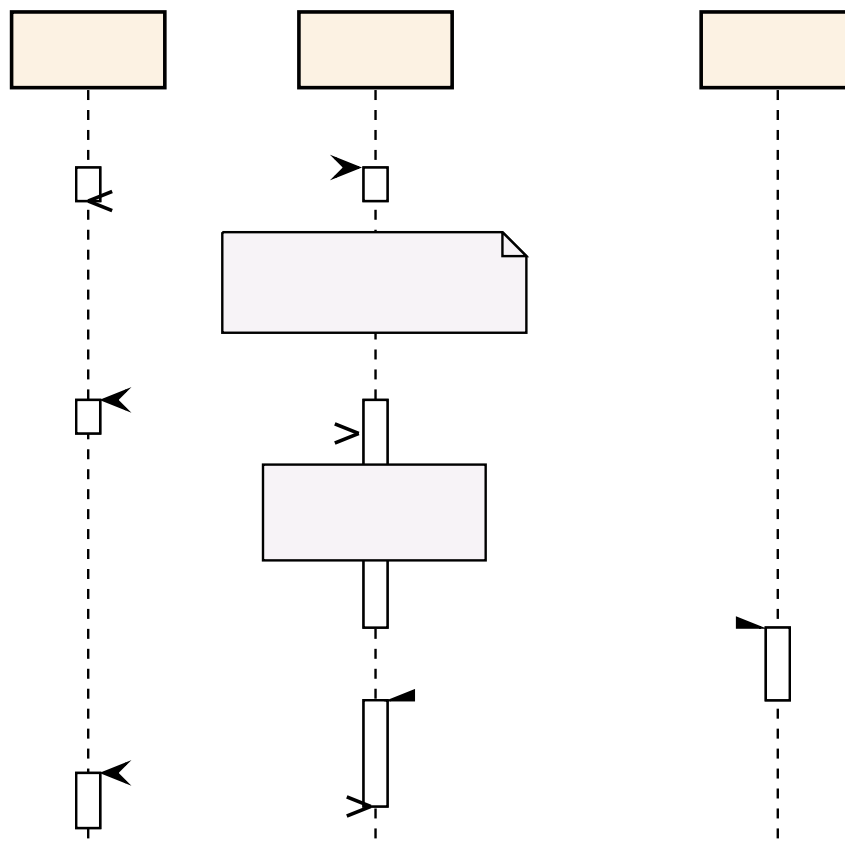
- Use-case: large data blocks frequently written with only small local changes
 - The actual algorithm is vendor-specific (block split, compression, delta,...)



Interfaces: Interaction of Layers – Example “Memory”

NvM块压缩

用例：频繁写入大型数据块，仅涉及局部小范围更改 ■ 实际算法因供应商而异
(数据块分割、压缩、差异编码等)



Interfaces: Interaction of Layers – Example “Memory”

Closer Look at Memory Hardware Abstraction

Architecture Description

The NVRAM Manager accesses drivers via the Memory Abstraction Interface. It addresses different memory devices using a device index.

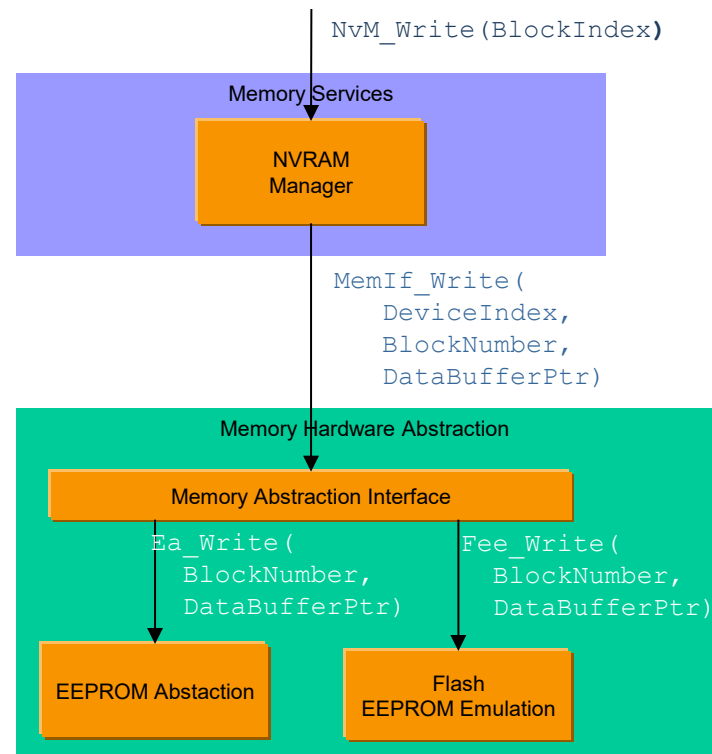
Interface Description

The Memory Abstraction Interface could have the following interface (e.g. for the write function):

```
Std_ReturnType MemIf_Write
(
    uint8          DeviceIndex,
    uint16         BlockNumber,
    uint8          *DataBufferPtr
)
```

The EEPROM Abstraction as well as the Flash EEPROM Emulation could have the following interface (e.g. for the write function):

```
Std_ReturnType Ea_Write
(
    uint16         BlockNumber,
    uint8          *DataBufferPtr
)
```



Interfaces: Interaction of Layers – Example “Memory”

深入探究内存硬件抽象

架构描述

TNVRAM管理器通过内存抽象接口访问驱动程序。
。它使用设备索引来寻址不同的内存设备。

接口描述

T内存抽象接口可以具有以下接口（例如对于写入函数）：

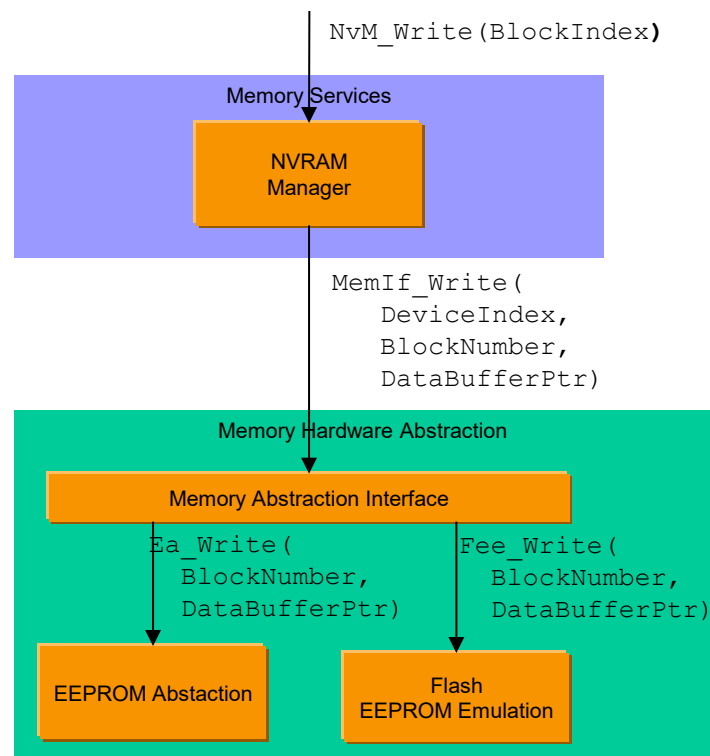
MemIf_Write

(uint8 设备索引, uint16 块编号, uint8 *数据缓冲区指针)

EEPROM抽象以及Flash EEPROM仿真可具备以下接口（例如针对写入函数）：

Std_ReturnType Ea_Write

(uint16 块编号, uint8 *数据缓冲区指针)



Interfaces: Interaction of Layers – Example “Memory”

Implementation of Memory Abstraction Interface

Situation 1: only one NV device type used

This is the usual use case. In this situation, the Memory Abstraction can, in case of source code availability, be implemented as a simple macro which neglects the [DeviceIndex](#) parameter. The following example shows the write function only:

File MemIf.h:

```
#include "Ea.h"          /* for providing access to the EEPROM Abstraction */  
...  
#define MemIf_Write(DeviceIndex, BlockNumber, DataBufferPtr) \  
    Ea_Write(BlockNumber, DataBufferPtr)
```

File MemIf.c:

Does not exist

Result:

No additional code at runtime, the NVRAM Manager virtually accesses the EEPROM Abstraction or the Flash Emulation directly.

Interfaces: Interaction of Layers – Example “Memory”

内存抽象接口的实现

情况1: 仅使用一种NV设备类型

这是常见的使用场景。在这种情况下，若源代码可用，内存抽象可通过忽略DeviceIndex参数的简单宏来实现。

以下示例仅展示写入函数：

文件 MemIf.h:

```
#include "Ea.h"                /* 用于提供对EEPROM抽象层的访问 */  
#define MemIf_Write(DeviceIndex, BlockNumber, DataBufferPtr) \ Ea_Write(BlockNum  
ber, DataBufferPtr)
```

文件 MemIf.c:

不存在

结果：

运行时无需额外代码，NVRAM管理器可直接虚拟访问EEPROM抽象层或闪存模拟层。

Interfaces: Interaction of Layers – Example “Memory”

Implementation of Memory Abstraction Interface

Situation 2: two or more different types of NV devices used

In this case the DeviceIndex has to be used for selecting the correct NV device. The implementation can also be very efficient by using an array of pointers to function. The following example shows the write function only:

File MemIf.h:

```
extern const WriteFctPtrType WriteFctPtr[2];

#define MemIf_Write(DeviceIndex, BlockNumber, DataBufferPtr) \
WriteFctPtr[DeviceIndex](BlockNumber, DataBufferPtr)
```

File MemIf.c:

```
#include "Ea.h"           /* for getting the API function addresses */
#include "Fee.h"           /* for getting the API function addresses */
#include "MemIf.h"         /* for getting the WriteFctPtrType */

const WriteFctPtrType WriteFctPtr[2] = {Ea_Write, Fee_Write};
```

Result:

The same code and runtime is needed as if the function pointer tables would be inside the NVRAM Manager. The Memory Abstraction Interface causes no overhead.

Interfaces: Interaction of Layers – Example “Memory”

内存抽象接口的实现

情况2：使用两种或更多种不同类型的NV设备

在这种情况下，必须使用DeviceIndex来选择正确的NV设备。通过使用函数指针数组，实现也可以非常高效。

以下示例仅展示写入函数：

文件 MemIf.h：

```
extern const WriteFctPtrType WriteFctPtr[2];
```

```
#define MemIf_Write(DeviceIndex, BlockNumber, DataBufferPtr) \WriteFctPtr[DeviceIndex](BlockNumber, DataBufferPtr)
```

文件 MemIf.c：

```
#include "Ea.h"    /* 用于获取API函数地址 */ #include "Fee.h"    /* 用于获取API函数地址 */  
#include "MemIf.h" /* 用于获取WriteFctPtrType类型 */
```

```
const WriteFctPtrType WriteFctPtr[2] {v*} {Ea_Write, Fee_Write};
```

结果：

所需的代码和运行时与函数指针表位于NVRAM管理器内部时相同。内存抽象接口不会产生任何开销。

Interfaces: Interaction of Layers – Example “Memory”

Conclusion

Conclusions:

- Abstraction Layers can be implemented very efficiently
- Abstraction Layers can be scaled
- The Memory Abstraction Interface eases the access of the NVRAM Manager to one or more EEPROM and Flash devices

Interfaces: Interaction of Layers – Example “Memory”

结论

结论:

抽象层可以实现非常高效的实现

抽象层可以进行扩展

内存抽象接口简化了NVRAM管理器对一个或多个EEPROM和闪存设备的访问。

Interfaces: Interaction of Layers – Example “Communication”

PDU Flow through the Layered Architecture

➤ Explanation of terms:

➤ SDU

SDU is the abbreviation of “Service Data Unit”. It is the data passed by an upper layer, with the request to transmit the data. It is as well the data which is extracted after reception by the lower layer and passed to the upper layer.

A SDU is part of a PDU.

➤ PCI

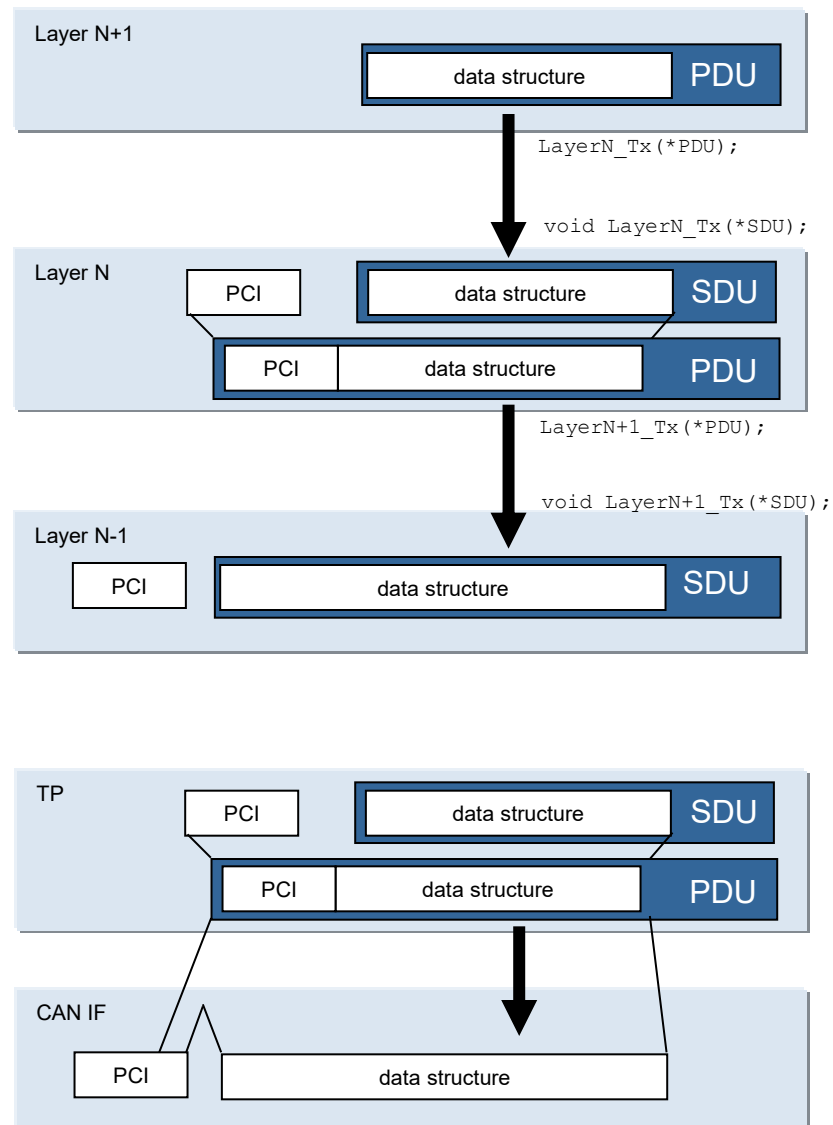
PCI is the abbreviation of “Protocol Control Information”. This Information is needed to pass a SDU from one instance of a specific protocol layer to another instance. E.g. it contains source and target information.

The PCI is added by a protocol layer on the transmission side and is removed again on the receiving side.

➤ PDU

PDU is the abbreviation of “Protocol Data Unit”. The PDU contains SDU and PCI.

On the transmission side the PDU is passed from the upper layer to the lower layer, which interprets this PDU as its SDU.



Interfaces: Interaction of Layers – Example “Communication”

PDU流经分层架构

术语解释：

山东大学

SDU是“服务数据单元”的缩写。它是上层传递的数据，带有传输该数据的请求。同样，它也是下层接收后提取并传递给上层的数据。

SDU是PDU的一部分。

PCI

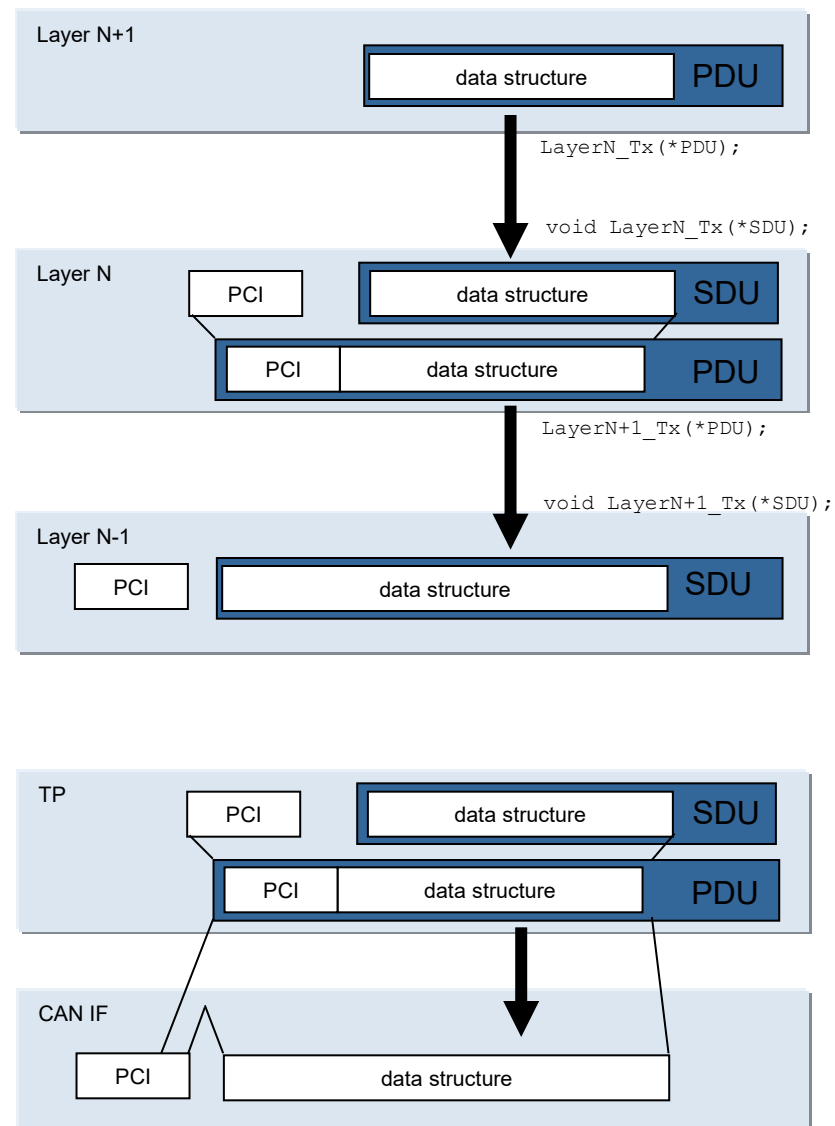
PCI是“协议控制信息”的缩写。此信息用于将SDU从一个特定协议层的实例传递到另一个实例。例如，它包含源和目标信息。

PCI由传输端的协议层添加，并在接收端再次移除。

协议数据单元

PDU是“协议数据单元”的缩写。PDU包含SDU和PCI。

在传输侧，PDU从上层传递到下层，下层将此PDU解释为其SDU。



Interfaces: Interaction of Layers

Example “Communication” (1)

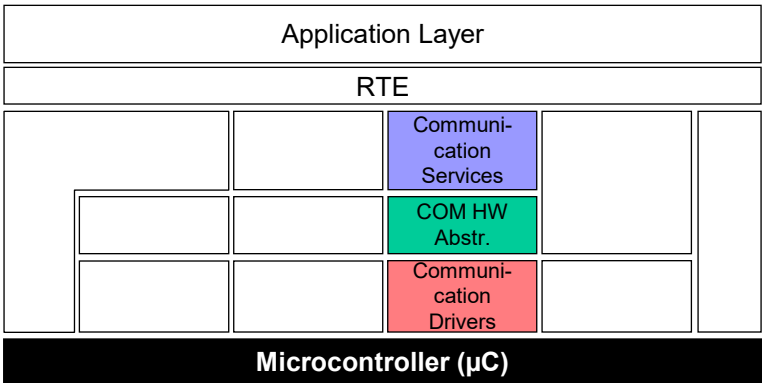
SDU and PDU Naming Conventions

The naming of PDUs and SDUs respects the following rules:

For PDU: *<bus prefix> <layer prefix>* - PDU

For SDU: *<bus prefix> <layer prefix>* - SDU

The *bus prefix* and *layer prefix* are described in the following table:



ISO Layer	Layer Prefix	AUTOSAR Modules	PDU Name	CAN prefix	LIN prefix	FlexRay prefix
Layer 6: Presentation (Interaction)	I	COM, DCM	I-PDU	N/A		
	I	PDU router, PDU multiplexer	I-PDU	N/A		
Layer 3: Network Layer	N	TP Layer	N-PDU	CAN SF CAN FF CAN CF CAN FC	LIN SF LIN FF LIN CF LIN FC	FR SF FR FF FR CF FR FC
Layer 2: Data Link Layer	L	Driver, Interface	L-PDU	CAN	LIN	FR

SF:
Single Frame
FF:
First Frame
CF:
Consecutive Frame
FC:
Flow Control

Examples:

- I-PDU or I-SDU
- CAN FF N-PDU or FR CF N-SDU
- LIN L-PDU or FR L-SDU

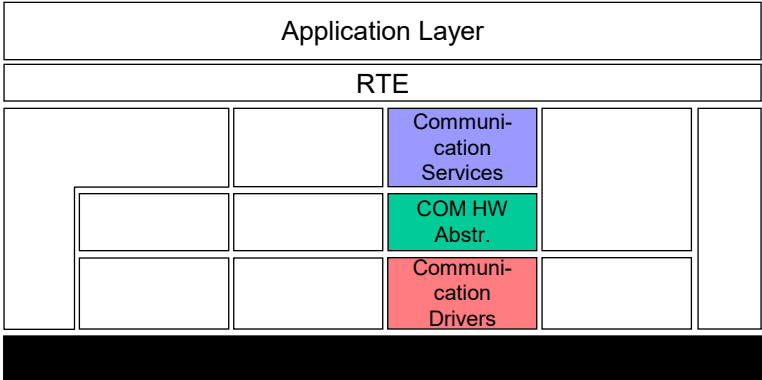
For details on the frame types, please refer to the AUTOSAR Transport Protocol specifications for CAN, LIN and FlexRay.

Interfaces: Interaction of Layers

示例“沟通”(1)

SDU与PDU命名规范

PDU和SDU的命名遵循以下规则： 对PDU: <bus prefix>
<layer prefix> - PDU 对SDU: <bus prefix> <layer prefix> - SDU
U bus prefix和layer prefix在下表中描述:



ISO Layer	Layer Prefix	AUTOSAR Modules	PDU Name	CAN prefix	LIN prefix	FlexRay prefix
Layer 6: Presentation (Interaction)	I	COM, DCM	I-PDU	N/A		
	I	PDU router, PDU multiplexer	I-PDU	N/A		
Layer 3: Network Layer	N	TP Layer	N-PDU	CAN SF CAN FF CAN CF CAN FC	LIN SF LIN FF LIN CF LIN FC	FR SF FR FF FR CF FR FC
Layer 2: Data Link Layer	L	Driver, Interface	L-PDU	CAN	LIN	FR

SF: Single 框架
FF: First 框架
CF: Cons执行 Frame
FC: Flow有关帧

请参见适用于CAN、LIN和FlexRay的AUTOSAR传输协议规范

示例: I-PDU 或 I-SDU CAN FF
N-PDU 或 FR CF N-SDU LIN L-P
DU 或 FR L-SDU



Interfaces: Interaction of Layers

Example “Communication” (2)

Components

- PDU Router:
 - Provides routing of PDUs between different abstract communication controllers and upper layers
 - Scale of the Router is ECU specific (down to no size if e.g. only one communication controller exists)
 - Provides TP routing on-the-fly. Transfer of TP data is started before full TP data is buffered
- COM:
 - Provides routing of individual signals or groups of signals between different I-PDUs
- NM Coordinator:
 - Synchronization of Network States of different communication channels connected to an ECU via the network managements handled by the NM Coordinator
- Communication State Managers:
 - Start and Shutdown the hardware units of the communication systems via the interfaces
 - Control PDU groups

Interfaces: Interaction of Layers

示例“通信”（2）

组件

PDU路由器：

- 在不同抽象通信控制器与上层之间提供PDU的路由
- 路由器的规模取决于ECU（例如，如果仅存在一个通信控制器，则可缩减至零尺寸）
- 提供动态TP路由。在完整TP数据缓冲完成前即开始传输TP数据

COM： ■ 在不同I-PDU之间提供独立信号或信号组的路由

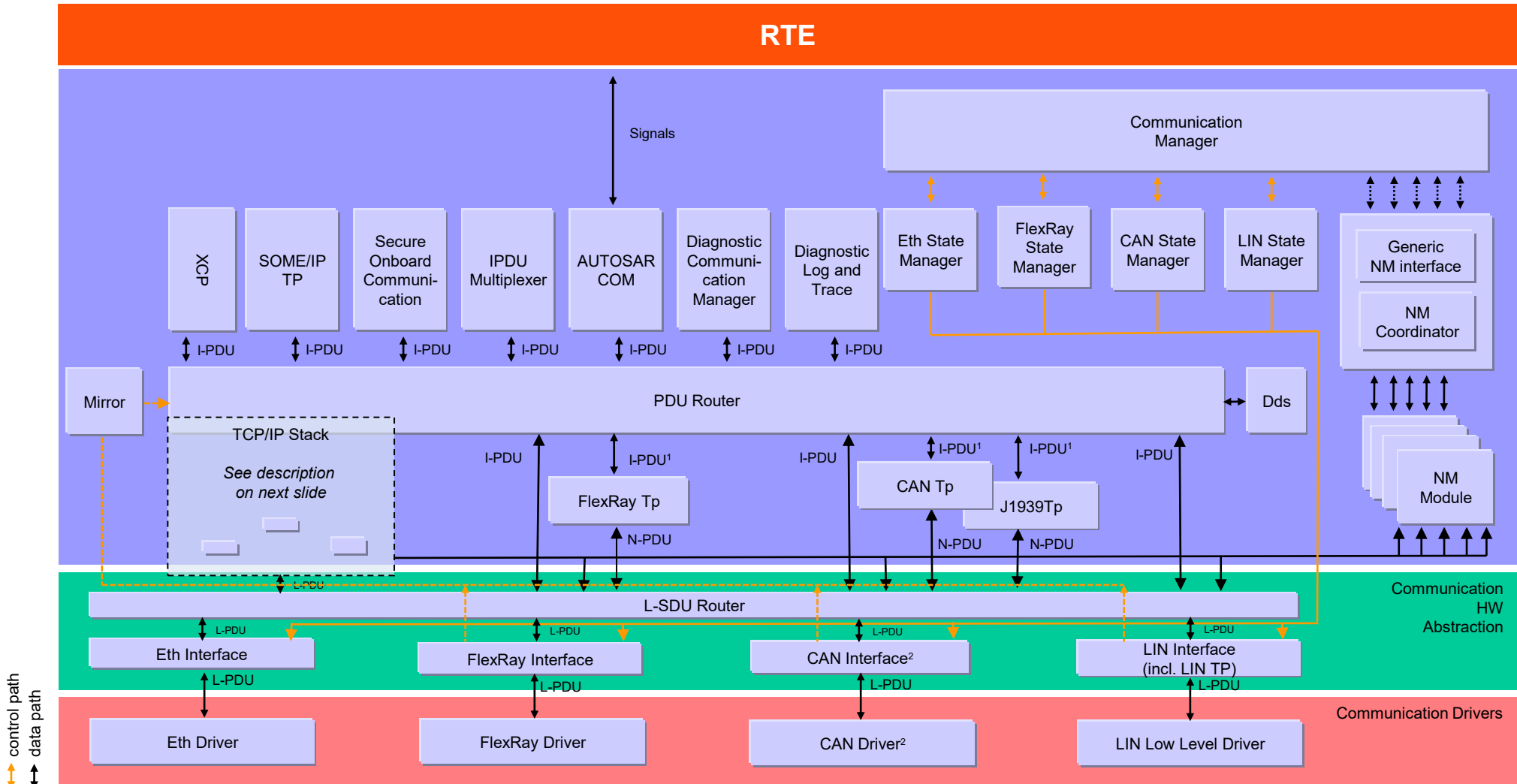
NM协调器： ■ 通过NM协调器处理的网络管理，同步连接到ECU的不同通信通道的网络状态

通信状态管理器： ■ 通过接口启动和关闭通信系统的硬件单元

■ 控制PDU组

Interfaces: Interaction of Layers

Example “Communication” (3)



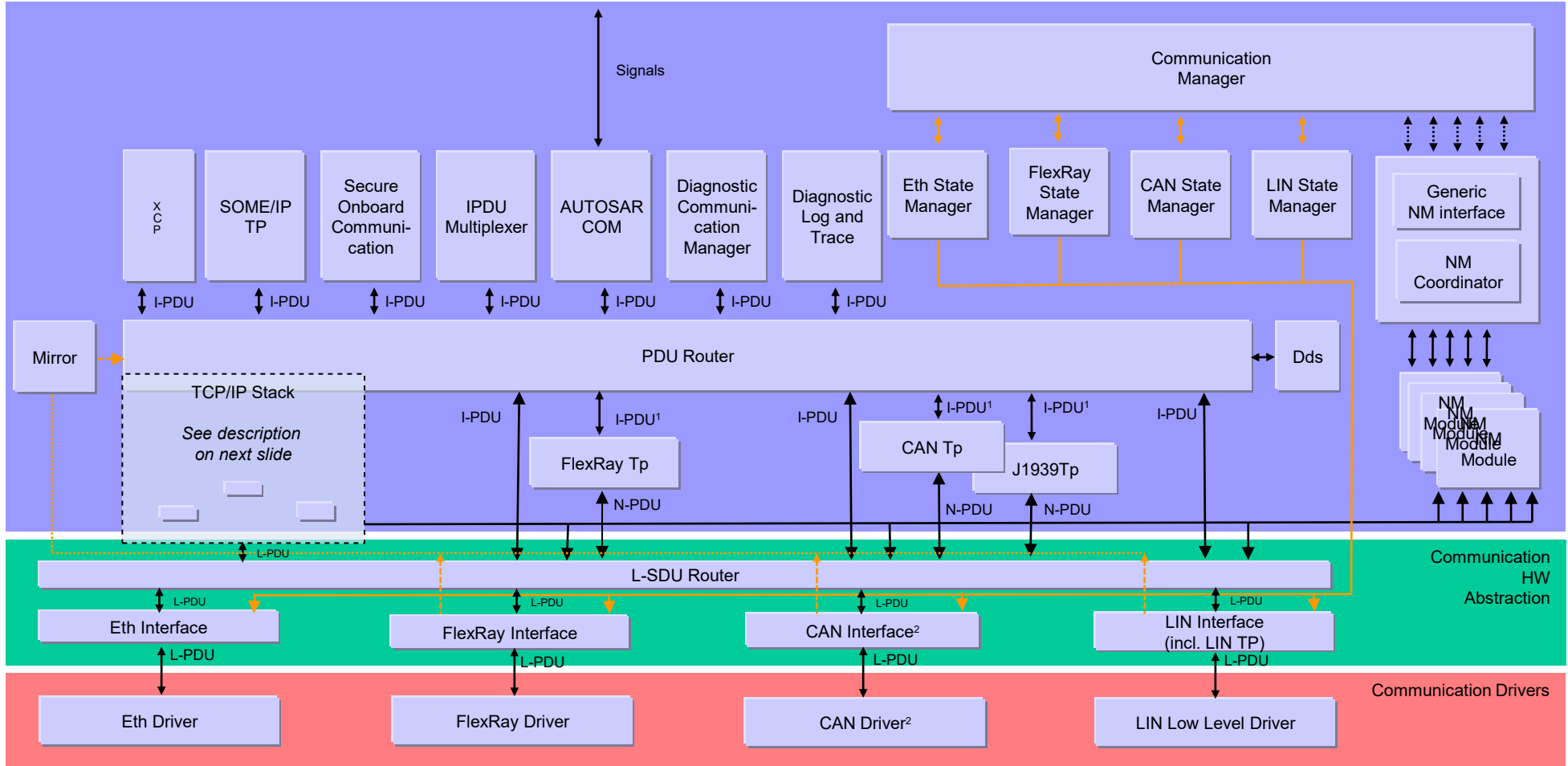
Note: This image is not complete with respect to all internal communication paths.

¹ The Interface between PduR and Tp differs significantly compared to the interface between PduR and the Ifs. In case of TP involvement a handshake mechanism is implemented allowing the transmission of I-PDUs > Frame size.

Interfaces: Interaction of Layers

示例“沟通” (3)

RTE

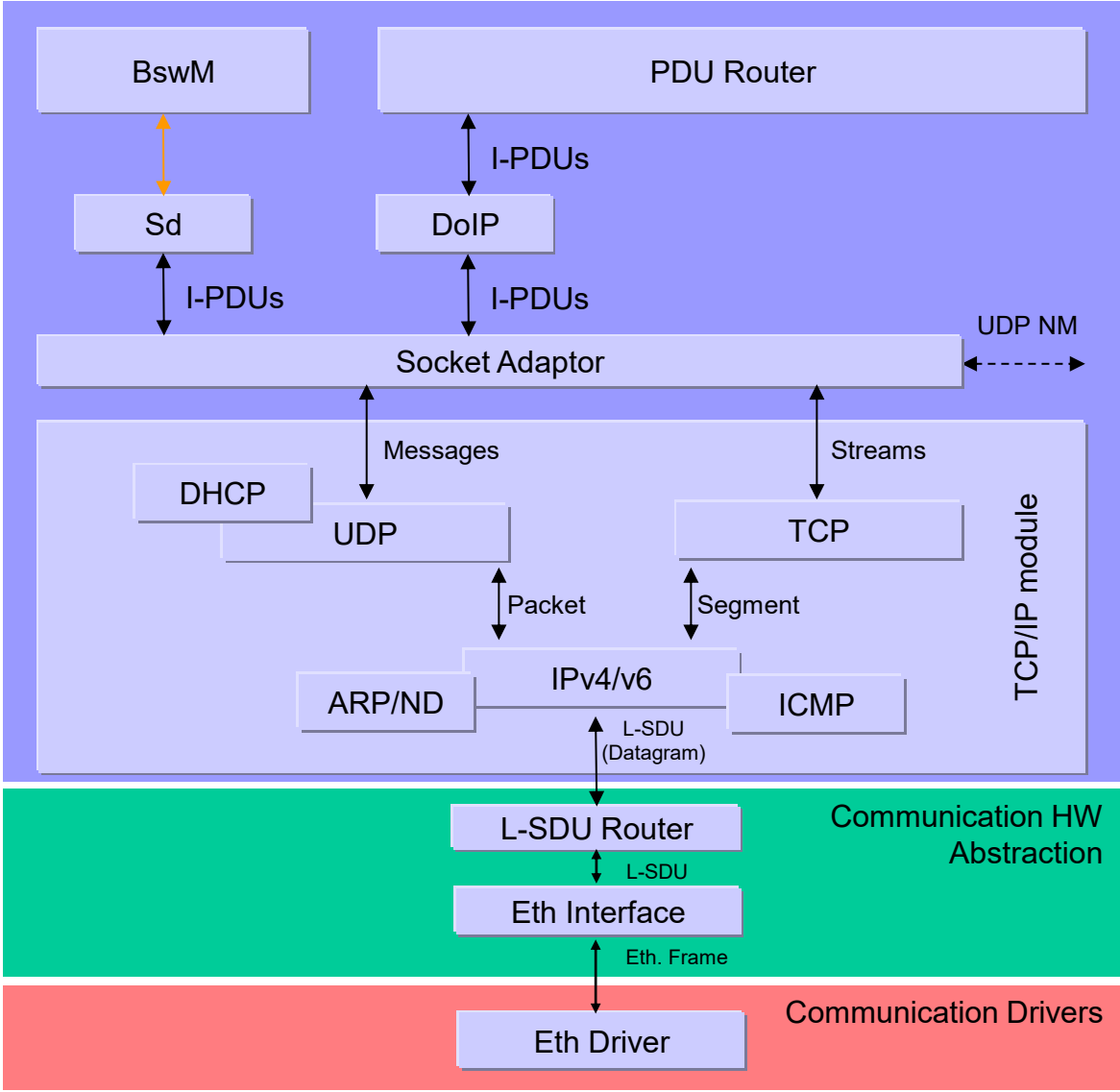


Note: This image is not complete with respect to all internal communication paths.

¹ The Interface between PduR and Tp differs significantly compared to the interface between PduR and the Ifs. In case of TP involvement a handshake mechanism is implemented allowing the transmission of I-PDUs > Frame size.

Interfaces: Interaction of Layers **Example “Communication” (4) – Ethernet Stack**

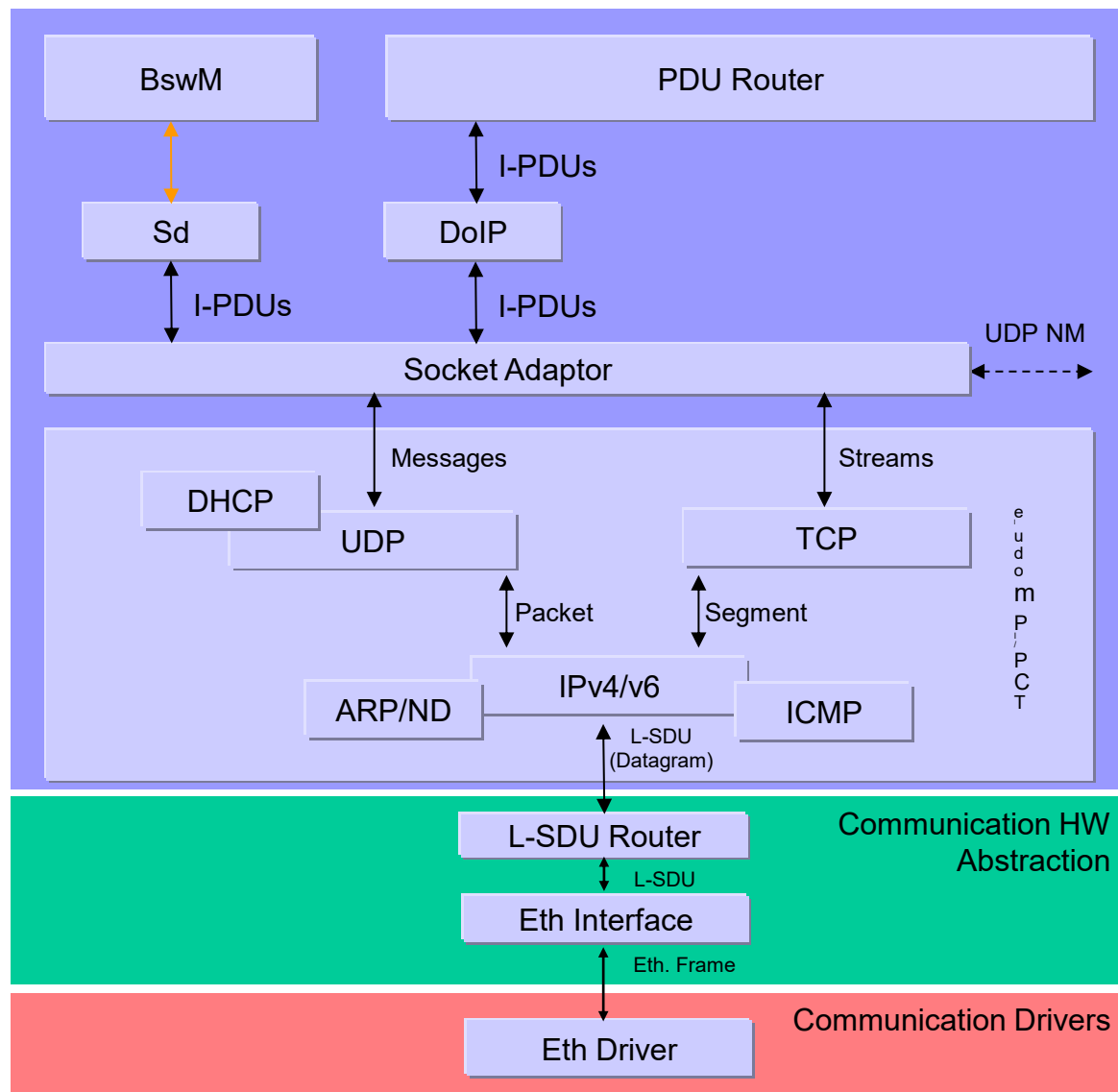
➤ This figure shows the interaction of and inside the Ethernet stack.



Interfaces: Interaction of Layers

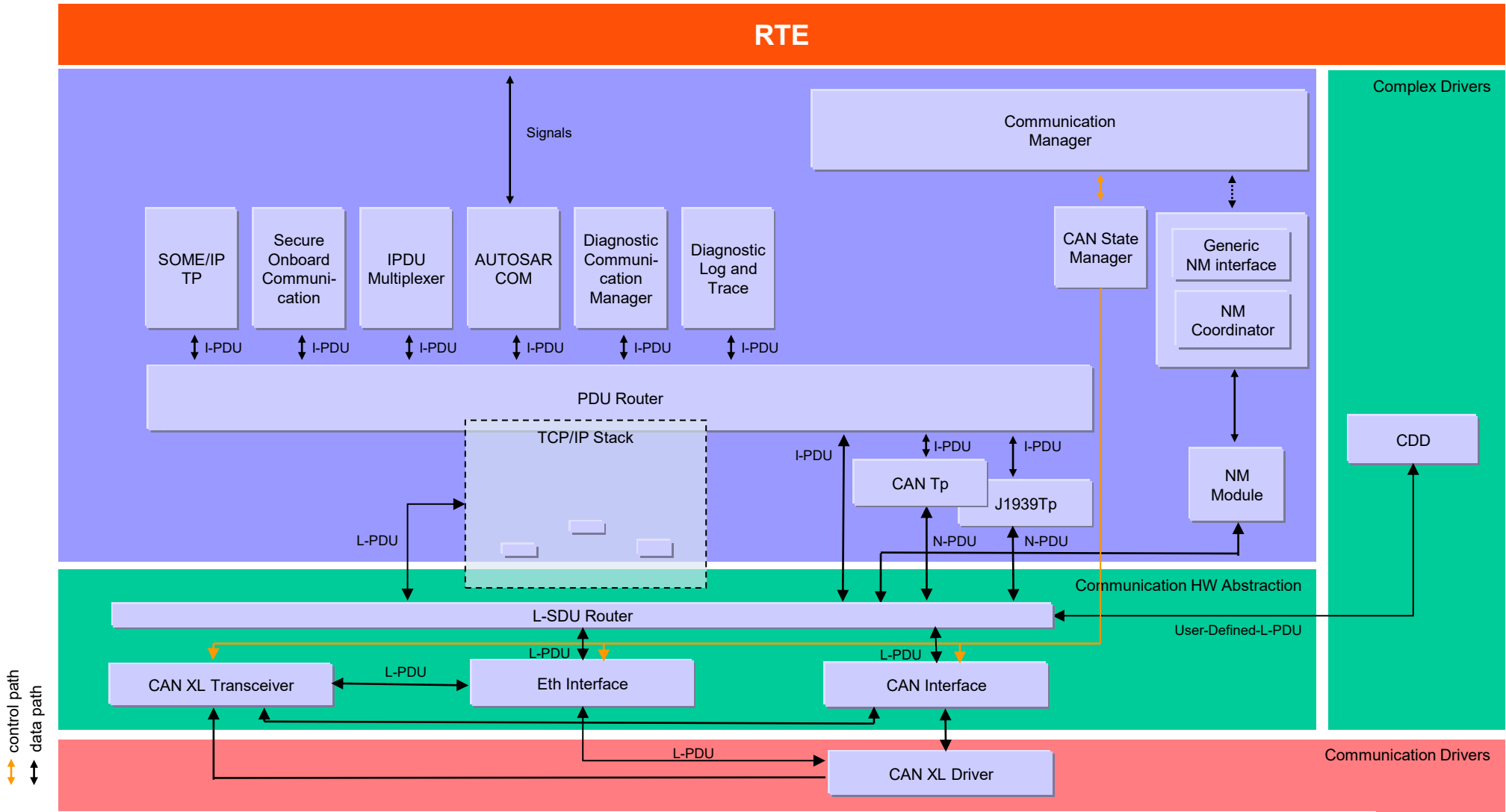
示例“通信”（4）——以太网协议栈

该图展示了以太网栈内部和之间的交互。



Interfaces: Interaction of Layers

Example “Communication” (5) - Ethernet and CAN communication using CAN XL



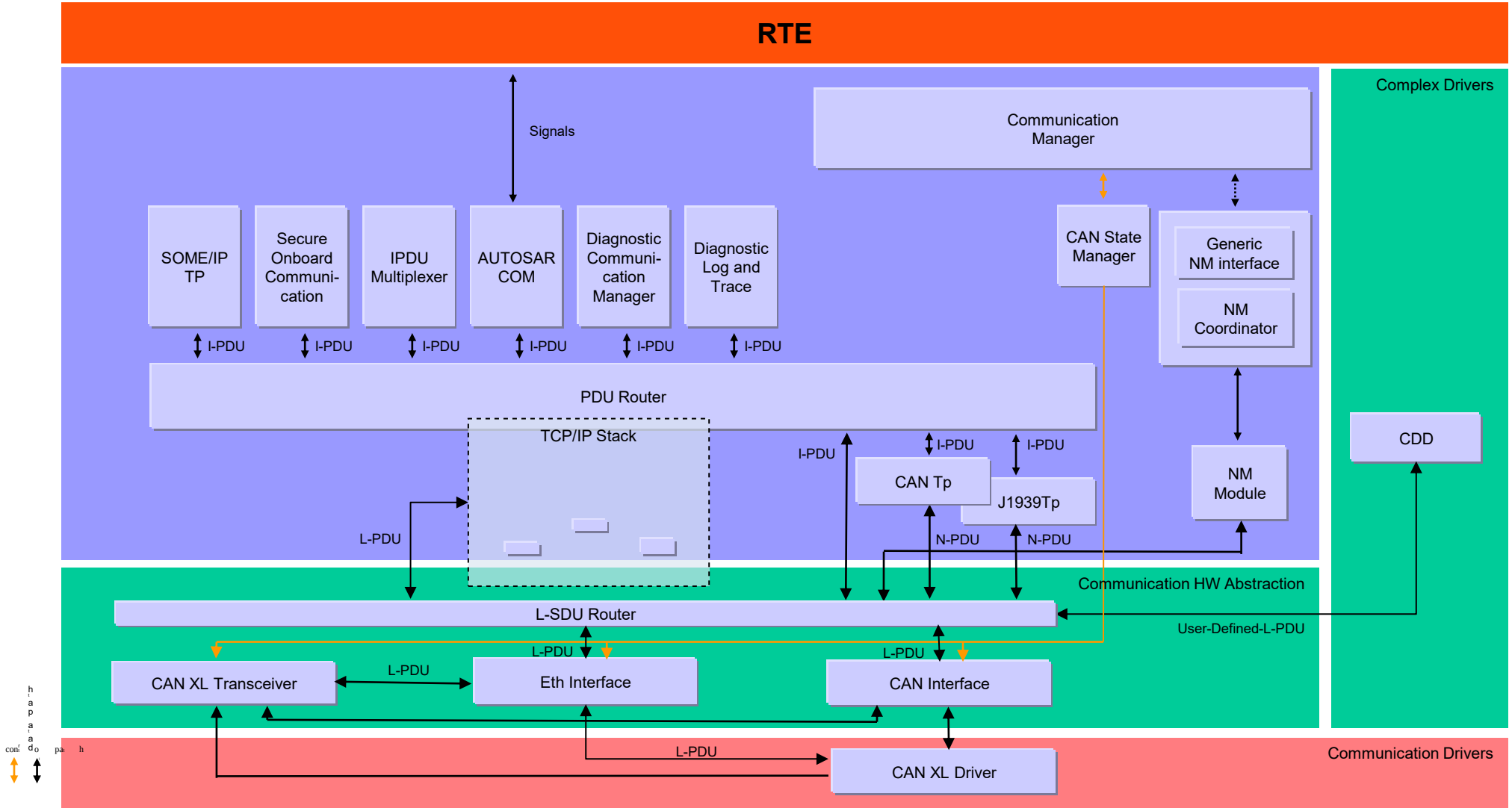
Note: This image is not complete with respect to all internal communication paths.

Interfaces: Interaction of Layers

示例“Communi 通信”(5) - 使用以太网和CAN通信

g CAN XL

8
d xmd w



注意：此图像并未完整展示所有内部通信路径。

Interfaces: Interaction of Layers

Example “Data Transformation” (1) – Introduction

The following pages explain communication with Data Transformation:

- How do the software layers interact?
- How do the software interfaces look like?

Interfaces: Interaction of Layers

示例“数据转换”（1）—— 引言

以下页面解释了与数据转换的通信：

软件各层之间如何交互？

软件界面看起来是怎样的？

Interfaces: Interaction of Layers

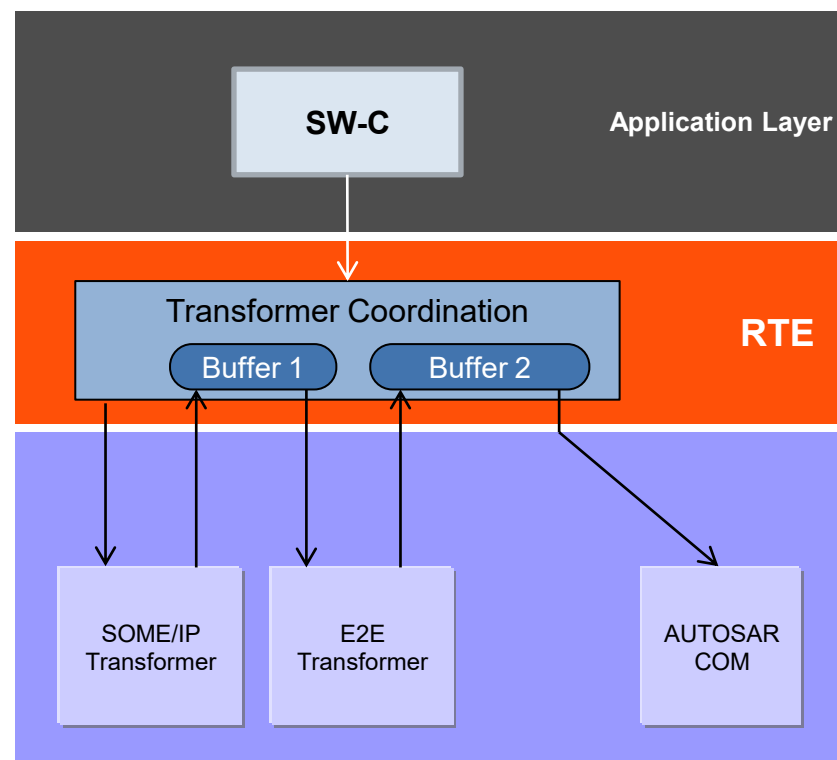
Example “Data Transformation” (2) – Example and First Look

This example shows the data flow if data transformation is used for inter-ECU communication.

A SW-C sends data configured to be transmitted to a remote ECU and subject to data transformation. This data transformation doesn't use in-place buffer handling.

Functionality

- The RTE calls the SOME/IP transformer as the first transformer in the chain and transfers the data from the SW-C.
- The SOME/IP transformer executes the transformation and writes the output (byte array) to a buffer provided by the RTE.
- Afterwards, the RTE executes the Safety transformer which is second in the transformer chain. The Safety transformer's input is the output of the SOME/IP transformer.
- The Safety transformer protects the data and writes the output into another buffer provided by the RTE. A new buffer is required because in-place buffer handling is not used.
- The RTE transfers the final output data as a byte array to the COM module.



Interfaces: Interaction of Layers

示例“数据转换”

(2) ——示例与初步了解

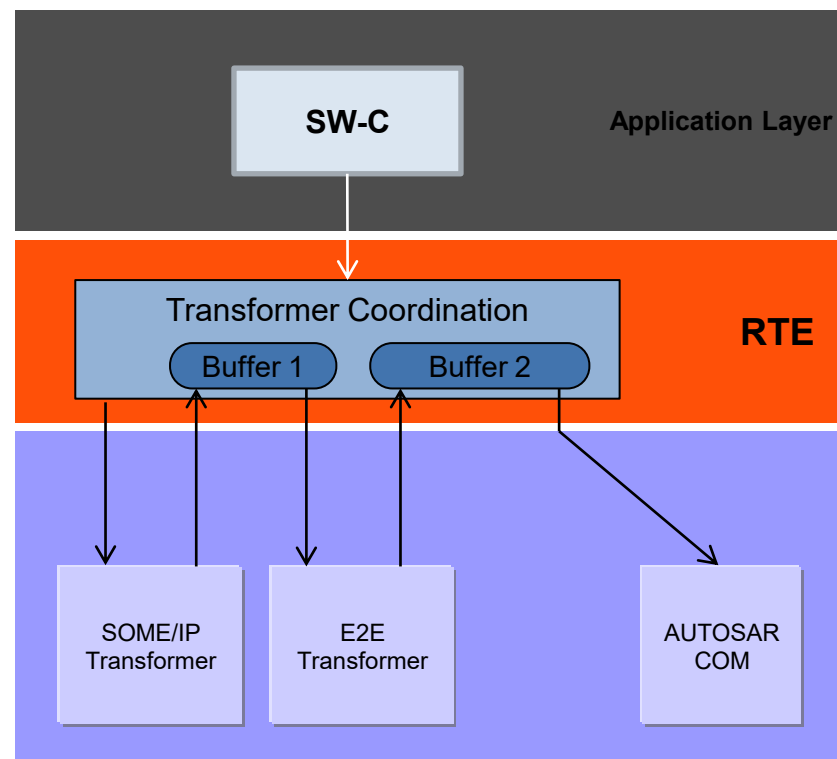
此示例展示了在ECU间通信中使用数据转换时的数据流。

一个软件组件（SW-C）发送配置为传输至远程ECU并需进行数据转换的数据。此数据转换不使用就地缓冲区处理。

功能性

RTE将SOME/IP转换器作为链中的首个转换器调用，并从SW-C传输数据。SOME/IP转换器执行转换并

将输出（字节数组）写入由RTE提供的缓冲区中。随后，RTE执行位于转换器链中第二位的安全转换器。安全转换器的输入是SOME/IP转换器的输出。安全转换器对数据进行保护，并将输出写入RTE提供的另一个缓冲区。由于未使用原地缓冲区处理，因此需要新的缓冲区。RTE将最终输出数据作为字节数组传输至COM模块。



Interfaces: Interaction of Layers

Example “Data Transformation” (3) – Closer Look at Interfaces

Architecture Description

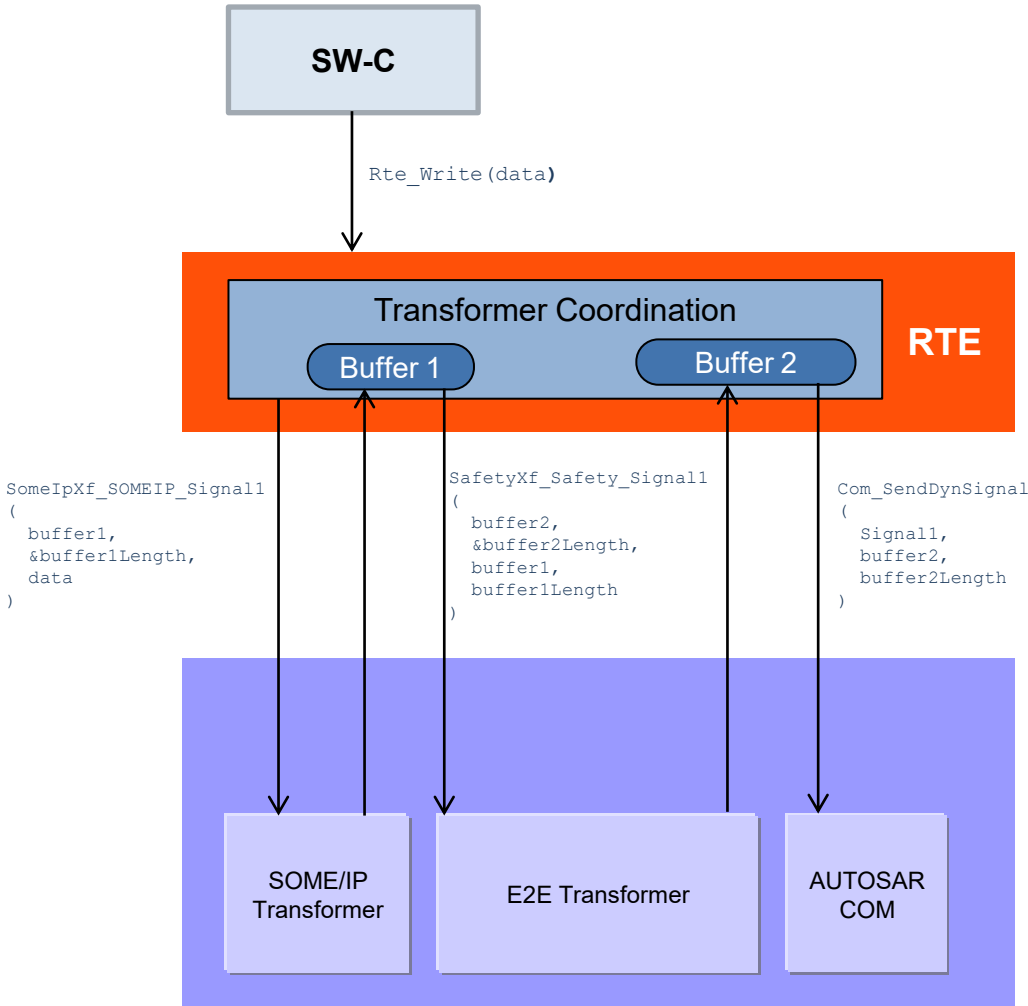
The RTE uses the transformer which are located in the System Service Layer.

Interface Description

The transformers in this example have the following interfaces:

```
SomeIpXf_SOMEIP_Signal1
(
    uint8      *buffer1,
    uint16     *buffer1Length,
    <type>     data
)
```

```
SafetyXf_Safety_Signal1
(
    uint8      *buffer2,
    uint16     *buffer2Length,
    uint8      *buffer1,
    uint16     buffer1Length
)
```



Interfaces: Interaction of Layers

示例“数据转换”（3）——深入探讨接口

架构描述

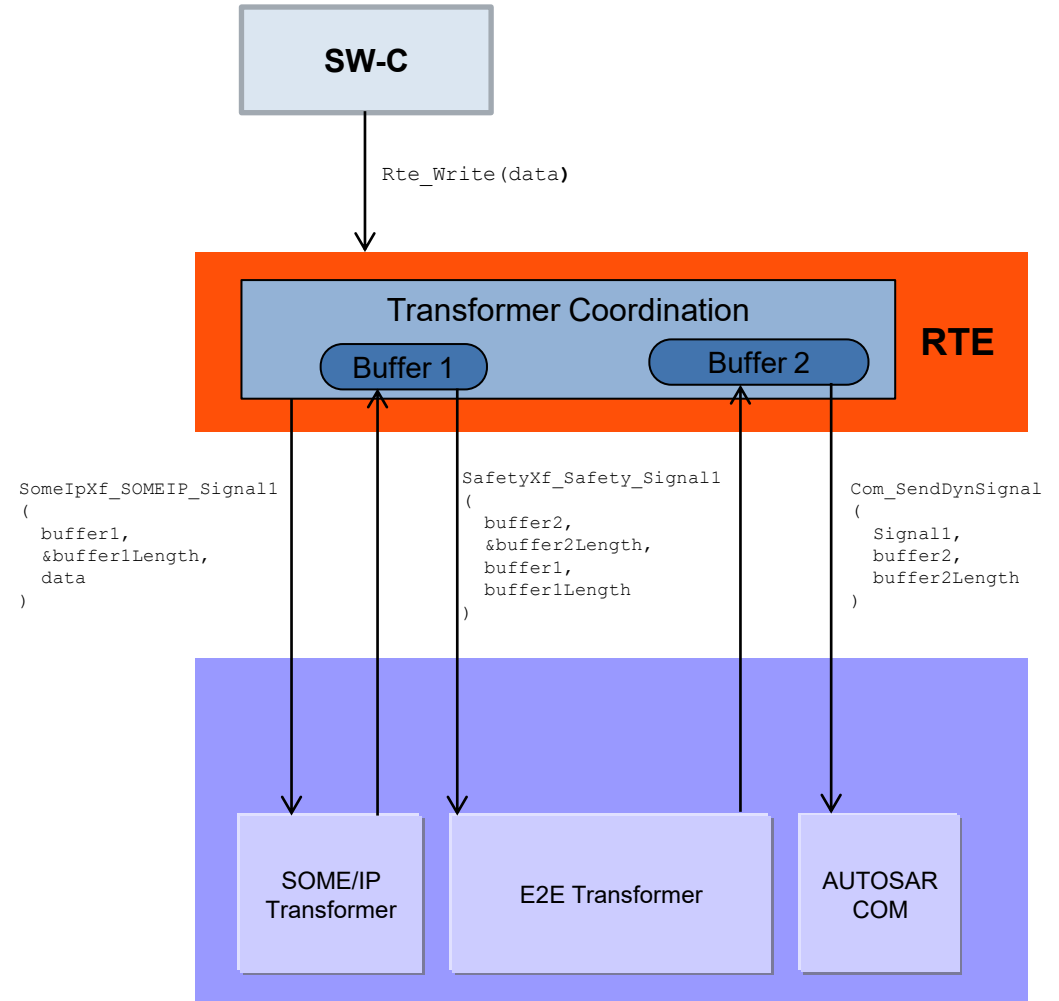
TRTE使用位于系统服务层中的变压器。

接口描述

本例中的变压器具有以下接口：

```
SomeIpXf_SOMEIP_Signal1 ( uint8  *buffer1, uint16 *buffer1Length, <type> data )
```

```
SafetyXf_安全信号1  
( uint8  *buffer2, uint16 *buffer2Length, uint8 *buffer1, uint16 *buffer1Length )
```



Interfaces: Interaction of Layers

Example “Data Transformation” (4) – COM Based Transformation

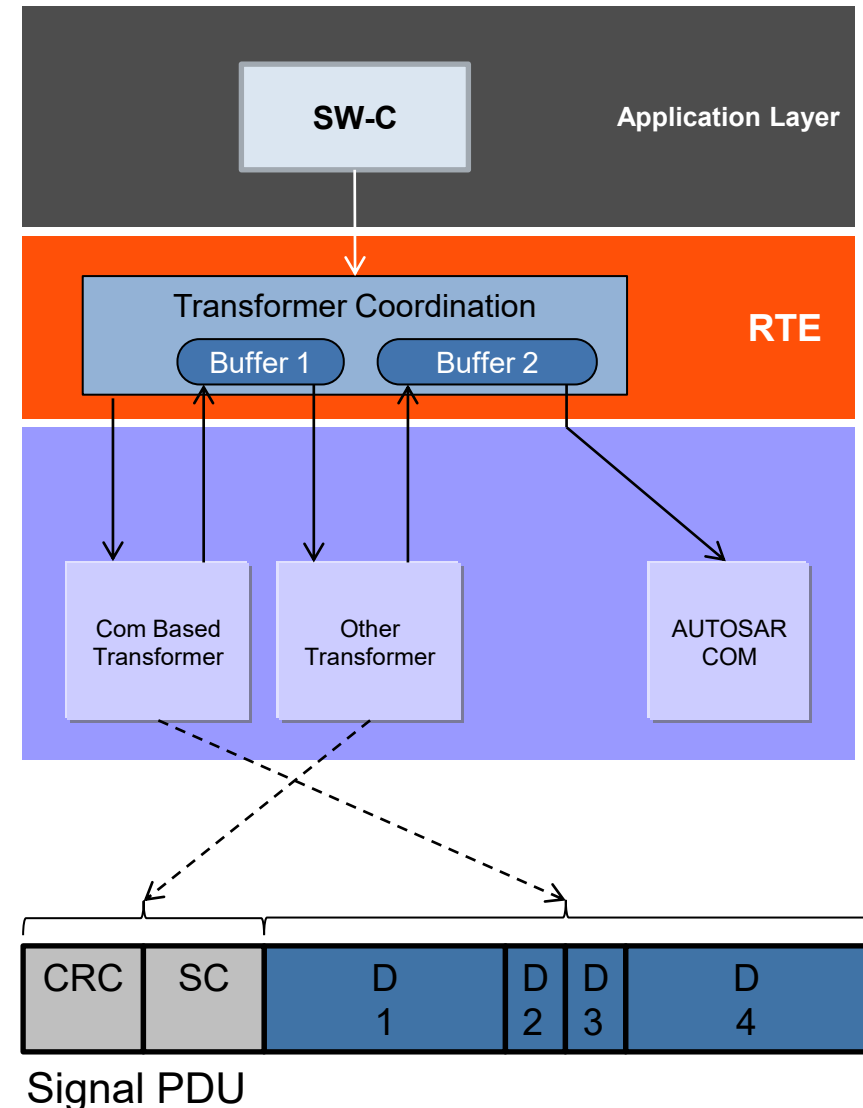
Goal

The *COM Based Transformer* provides serialization functionality to the transformer chain based on a fixed communication matrix.

The fixed communication matrix allows an optimized placement of signals into PDUs (e.g. a Boolean data can be configured to only occupy one bit in the PDU). This enables the usage of transformer chains in low payload networks like Can or Lin.

Functionality

- The *COM Based Transformer* is the first transformer (serializer) and gets the data from the application via the RTE.
- Based on the COM configuration (communication matrix) the data is serialized exactly in the same way as the COM module would have done it (endianness, sign extension).
- Other transformers may enhance the payload to have CRCs and sequence counters (SC).
- The transformer payload is passed to the COM module as one array of byte via the `Com_SendSignalGroupArray` API.
- The COM module can be configured to perform transmission mode selection based on the communication matrix definition.



Interfaces: Interaction of Layers

示例“数据转换”（4）——基于COM的转换

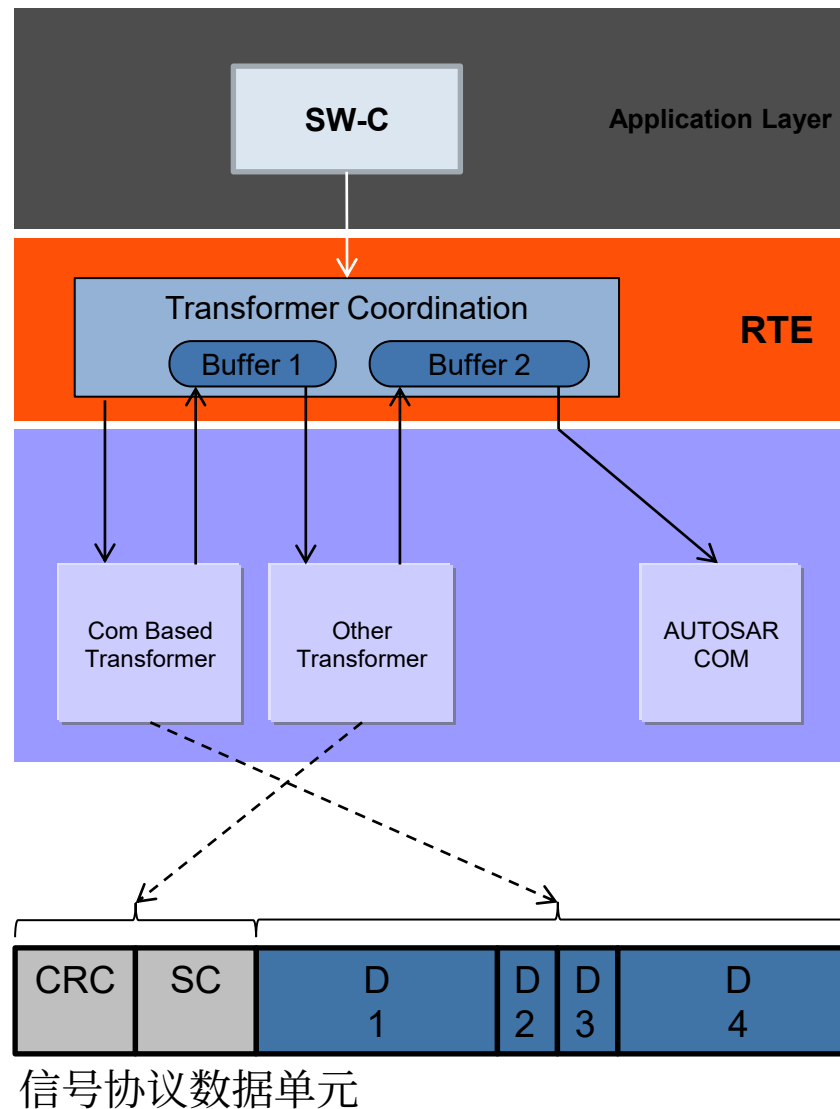
目标

COM Based Transformer 基于固定的通信矩阵为变压器链提供序列化功能。

固定的通信矩阵允许将信号优化地放置到PDU中（例如，布尔数据可配置为仅占用PDU中的一位）。这使得在Can或Lin等低负载网络中能够使用转换器链。

功能性

COM Based Transformer 是第一个转换器（序列化器），通过RTE从应用程序获取数据。基于COM配置（通信矩阵），数据以与COM模块完全相同的方式进行序列化（字节序、符号扩展）。其他转换器可增强有效载荷以包含CRC和序列计数器（SC）。转换器有效载荷通过Com_SendSignalGroupArray API以字节数组形式传递给COM模块。COM模块可配置为基于通信矩阵定义执行传输模式选择。



Interfaces: Interaction of Layers

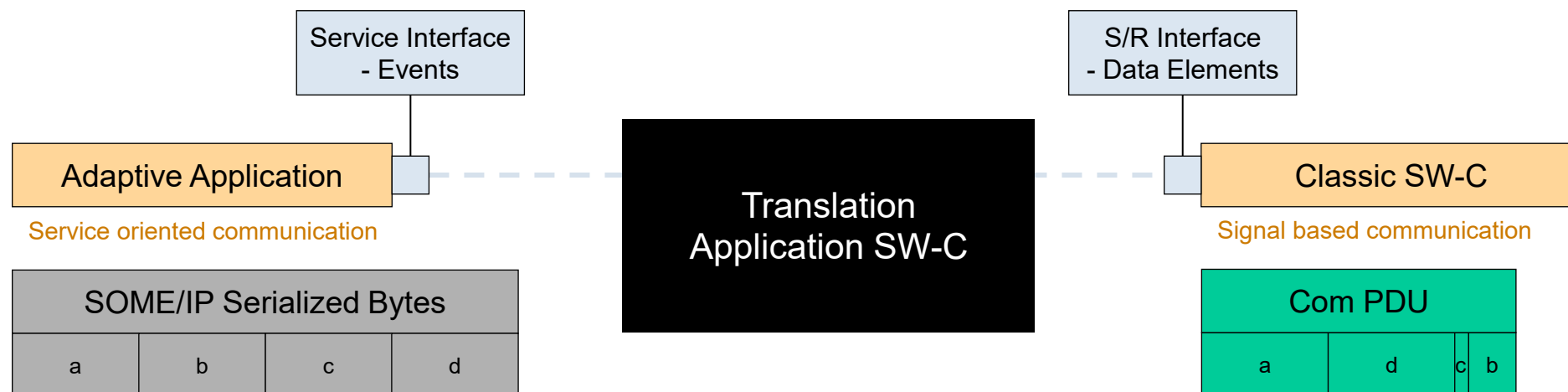
Signal-Service-Translation (1)

Goal

Adaptive Platform restricts communication to Service-oriented communication, the rest of the vehicle however still uses Signal-based communication means - therefore a translation of these two approaches has to be performed in order to allow an interaction between Classic and Adaptive Platform.

Functionality

- The definition and implementation of the Classic platform signal-service-translation shall be done inside an Application Software Component, the so called **Translation Software Component**.
- The **Translation Software Component** has Ports defined and the payload is described using **PortInterfaces**
 - Signal-to-service: Ports for incoming signals and Ports for outgoing events
 - Service-to-signal: Ports for incoming events and Ports for outgoing signals



Interfaces: Interaction of Layers

信号-服务-翻译 (1)

目标

自适应平台将通信限制在面向服务的通信，然而车辆的其他部分仍使用基于信号的通信方式——因此，为了允许经典平台与自适应平台之间的交互，必须对这两种通信方式进行转换。

功能性

经典平台信号服务转换的定义和实现应在应用软件组件内部完成，即所谓的转换软件组件。

翻译软件组件定义了端口，并且使用端口接口描述有效载荷。

- 信号到服务：用于传入信号的端口和用于传出事件的端口
- 服务到信号：用于传入事件的端口和用于传出信号的端口

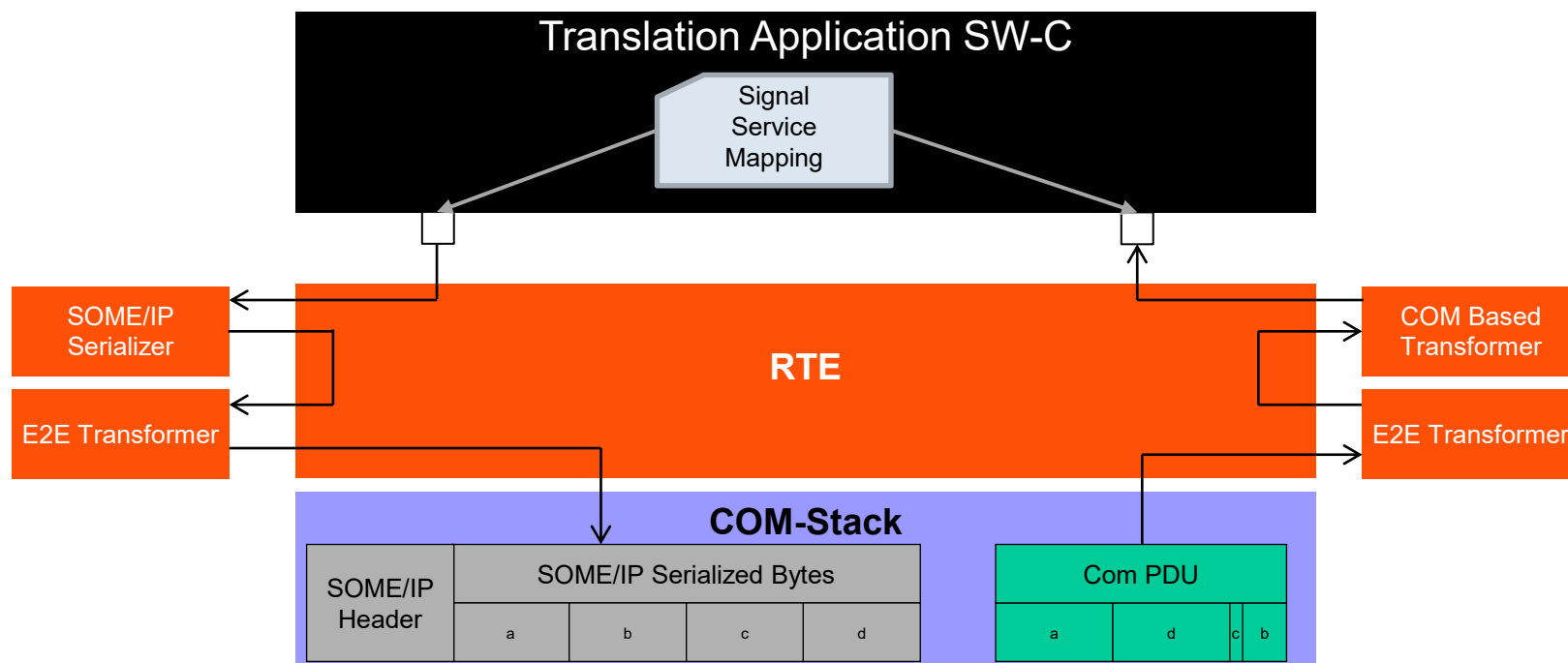


Interfaces: Interaction of Layers

Signal-Service-Translation (2)

Functionality

- For the signal-based part the full functionality of the Classic platform **COM-Stack** is available and may be configured such that the signal-based ISignalIPdus may originate from a variety of sources (Can, Lin, FlexRay) and the ISignalIPdus may be safety and security protected.
- For the service-oriented part it has to be guaranteed that the defined SOME/IP Service actually is compatible to the Adaptive platform. This applies for the payload part (e.g. the SOME/IP serializer has to be used) as well as for the control path using BswM and ServiceDiscovery.
- The behavioral part of the Translation Software Component itself defines how the data from signal-based side is transported to the service-oriented side, and vice versa.



Interfaces: Interaction of Layers

信号-服务-翻译 (2)

功能性

对于基于信号的部分，经典平台COM-Stack的全部功能均可使用，并可配置为使得基于信号的ISignalIPdu可源自多种来源（Can、Lin、FlexRay），且ISignalIPdu可受到安全与防护保护。对于面向服务的部分，必须确保所定义的SOME/IP服务实际上与自适应平台兼容。这适用于有效载荷部分（例如，SOME/IP序列化器必须能够

以及使用BswM和ServiceDiscovery的控制路径。翻译软件组件本身的行为部分定义了如何将来自基于信号侧的数据传输到面向服务侧，反之亦然。

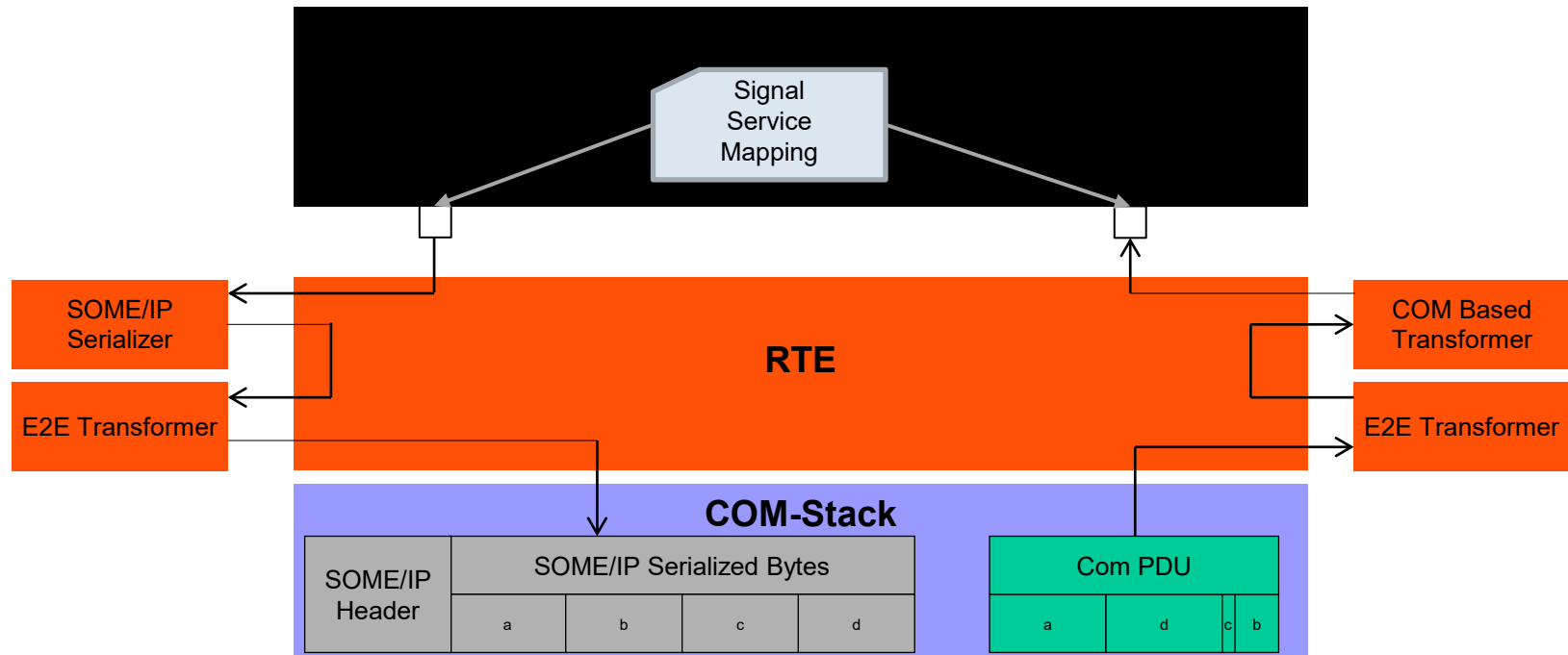


Table of contents

1. Architecture

1. Overview of Software Layers
2. Content of Software Layers
3. Content of Software Layers in Multi-Core Systems
4. Content of Software Layers in Mixed-Critical Systems
5. Overview of Modules
6. Interfaces: General Rules
7. Interfaces: Interaction of Layers
8. Overview of CP Software Clusters

2. Configuration

3. Integration and Runtime Aspects

Table of contents

1. 架构

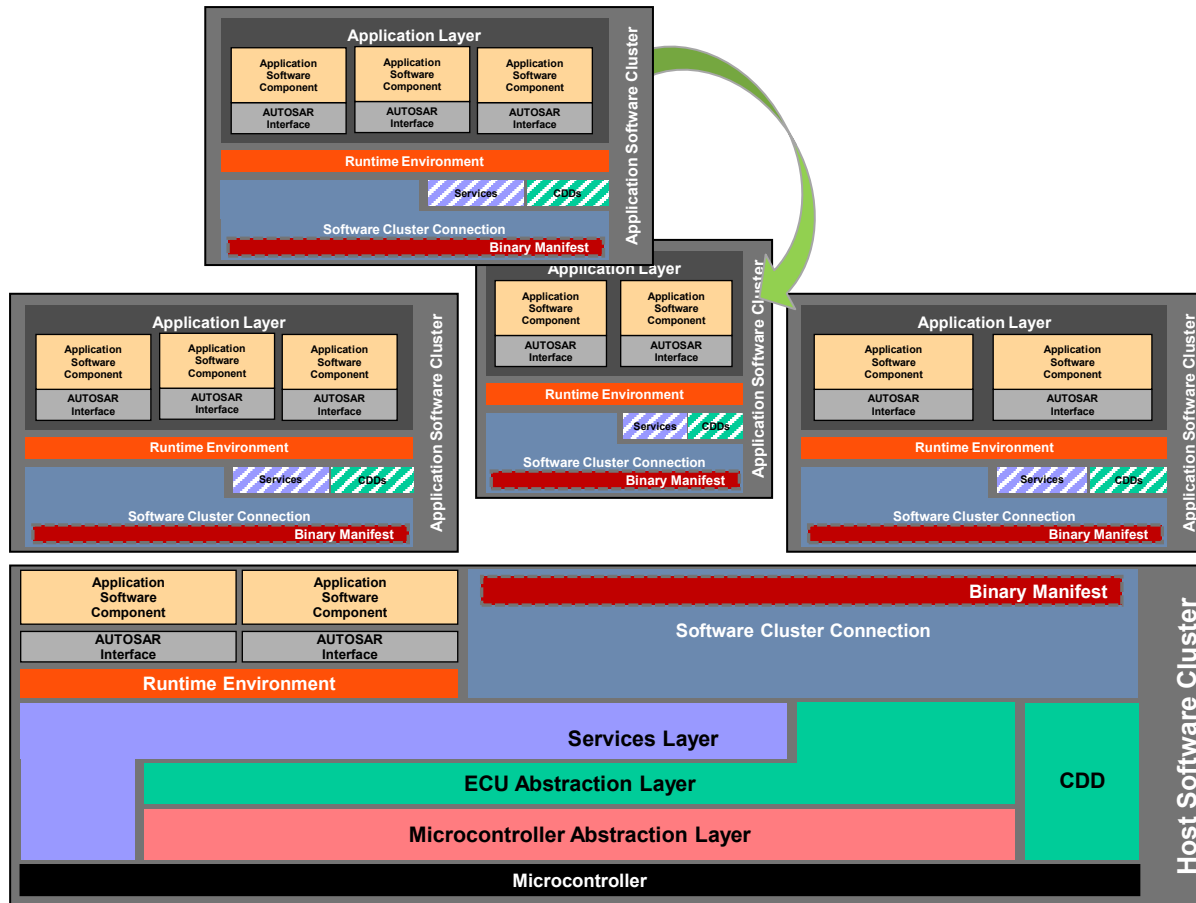
1. 软件层概述 2. 软件层内容 3. 多核系统中的软件层内容 4
. 混合关键系统中的软件层内容 5. 模块概述 6. 接口：通用
规则 7. 接口：层间交互 8. CP软件集群概述

2. 配置 3. 集成与运行时方面

Overview of CP Software Clusters

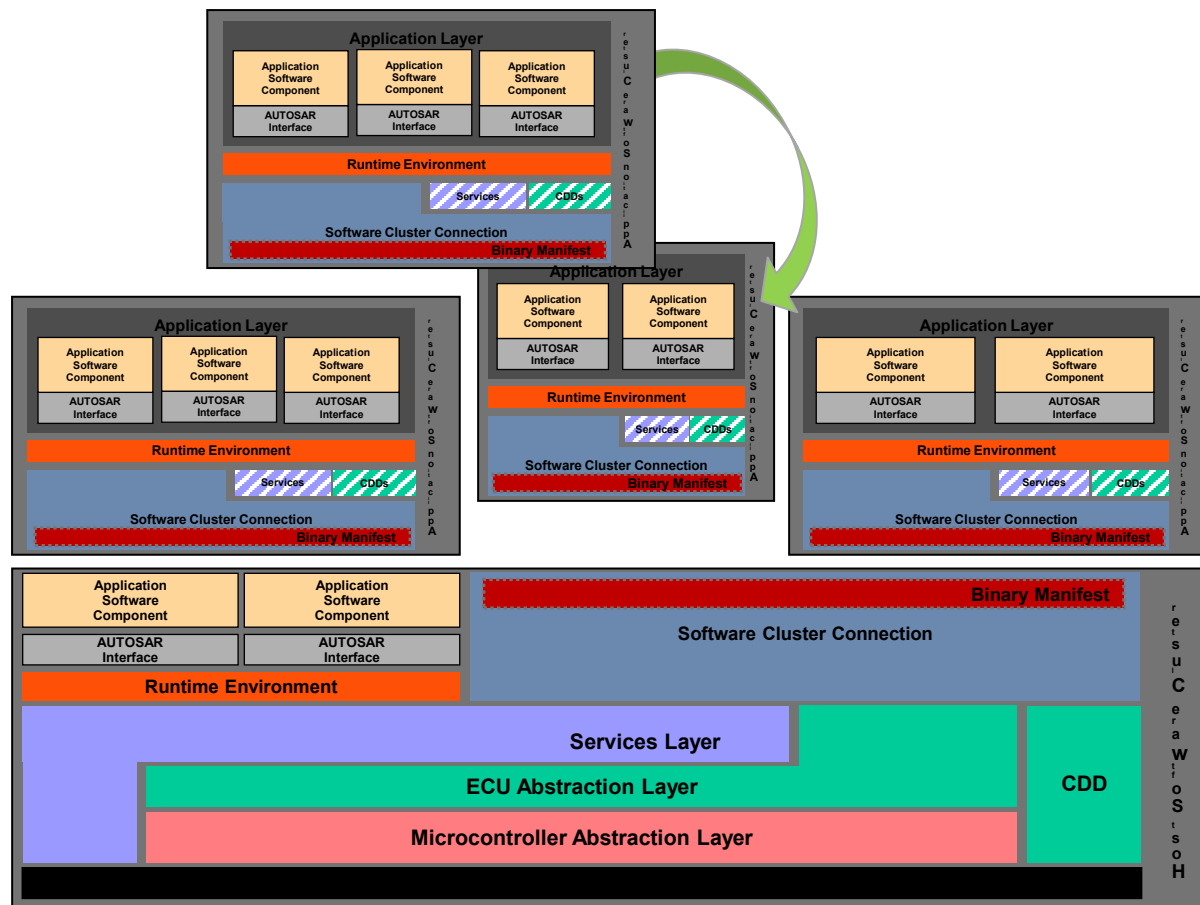
Concept overview

The approach in a nutshell



- Software Cluster enable to split the monolithic Classic Platform Architecture into smaller units
- Each CP Software Cluster is separately buildable
- Software Clusters can be independently updated
- Connections between Software Clusters are created on basis of Binary Objects and the information hold in the Binary Manifest
- Considers the limitation of current micro controller architectures, e.g. no address virtualization
- In an *Application Software Cluster*, Application SW-Cs and BSW modules (with limitations) can be integrated
- The *Host Software Cluster* contains the major part of the BSW Stack, especially micro controller dependent modules including the **Operating System**.

The approach in a nutshell



软件集群能够将单一的传统平台架构拆分为更小的单元

每个CP软件集群均可独立构建

软件集群可独立更新

软件集群之间的连接基于二进制对象及二进制清单中所包含的信息而建立。

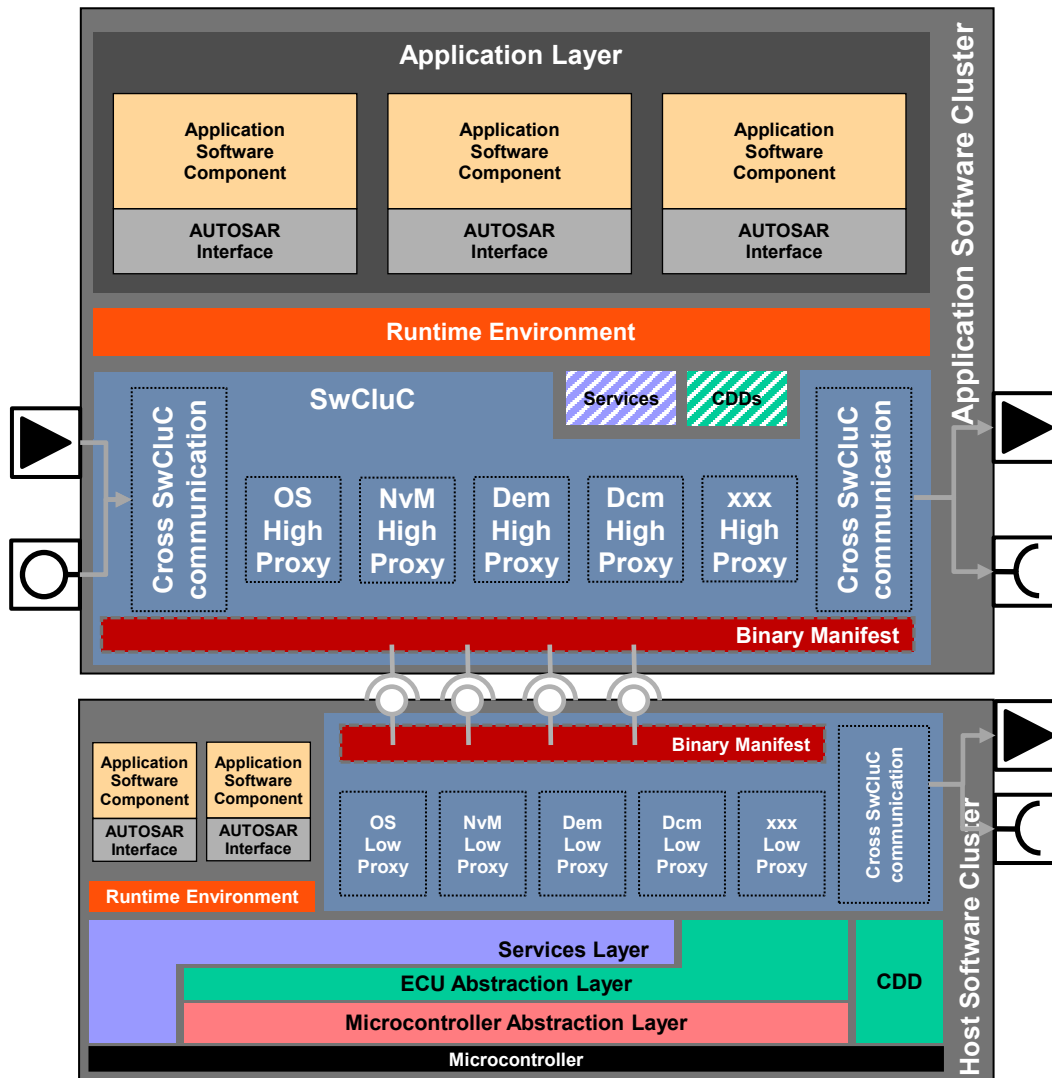
考虑到当前微控制器架构的局限性，例如缺乏地址虚拟化功能

在 *Application Software Cluster* 中，应用软件组件和基础软件模块（有限制地）可被集成

Host Software Cluster 包含了BSW堆栈的主要部分，特别是依赖于微控制器的模块，包括操作系统。

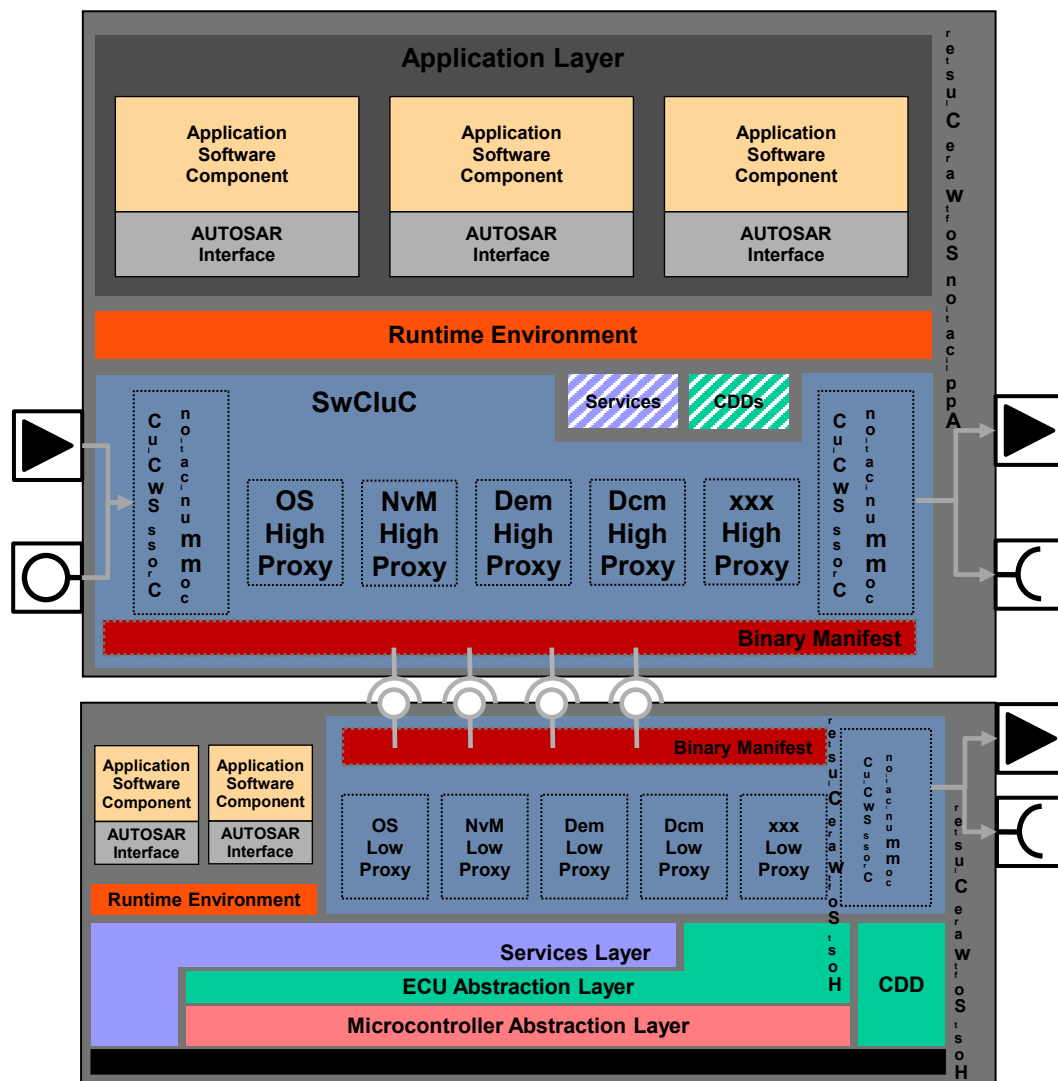
Overview of CP Software Clusters

Software Cluster Connection (1)



The module **Software Cluster Connection (SwCluC)** has 3 parts:

- **Cross Software Cluster Communication (SwCluC_Xcc)** provides the features in Classic Platform
 - to enable the connection of software clusters based on binary manifest
 - for cross interaction and communication of software clusters
- Abstraction of non-software cluster-local BSW modules and their APIs in the corresponding **proxy modules**
 - High Proxies substitute non-local BSW and provide the according APIs
 - Lower Proxy modules connect to regular BSW modules of the Host Software Cluster
- The **Binary Manifest** (BManif) provides binary meta information for interfaces to be able to connect software clusters.



软件集群连接模块 (SwCluC) 包含3个部分:

跨软件集群通信 (SwCluC_Xcc) 提供经典平台中的功能

- 实现基于二进制清单■的软件集群连接，以支持软件集群间的跨交互与通信。

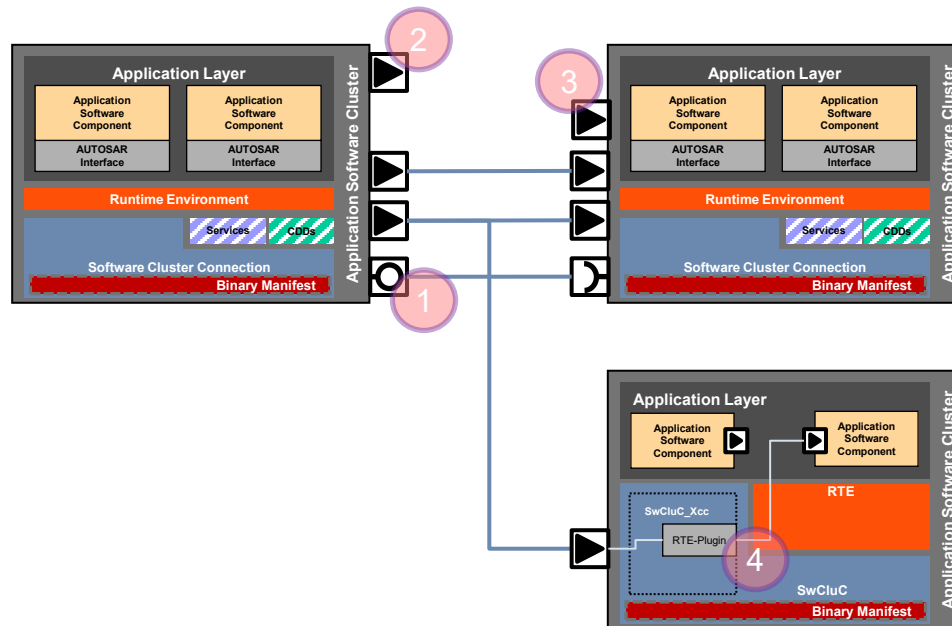
在相应的代理模块中对非软件集群本地BSW模块及其API进行抽象

- 高代理替代非本地基础软件 (BSW) 并提供相应的API ■，低代理模块则连接到主机软件集群的常规BSW模块。

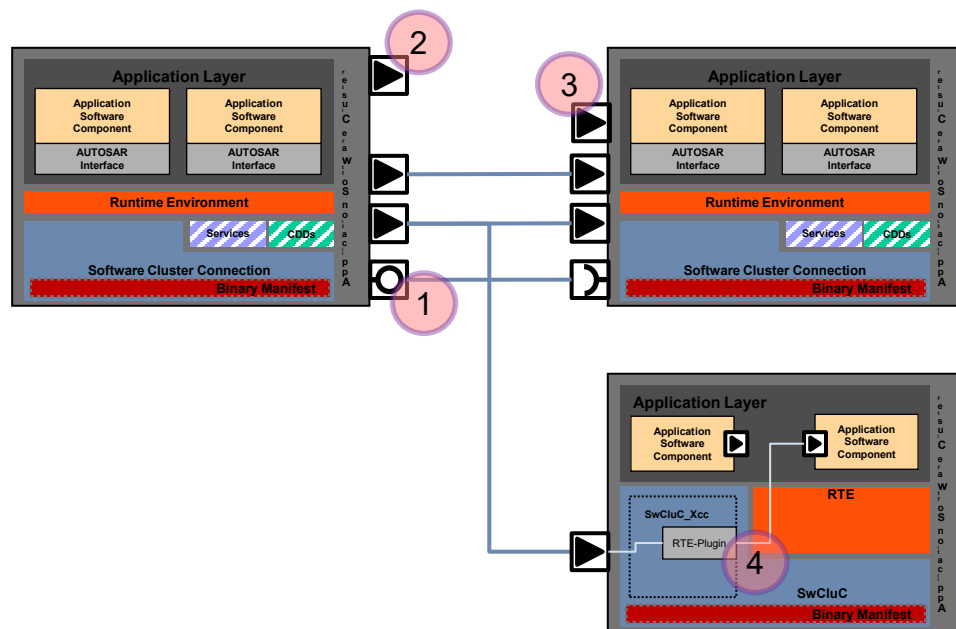
二进制清单 (BManif) 为接口提供二进制元信息，以便能够连接软件集群。

Overview of CP Software Clusters

Software Cluster Connection (2)



- **Software Cluster Connection (SwCluC)** enables a flexible handling of interfaces
 - Interfaces will be connected in a link process, based on Binary Manifest and match of required and provided entries
 - If a match is found the connection is established
 - If no requester is found the interface stays open
 - If no provider is found, the interface stays open, and default values are provided
 - This enables update of Software Clusters with interface changes
- **Cross Software Cluster Communication (SwCluC_Xcc)** implements the communication pattern and the interface to the RTE
 - RTE interface: RIPS-Plugin



软件集群连接 (SwCluC) 实现了接口的灵活处理

- 接口将通过链接过程进行连接，该过程基于二进制清单以及所需条目与提供条目之间的匹配■。若找到匹配项，则建立连接■。若未找到请求方，接口将保持开放状态■。若未找到提供方，接口将保持开放，并提供默认值■。这使得软件集群能够随接口变更而更新

跨软件集群通信

(SwCluC_Xcc) 实现了通信模式以及与RTE的接口

- RTE接口：RIPS-插件

Table of contents

1. Architecture

1. Overview of Software Layers
2. Content of Software Layers
3. Content of Software Layers in Multi-Core Systems
4. Content of Software Layers in Mixed-Critical Systems
5. Overview of Modules
6. Interfaces
 1. General
 2. Interaction of Layers (Examples)

2. Configuration

3. Integration and Runtime Aspects

Table of contents

1. 架构

1. 软件层概述 2. 软件层内容 3. 多核系统中的软件层内容 4. 混合关键系统中的软件层内容 5. 模块概述 6. 接口 1. 概述 2. 层间交互（示例）

2. 配置

3. 集成与运行时方面

Configuration Overview

The AUTOSAR Basic Software supports the following configuration classes:

1. Pre-compile time

- Preprocessor instructions
- Code generation (selection or synthetization)

2. Link time

- Constant data outside the module; the data can be configured after the module has been compiled

3. Post-build time

- Loadable constant data outside the module. Very similar to [2], but the data is located in a specific memory segment that allows reloading (e.g. reflashing in ECU production line)

Independent of the configuration class, single or multiple configuration sets can be provided by means of variation points. In case that multiple configuration sets are provided, the actually used configuration set is to be chosen at runtime in case the variation points are bound at run-time.

In many cases, the configuration parameters of one module will be of different configuration classes. Example: a module providing Post-build time configuration parameters will still have some parameters that are Pre-compile time configurable.

Note: Multiple configuration sets were modeled as a sub class of the Post-build time configuration class up to AUTOSAR 4.1.x.

Configuration Overview

AUTOSAR基础软件支持以下配置类别：

1. 预编译时间

- 预处理器指令 ■ 代码生成（选择或合成）

2. 链接时间

- 模块外部的常量数据；该数据可以在模块编译后进行配置

3. 构建后时间

- 模块外可加载的常量数据。与[2]非常相似，但数据位于允许重新加载的特定内存段中（例如在ECU生产线上重新刷写）

独立于配置类别，可以通过变点提供单个或多个配置集。若提供了多个配置集，当变点在运行时绑定时，实际使用的配置集需在运行时选择。

在许多情况下，一个模块的配置参数会属于不同的配置类别。例如：一个提供构建后时间配置参数的模块，仍可能包含一些支持预编译时间配置参数。

注意：在AUTOSAR 4.1.x版本之前，多个配置集被建模为构建后配置类的一个子类。

Configuration

Pre-compile time (1)

Use cases

Pre-compile time configuration would be chosen for

- Enabling/disabling optional functionality
This allows to exclude parts of the source code that are not needed
- Optimization of performance and code size
Using `#defines` results in most cases in more efficient code than access to constants or even access to constants via pointers.
Generated code avoids code and runtime overhead.

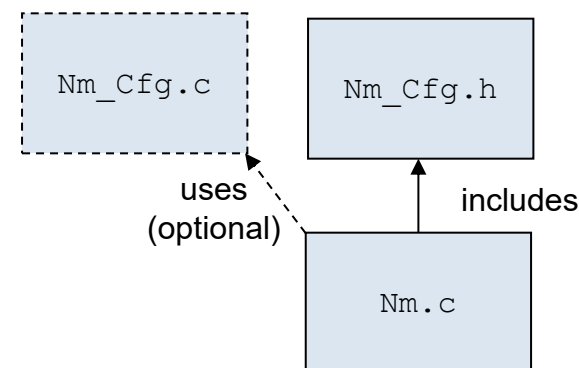
Restrictions

- The module must be available as source code
- The configuration is static and it may consist of one or more configuration sets identified by means of variation points. To update any configuration set (e.g. change the value of certain parameters), the module has to be recompiled.

Required implementation

Pre-compile time configuration shall be done via the module's two configuration files (`*_Cfg.h`, `*_Cfg.c`) and/or by code generation:

- `*_Cfg.h` stores e.g. macros and/or `#defines`
- `*_Cfg.c` stores e.g. constants



Configuration

Pre-compile time (1)

使用案例

预编译时配置将被选择用于

启用/禁用可选功能 这允许排除不需要的源代码部分 优化性能和代码大小 使用 `#defines` 在大多数情况下能生成比访问常量（甚至通过指针访问常量）更高效的代码。生成的代码避免了代码和运行时开销。

限制条件

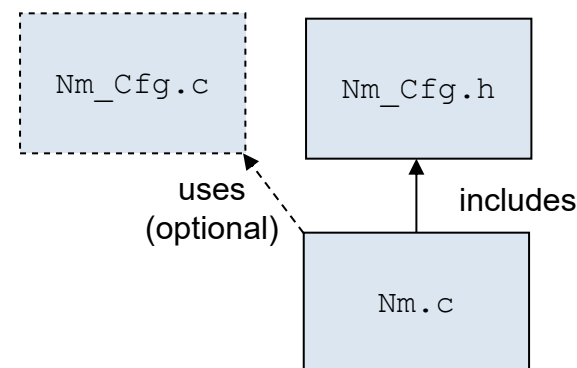
模块必须提供源代码形式。

配置是静态的，它可能包含一个或多个通过变点（variation points）标识的配置集。要更新任何配置集（例如更改某些参数的值），必须重新编译该模块。

所需实现

预编译时配置应通过模块的两个配置文件(*_Cfg.h, *_Cfg.c)和/或代码生成来完成：

- *_Cfg.h 存储例如宏和/或 `#defines`
- *_Cfg.c 存储例如常量



Configuration

Pre-compile time (2)

Example 1: Enabling/disabling functionality

File Spi_Cfg.h:

```
#define SPI_DEV_ERROR_DETECT    ON
```

File Spi_Cfg.c:

```
const uint8 myconstant = 1U;
```

File Spi.c (available as source code):

```
#include "Spi_Cfg.h"          /* for importing the configuration parameters */

extern const uint8 myconstant;

#if (SPI_DEV_ERROR_DETECT == ON)
Det_ReportError(Spi_ModuleId, 0U, 3U, SPI_E_PARAM_LENGTH); /* only one instance available */
#endif
```

Note: The Memory Abstraction (as specified by AUTOSAR) is not used to keep the example simple.

Configuration

Pre-compile time (2)

示例1: 启用/禁用功能 文件 Spi_Cfg.h: #define
SPI_DEV_ERROR_DETECT ON

文件 Spi_Cfg.c:
const uint8 我的常量 = 1U;

文件 Spi.c (提供源代码) : #include "Spi_Cfg.h" /* 用于导入配置参数 */

```
extern const uint8 myconstant;
```

```
#if (SPI_DEV_ERROR_DETECT == ON) Det_ReportError(Spi_ModuleId, 0U, 3U, SPI_E_PARAM_LENGTH); /* 仅有一个实例可用  
*/ #endif
```

Note: The Memory Abstraction (as specified by AUTOSAR) is not used to keep the example simple.

Configuration

Pre-compile time (3)

Example 2: Event IDs reported to the Dem

XML configuration file of the NVRAM Manager:

Specifies that it needs the event symbol `NVM_E_REQ_FAILED` for production error reporting.

File `Dem_Cfg.h` (generated by Dem configuration tool):

```
typedef uint8 Dem_EventIdType; /* total number of events = 46 => uint8 sufficient */

#define DemConf_DemEventParameter_NVM_E_REQ_FAILED      5U
#define DemConf_DemEventParameter_CANSM_E_BUS_OFF      6U
...
```

File `Dem.h`:

```
#include "Dem_Cfg.h" /* for providing access to event symbols */
```

File `NvM.c` (available as source code):

```
#include "Dem.h" /* for reporting production errors */

Dem_SetEventStatus(DemConf_DemEventParameter_NVM_E_REQ_FAILED, DEM_EVENT_STATUS_PASSED);
```

Configuration

Pre-compile time (3)

示例2: 报告给Dem的事件ID

NVRAM管理器的XML配置文件:

指定其需要事件符号NVM_E_REQ_FAILED用于生产错误报告。

文件 Dem_Cfg.h (由 Dem 配置工具生成) :

```
typedef uint8 Dem_EventIdType; /* 事件总数 = 46 => uint8 足够 */
```

```
#define DemConf_DemEventParameter_NVM_E_REQ_FAILED    5U#define DemC  
onf_DemEventParameter_CANSM_E_BUS_OFF    6U...
```

文件 Dem.h:

```
#include "Dem_Cfg.h" /* 用于提供对事件符号的访问 */
```

文件 NvM.c (提供源代码形式) : #include "Dem.h" /* 用于报告生产错误 */

```
Dem_SetEventStatus (DemConf_DemEventParameter_NVM_E_REQ_FAILED, DEM_EVENT_STATUS_PASSED);
```

Configuration

Link time (1)

Use cases

Link time configuration would be chosen for

- Configuration of modules that are only available as object code (e.g. IP protection or warranty reasons)
- Creation of configuration after compilation but before linking.

Required implementation

1. One configuration set, no runtime selection
Configuration data shall be captured in external constants. These external constants are located in a separate file. The module has direct access to these external constants.
2. 2..n configuration sets, runtime selection possible
Configuration data shall be captured within external constant structs. The module gets a pointer to one of those structs at initialization time. The struct can be selected at each initialization.

Configuration

Link time (1)

使用案例

链接时配置将被选择用于

仅以目标代码形式提供的模块配置（例如，出于知识产权保护或保修原因）

编译后但链接前创建配置。

所需实现

1. 一个配置集，无运行时选择 配置数据应通过外部常量获取。这些外部常量位于独立的文件中。模块可直接访问这些外部常量。
2. 2..n 配置集，支持运行时选择 配置数据应在外部常量结构体中捕获。模块在初始化时获取指向其中一个结构体的指针。每次初始化时均可选择不同的结构体。

Configuration

Link time (2)

Example 1: Event IDs reported to the Dem by a multiple instantiated module (example “Foo” module) only available as object code

XML configuration file of the Foo module:

Specifies that it needs the event symbol `FOO_E_WRITE_FAILED` for production error reporting.

File `Dem_Cfg.h` (generated by Dem configuration tool):

```
typedef uint16 Dem_EventIdType; /* total number of events = 380 => uint16 required */

#define DemConf_DemEventParameter_FOO_E_ERASE_FAILED_0    1U
#define DemConf_DemEventParameter_FOO_E_ERASE_FAILED_1    2U
#define DemConf_DemEventParameter_FOO_E_WRITE_FAILED_0    3U
#define DemConf_DemEventParameter_FOO_E_WRITE_FAILED_1    4U
#define DemConf_DemEventParameter_NVM_E_REQ_FAILED        5U
#define DemConf_DemEventParameter_CANSM_E_BUS_OFF        6U
...
```

File `Foo_Lcfg.c`:

```
#include "Dem_Cfg.h" /* for providing access to event symbols */

const Dem_EventIdType Foo_WriteFailed[2] = {DemConf_DemEventParameter_FOO_E_WRITE_FAILED_1,
      DemConf_DemEventParameter_FOO_E_WRITE_FAILED_2};
```

File `Foo.c` (available as object code):

```
#include "Dem.h" /* for reporting production errors */
extern const Dem_EventIdType Foo_WriteFailed[];

Dem_SetEventStatus(Foo_WriteFailed[instance], DEM_EVENT_STATUS_FAILED);
```

Note: the complete include file structure with all forward declarations is not shown here to keep the example simple.

Configuration

Link time (2)

示例1：报告给演示的实体ID仅作为对象代码可用，例如“Foo”模块，这是一个多次实例化的模块。

Foo模块的XML配置文件：指定其需要事件符号FOO_E_WRITE_FAILED用于生产错误报告。

文件 Dem_Cfg.h（由 Dem 配置工具生成）：

```
typedef uint16 Dem_EventIdType; /* 事件总数 = 380 => uint16 必需 */
```

```
#define DemConf_DemEventParameter_FOO_E_ERASE_FAILED_0 1U#define DemConf_DemEventParameter_FOO_E_ERASE_FAILED_1 2U#define DemConf_DemEventParameter_FOO_E_WRITE_FAILED_0 3U#define DemConf_DemEventParameter_FOO_E_WRITE_FAILED_1 4U#define DemConf_DemEventParameter_NVM_EREQ_FAILED 5U#define DemConf_DemEventParameter_CANSME_BUS_OFF 6U
```

...

文件 Foo_Lcfg.c：

```
#include "Dem_Cfg.h" /* 用于提供对事件符号的访问 */
```

```
const Dem_EventIdType Foo_WriteFailed[2] = {DemConf_DemEventParameter_FOO_E_WRITE_FAILED_1, DemConf_DemEventParameter_FOO_E_WRITE_FAILED_2};
```

文件 Foo.c（以目标代码形式提供）：

```
#include "Dem.h" /* 用于报告生产错误 */ /*  
extern const Dem_EventIdType Foo_WriteFailed[];
```

```
Dem_SetEventStatus(Foo_WriteFailed[instance], DEM_EVENT_STATUS_FAILED);
```

注意：为了保持示例简洁，此处未展示包含所有前向声明的完整头文件结构。

Configuration

Link time (3)

Example 2: Event IDs reported to the Dem by a module (Flash Driver) that is available as object code only

Problem

`Dem_EventIdType` is also generated depending of the total number of event IDs on this ECU. In this example it is represented as `uint16`. The Flash Driver uses this type, but is only available as object code.

Solution

In the contract phase of the ECU development, a bunch of variable types (including `Dem_EventIdType`) have to be fixed and distributed for each ECU. The object code suppliers have to use those types for their compilation and deliver the object code using the correct types.

Configuration

链接时间 (3)

示例2：由仅以目标代码形式提供的模块（Flash Driver）报告给Dem的事件ID

问题

Dem_EventIdType 也根据该ECU上事件ID的总数生成。在此示例中，它表示为 uint16。Flash Driver 使用此类型，但仅以目标代码形式提供。

解决方案

在在ECU开发的合同阶段，必须为每个ECU确定并分配一系列变量类型（包括Dem_EventIdType）。目标代码供应商必须使用这些类型进行编译，并交付使用正确类型的对象代码。

Configuration

Post-build time (1)

Use cases

Post-build time configuration would be chosen for

- Configuration of data where only the structure is defined but the contents not known during ECU-build time
- Configuration of data that is likely to change or has to be adapted after ECU-build time (e.g. end of line, during test & calibration)
- Reusability of ECUs across different car versions (same application, different configuration), e.g. ECU in a low-cost car version may transmit less signals on the bus than the same ECU in a luxury car version.

Restrictions

- Implementation requires storing all possibly relevant configuration items in a flash able area and requires pointer dereferencing upon config access. Implementation precludes generation of code, which has impact on performance, code and data size.

Required implementation

1. One configuration set, no runtime selection
Configuration data shall be captured in external constant structs. These external structs are located in a separate memory segment that can be individually reloaded. The module gets a pointer to a base struct at initialization time.
2. 2..n configuration sets, runtime selection possible
Configuration data shall be captured within external constant structs. These external structs are located in a separate memory segment that can be individually reloaded. The module gets a pointer to one of several base structs at initialization time. The struct can be selected at each initialization.

Configuration

Post-build time (1)

使用案例

构建后时间配置将被选择用于

数据配置，其中仅定义了结构，但在ECU构建时内容未知

可能在ECU构建后需要更改或调整的数据配置（例如，生产线末端、测试与标定期间）

ECU在不同车型版本间的可重用性（相同应用，不同配置），例如低成本车型中的ECU在总线上传输的信号可能比豪华车型中的同一ECU更少。

限制条件

实现需要在可擦写区域存储所有可能相关的配置项，并在访问配置时进行指针解引用。该实现方式排除了代码生成，这对性能、代码及数据大小均有影响。

所需实现

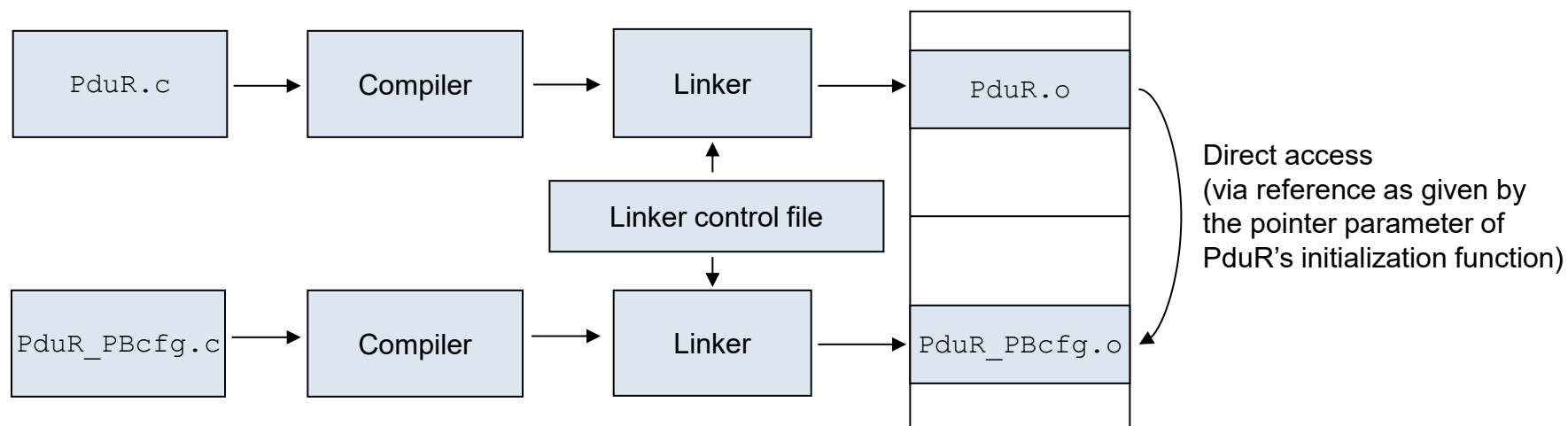
1. 一个配置集，无运行时选择 配置数据应存储在外部常量结构体中。这些外部结构体位于可单独重载的独立内存段中。模块在初始化时获取指向基础结构体的指针。
2. 2..n 配置集，支持运行时选择 配置数据应在外部常量结构体中捕获。这些外部结构体位于独立的内存段中，可单独重新加载。模块在初始化时获取指向多个基础结构体之一的指针。每次初始化时均可选择不同的结构体。

Configuration

Post-build time (2)

Example 1

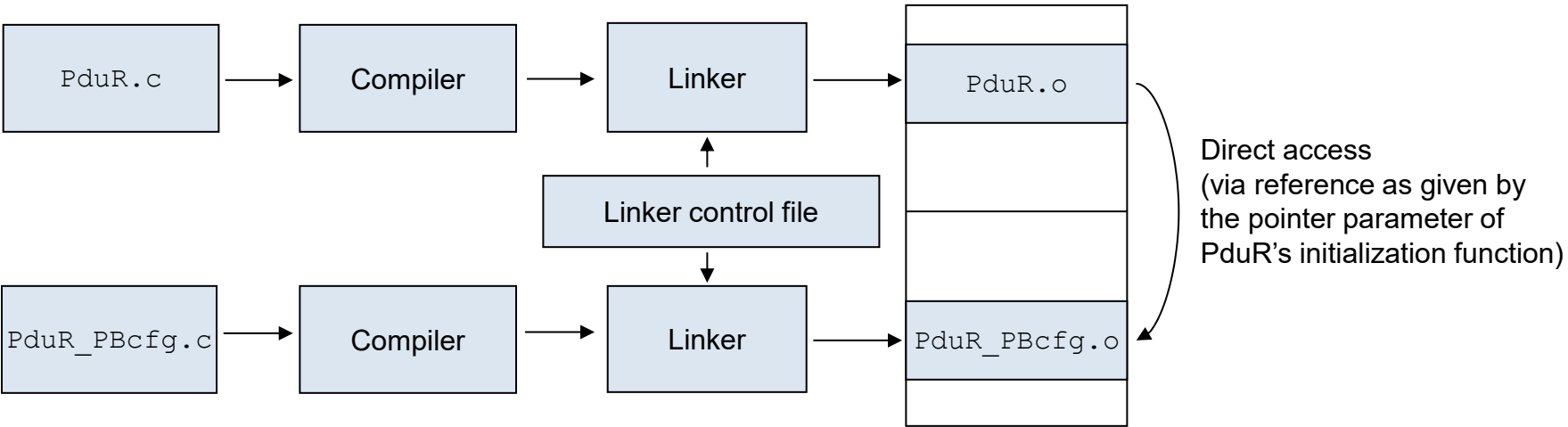
If the configuration data is fix in memory size and position, the module has direct access to these external structs.



Configuration

构建后时间 (2)

示例 1
 如果配置数据在内存大小和位置上固定，模块可以直接访问这些外部结构。



Configuration

Post-build time (3)

Required implementation 2: Configuration of CAN Driver that is available as object code only; a configuration set can be selected out of multiple configuration sets during initialization time.

File Can_PBcfg.c:

```
#include "Can.h" /* for getting Can_ConfigType */
const Can_ConfigType MySimpleCanConfig [2] =
{
    {
        Can_BitTiming      = 0xDF,
        Can_AcceptanceMask1 = 0xFFFFFFFF,
        Can_AcceptanceMask2 = 0xFFFFFFFF,
        Can_AcceptanceMask3 = 0x00034DFF,
        Can_AcceptanceMask4 = 0x00FF0000
    },
    { ... }
};
```

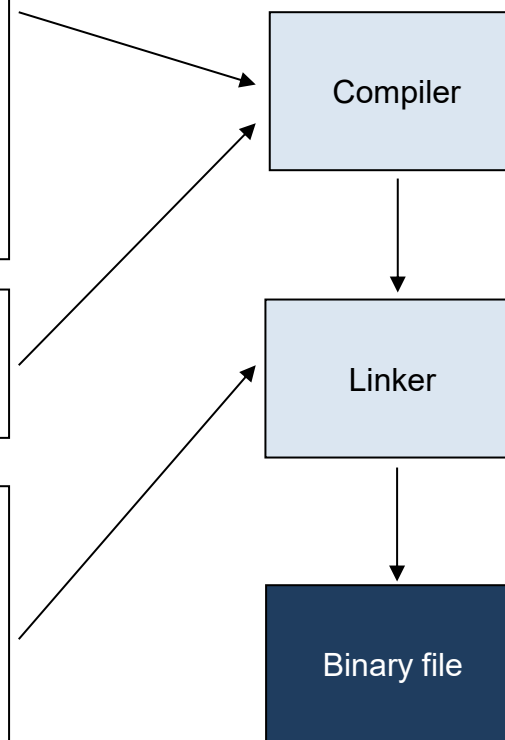
File EcuM.c:

```
#include "Can.h" /* for initializing the CAN Driver */
Can_Init(&MySimpleCanConfig[0]);
```

File Can.c (available as object code):

```
#include "Can.h" /* for getting Can_ConfigType */

void Can_Init(Can_ConfigType* Config)
{
    /* write the init data to the CAN HW */
};
```



Configuration

构建后时间 (3)

必需实现2: 仅以目标代码形式提供的CAN驱动程序的配置；在初始化期间可以从多个配置集中选择一个配置集。

File Can_PBcfg.c:

```
#include "Can.h" /* for getting Can_ConfigType */
const Can_ConfigType MySimpleCanConfig [2] =
{
    {
        Can_BitTiming      = 0xDF,
        Can_AcceptanceMask1 = 0xFFFFFFFF,
        Can_AcceptanceMask2 = 0xFFFFFFFF,
        Can_AcceptanceMask3 = 0x00034DFF,
        Can_AcceptanceMask4 = 0x00FF0000
    },
    { ... }
};
```

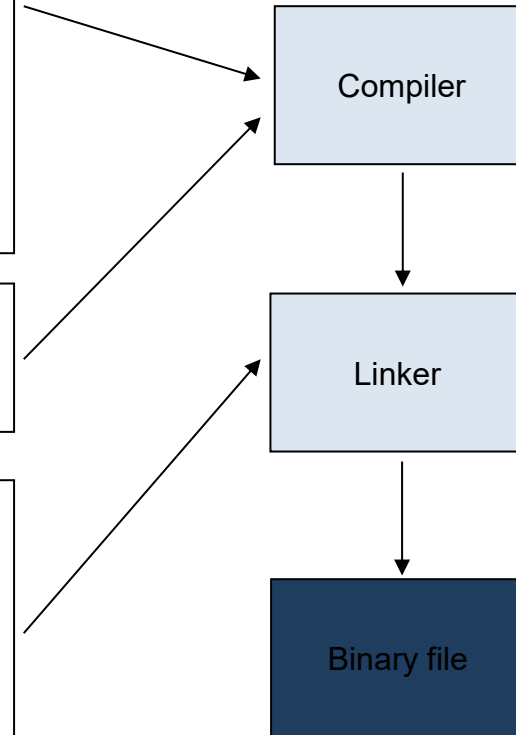
File EcuM.c:

```
#include "Can.h" /* for initializing the CAN Driver */
Can_Init(&MySimpleCanConfig[0]);
```

File Can.c (available as object code):

```
#include "Can.h" /* for getting Can_ConfigType */

void Can_Init(Can_ConfigType* Config)
{
    /* write the init data to the CAN HW */
};
```



Configuration Variants

Different use cases require different kinds of configurability. Therefore the following configuration variants are provided:

- VARIANT-PRE-COMPILE

Only parameters with "Pre-compile time" configuration are allowed in this variant.

- VARIANT-LINK-TIME

Only parameters with "Pre-compile time" and "Link time" are allowed in this variant.

- VARIANT-POST-BUILD

Parameters with "Pre-compile time", "Link time" and "Post-build time" are allowed in this variant.

Example use cases:

- Reprogrammable PDU routing tables in gateway (Post-build time configurable PDU Router required)
- Statically configured PDU routing with no overhead (Pre-compile time configuration of PDU Router required)

To allow the implementation of such different use cases in each BSW module, up to 3 variants can be specified:

- A variant is a dedicated assignment of the configuration parameters of a module to configuration classes
- Within a variant a configuration parameter can be assigned to only ONE configuration class
- Within a variant a configuration class for different configuration parameters can be different (e.g. Pre-Compile for development error detection and post-build for reprogrammable PDU routing tables)
- It is possible and intended that specific configuration parameters are assigned to the same configuration class for all variants (e.g. development error detection is in general Pre-compile time configurable).

Configuration

变体

不同的使用场景需要不同种类的可配置性。因此，以下配置变体是

提供：
变体-预编译 此变体中仅允许使用具有“预编译时”配置的参数。
变体-链接时 此变体中仅允许使用具有“预编译时”和“链接时”的参数。
变体-构建后 此变体中允许使用具有“预编译时”、“链接时”和“构建后时”的参数。

示例用例：

网关中可重新编程的PDU路由表（需支持构建后配置的PDU路由器） 静态配置的PDU路由，无额外开销（需支持预编译时配置的PDU路由器）

为了在每个BSW模块中实现这些不同的用例，最多可以指定3种变体：

变体是指将模块的配置参数专门分配给配置类的一种特定分配方式。在一个变体中，一个配置参数只能分配给一个配置类。在一个变体中，不同配置参数的配置类可以不同（例如：开发错误检测采用预编译配置，而可重编程PDU路由表采用构建后配置）。某些特定配置参数在所有变体中均可分配至相同的配置类，这是可能且有意设计的（例如：开发错误检测通常可在预编译时配置）。

Configuration

Memory Layout Example: Post-build configuration

EcuM defines the index:

0x8000	&index (=0x8000)
0x8000	&xx_configuration = 0x4710
0x8002	&yy_configuration = 0x4720
0x8004	&zz_configuration = 0x4730
...	

Xx defines the modules configuration data:

0x4710	&the_real_xx_configuration
0x4710	lower = 2
0x4712	upper =7
0x4714	more_data
...	

Yy defines the modules configuration data:

0x4720	&the_real_yy_configuration
0x4720	Xx_data1=0815
0x4722	Yy_data2=4711
0x4724	more_data
...	

Description where to find what is an overall agreement:

1. EcuM needs to know all addresses including index
2. The modules (xx, yy, zz) need to know their own start address: in this case: 0x4710, 0x4720 ...
3. The start addresses might be dynamic i.e. changes with new configuration
4. When initializing a module (e.g. xx, yy, zz), EcuM passes the base address of the configuration data (e.g. 0x4710, 0x4720, 0x4730) to the module to allow for variable sizes of the configuration data.

The module data is agreed locally (in the module) only

1. The module (xx, yy) knows its own start address (to enable the implementer to allocate data section)
2. Only the module (xx, yy) knows the internals of its own configuration

Configuration

内存布局示例：构建后配置

EcuM 定义了索引：

0x8000	&index (=0x8000)
0x8000	&xx_configuration = 0x4710
0x8002	&yy_configuration = 0x4720
0x8004	&zz_configuration = 0x4730
...	

Xx定义了模块的配置数据：

0x4710	&the_real_xx_configuration
0x4710	lower = 2
0x4712	upper =7
0x4714	more_data
...	

Yy定义了模块的配置数据：

0x4720	&the_real_yy_configuration
0x4720	Xx_data1=0815
0x4722	Yy_data2=4711
0x4724	more_data
...	

描述在哪里找到总体协议的内容：

1. EcuM需要知道包括索引在内的所有地址。
2. 模块（xx、yy、zz）需要知道各自的起始地址：在本例中为0x4710、0x4720……
3. 起始地址可能是动态的，即会随新配置而变化。
4. 初始化模块（例如xx、yy、zz）时，EcuM会将配置数据的基地址（例如0x4710、0x4720、0x4730）传递给模块，以适应配置数据大小的变化。

模块数据仅在本地（模块内）达成一致

1. 模块（xx,yy）知晓自身的起始地址（以便实现者能够分配数据段）
2. 仅模块（xx,yy）知晓其自身配置的内部细节

Configuration

Memory Layout Example: Multiple configuration sets

FL	0x8000	&index[] (=0x8000)
	0x8000	&xx_configuration = 0x4710
	0x8002	&yy_configuration = 0x4720
	0x8004	&zz_configuration = 0x4730
FR	...	
	0x8008	&xx_configuration = 0x5000
	0x800a	&yy_configuration = 0x5400
	0x800c	&zz_configuration = 0x5200
RL	...	
	0x8010	&xx_configuration = ...
	0x8012	&yy_configuration = ...
	0x8014	&zz_configuration = ...
	...	

As before, the description where to find what is an overall agreement

1. The index contains more than one description (FL, FR,..) in an array
(here the size of an array element is agreed to be 8)
2. There is an agreed variable containing the position of one description
selector = CheckPinCombination()
3. Instead of passing the pointer directly there is one indirection:
(struct EcuM_ConfigType *) &index[selector];
4. Everything else works as in conventional single configuration case.

0x8002 &yy配置 = 0x4720 0x8004 &zz配置 = 0x4730 ... 0x8008
&xx配置 = 0x5000 0x800a &yy配置 = 0x5400 0x800c &zz配置
= 0x5200 ... 0x8010 &xx配置 = ... 0x8012 &yy配置 = ... 0x801
4 &zz配置 = FL FBRL

1. 索引中包含多个描述 (FL、FR等) 组成的数组 (此处约定每个数组元素的大小为8)
2. 存在一个约定变量, 用于存储描述选择器 = CheckPinCombination() 的位置
3. 不直接传递指针, 而是通过一次间接引用: (struct Ec uM_ConfigType *) &index[selector];
4. 其余机制与传统单一配置情况相同。

Table of contents

1. Architecture
2. Configuration
3. **Integration and Runtime Aspects**
 1. **Mapping of Runnables**
 2. Partitioning
 3. Scheduling
 4. Mode Management
 5. Error Handling, Reporting and Diagnostic
 6. Measurement and Calibration
 7. Security
 8. Energy Management
 9. Global Time Synchronization

Table of contents

1. 架构 2. 配置

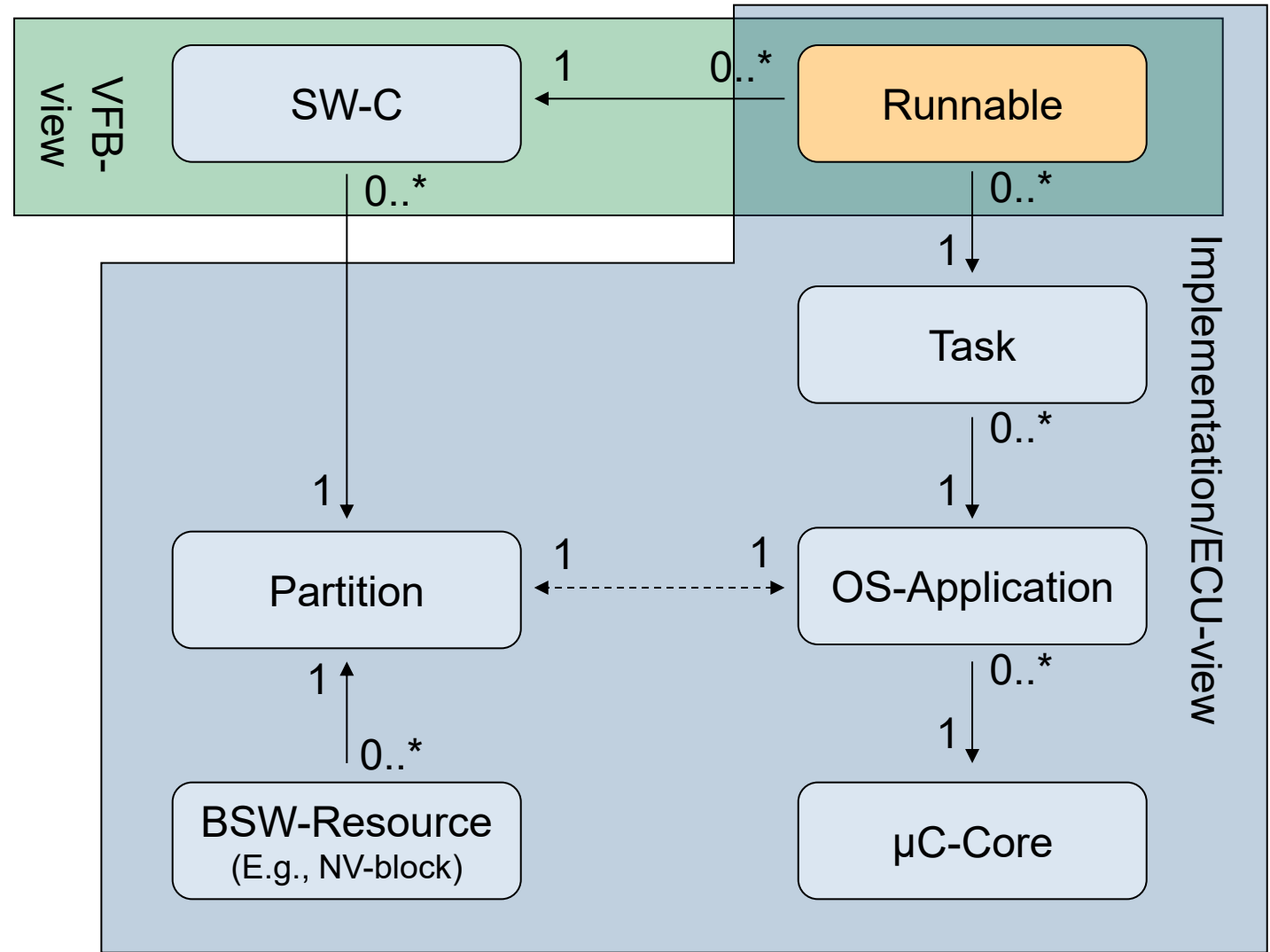
3. 集成与运行时方面

1. 可运行实体的映射 2. 分区管理 3. 调度机制 4. 模式管理 5. 错误处理、报告与诊断 6. 测量与标定 7. 安全机制 8. 能源管理 9. 全局时间同步

Integration and Runtime Aspects

Mapping of Runnables

- Runnables are the active parts of Software Components
- They can be executed concurrently, by mapping them to different Tasks.
- The figure shows further entities like OS-applications, Partitions, μ C-Cores and BSW-Resources which have to be considered for this mapping.



Integration and Runtime Aspects

可运行对象的映射

可运行实体是软件组件的活动部分。它们可通过映射到不同任务实现并发执行。图示进一步展示了在此映射过程中需考虑的其他实体，如操作系统应用、分区、微控制器内核及基础软件资源。

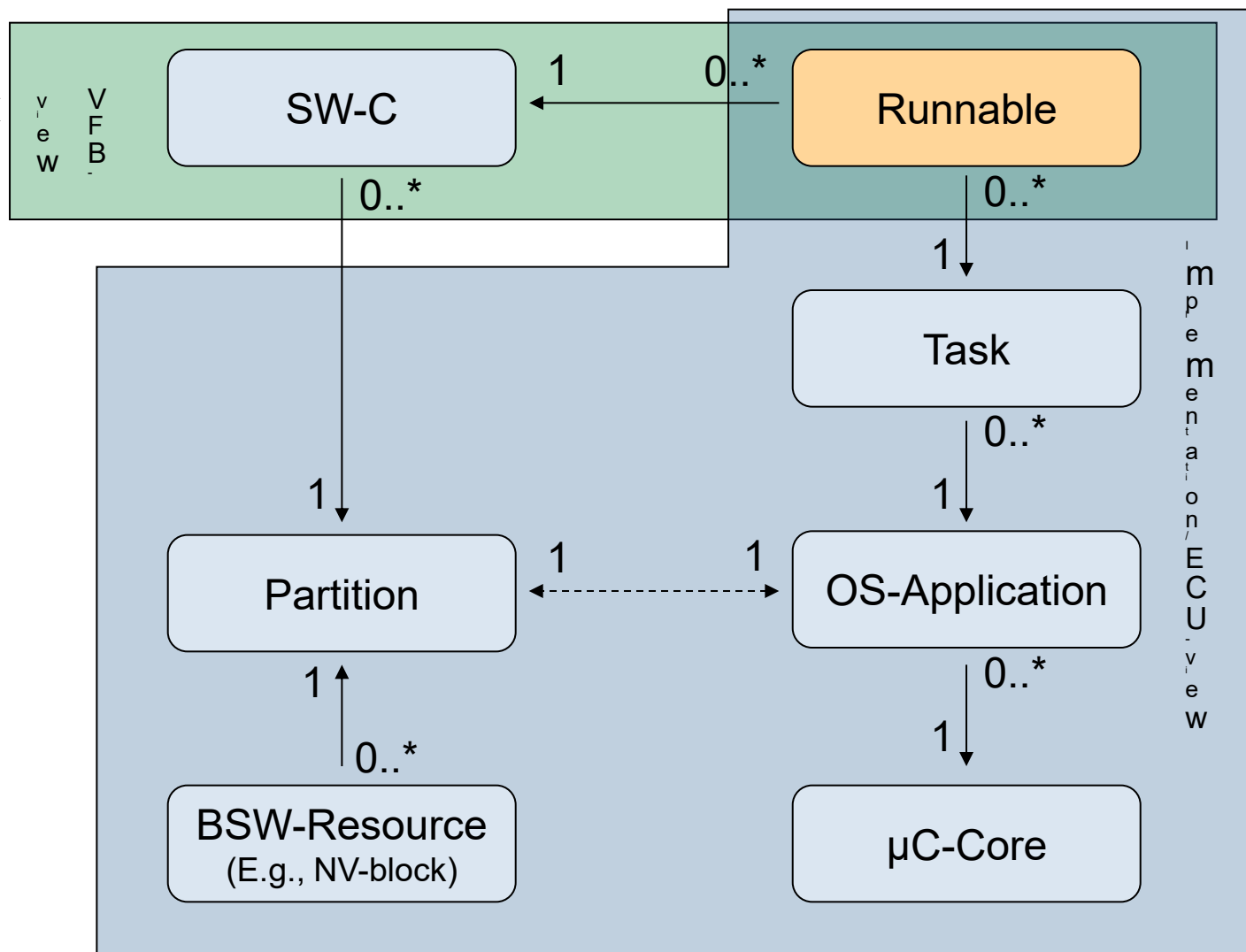


Table of contents

1. Architecture
2. Configuration
3. **Integration and Runtime Aspects**
 1. Mapping of Runnables
 2. **Partitioning**
 3. Scheduling
 4. Mode Management
 5. Error Handling, Reporting and Diagnostic
 6. Measurement and Calibration
 7. Security
 8. Energy Management
 9. Global Time Synchronization

Table of contents

1. 架构 2. 配置

3. 集成与运行时方面

1. 可运行实体的映射2. 分区管理3. 调度机制4.
模式管理5. 错误处理、报告与诊断6. 测量与
标定7. 安全机制8. 能源管理9. 全局时间同步

Integration and Runtime Aspects - Partitioning

Introduction

- Partitioning is implemented by using OS-Applications within the OS
- OS-Applications are used as error containment regions:
 - Permit logical grouping of SW-Cs and resources
 - Recovery policies defined individually for each OS-Application
- OS-Application consistency is ensured by the system/platform, for instance for:
 - Memory access violation
 - Time budget violation
- OS-Applications can be terminated during run-time as a result of a detected error:
 - All BSW modules are placed in privileged OS-Applications
 - These OS-Applications should not be terminated
- OS-Applications are configured in the ECU configuration:
 - SW-Cs are mapped to OS-Applications (Consequence: restricts runnable to task mapping)
- Communication across OS-Application boundaries is realized by the IOC

分区是通过在操作系统内使用OS-Applications来实现的。

操作系统应用程序被用作错误隔离区域： ■ 允许对软件组件和资源进行逻辑分组 ■ 为每个操作系统应用程序单独定义恢复策略

OS-应用一致性由系统/平台确保，例如针对： ■ 内存访问违规 ■ 时间预算违规

OS-应用程序在运行时可能因检测到错误而被终止：

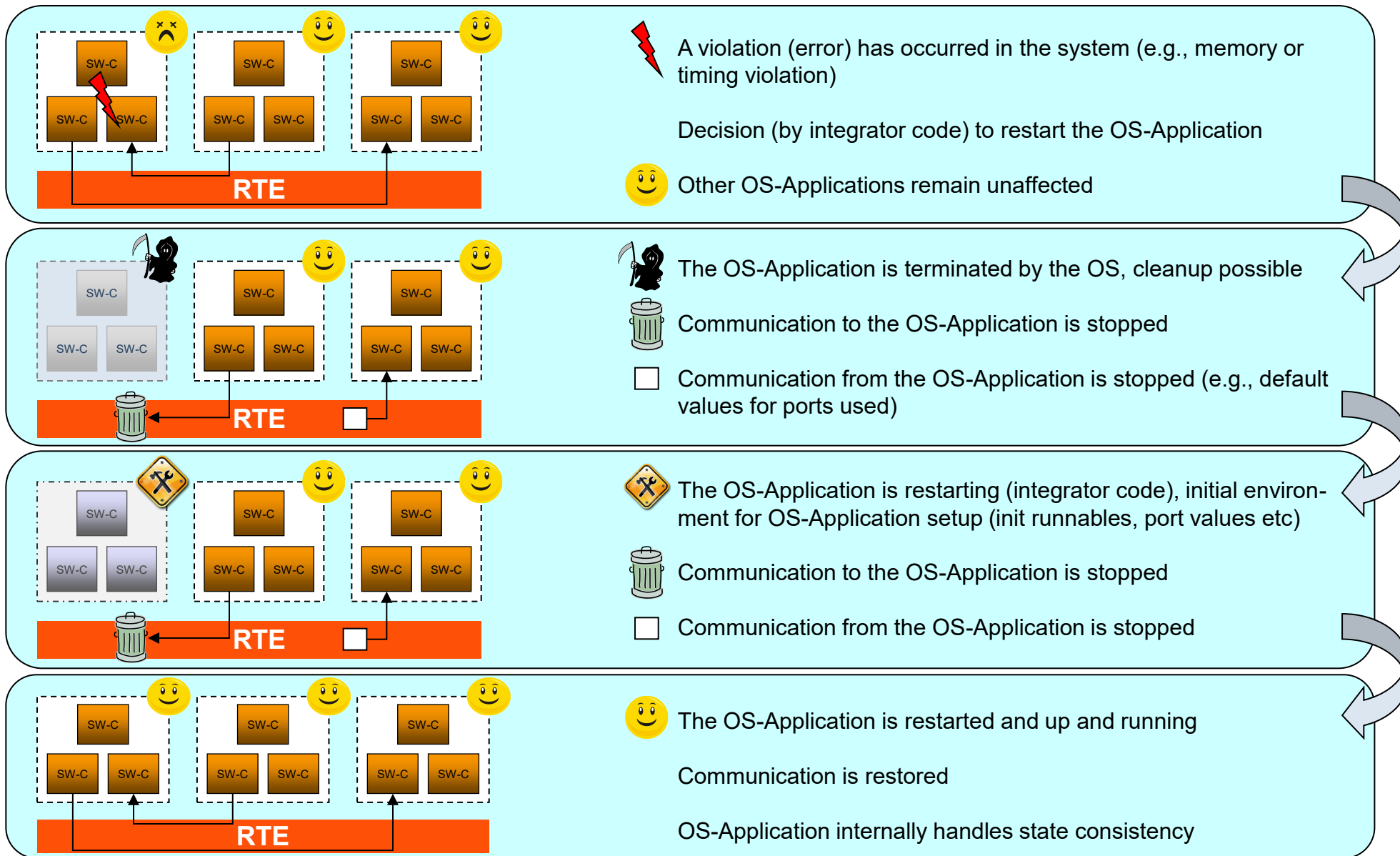
- 所有BSW模块都放置在特权操作系统应用程序■中，这些操作系统应用程序不应被终止。

OS-应用程序在ECU配置中配置： ■ 个SW-C被映射到OS-应用程序（结果：限制了可运行实体到任务的映射）

跨操作系统-应用程序边界的通信通过IOC实现

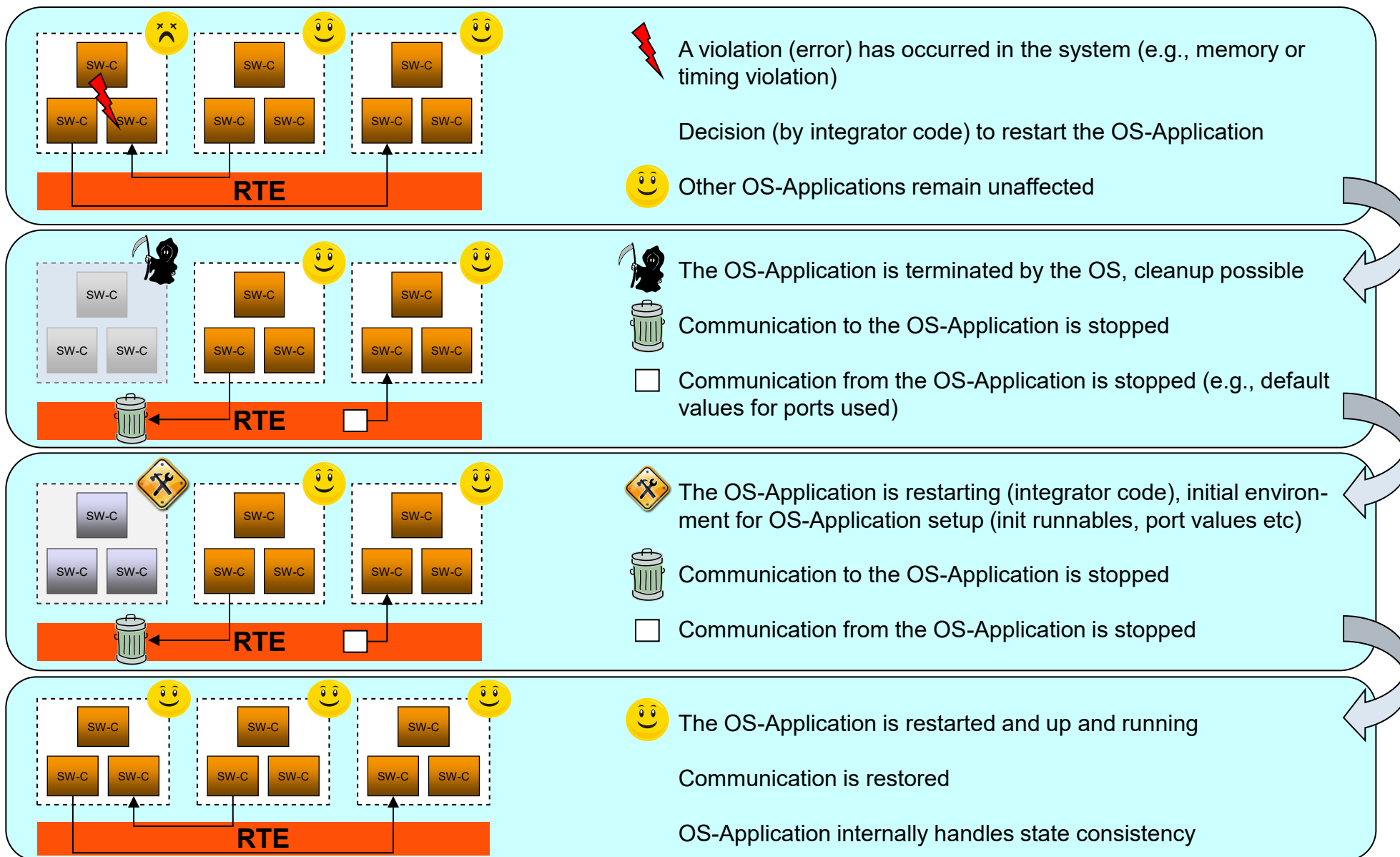
Integration and Runtime Aspects - Partitioning

Example of restarting OS-Application



Integration and Runtime Aspects - Partitioning

Example of restarting OS-Application



Integration and Runtime Aspects - Partitioning Involved components

- Protection Hook
 - Executed on protection violation (memory or timing)
 - Decides what the action is (Terminate, Restart, Shutdown, Nothing)
 - Provided by integrator
 - OS acts on decision by inspecting return value
- OsRestartTask
 - Started by OS in case Protection Hook returns Restart
 - Provided by integrator
 - Runs in the OS-Application's context and initiates necessary cleanup and restart activities, such as:
 - Stopping communication (ComM)
 - Updating NvM
 - Informing Watchdog, CDDs etc.
- RTE
 - Functions for performing cleanup and restart of RTE in OS-Application
 - Triggers init runnables for restarted OS-Application
 - Handles communication consistency for restarting/terminated OS-Applications
- Operating System
 - OS-Applications have states (APPLICATION_ACCESSIBLE, APPLICATION_RESTART, APPLICATION_TERMINATED)
 - OS provides API to terminate other OS-Applications (for other errors than memory/timing)

Integration and Runtime Aspects - Partitioning

Involved components

保护钩

- 在保护违规（内存或时序）时执行
- 决定采取何种行动（终止、重启、关机、不操作）
- 由集成商提供
- 操作系统通过检查返回值来执行决策

OsRestartTask

- 由操作系统启动，以防保护钩返回重启
- 由集成商提供
- 在操作系统应用程序的上下文中运行，并启动必要的清理和重启活动，例如：
 - 停止通信（ComM）
 - 更新NvM
 - 通知看门狗、CDD等。

实时评估

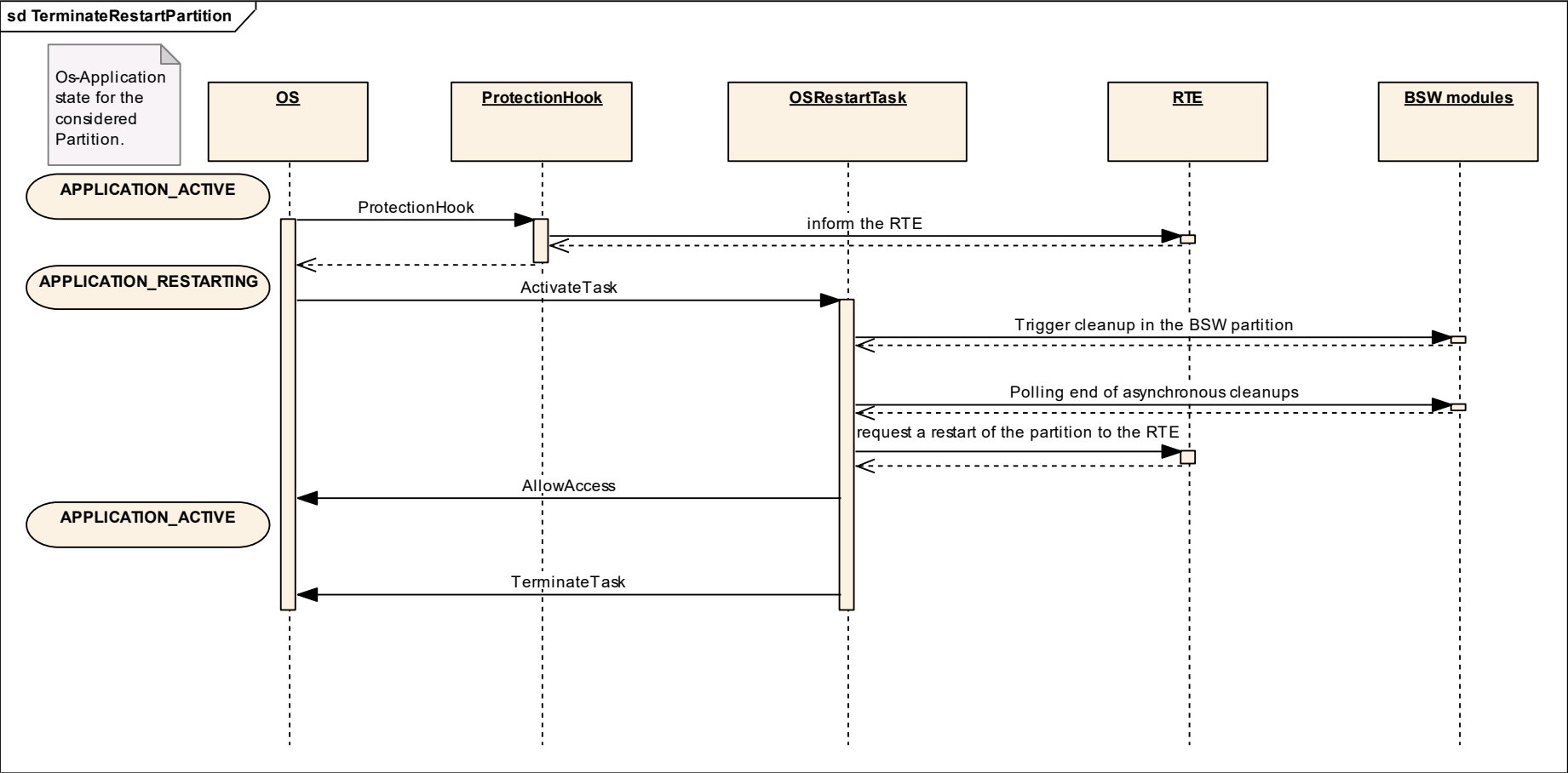
- 用于执行OS-Application中RTE清理和重启的函数
- 为重启的OS-Application触发初始化可运行实体
- 处理重启/终止的OS-Applications的通信一致性

操作系统

- OS-应用程序具有状态（可访问、重启中、已终止）
- OS提供API以终止其他OS-应用程序（适用于内存/时序之外的其他错误）

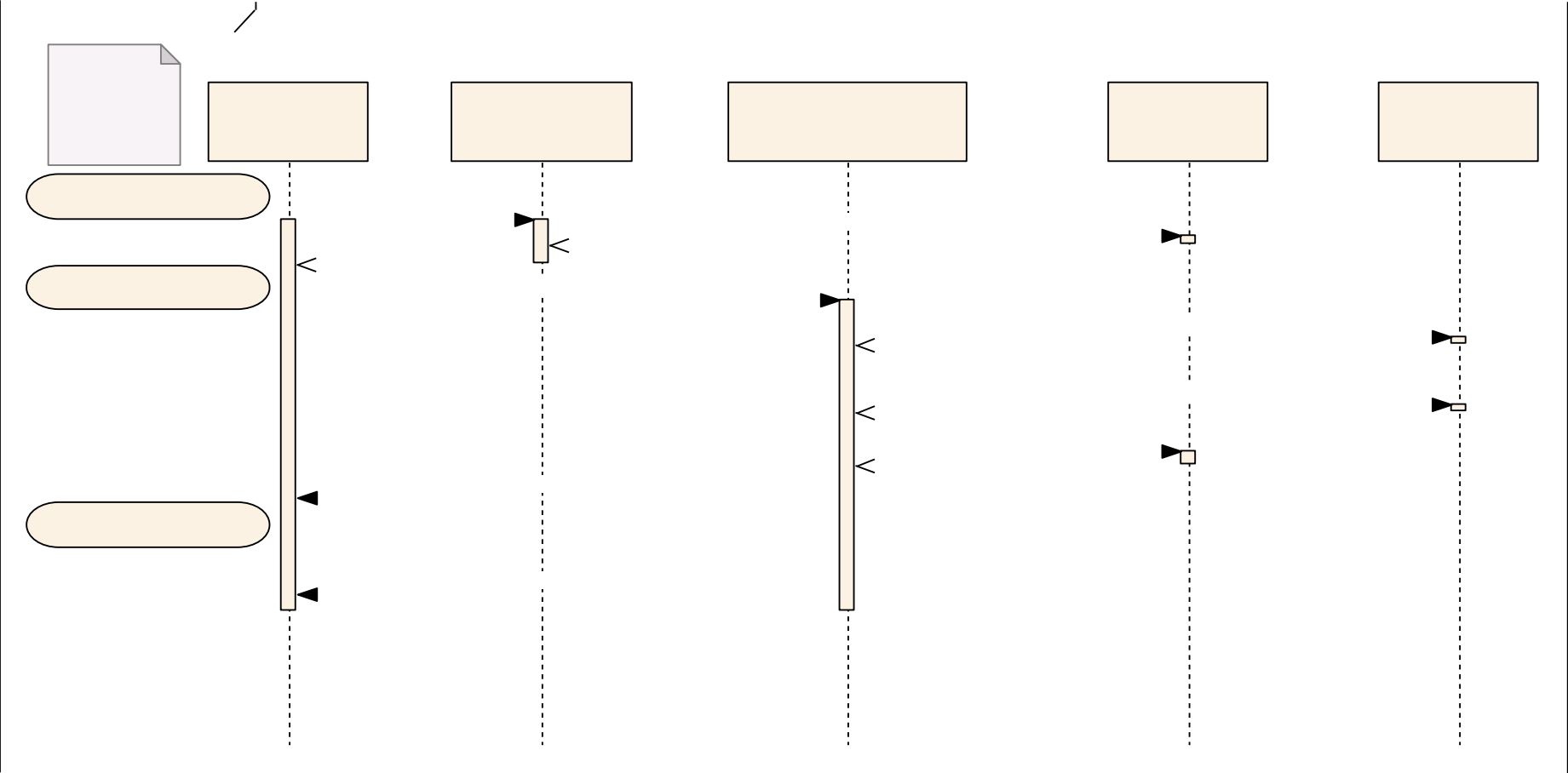
Integration and Runtime Aspects - Partitioning

Restart example



Integration and Runtime Aspects - Partitioning

Restart example



Integration and Runtime Aspects - Partitioning

Other examples

- Termination
 - An OS-Application can be terminated directly
 - Also for termination, some cleanup may be needed, and this shall be performed in the same way as when restarting an OS-Application

- Error detection in applications
 - SW-Cs may require restart for other reasons than memory or timing violation
 - A termination/restart can be triggered from a SW-C using the OS service `TerminateApplication()`
 - Example: a distributed application requires restart on multiple ECUs

终止

- 操作系统应用程序可以直接终止 ■。同样，在终止时可能需要进行一些清理工作，且该过程应与重启操作系统应用程序时采用相同方式执行。

应用程序中的错误检测

- SW-C可能因内存或时序违规之外的其他原因需要重启 ■ 终止/重启可通过SW-C调用操作系统服务TerminateApplication()来触发 ■ 示例：分布式应用需要在多个ECU上重启

Table of contents

1. Architecture
2. Configuration
3. **Integration and Runtime Aspects**
 1. Mapping of Runnables
 2. Partitioning
 3. **Scheduling**
 4. Mode Management
 5. Error Handling, Reporting and Diagnostic
 6. Measurement and Calibration
 7. Security
 8. Energy Management
 9. Global Time Synchronization

1. 架构 2. 配置

3. 集成与运行时方面

1. 可运行实体的映射 2. 分区管理 3. 调度机制 4. 模式管理 5. 错误处理、报告与诊断 6. 测量与标定 7. 安全机制 8. 能源管理 9. 全局时间同步

Integration and Runtime Aspects - Scheduling

General Architectural Aspects

- Basic Software Scheduler and the RTE are generated together.
- This enables
 - that the same OS Task schedules BSW Main Functions and Runnable Entities of Software Components
 - to optimize the resource consumption
 - to configure interlaced execution sequences of Runnable Entities and BSW Main functions.
 - a coordinated switching of a Mode affecting BSW Modules and Application Software Components
 - the synchronized triggering of both, Runnable Entities and BSW Main Functions by the same External Trigger Occurred Event.

基础软件调度器与RTE一同生成。这使得 ■ 同一操作系统任务能够调度基础软件主功能与软件组件的可运行实体 ■ 以优化资源消耗 ■ 配置可运行实体与基础软件主功能的交错执行序列。

- 一种影响BSW模块和应用软件组件的模式协调切换
- 由同一外部触发发生事件同步触发可运行实体和BSW主函数。

Integration and Runtime Aspects - Scheduling

Basic Scheduling Concepts of the BSW

BSW Scheduling shall

- Assure correct timing behavior of the BSW, i.e., correct interaction of all BSW modules with respect to time

Data consistency mechanisms

- Applied data consistency mechanisms shall be **configured by the ECU/BSW integrator dependent from the configured scheduling.**

Single BSW modules do not know about

- ECU wide timing dependencies
- Scheduling implications
- Most efficient way to implement data consistency

Centralize the BSW schedule in the BSW Scheduler configured by the ECU/BSW integrator and generated by the RTE generator together with the RTE

- Eases the integration task
- Enables applying different scheduling strategies to schedulable objects
 - Preemptive, non-preemptive, ...
- Enables applying different data consistency mechanisms
- Enables reducing resources (e.g., minimize the number of tasks)
- Enables interlaced execution sequences of Runnable Entities and BSW Main functions

Restrict the usage of OS functionality

- Only the BSW Scheduler and the RTE shall use OS objects or OS services (exceptions: EcuM, Complex Drivers and services: `GetCounterValue` and `GetElapsedCounterValue` of OS; MCAL modules may enable/disable interrupts)
- Rationale:
 - Scheduling of the BSW shall be transparent to the system (integrator)
 - Enables reducing the usage of OS resources (Tasks, Resources,...)
 - Enables re-using modules in different environments

Integration and Runtime Aspects - Scheduling

Basic Scheduling Concepts of the BSW

BSW调度应

确保BSW的正确时序行为，即所有BSW模块在时间上的正确交互

数据一致性机制

应用的数据一致性机制应由ECU/BSW集成商根据配置的调度策略进行配置。

单个BSW模块不了解

ECU范围内的时序依赖关系 调度影响 实现数

据一致性的最高效方式

将BSW调度集中在由ECU/BSW集成商配置并由RTE生成器与RTE共同生成的BSW调度器中。

简化集成任务 支持对可调度对象 ■ 应用不同的调度策略（抢占式、非抢占式等）

支持应用不同的数据一致性机制 支持减少资源占用（例如最小化任务数量） 支持可
运行实体与基础软件主函数的交错执行序列

限制操作系统功能的使用

只有BSW调度器和RTE可以使用OS对象或OS服务（例外情况：EcuM、复杂驱动程序以及OS的GetCounterValue和GetElapsed
CounterValue服务；MCAL模块可以启用/禁用中断） 设计依据：

- BSW的调度应对系统（集成商）透明 ■ 能够减少操作系统资源（任务、资源等）的使用 ■ 支持在不同环境中复用模块

Integration and Runtime Aspects - Scheduling

Scheduling Objects, Triggers and Mode Disabling Dependencies

BSW Scheduling objects

- Main functions
 - n per module
 - located in all layers

BSW Events

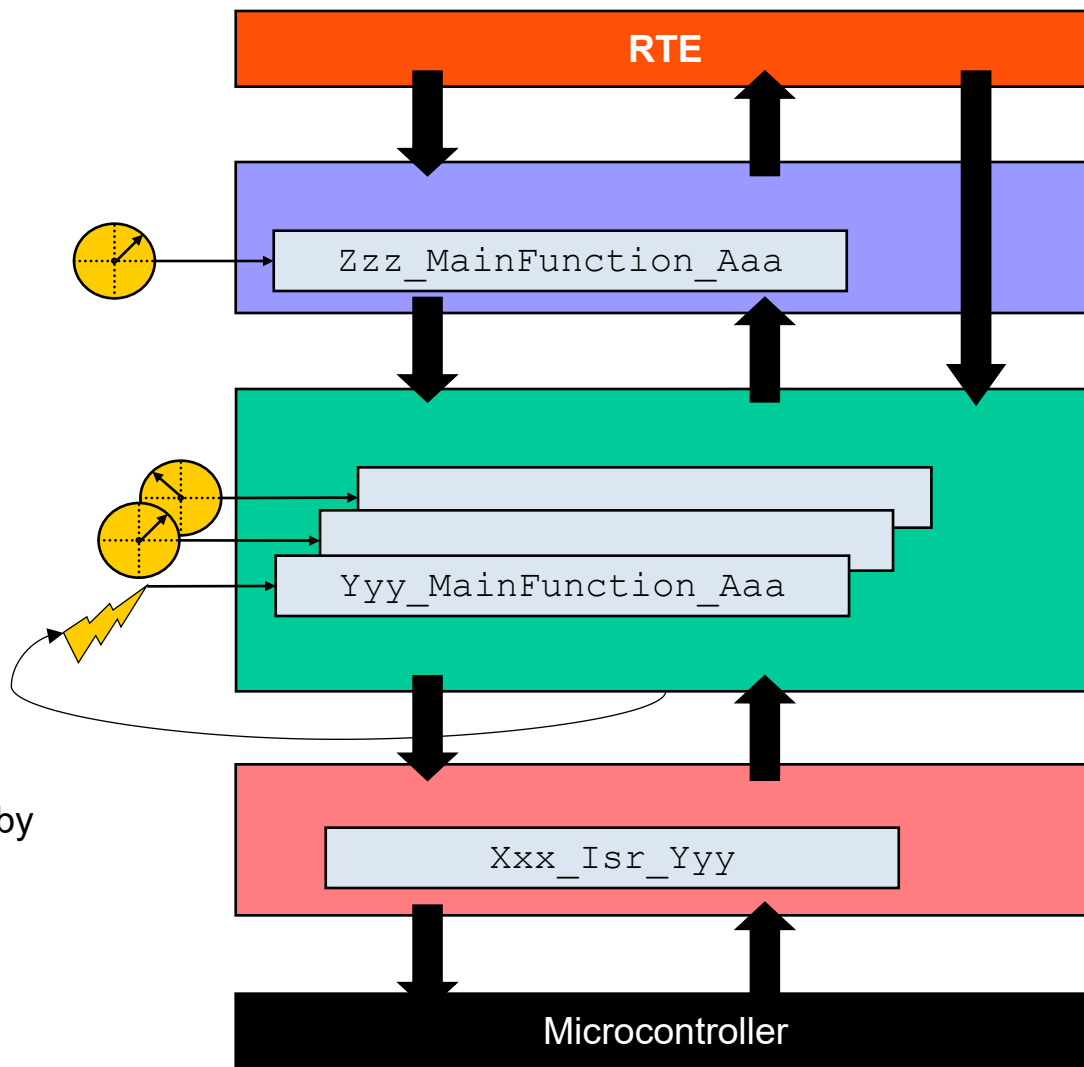
- BswTimingEvent
- BswBackgroundEvent
- BswModeSwitchEvent
- BswModeSwitchedAckEvent
- BswInternalTriggerOccuredEvent
- BswExternalTriggerOccuredEvent
- BswOperationInvokedEvent

Triggers

- Main functions can be triggered in all layers by the listed BSW Events

Mode Disabling Dependencies

- The scheduling of Main functions can be disabled in particular modes.



Integration and Runtime Aspects - Scheduling

调度对象、触发器与模式禁用依赖关系

BSW Scheduling objects

- Main functions
 - n per module
 - located in all layers

BSW Events

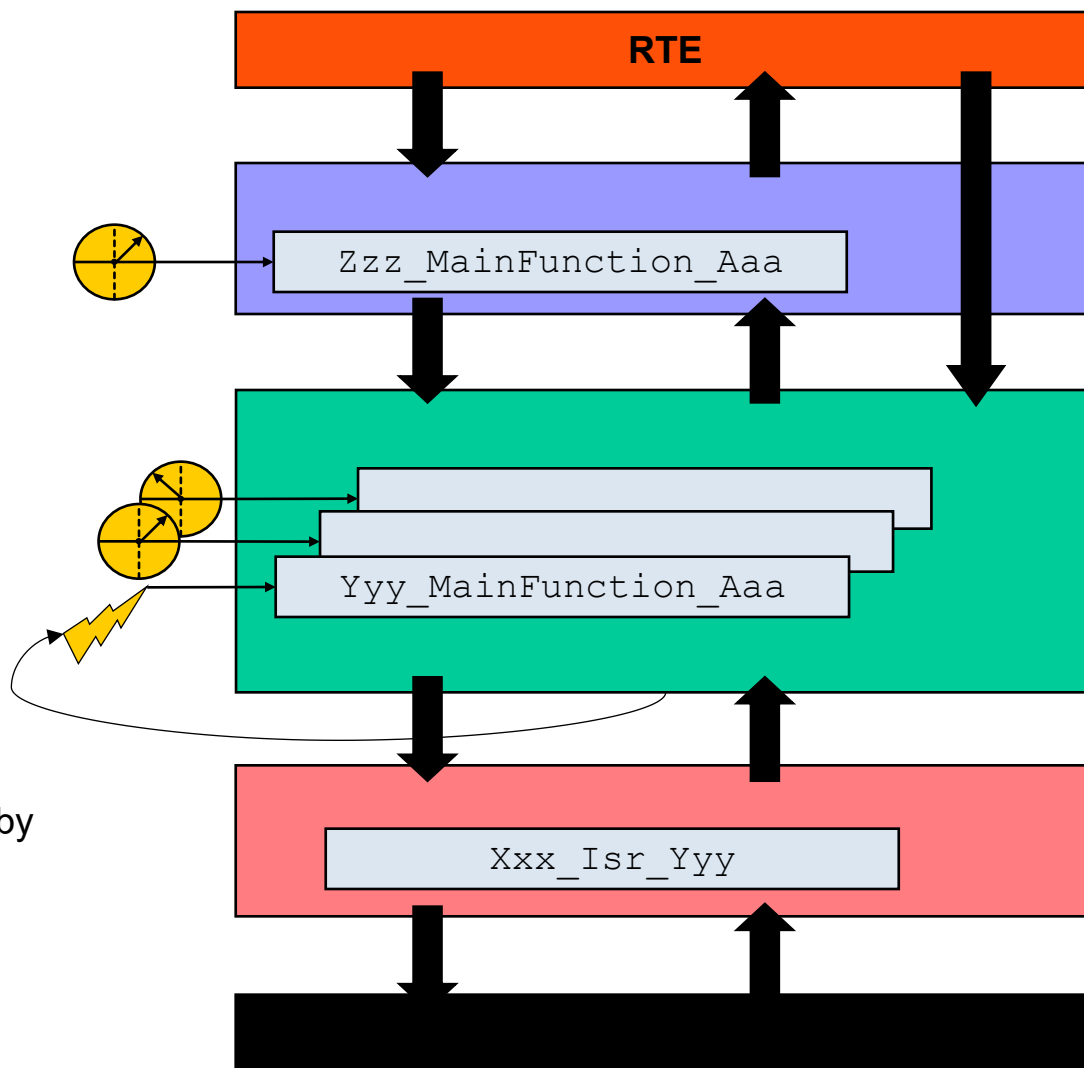
- BswTimingEvent
- BswBackgroundEvent
- BswModeSwitchEvent
- BswModeSwitchedAckEvent
- BswInternalTriggerOccuredEvent
- BswExternalTriggerOccuredEvent
- BswOperationInvokedEvent

Triggers

- Main functions can be triggered in all layers by the listed BSW Events

Mode Disabling Dependencies

- The scheduling of Main functions can be disabled in particular modes.



Integration and Runtime Aspects - Scheduling Transformation Process

Logical Architecture (Model)

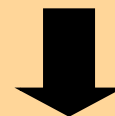
- Ideal concurrency
- Unrestricted resources
- Only real data dependencies



- Scheduling objects
- Trigger
 - BSW Events
- Sequences of scheduling objects
- Scheduling Conditions
- ...

Technical Architecture (Implementation)

- Restricted concurrency
- Restricted resources
- Real data dependencies
- Dependencies given by restrictions



- OS objects
 - Tasks
 - ISRs
 - Alarms
 - Resources
 - OS services
- Sequences of scheduling objects within tasks
- Sequences of tasks
- ...

Transformation

- Mapping of scheduling objects to OS Tasks
- Specification of sequences of scheduling objects within tasks
 - Specification of task sequences
 - Specification of a scheduling strategy
- ...

Integration and Runtime Aspects - Scheduling Transformation Process

Logical Architecture (Model)

- Ideal concurrency
- Unrestricted resources
- Only real data dependencies



- Scheduling objects
- Trigger
 - BSW Events
- Sequences of scheduling objects
- Scheduling Conditions
- ...

Technical Architecture (Implementation)

- Restricted concurrency
- Restricted resources
- Real data dependencies
- Dependencies given by restrictions



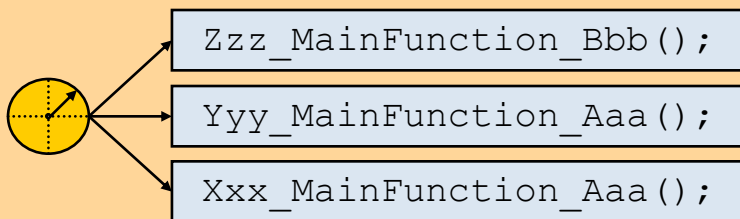
- OS objects
 - Tasks
 - ISRs
 - Alarms
 - Resources
 - OS services
- Sequences of scheduling objects within tasks
- Sequences of tasks
- ...

Transformation

- Mapping of scheduling objects to OS Tasks
- Specification of sequences of scheduling objects within tasks
 - Specification of task sequences
 - Specification of a scheduling strategy
 - ...

Integration and Runtime Aspects - Scheduling Transformation Process – Example 1

Logical Architecture (Model)



Technical Architecture (Schedule Module)

```

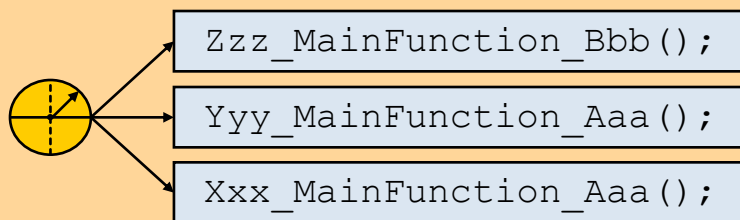
Task1 {
    Zzz_MainFunction_Bbb();
    Yyy_MainFunction_Aaa();
    glue code
    Xxx_MainFunction_Aaa();
    glue code
    ...
}
  
```

Transformation

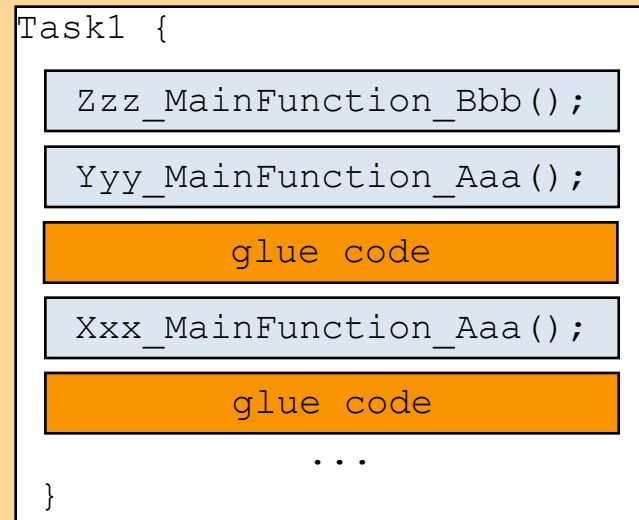
- Mapping of scheduling objects to OS Tasks
- Specification of sequences of scheduling objects within tasks

Integration and Runtime Aspects - Scheduling Transformation Process – Example 1

Logical Architecture (Model)



Technical Architecture (Schedule Module)

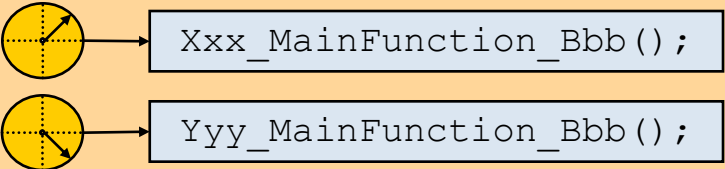


Transformation

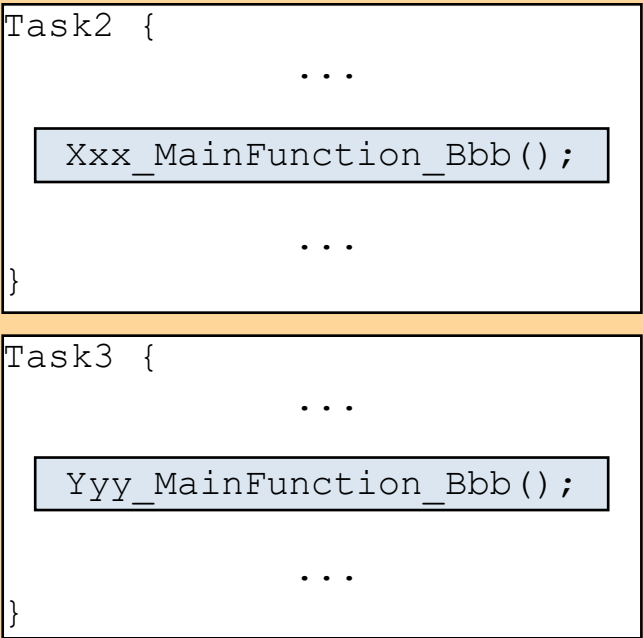
- Mapping of scheduling objects to OS Tasks
- Specification of sequences of scheduling objects within tasks

Integration and Runtime Aspects - Scheduling **Transformation Process – Example 2**

Logical Architecture (Model)



Technical Architecture (Schedule Module)

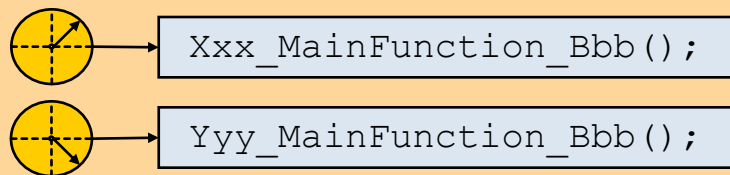


Transformation

➤ Mapping of scheduling objects to OS Tasks

Integration and Runtime Aspects - Scheduling Transformation Process – Example 2

Logical Architecture (Model)



Technical Architecture (Schedule Module)

```
Task2 {  
    ...  
    Xxx_MainFunction_Bbb();  
    ...  
}  
  
Task3 {  
    ...  
    Yyy_MainFunction_Bbb();  
    ...  
}
```

Transformation

- Mapping of scheduling objects to OS Tasks

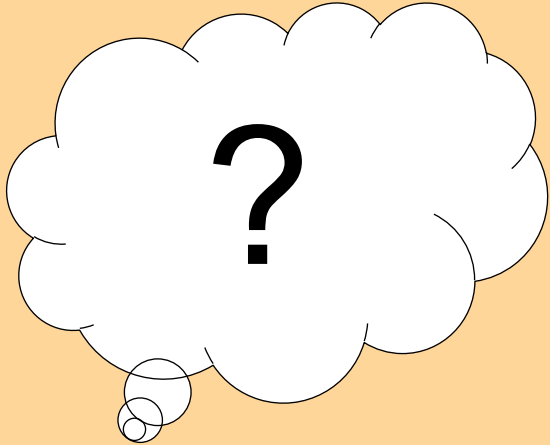
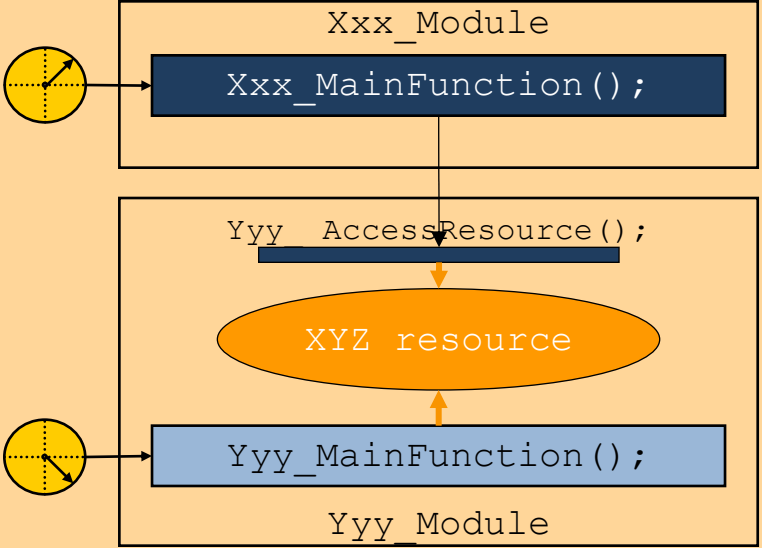
Integration and Runtime Aspects - Scheduling

Data Consistency – Motivation

Logical Architecture (Model)

Technical Architecture (Schedule Module)

- Access to resources by different and concurrent entities of the implemented technical architecture (e.g., main functions and/or other functions of the same module out of different task contexts)



Transformation

Data consistency strategy to be used:

- Sequence, Interrupt blocking, Cooperative Behavior, Semaphores (OSEK Resources), Copies of ...

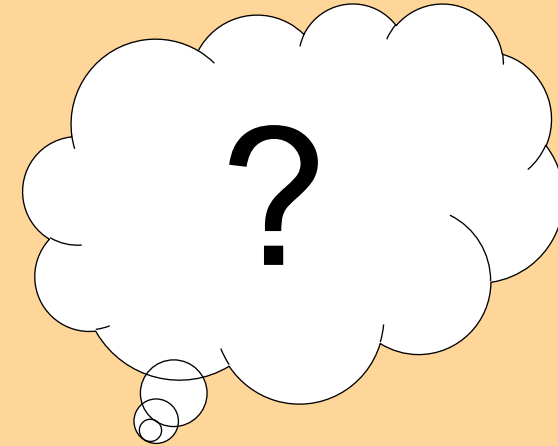
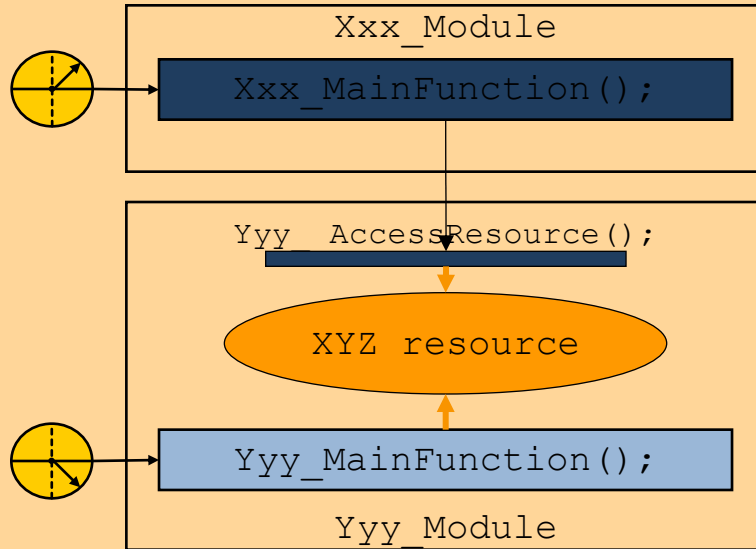
Integration and Runtime Aspects - Scheduling

Data Consistency – Motivation

Logical Architecture (Model)

Technical Architecture (Schedule Module)

- Access to resources by different and concurrent entities of the implemented technical architecture (e.g., main functions and/or other functions of the same module out of different task contexts)



Transform

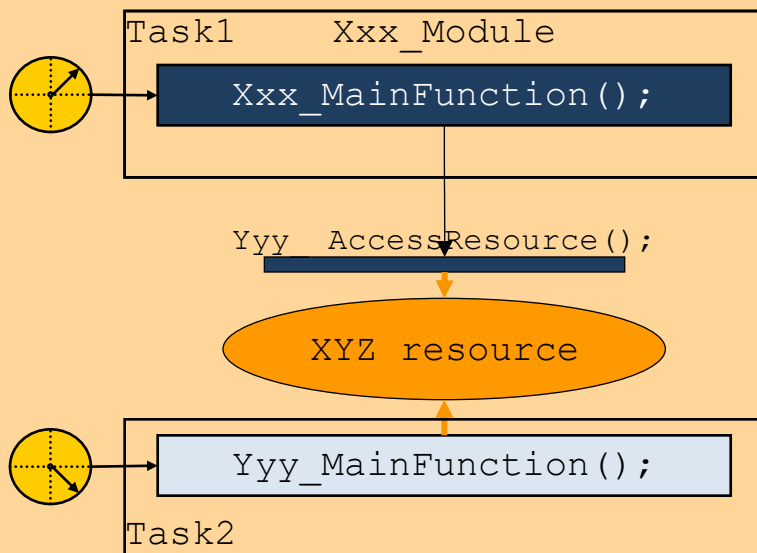
Data consistency strategy to be used:

- Sequence, Interrupt blocking, Cooperative Behavior, Semaphores (OSEK Resources), Copies of ...

Integration and Runtime Aspects - Scheduling

Data Consistency – Example 1 – “Critical Sections” Approach

Logical Architecture (Model)/ Technical Architecture (Schedule Module)



Implementation of Schedule Module

```

#define SchM_Enter_<mod>_<name> \
    DisableAllInterrupts
#define SchM_Exit_<mod>_<name> \
    EnableAllInterrupts

Yyy_AccessResource() {
    ...
    SchM_Enter_Xxx_XYZ();
    <access_to_shared_resource>
    SchM_Exit_Xxx_XYZ();
    ...
}

Yyy_MainFunction() {
    ...
    SchM_Enter_Yyy_XYZ();
    <access_to_shared_resource>
    SchM_Exit_Yyy_XYZ();
    ...
}
  
```

Transformation

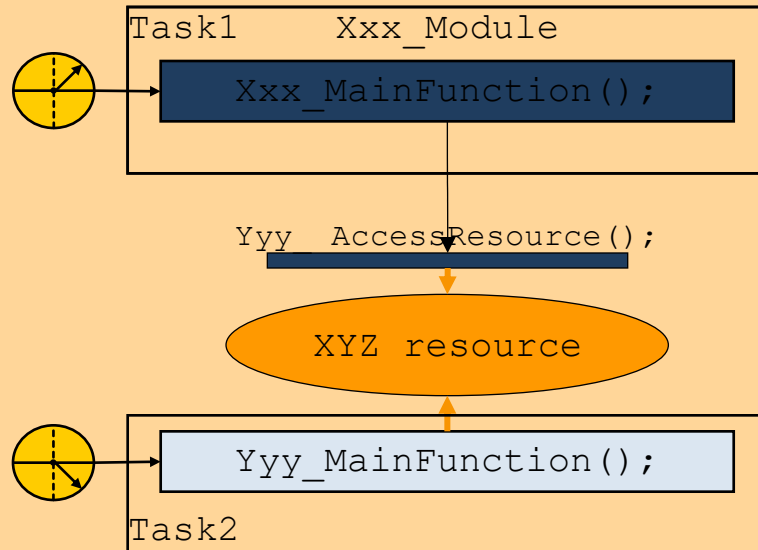
Data consistency is ensured by:

- Interrupt blocking

Integration and Runtime Aspects - Scheduling

Data Consistency – Example 1 – “Critical Sections” Approach

Logical Architecture (Model)/ Technical Architecture (Schedule Module)



Implementation of Schedule Module

```

#define SchM_Enter_<mod>_<name> \
    DisableAllInterrupts
#define SchM_Exit_<mod>_<name> \
    EnableAllInterrupts

Yyy_AccessResource() {
    ...
    SchM_Enter_Xxx_XYZ();
    <access_to_shared_resource>
    SchM_Exit_Xxx_XYZ();
    ...
}

Yyy_MainFunction() {
    ...
    SchM_Enter_Yyy_XYZ();
    <access_to_shared_resource>
    SchM_Exit_Yyy_XYZ();
    ...
}
  
```

Transform

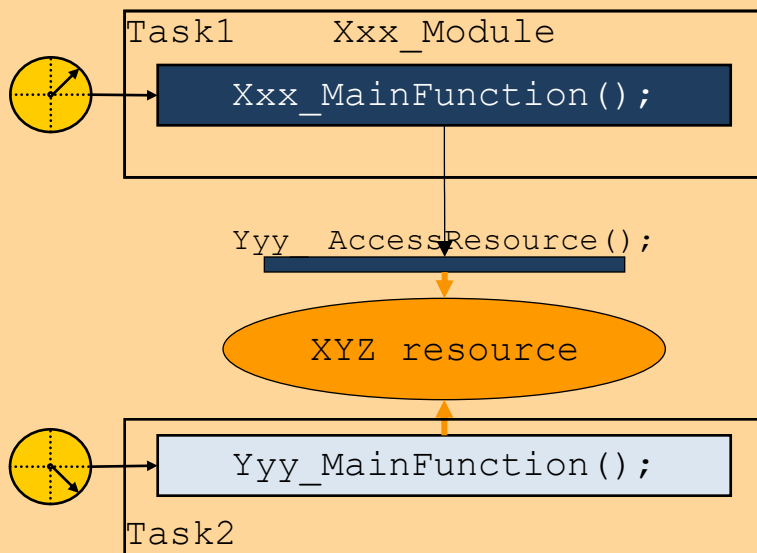
Data consistency is ensured by:

- Interrupt blocking

Integration and Runtime Aspects - Scheduling

Data Consistency – Example 1 – “Critical Sections” Approach

Logical Architecture (Model)/ Technical Architecture (Schedule Module)



Implementation of Schedule Module

```

#define SchM_Enter_<mod>_<name> \
    /* nothing required */
#define SchM_Exit_<mod>_<name> \
    /* nothing required */

Yyy_AccessResource() {
    ...
    SchM_Enter_Xxx_XYZ();
    <access_to_shared_resource>
    SchM_Exit_Xxx_XYZ();
    ...
}

Yyy_MainFunction() {
    ...
    SchM_Enter_Yyy_XYZ();
    <access_to_shared_resource>
    SchM_Exit_Yyy_XYZ();
    ...
}
  
```

Transformation

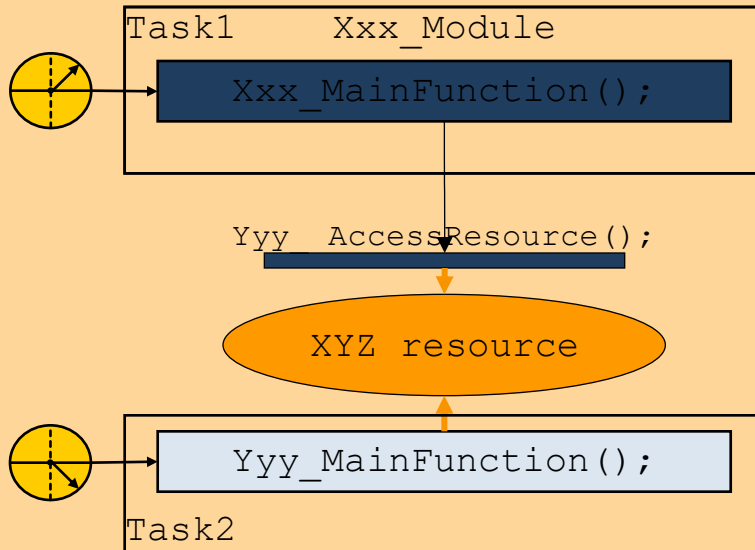
Data consistency is ensured by:

- Sequence

Integration and Runtime Aspects - Scheduling

Data Consistency – Example 1 – “Critical Sections” Approach

Logical Architecture (Model)/ Technical Architecture (Schedule Module)



Implementation of Schedule Module

```
#define SchM_Enter_<mod>_<name> \
/* nothing required */
#define SchM_Exit_<mod>_<name> \
/* nothing required */

Yyy_AccessResource() {
    ...
    SchM_Enter_Xxx_XYZ();
    <access_to_shared_resource>
    SchM_Exit_Xxx_XYZ();
    ...
}

Yyy_MainFunction() {
    ...
    SchM_Enter_Yyy_XYZ();
    <access_to_shared_resource>
    SchM_Exit_Yyy_XYZ();
    ...
}
```

Transform

Data consistency is ensured by:

- Sequence

Integration and Runtime Aspects

Mode Communication / Mode Dependent Scheduling

- The mode dependent scheduling of BSW Modules is identical to the mode dependent scheduling of runnables of software components.
- A mode manager defines a `Provide ModeDeclarationGroupPrototype` in its Basic Software Module Description, and the BSW Scheduler provides an API to communicate mode switch requests to the BSW Scheduler
- A mode user defines a `Required ModeDeclarationGroupPrototype` in its Basic Software Module Description. On demand the BSW Scheduler provides an API to read the current active mode
- If the Basic Software Module Description defines Mode Disabling Dependencies, the BSW Scheduler suppresses the scheduling of BSW Main functions in particular modes.

Integration and Runtime Aspects

模式通信 / 模式相关调度

BSW模块的模式依赖调度与软件组件可运行实体的模式依赖调度相同。模式管理器在其基础软件模块描述中定义提供模式声明组原型，BSW调度器提供API以向BSW调度器传递模式切换请求。模式用户在其基础软件模块描述中定义所需模式声明组原型。BSW调度器按需提供API以读取当前活动模式。若基础软件模块描述定义了模式禁用依赖关系，BSW调度器将在特定模式下抑制BSW主函数的调度。

Table of contents

1. Architecture
2. Configuration
3. **Integration and Runtime Aspects**
 1. Mapping of Runnables
 2. Partitioning
 3. Scheduling
 4. **Mode Management**
 5. Error Handling, Reporting and Diagnostic
 6. Measurement and Calibration
 7. Security
 8. Energy Management
 9. Global Time Synchronization

Table of contents

1. 架构 2. 配置

3. 集成与运行时方面

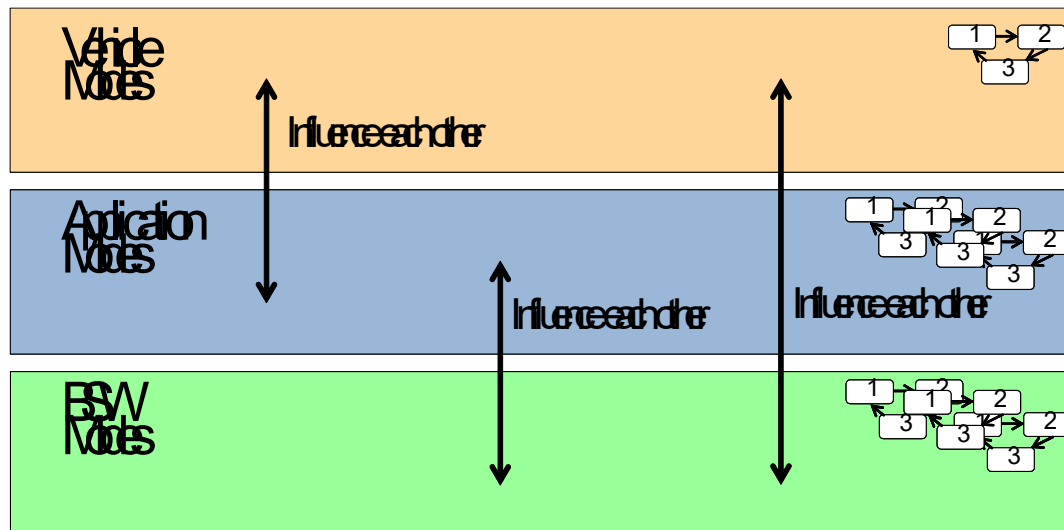
1. 可运行实体的映射2. 分区管理3. 调度机制4.
模式管理5. 错误处理、报告与诊断6. 测量与
标定7. 安全机制8. 能源管理9. 全局时间同步

Integration and Runtime Aspects

Vehicle and application mode management (1)

Relation of Modes:

- Every system contains Modes at different levels of granularity. As shown in the figure, there are vehicle modes and several applications with modes and ECUs with local BSW modes.
- Modes at all this levels influence each other.



Therefore:

- Depending on vehicle modes, applications may be active or inactive and thus be in different application modes.
- Vice versa, the operational state of certain applications may cause vehicle mode changes.
- Depending on vehicle and application modes, the BSW modes may change, e.g. the communication needs of an application may cause a change in the BSW mode of a communication network.
- Vice versa, BSW modes may influence the modes of applications and even the whole vehicle, e.g. when a communication network is unavailable, applications that depend on it may change into a limp-home mode.

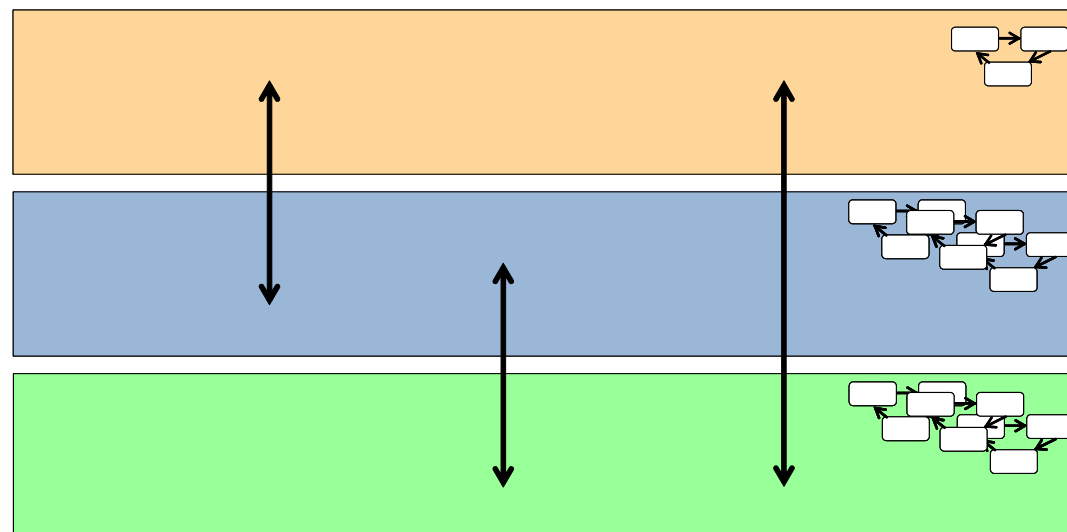
Integration and Runtime Aspects

车辆与应用模式管理 (1)

模式关系：

每个系统都包含不同粒度级别的模式。如图所示，存在车辆模式、若干带模式的应用程序以及具有本地BSW模式的ECU。

所有这些层面的模式都相互影响。



因此：

根据车辆模式的不同，应用程序可能处于激活或非激活状态，从而处于不同的应用模式。反之，特定应用程序的运行状态也可能引发车辆模式的变化。依据车辆及应用模式，基础软件（BSW）模式可能发生变化，例如：应用程序的通信需求可能导致通信网络的BSW模式改变。反之，BSW模式也可能影响应用程序乃至整车的模式，例如：当通信网络不可用时，依赖该网络的应用程序可能切换至跛行回家模式。

Integration and Runtime Aspects

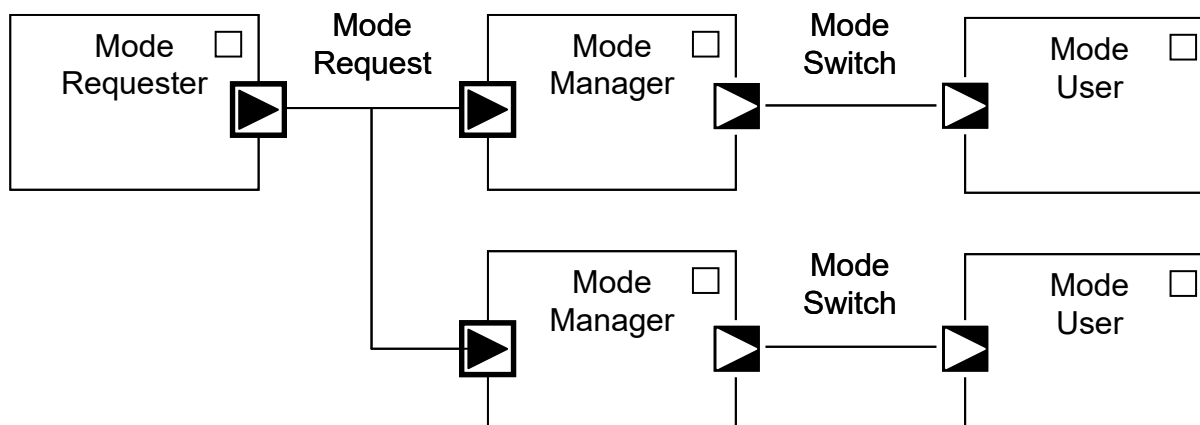
Vehicle and application mode management (2)

Processing of Mode Requests

The basic idea of vehicle mode management is to distribute and arbitrate mode requests and to control the BSW locally based on the results.

This implies that in each OS-Application, there has to be a mode manager that switches the modes for its local mode users and controls the BSW. Of course there can also be multiple mode managers that switch different Modes.

The mode request is a “normal” sender/receiver communication (system wide) while the mode switch always a local service.



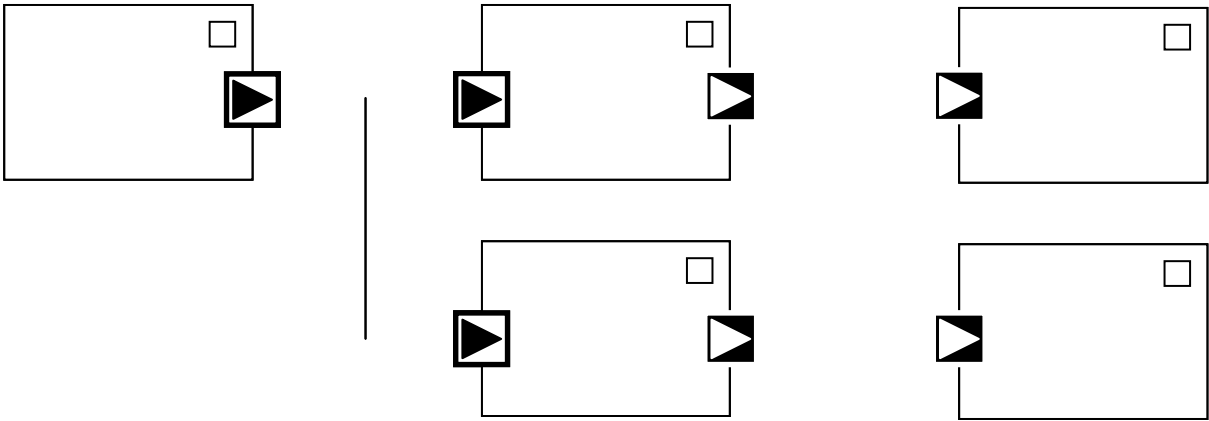
Integration and Runtime Aspects 车辆与应用模式管理 (2)

模式请求的处理

车辆模式管理的基本思想是分发和仲裁模式请求，并根据结果在本地控制BSW。

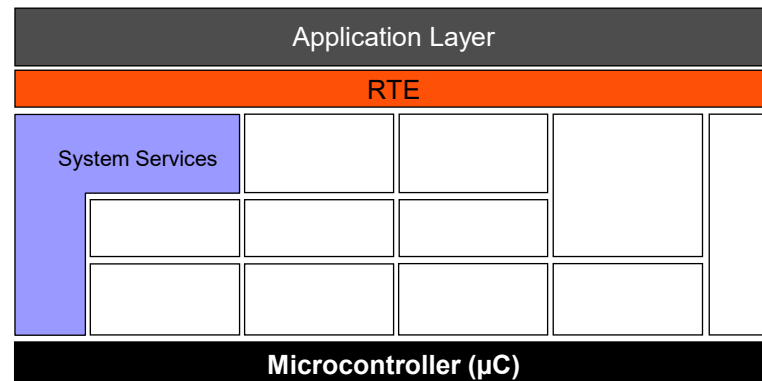
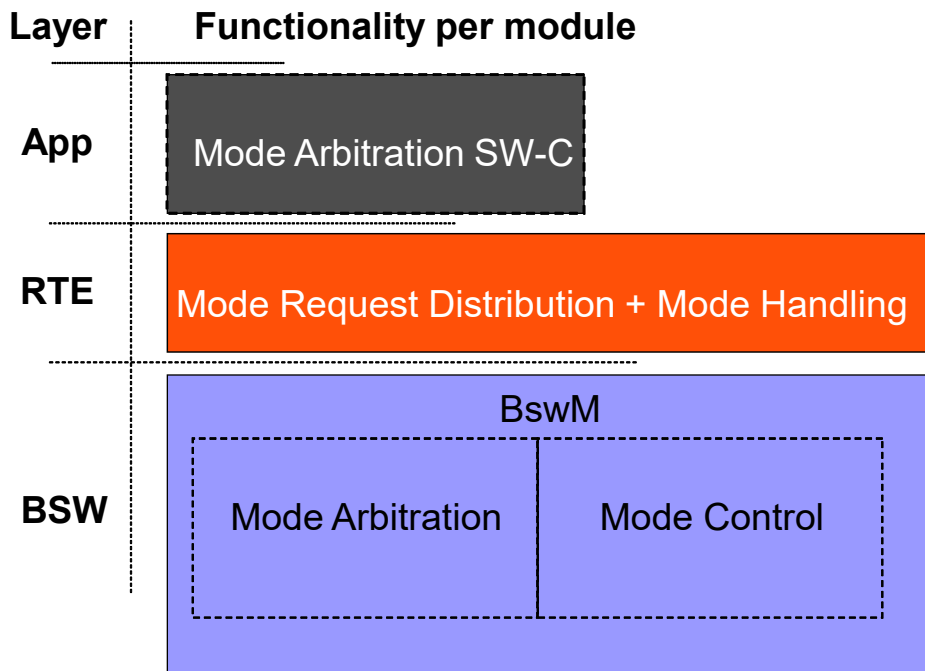
这意味着在每个OS-Application中，必须有一个模式管理器，用于为其本地模式用户切换模式并控制BSW。当然，也可以有多个模式管理器来切换不同的模式。

模式请求是一种“正常”的发送者/接收者通信（系统范围内），而模式切换始终是本地服务。



Integration and Runtime Aspects

Vehicle and application mode management (3)

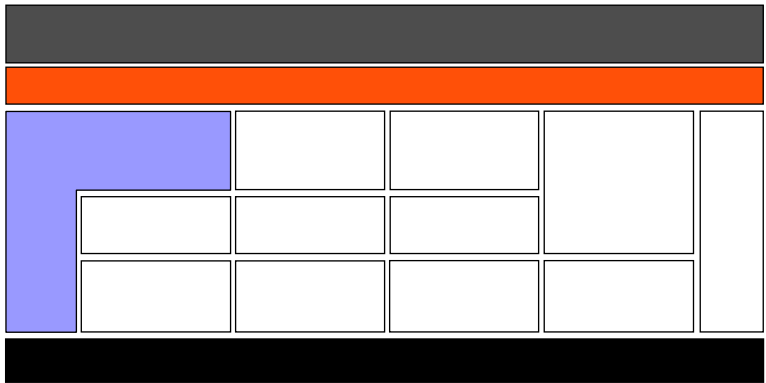
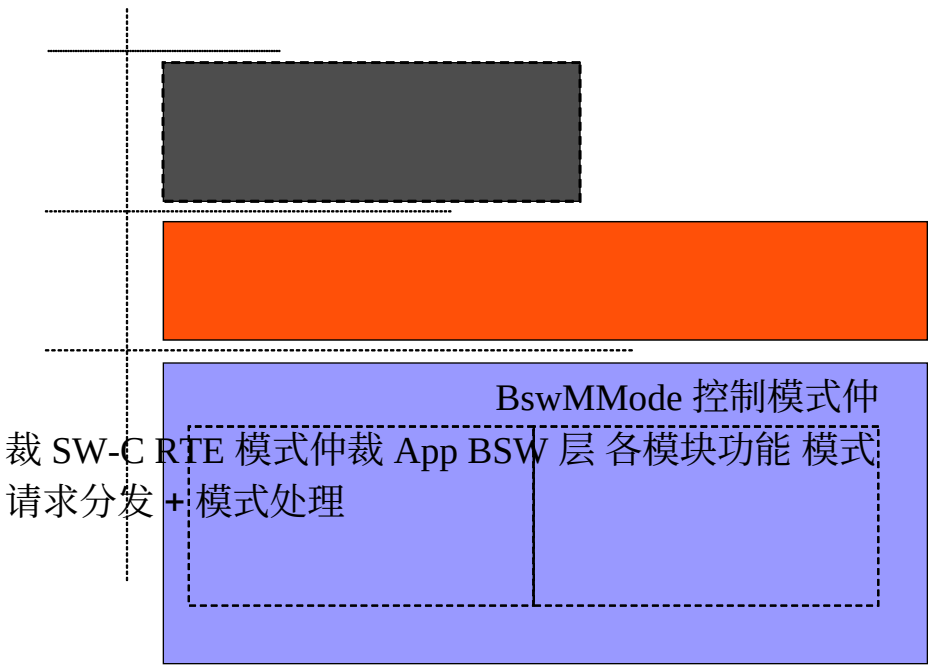


- The major part of the needed functionality is placed in the Basic Software Mode Manager (BswM for short). Since the BswM is located in the BSW, it is present in every OS-Application and local to the mode users as well as the controlled BSW modules.

- The distribution of mode requests is performed by the RTE and the RTE also implements the handling of mode switches.
- E.g. for vehicle modes, a mode request originates from one central mode requestor SW-C and has to be received by the BswMs in many ECUs. This is an exception of the rule that SW-Cs may only communicate to local BSW.
- BswMs running in different OS-Applications can propagate mode requests by Sender-Receiver communication (SchM_Send, SchM_Receive).

Integration and Runtime Aspects

车辆与应用模式管理 (3)



系统服务应用层

所需功能的主要部分被置于基础软件模式管理器（简称BswM）中。由于BswM位于基础软件层，它存在于每一个操作系统应用中，并且对模式用户以及受控的基础软件模块而言都是本地的。

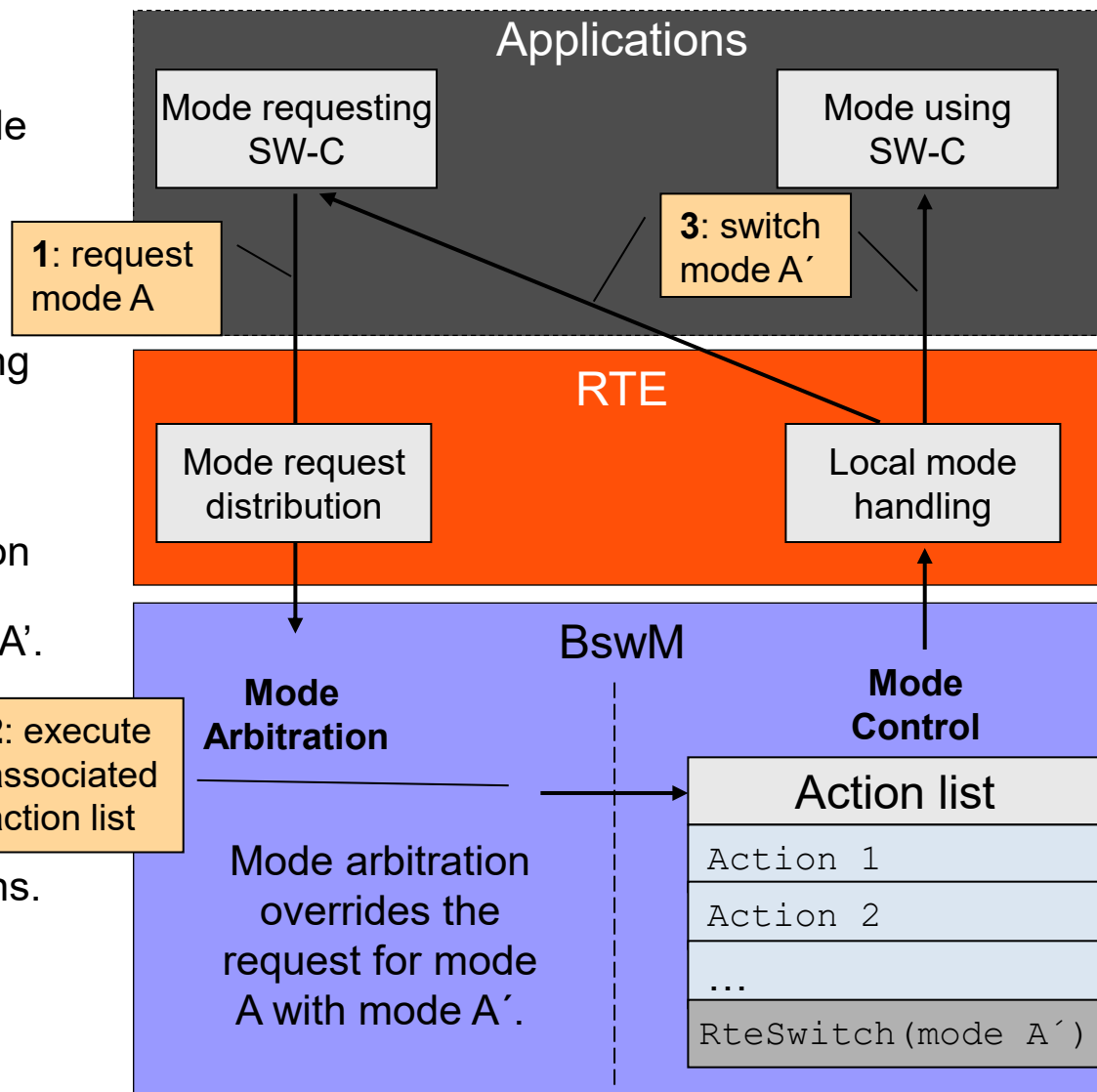
模式请求的分布由RTE执行，且RTE也负责处理模式切换。例如，对于车辆模式，模式请求源自一个中央模式请求者SW-C，并需由多个ECU中的BswMs接收。这是SW-C仅能与本地BSW通信规则的一个例外。运行在不同OS-Applications中的BswMs可通过发送-接收通信（SchM_Send、SchM_Receive）传播模式请求。

Integration and Runtime Aspects

Vehicle and application mode management (4)

Mode Processing Cycle

- The mode requester SW-C requests mode A through its sender port. The RTE distributes the request and the BswM receives it through its receiver port.
- The BswM evaluates its rules and if a rule triggers, it executes the corresponding action list.
- When executing the action list, the BswM may issue a (configurable optional) RTE call to the mode switch API as a last action to inform the mode users about the arbitration result, e.g. the resulting mode A'.
- Any SW-C, especially the mode requester can register to receive the mode switch indication.
- The mode requests can originate from local and remote ECUs or OS-Applications.
- Note that the mode requestor can only receive the mode switch indications from the local BswM, even if the requests are sent out to multiple OS-Applications.



Mode Processing Cycle

- The mode requester SW-C requests mode A through its sender port. The RTE distributes the request and the BswM receives it through its receiver port.
- The BswM evaluates its rules and if a rule triggers, it executes the corresponding action list.
- When executing the action list, the BswM may issue a (configurable optional) RTE call to the mode switch API as a last action to inform the mode users about the arbitration result, e.g. the resulting mode A'.
- Any SW-C, especially the mode requester can register to receive the mode switch indication.
- The mode requests can originate from local and remote ECUs or OS-Applications.
- Note that the mode requestor can only receive the mode switch indications from the local BswM, even if the requests are sent out to multiple OS-Applications.

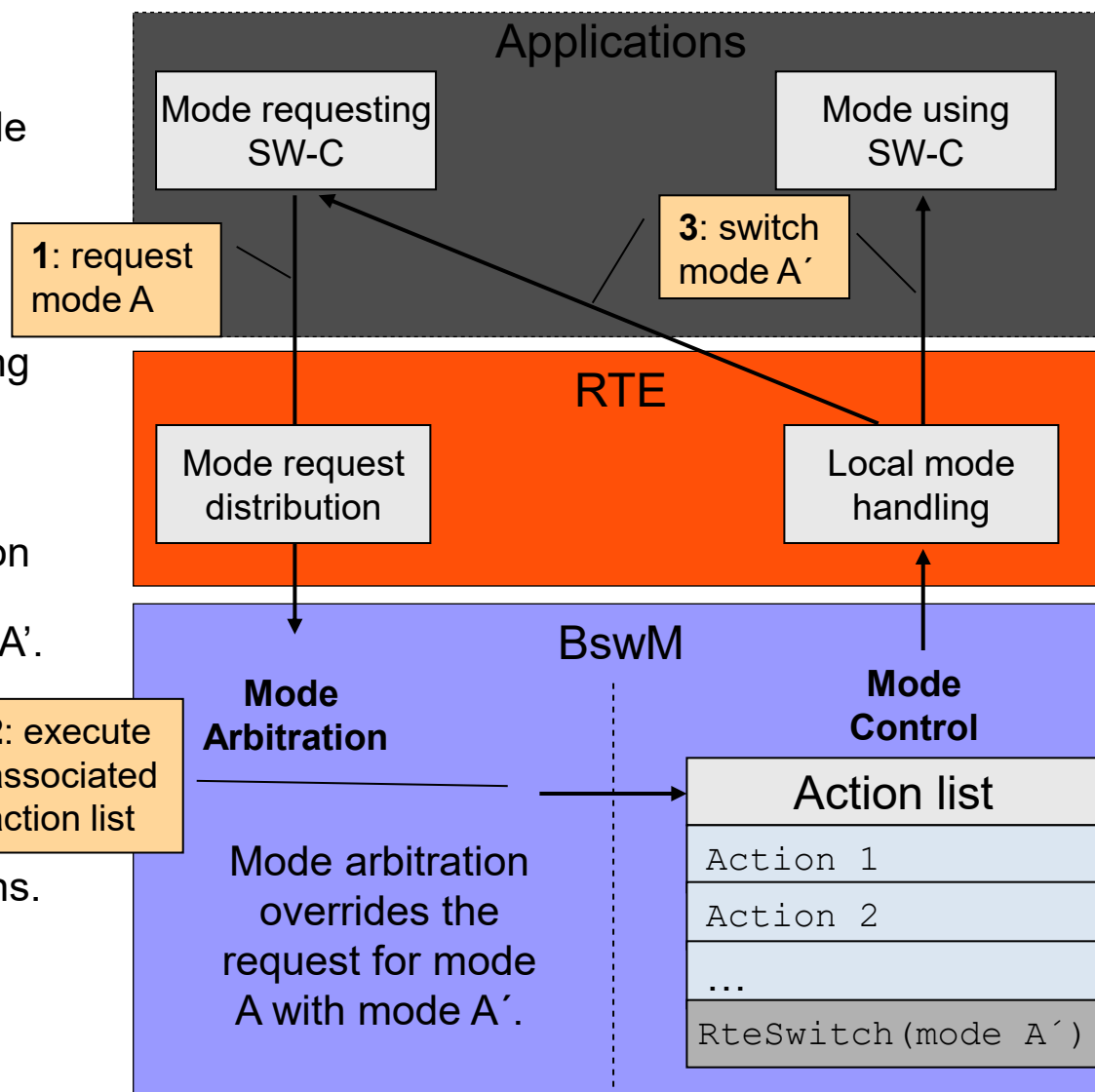


Table of contents

1. Architecture
2. Configuration
- 3. Integration and Runtime Aspects**
 1. Mapping of Runnables
 2. Partitioning
 3. Scheduling
 4. Mode Management
 - 5. Error Handling, Reporting and Diagnostic**
 6. Measurement and Calibration
 7. Security
 8. Energy Management
 9. Global Time Synchronization

Table of contents

1. 架构 2. 配置

3. 集成与运行时方面

1. 可运行实体的映射 2. 分区管理 3. 调度机制 4. 模式管理 5. 错误处理、报告与诊断 6. 测量与标定 7. 安全机制 8. 能源管理 9. 全局时间同步

Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic Error Classification (1)

Types of errors

Hardware errors / failures

- Root cause: Damage, failure or ,value out of range', detected by software
- Example 1: EEPROM cell is not writable any more
- Example 2: Output voltage of sensor out of specified range

Software errors

- Root cause: Wrong software or system design, because software itself can never fail.
- Example 1: wrong API parameter (EEPROM target address out of range)
- Example 2: Using not initialized data

System errors

- Example 1: CAN receive buffer overflow
- Example 2: time-out for receive messages

错误分类 (1)**错误类型****硬件错误/故障**

- 根本原因：软件■检测到损坏、故障或“数值超出范围” 示例 EEPROM单元无法再写入■ 示例 传感器输出电压超出规定范围

软件错误

- 根本原因：错误的软件或系统设计，因为软件本身永远不会失效。■ 示例 错误的API参数（EEPROM目标地址超出范围）■ 示例 使用未初始化的数据

系统错误

- 示例1：CAN接收缓冲区溢出■ 示例 接收消息超时

Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

Error Reporting – Alternatives

There are several alternatives to report an error (detailed on the following slides):

Via API

Inform the caller about success/failure of an operation.

Via statically definable callback function (notification)

Inform the caller about failure of an operation

Via central Error Hooks (Default Error Tracer, Det)

For logging and tracing errors during product development. Can be switched off for production code.

Via central Callouts (Default Error Tracer, Det)

For handling errors during product life time.

Via central Error Function (AUTOSAR Diagnostic Event Manager)

For error reaction and logging in series (production code)

Each application software component (SW-C) can report errors to Diagnostic Event Manager (Dem).

报告错误有几种替代方案（详见以下幻灯片）：

通过API

通知调用者操作的成功/失败情况。

通过静态可定义的回调函数（通知）向调用者报告操作失败

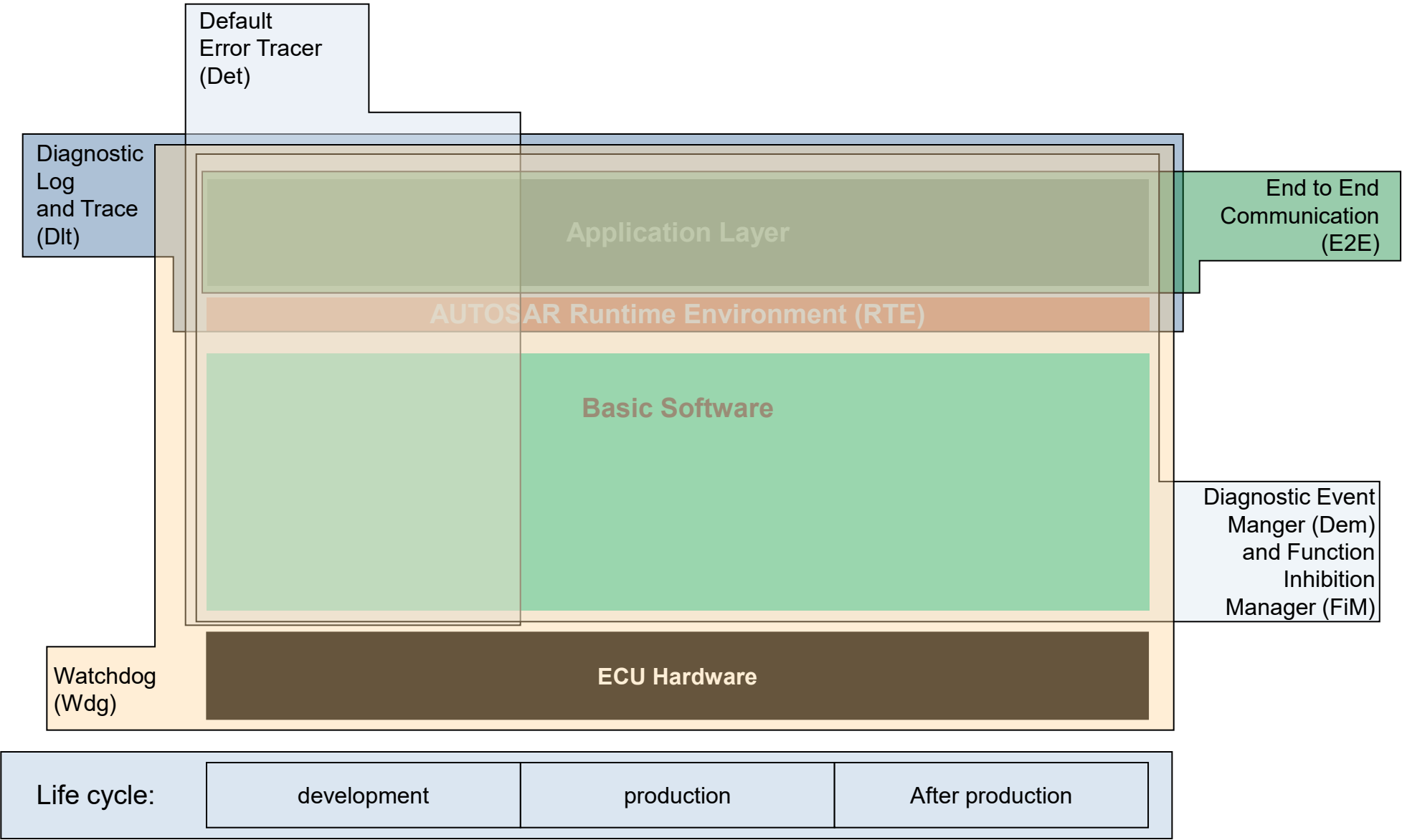
通过中央错误钩子（默认错误追踪器，Det）用于在产品开发期间记录和追踪错误。可在生产代码中关闭此功能。

通过中央标注（默认错误追踪器，Det）用于处理产品生命周期中的错误。

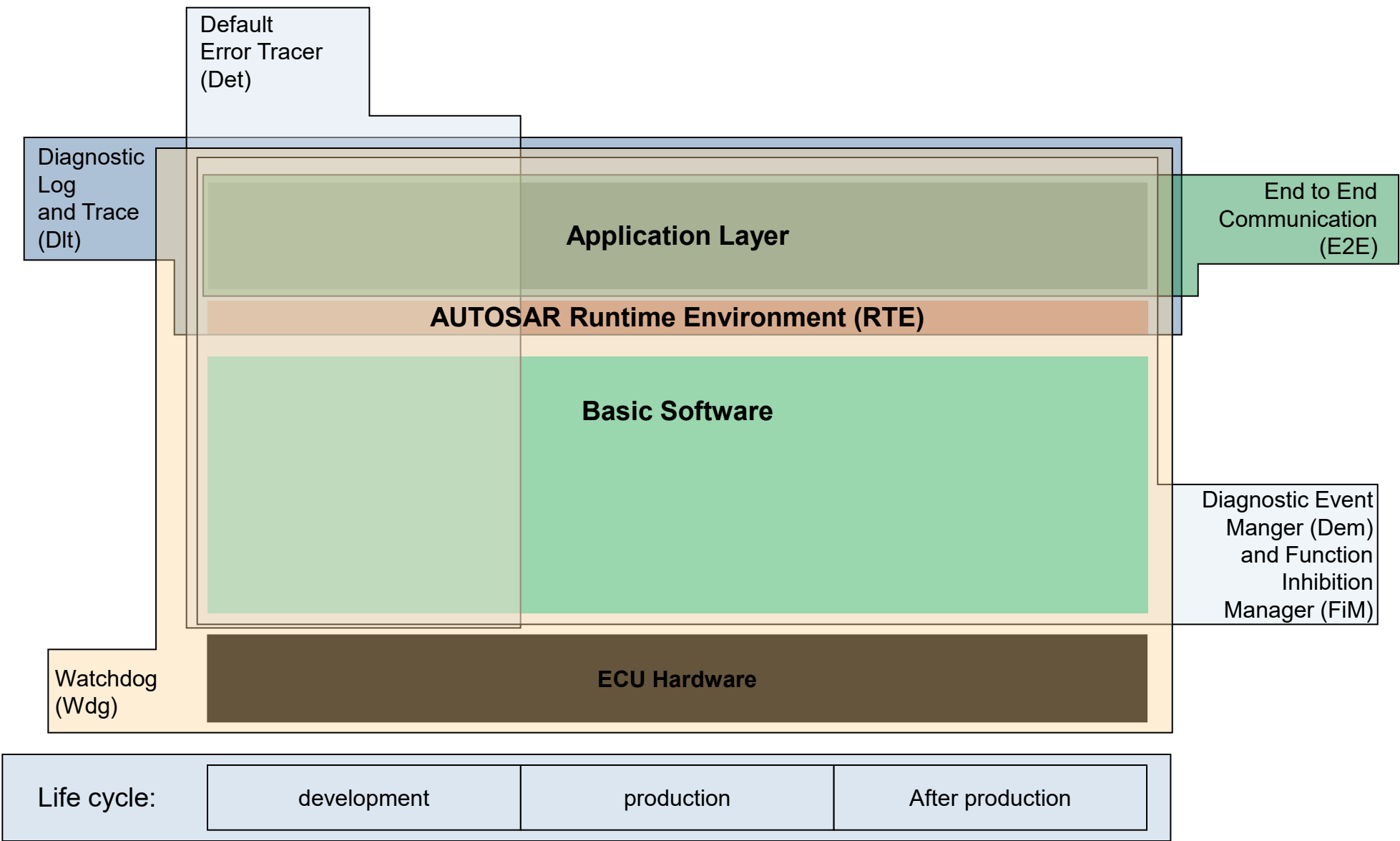
通过中央错误函数（AUTOSAR诊断事件管理器）用于系列中的错误反应和记录（生产代码）

每个应用软件组件（SW-C）都可以向诊断事件管理器（Dem）报告错误。

Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic Mechanism in relation to AUTOSAR layers and system life time



Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic 与AUTOSAR层级及系统生命周期相关的机制



Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic Error Classification (2)

Error Classes

➤ Development Errors

Development errors are software errors. They shall be detected like assertions and fixed during development phase. The detection of errors that shall only occur during development can be switched off per module for production code (by static configuration namely preprocessor switches). The according API is specified within AUTOSAR, but the functionality can be chosen/implemented by the developer according to specific needs.

➤ Runtime Errors

Runtime errors are systematic software errors. They indicate severe exceptions that hinder correct execution of the code. The monitors may stay in code even for a deployed systems. Synchronous handling of these errors can be done optionally in integrator code.

➤ Production Errors

Those errors are stored in fault memory for repair actions in garages. Their occurrence can be anticipated and cannot be avoided in production code. Production errors have a detection and a healing condition.

错误分类 (2)**错误类别****开发错误**

开发错误属于软件错误。它们应像断言一样在开发阶段被检测并修复。对于生产代码，可按模块关闭仅应在开发期间出现的错误检测（通过静态配置，即预处理器开关）。相关API在AUTOSAR中已有规范，但具体功能可由开发者根据实际需求选择或实现。

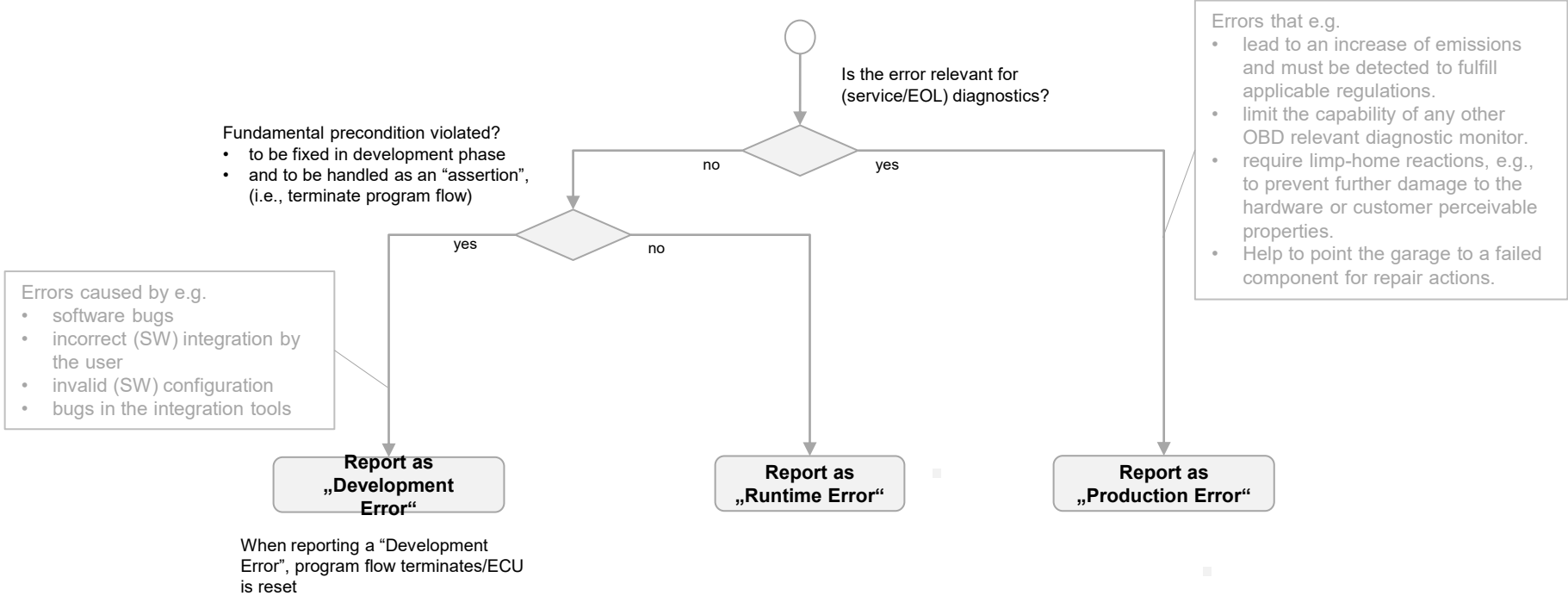
运行时错误

运行时错误是系统性的软件错误。它们指示了阻碍代码正确执行的严重异常。即使在已部署的系统中，监视器也可能停留在代码中。这些错误的同步处理可选择在集成器代码中完成。

生产错误

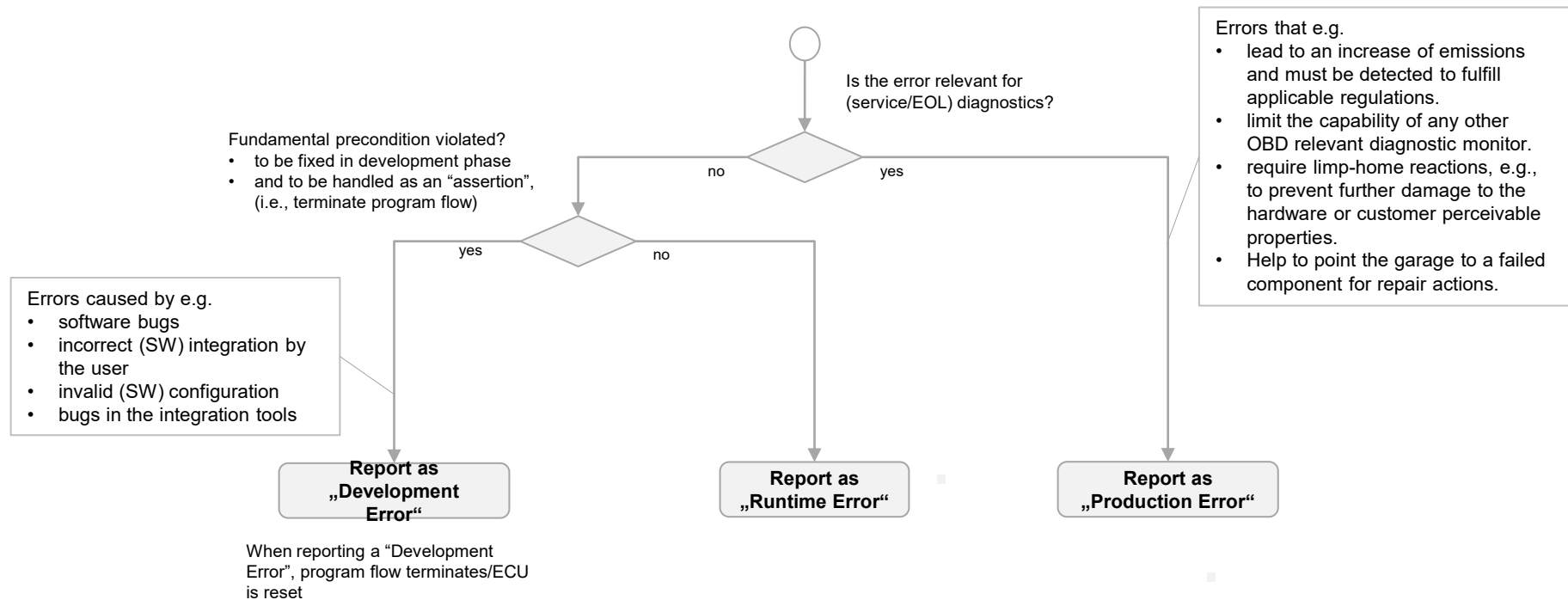
这些错误被存储在故障存储器中，以便在维修车间进行修复操作。它们的发生是可以预见的，并且在生产代码中无法避免。生产错误具有检测条件和恢复条件。

Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic Error Classification (3)



Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

错误分类 (3)



Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

Error Classification Summary

Error Type	Main purpose	Detection and reporting	Configuration	Detection mechanism	Program flow	Reported to
Error Return Code	Inform API caller about current state of an error Leave error reaction to calling module, which has more context to decide on proper error reaction	During all life cycles	none	Event based (set condition only)	continues	To calling BSW module (synchronously via API return code or asynchronously via notification callback)
Development Error	Detect SW/integration issues early during development.	During „Development“ only	Module specific overall enabling/disabling of Detection + Reporting of development errors via pre-compile switch DevErrorDetect provides Callout for integration code in Det_ReportError	Event based (set condition only)	terminates	DET (via Det_ReportError; integration code in Callout allows some preparation before ECU is reset)
Runtime Error	Detect errors during normal operation, e.g. systematic software faults, timing faults or hardware issues	During all life cycles	Always active provides Callout for integration code in Det_ReportRuntimeError	Event based (set condition only)	continues	DET (via Det_ReportRuntimeError; integration code in Callout allows further reporting)
Production Error	Detect issues (and absense of issues) relevant e.g. for service/repair actions or system degradation	During all life cycles	<Ma>DemEventParameterRefs parameter allows to enable/disable the error	Maturing/healing (set/reset conditions)	continues	DEM (via Dem_SetEventStatus to the event memory)

Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

错误分类总结

Error Return Code	Inform API caller about current state of an error Leave error reaction to calling module, which has more context to decide on proper error reaction	During all life cycles	none	Event based (set condition only)	continues	To calling BSW module (synchronously via API return code or asynchronously via notification callback)
Development Error	Detect SW/integration issues early during development.	During „Development“ only	Module specific overall enabling/disabling of Detection + Reporting of development errors via pre-compile switch DevErrorDetect provides Callout for integration code in Det_ReportError	Event based (set condition only)	terminates	DET (via Det_ReportError; integration code in Callout allows some preparation before ECU is reset)
Runtime Error	Detect errors during normal operation, e.g. systematic software faults, timing faults or hardware issues	During all life cycles	Always active provides Callout for integration code in Det_ReportRuntimeError	Event based (set condition only)	continues	DET (via Det_ReportRuntimeError; integration code in Callout allows further reporting)
Production Error	Detect issues (and absense of issues) relevant e.g. for service/repair actions or system degradation	During all life cycles	<Ma>DemEventParameterRefs parameter allows to enable/disable the error	Maturing/healing (set/reset conditions)	continues	DEM (via Dem_SetEventStatus to the event memory)

Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

Error Reporting via API

Error reporting via API

Informs the caller about failure of an operation by returning an error status.

Basic return type

Success: `E_OK` (value: 0)

Failure: `E_NOT_OK` (value: 1)

Specific return type

If different errors have to be distinguished for production code, own return types have to be defined. Different errors shall only be used if the caller can really handle these. Specific development errors shall not be returned via the API. They can be reported to the Default Error Tracer (Det).

Example: services of ADC driver

Success: `ADC_E_OK`

General error (service not accepted): `ADC_E_NOT_OK`

ADC is busy with current conversion: `ADC_E_BUSY`

通过API报告错误

通过API报告错误

通过返回错误状态来通知调用者操作失败。

基本返回类型

成功：E_OK（值：0）失败：

E_NOT_OK（值：1）

特定返回类型

如果生产代码需要区分不同的错误，必须定义自己的返回类型。只有当调用者确实能够处理这些错误时，才应使用不同的错误类型。具体的开发错误不应通过API返回。它们可以报告给默认错误跟踪器（Det）。

示例：ADC驱动服务 成功ADC_E_OK 一般错误（
服务未被接受）：ADC_E_NOT_OK ADC正忙于当前
转换：ADC_E_BUSY

Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

Error Reporting – Introduction

Error reporting via Diagnostic Event Manager (Dem)

For reporting production / series errors.

Those errors have a defined reaction depending on the configuration of this ECU, e.g.:

- Writing to error memory
- Disabling of ECU functions (e.g. via Function Inhibition Manager)
- Notification of SW-Cs

The Diagnostic Event Manager is a standard AUTOSAR module which is always available in production code and whose functionality is specified within AUTOSAR.

Error reporting via Default Error Tracer (Det)

For reporting development/runtime errors.

The Default Error Tracer is mainly intended for handling errors during development time but also for handling systematic errors in production code. Within the Default Error Tracer many mechanisms are possible, e.g.:

- Count errors
- Write error information to ring buffer in RAM
- Send error information via serial interface to external logger
- Infinite Loop, Breakpoint

The detection and reporting of development errors to the Default Error Tracer can be statically switched on/off per module (preprocessor switch or different object code builds of the module) but not for Runtime errors.

错误报告 – 简介

通过诊断事件管理器 (Dem) 进行错误报告

用于报告生产/系列错误。

这些错误根据该ECU的配置有明确的反应，例如：

- 写入错误内存

- 禁用ECU功能（例如通过功能抑制管理器）

- SW-Cs通知

诊断事件管理器是一个标准的AUTOSAR模块，在生产代码中始终可用，其功能在AUTOSAR规范内定义。

通过默认错误追踪器 (Det) 进行错误报告

用于报告开发/运行时错误。

默认错误追踪器主要用于处理开发期间的错误，同时也用于处理生产代码中的系统性错误。在默认错误追踪器中可实现多种机制，例如：

- 计数错误

- 将错误信息写入RAM中的环形缓冲区

- 通过串行接口将错误信息发送至外部记录器

- 无限循环，断点

每个模块（预处理器开关或模块的不同对象代码构建）可以静态地开启/关闭向默认错误追踪器报告开发错误的功能，但运行时错误不在此列。

Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

Diagnostic Event Manager – Diagnostic Error Reporting

API

The **Diagnostic Event Manager** has the following API:

```
Dem_SetEventStatus(EventId, EventStatus)
```

Problem: the error IDs passed with this API have to be ECU wide defined, have to be statically defined and have to occupy a compact range of values for efficiency reasons. Reason: The Diagnostic Event Manager uses this ID as index for accessing ROM arrays.

Error numbering concept: XML based error number generation

Properties:

- Source and object code compatible
- Single name space for all production relevant errors
- Tool support required
- Consecutive error numbers → Error manager can easily access ROM arrays where handling and reaction of errors is defined

Process:

- Each BSW Module declares all production code relevant error variables it needs as “extern”
- Each BSW Module stores all error variables that it needs in the ECU configuration description (e.g. `CANSM_E_BUS_OFF`)
- The configuration tool of the Diagnostic Event Manager parses the ECU configuration description and generates a single file with global constant variables that are expected by the SW modules (e.g. `const Dem_EventIdType DemConf_DemEventParameter_CANSM_E_BUS_OFF=7U; or #define DemConf_DemEventParameter_CANSM_E_BUS_OFF ((Dem_EventIdType) 7)`)
- The reaction to the errors is also defined in the Error Manager configuration tool. This configuration is project specific.

Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

诊断事件管理器 – 诊断错误报告

应用程序编程接口

诊断事件管理器具有以下API:

Dem_SetEventStatus(事件ID, 事件状态)

问题: 出于效率考虑, 通过此API传递的错误ID必须为ECU全局定义、静态定义, 且需占用紧凑的数值范围。原因: 诊断事件管理器将此ID作为索引来访问ROM数组。

错误编号概念: 基于XML的错误编号生成

Properties:

- 源与目标代码兼容
- 所有生产相关错误采用单一命名空间
- 需要工具支持
- 连续错误编号 → 错误管理器可轻松访问定义错误处理与应对措施的ROM数组

Process:

- 每个BS模块将其所需的所有生产代码相关错误变量声明为“extern”
- 每个BSW模块将其所需的所有错误变量存储在ECU配置描述中(例如CANSM_E_BUS_OFF)
- 诊断事件管理器的配置工具解析ECU配置描述, 并生成一个包含SW模块所需的全局常量变量的单一文件(例如const Dem_EventIdType DemConf_DemEventParameter_CANSM_E_BUS_OFF=7U; 或#define DemConf_DemEventParameter_CANSM_E_BUS_OFF ((Dem_EventIdType)7))
- 对错误的响应也在错误管理器配置工具中定义, 此配置是项目特定的。

Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

Default Error Tracer – Example: Development Error Reporting

API

The **Default Error Tracer** has the following API for reporting development errors (runtime errors use identical APIs with different names):

```
Det_ReportError(uint16 ModuleId, uint8 InstanceId, uint8 ApiId, uint8 ErrorId)
```

Error numbering concept

`ModuleId (uint16)`

The Module ID contains the AUTOSAR module ID from the CP SWS BSWGeneral.

As the range is 16 Bit, future extensions for development error reporting of application SW-C are possible. The Basic SW uses only the range from 0..255.

`InstanceId (uint8)`

The Instance ID represents the identifier of an indexed based module starting from 0. If the module is a single instance module it shall pass 0 as an instance ID.

`ApiId (uint8)`

The API-IDs are specified within the software specifications of the BSW modules. They can be #defines or constants defined in the module starting with 0.

`ErrorId (uint8)`

The Error IDs are specified within the software specifications of the BSW modules. They can be #defines defined in the module's header file.

If there are more errors detected by a particular software module which are not specified within the AUTOSAR module software specification, they have to be documented in the module documentation.

All Error-IDs have to be specified in the BSW description.

Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

默认错误追踪器 – 示例：开发错误报告

应用程序编程接口

默认错误追踪器提供以下API用于报告开发错误（运行时错误使用相同但名称不同的API）：

Det_报告错误(uint16 模块标识, uint8 实例标识, uint8 接口标识, uint8 错误标识)

错误编号概念

模块ID (uint16)

模块ID包含来自CP SWS BSWGeneral的AUTOSAR模块ID。由于范围为16位，未来可扩展用于应用软件组件（SW-C）的开发错误报告。基础软件仅使用0..255的范围。

实例ID (uint8)

实例ID表示从0开始的基于索引的模块标识符。如果模块是单实例模块，则应传递0作为实例ID。

ApiId (uint8)

API-ID在BSW模块的软件规范中指定。它们可以是模块内定义的以0开头的#define或常量。

错误ID (uint8)

错误ID在BSW模块的软件规范中指定。它们可以是模块头文件中定义的#define指令。

如果某个特定软件模块检测到的错误未在AUTOSAR模块软件规范中明确规定，则必须在模块文档中予以记录。

所有错误ID必须在BSW描述中指定。

Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

Diagnostic Log and Trace (1)

The module **Diagnostic Log and Trace (Dlt)** collects log messages and converts them into a standardized format. The Dlt module forwards the data to the PduR, which sends it to the configured communications bus.

Therefore the Dlt provides the following functionalities:

- Logging
 - logging of errors, warnings and info messages from AUTOSAR SW-Cs, providing a standardized AUTOSAR interface,
 - gathering all log and trace messages from all AUTOSAR SW-Cs in a centralized AUTOSAR service component (Dlt) in the BSW,
 - logging of messages from Dem.
- Tracing
 - of RTE activities
- Control
 - individual log and trace messages can be enabled/disabled and
 - Log levels can be controlled individually by back channel.
- Generic
 - Dlt is available during development and production phase,
 - access over standard diagnosis or platform specific test interface is possible and
 - security mechanisms to prevent misuse in production phase are provided.

Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic Diagnostic Log and Trace (1)

诊断日志与追踪（Dlt）模块负责收集日志信息并将其转换为标准化格式。Dlt模块将数据转发至PduR，再由PduR将其发送至已配置的通信总线。

因此，Dlt 提供以下功能

： 日志记录

- 记录来自AUTOSAR SW-C的错误、警告和信息消息，提供标准化的AUTOSAR接口，
- 将所有AUTOSAR软件组件（SW-C）的日志和跟踪消息集中收集到基础软件（BSW）中的统一AUTOSAR服务组件（Dlt）内，
- 记录来自Dem的消息。

追踪

- RTE活动的

控制

- 单个日志和跟踪消息可以启用/禁用，且通过反向通道可独立控制
- 日志级别。

通用

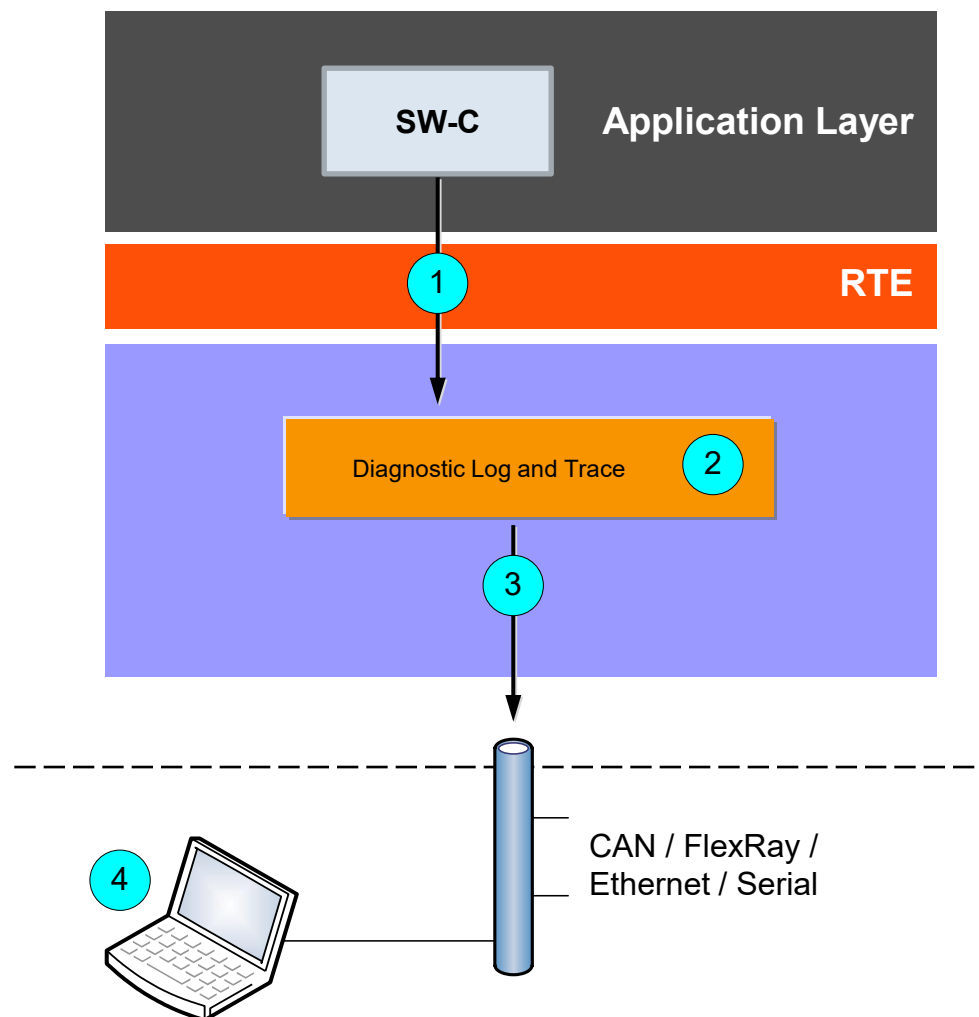
- Dlt在开发和生产阶段均可使用，
- 可通过标准诊断或平台特定测试接口进行访问，并且
- 提供了防止在生产阶段滥用的安全机制。

Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

Diagnostic Log and Trace (2)

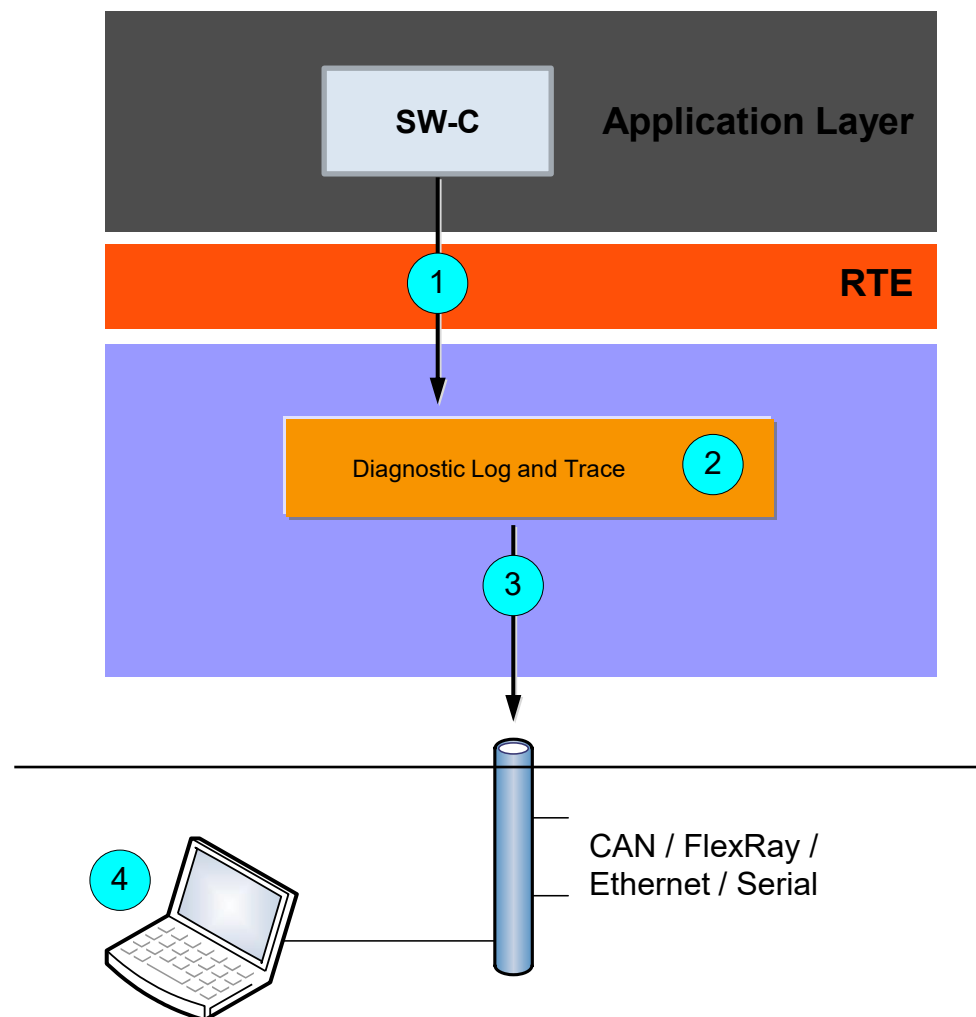
The Dlt communication module is enabled by an external client.

- (1) A **SW-C** is generating a log message. The log message is sent to Dlt by calling the Interface provided by Dlt
- (2) Dlt implements the Dlt protocol
- (3) Dlt sends the encoded log message to the communication bus
- (4) An external Dlt client collects the log message and provides it for later analysis



Dlt通信模块由外部客户端启用。

(1) 一个SW-C正在生成日志信息。该日志信息通过调用Dlt提供的接口发送至Dlt (2) Dlt实现Dlt协议 (3) Dlt将编码后的日志信息发送至通信总线 (4) 外部Dlt客户端收集日志信息，并为后续分析提供支持



Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

Diagnostic Log and Trace (3)

API

The **Diagnostic Log and Trace** has syntactically the following API:

```
Dlt_SendLogMessage(Dlt_SessionIDType session_id, Dlt_MessageLogInfoType log_info, uint8
*log_data,
uint16 log_data_length)
```

Log message identification :

`session_id`

Session ID is the identification number of a log or trace session. A session is the logical entity of the source of log or trace messages. If a SW-C is instantiated several times or opens several ports to Dlt, a new session with a new Session ID for every instance is used. A SW-C additionally can have several log or trace sessions if it has several ports opened to Dlt.

`log_info` contains:

Application ID / Context ID

Application ID is a short name of the SW-C. It identifies the SW-C in the log and trace message. Context ID is a user defined ID to group log and trace messages produced by a SW-C to distinguish functionality. Each Application ID can own several Context IDs. Context ID's are grouped by Application ID's. Both are composed by four 8 bit ASCII characters.

Message ID

Message ID is the ID to characterize the information, which is transported by the message itself. It can be used for identifying the source (in source code) of a message and shall be used for characterizing the payload of a message. A message ID is statically fixed at development or configuration time.

`log_data`

Contain the log or trace data it self. The content and the structure of this provided buffer is specified by the Dlt transmission protocol.

Description File

Normally the `log_data` contains only contents of not fixed variables or information (e.g. no static strings are transmitted). Additionally a description file shall be provided. Within this file the same information for a log messages associated with the Message ID are posted. These are information how to interpret the `log_data` buffer and what fixed entries belonging to a log message.

应用程序编程接口

诊断日志与跟踪在语法上具有以下API:

Dlt_SendLogMessage(Dlt_SessionIDType 会话ID, Dlt_MessageLogInfoType 日志信息, uint8 *日志数据, uint16 日志数据长度)

日志消息识别:

会话ID

会话ID是日志或跟踪会话的识别号码。会话是日志或跟踪消息来源的逻辑实体。如果一个SW-C被多次实例化或向Dlt开放多个端口，则每个实例都会使用一个具有新会话ID的新会话。此外，如果SW-C向Dlt开放了多个端口，它还可以拥有多个日志或跟踪会话。

log_info 包含:

应用ID / 上下文ID

应用ID是SW-C的简称，用于在日志和追踪消息中标识SW-C。上下文ID是用户定义的ID，用于对SW-C产生的日志和追踪消息进行分组以区分功能。每个应用ID可拥有多个上下文ID。上下文ID按应用ID分组。两者均由四个8位ASCII字符组成。

消息ID

消息ID是用于表征信息本身的标识符，它随消息一同传输。该ID可用于识别消息的源代码来源，并应用于描述消息的有效载荷。消息ID在开发或配置阶段静态固定。

日志数据

包含log 或跟踪数据本身。所提供缓冲区的内容和结构

fer 由 Dlt 传输指定

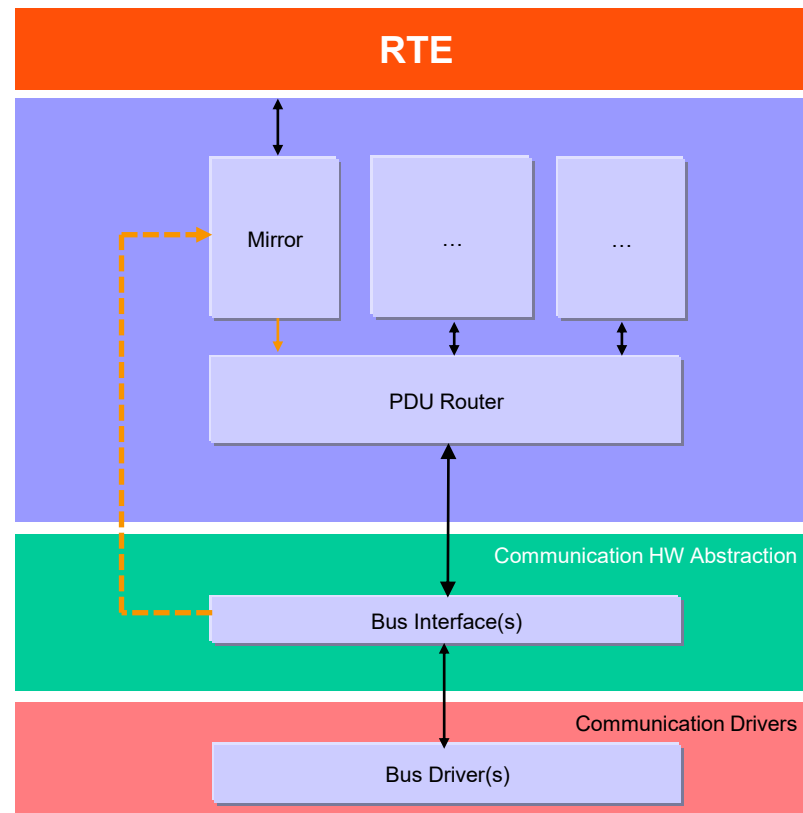
描述文件

通常，日志数据仅包含非固定变量或信息的内容（例如，不传输静态字符串）。此外，应提供一个描述文件。在该文件中，会发布与消息ID关联的日志消息的相同信息。这些信息说明了如何解释日志数据缓冲区以及哪些固定条目属于日志消息。

Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

Mirroring bus communication

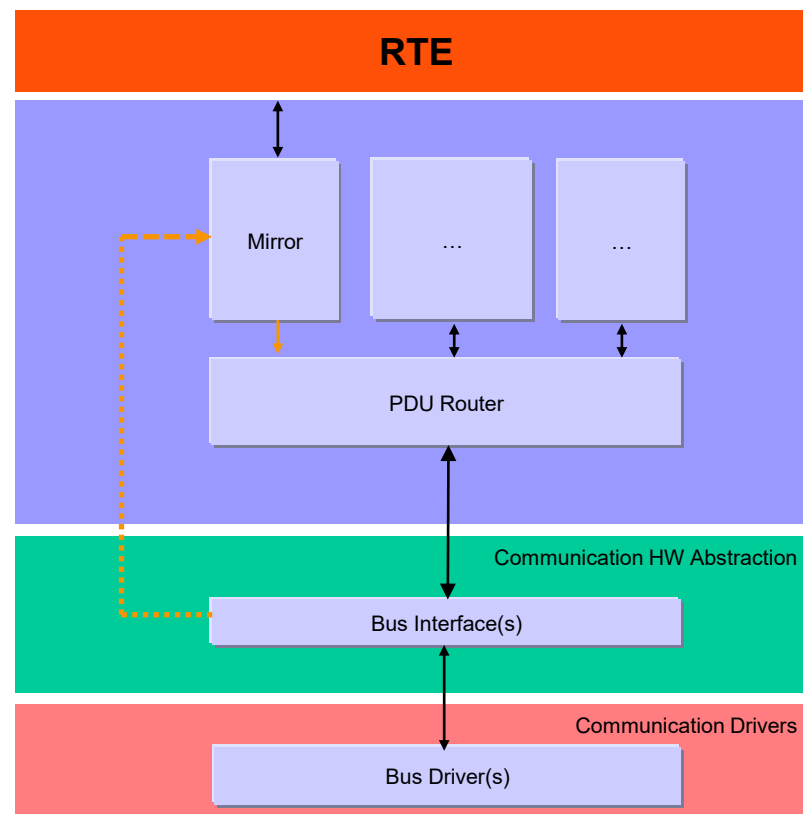
- The Bus Mirroring module (Mirror) collects frames from several buses and forwards these to a common destination bus, either one by one or collected into a stream.
- The Mirror module defines the protocol to be used inside the stream of mirrored frames.
- The Mirror module shall only allow for uni-directional forwarding to avoid vulnerability of the internal networks from an external link (that needs to be configured as destination of mirroring).



Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

镜像总线通信

- 总线镜像模块（Mirror）从多条总线收集帧，并将这些帧逐一或汇总成流后转发至一个共同的目标总线。
- 镜像模块定义了镜像帧流内部使用的协议。
- 镜像模块应仅允许单向转发，以避免内部网络因外部链接（需配置为镜像目标）而暴露于安全漏洞。



Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

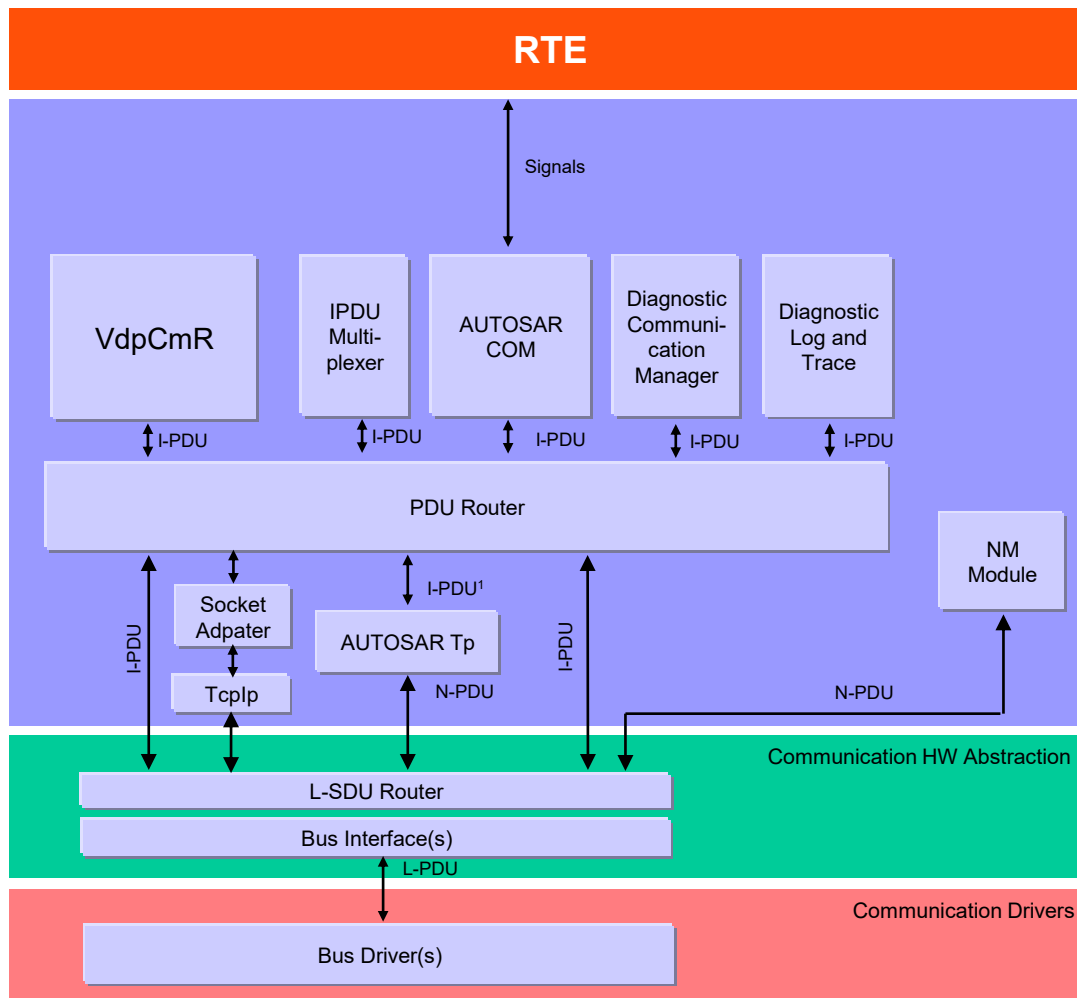
Data collection during operation: VDP

VDP is the **Vehicle Data Protocol** of AUTOSAR which is realized by the **VdpCmR** module.

VDP is an AUTOSAR standard for the purpose of enabling ECU data collection in production vehicles.

VDP within AUTOSAR provides the following features

- A systematic decoupling of data acquisition (sampling) in the remote ECU and data transmission.
- Dynamic configuration of which data shall be collected.
- Activation and deactivation of data points to stabilize temporary pausing of data collection.



Integration and Runtime Aspects - Error Handling, Reporting and Diagnostic

运行期间的数据收集：VDP

VDP是AUTOSAR的车辆数据协议，由VdpCmR模块实现。

VDP是一种AUTOSAR标准，旨在实现量产车辆中ECU数据的采集。

AUTOSAR中的VDP提供以下功能

- 远程ECU中数据采集（采样）与数据传输的系统性解耦。
- 动态配置应收集哪些数据。
- 数据点的激活与停用以稳定数据收集的临时暂停。

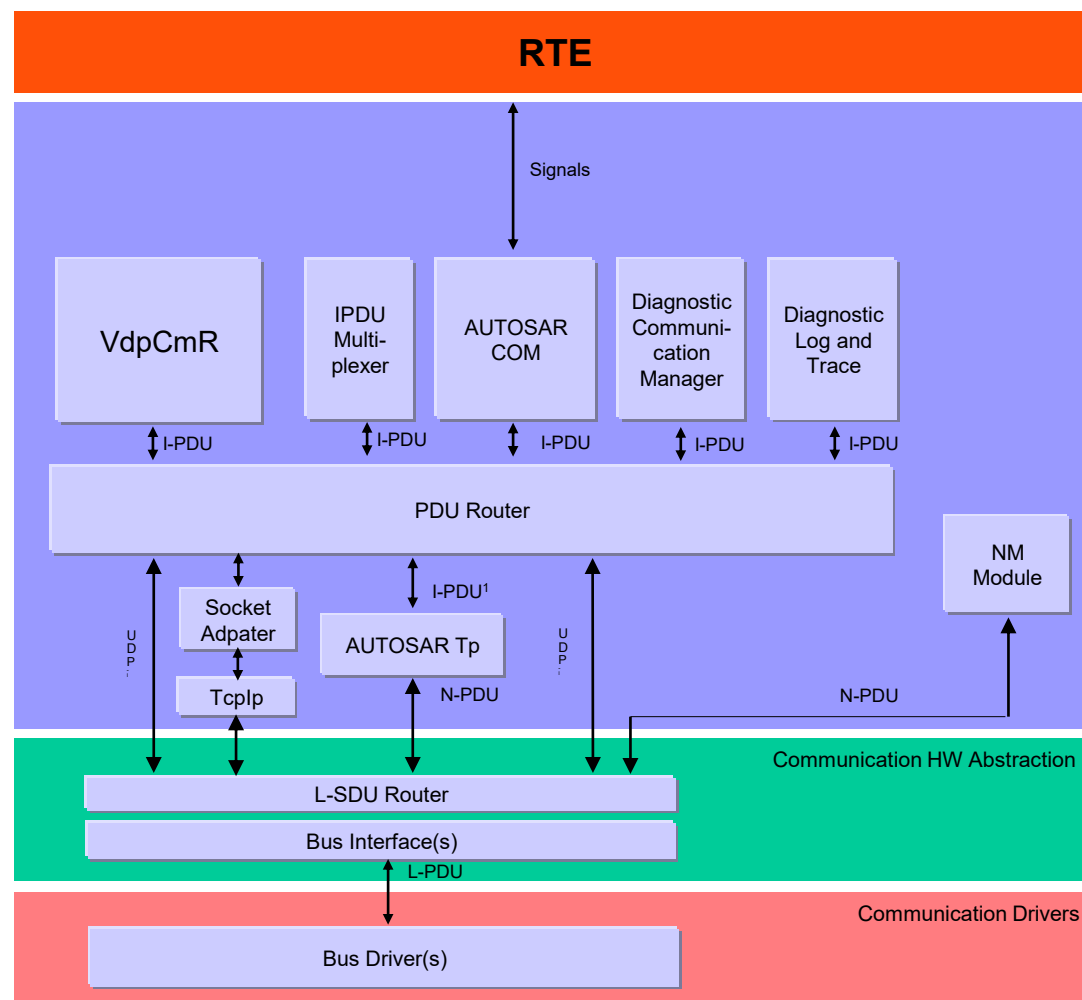


Table of contents

1. Architecture
2. Configuration
- 3. Integration and Runtime Aspects**
 1. Mapping of Runnables
 2. Partitioning
 3. Scheduling
 4. Mode Management
 5. Error Handling, Reporting and Diagnostic
- 6. Measurement and Calibration**
 7. Security
 8. Energy Management
 9. Global Time Synchronization

Table of contents

1. 架构 2. 配置

3. 集成与运行时方面

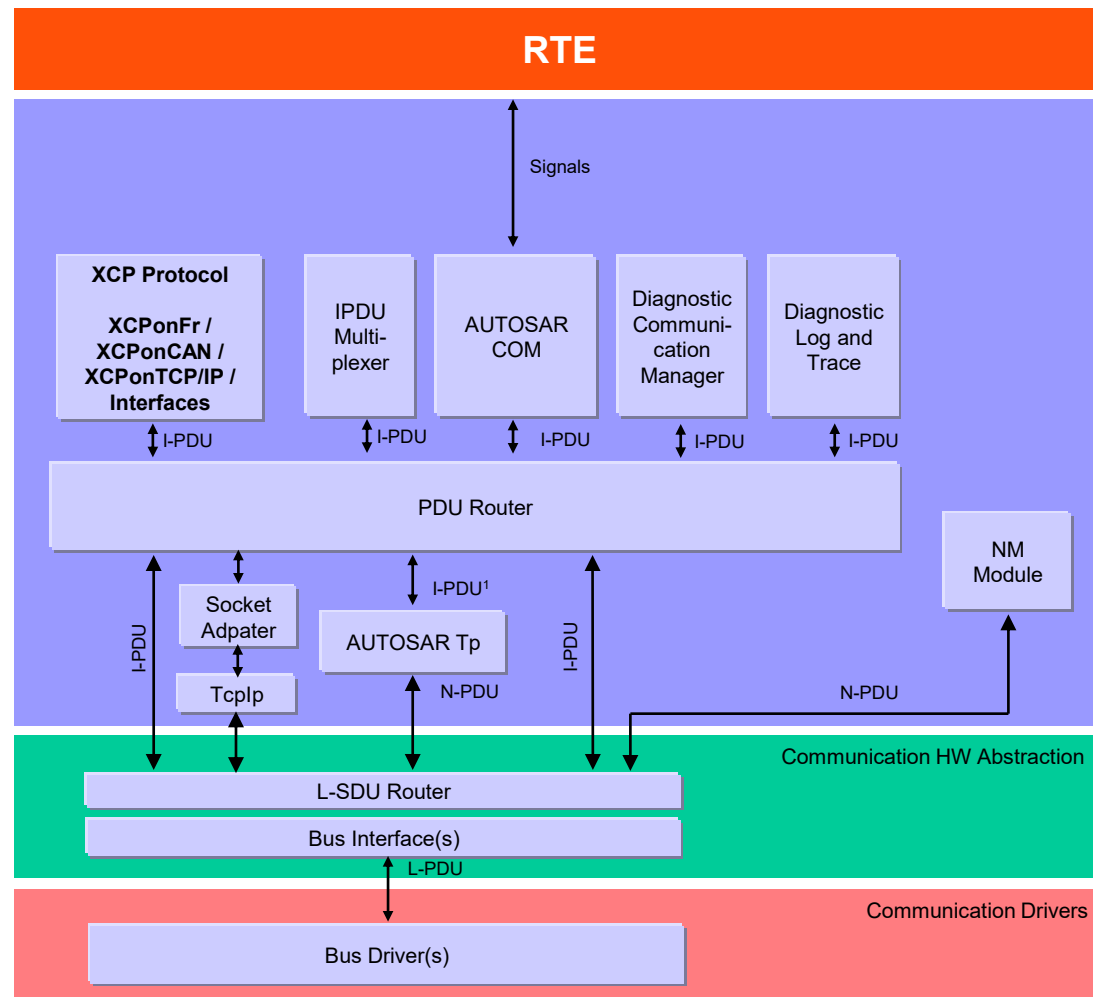
1. 可运行实体的映射2. 分区管理3. 调度机制4.
模式管理5. 错误处理、报告与诊断6. 测量与
标定7. 安全机制8. 能源管理9. 全局时间同步

Integration and Runtime Aspects - Measurement and Calibration XCP

XCP is an **ASAM** standard for calibration purpose of an ECU.

XCP within AUTOSAR provides the following basic **features**:

- Synchronous data acquisition
- Synchronous data stimulation
- Online memory calibration (read / write access)
- Calibration data page initialization and switching
- Flash Programming for ECU development purposes



Integration and Runtime Aspects - Measurement and Calibration

XCP

XCP是用于ECU校准的ASAM标准。

AUTOSAR中的XCP提供以下基本功能：

同步数据采集

同步数据刺激

在线内存校准（读/写访问）

校准数据页初始化和切换

用于ECU开发目的的Flash编程

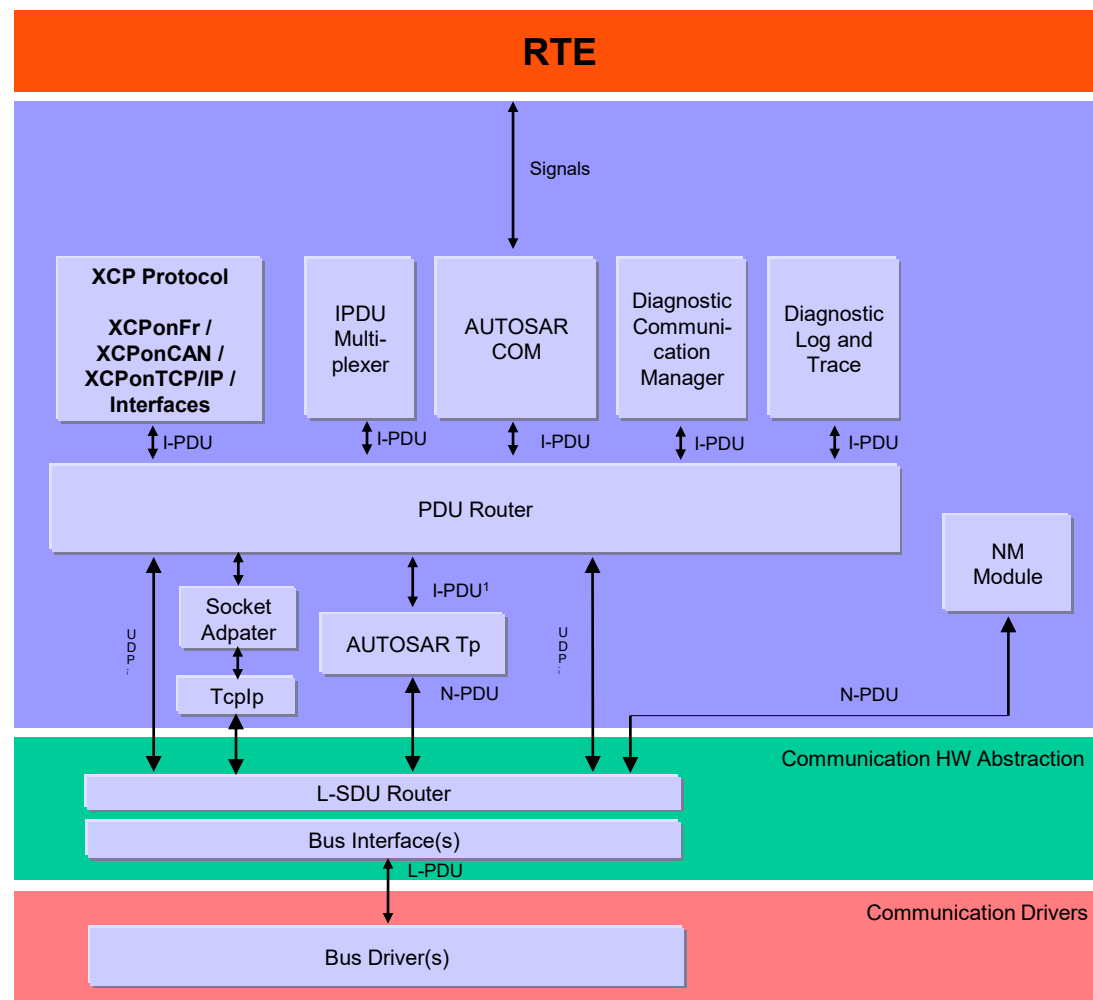


Table of contents

1. Architecture
2. Configuration
3. **Integration and Runtime Aspects**
 1. Mapping of Runnables
 2. Partitioning
 3. Scheduling
 4. Mode Management
 5. Error Handling, Reporting and Diagnostic
 6. Measurement and Calibration
 7. **Security**
 8. Energy Management
 9. Global Time Synchronization

Table of contents

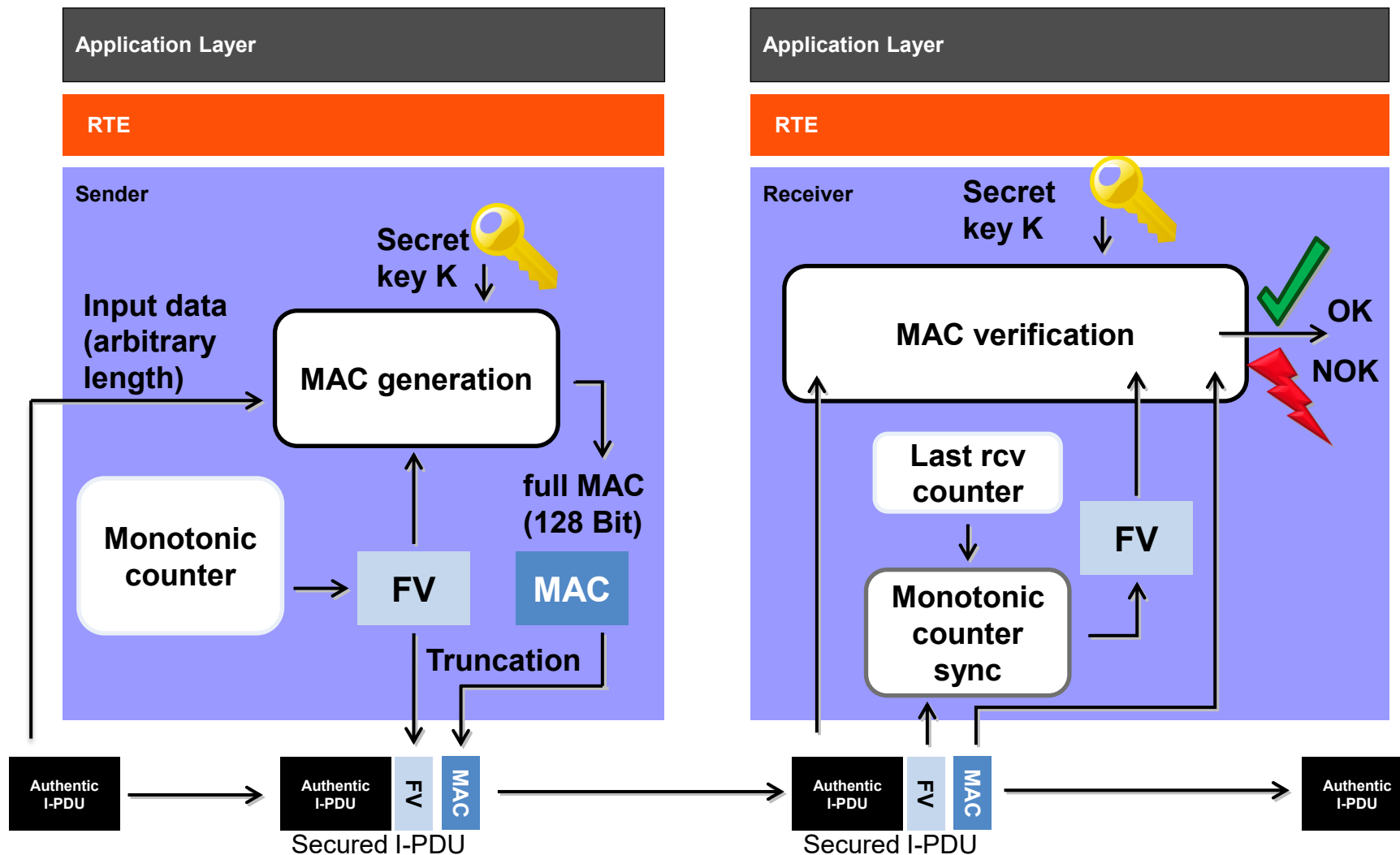
1. 架构 2. 配置

3. 集成与运行时方面

1. 可运行实体的映射2. 分区管理3. 调度机制4.
模式管理5. 错误处理、报告与诊断6. 测量与
标定7. 安全机制8. 能源管理9. 全局时间同步

Integration and Runtime Aspects – Secure Onboard Communication

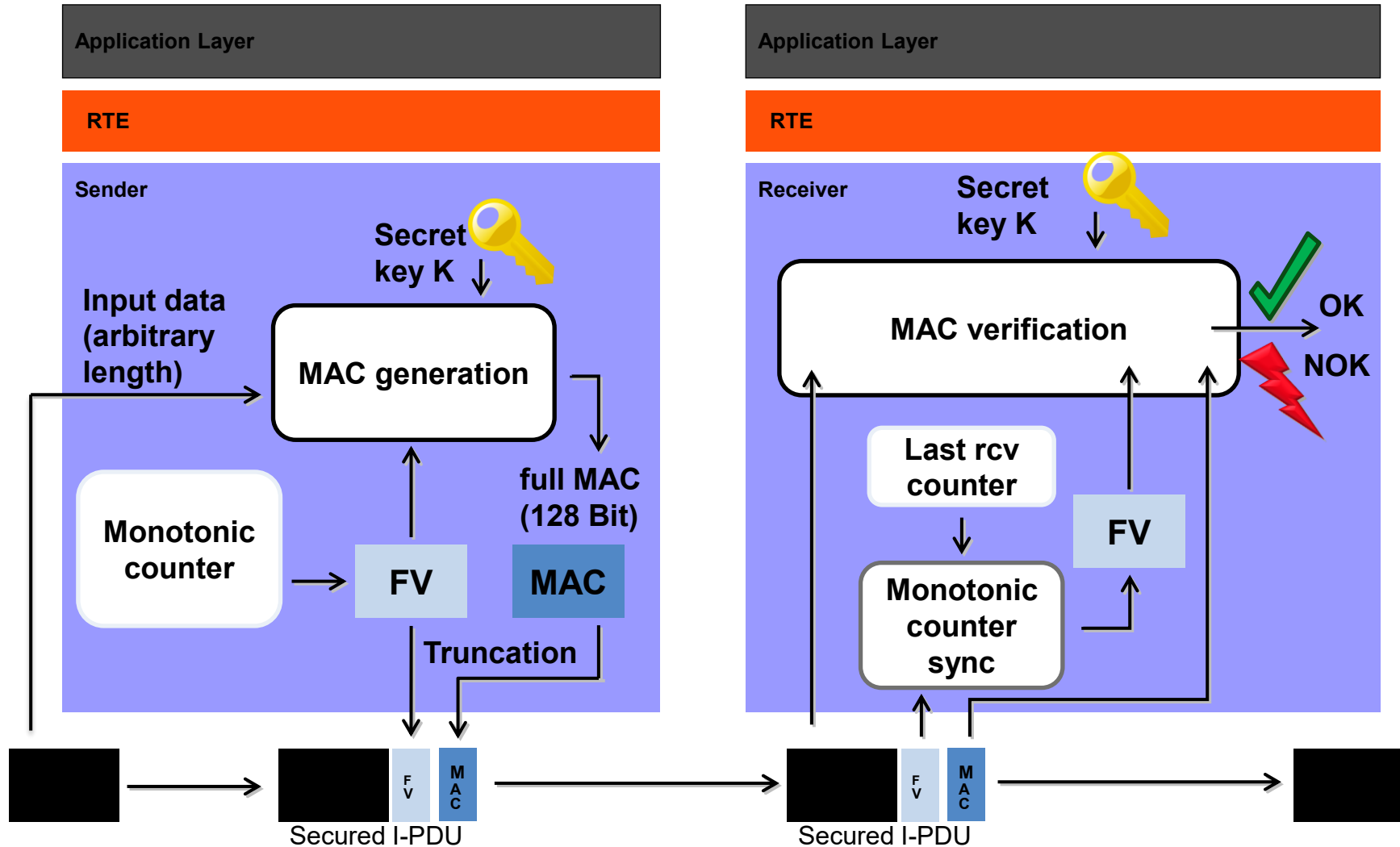
Overview - Message Authentication and Freshness Verification



MAC: Message Authentication Code
FV: Freshness Counter Value

Integration and Runtime Aspects – Secure Onboard Communication

概述 - 消息认证与新鲜性验证

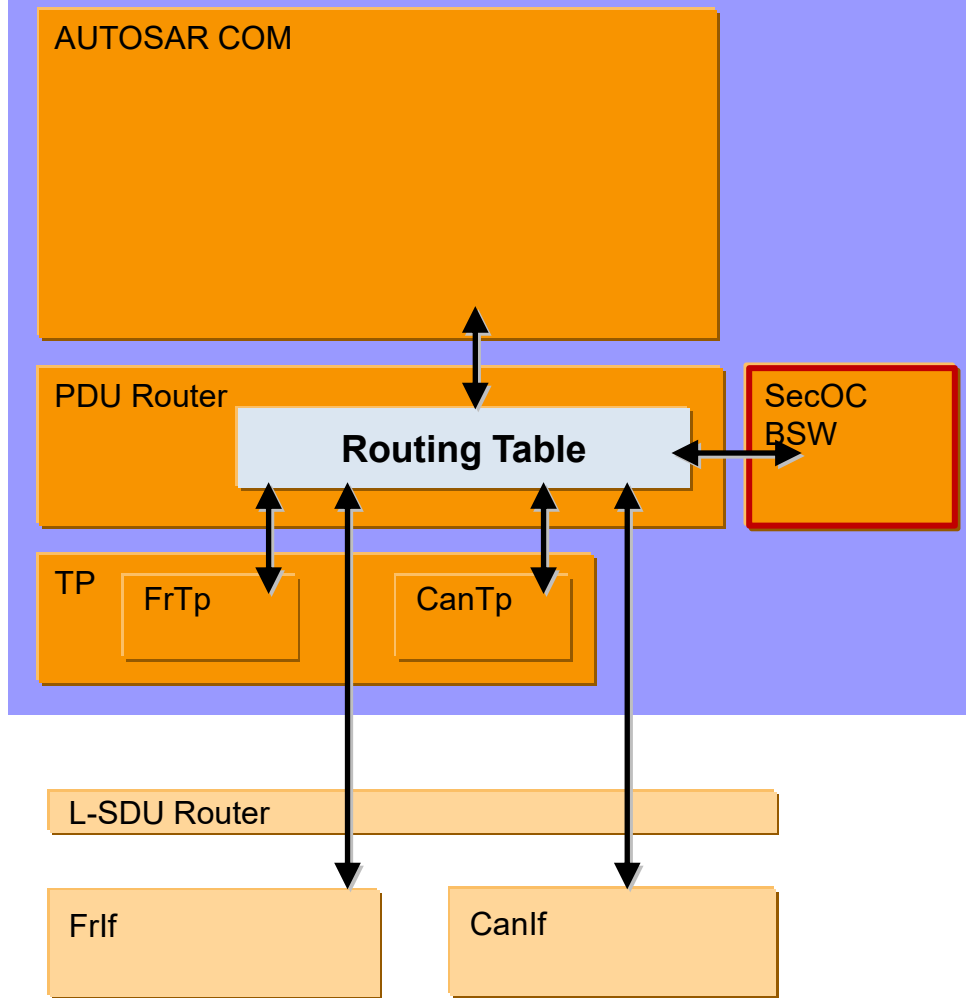


MAC: 消息认证码
FV: Freshness Counter Value

Integration and Runtime Aspects – Secure Onboard Communication

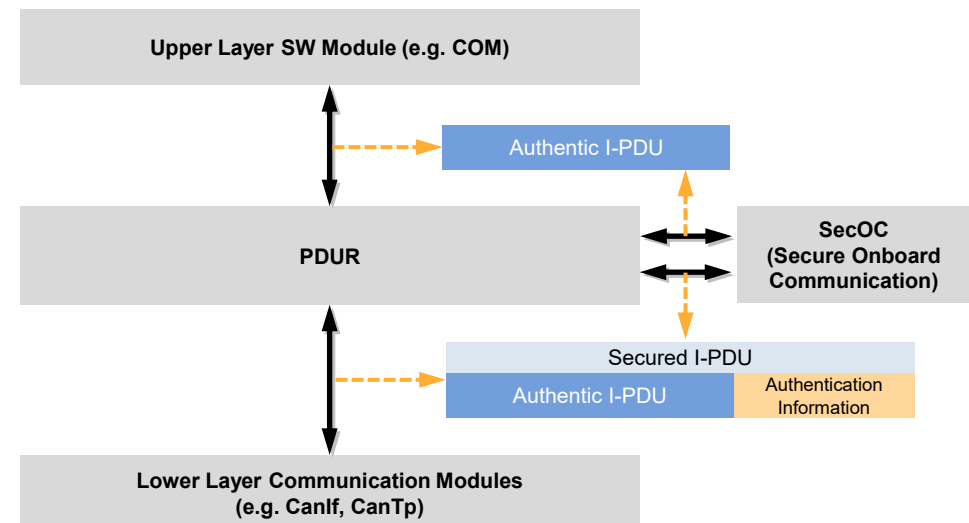
Integration as communication service

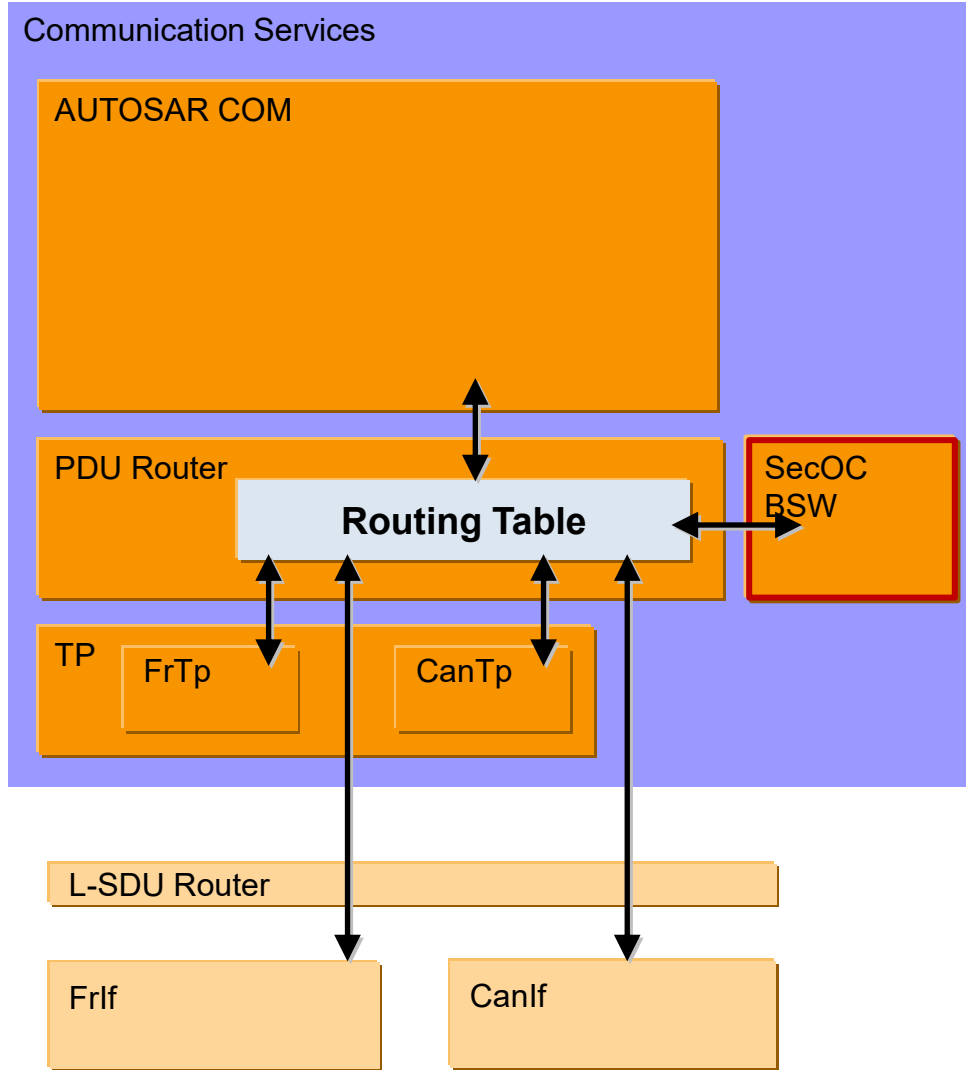
Communication Services



SecOC BSW:

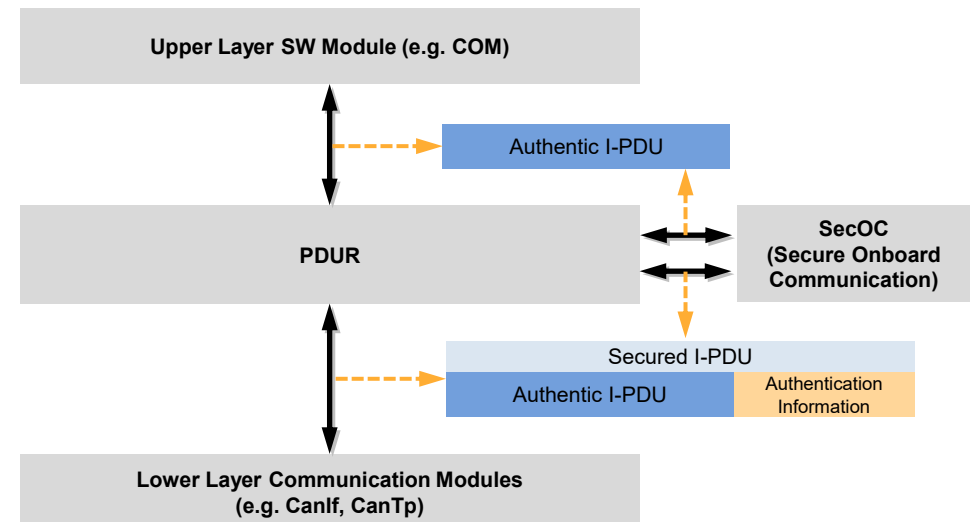
- adds/verifies authentication information (for/from lower layer)
- realizes interface of upper and lower layer modules
- is addressed by PduR routing configuration
- maintains buffers to store and modify secured I-PDUs



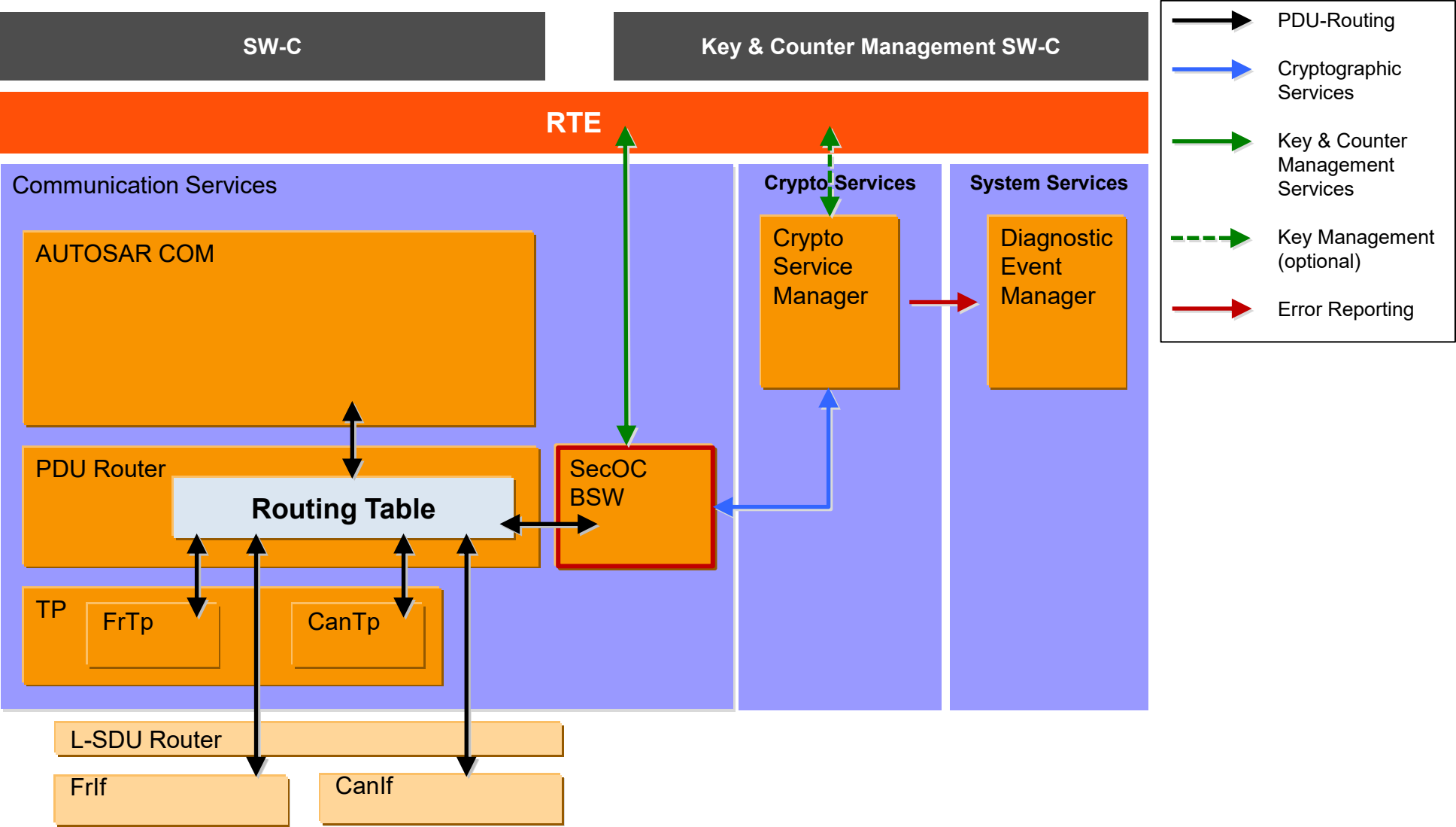


SecOC BSW:

- adds/verifies authentication information (for/from lower layer)
- realizes interface of upper and lower layer modules
- is addressed by PduR routing configuration
- maintains buffers to store and modify secured I-PDUs

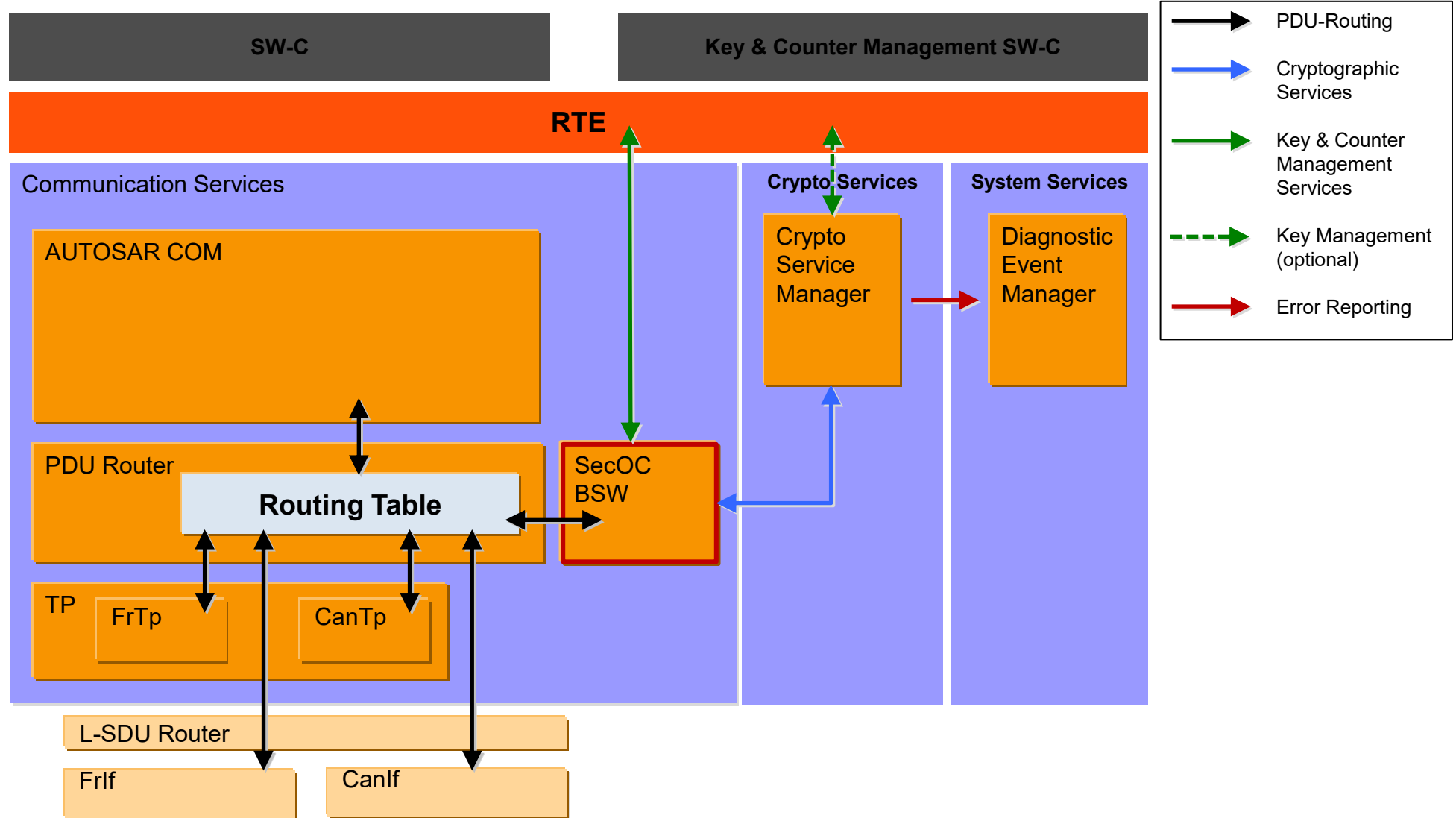


Integration and Runtime Aspects – Secure Onboard Communication **Integration with other services**



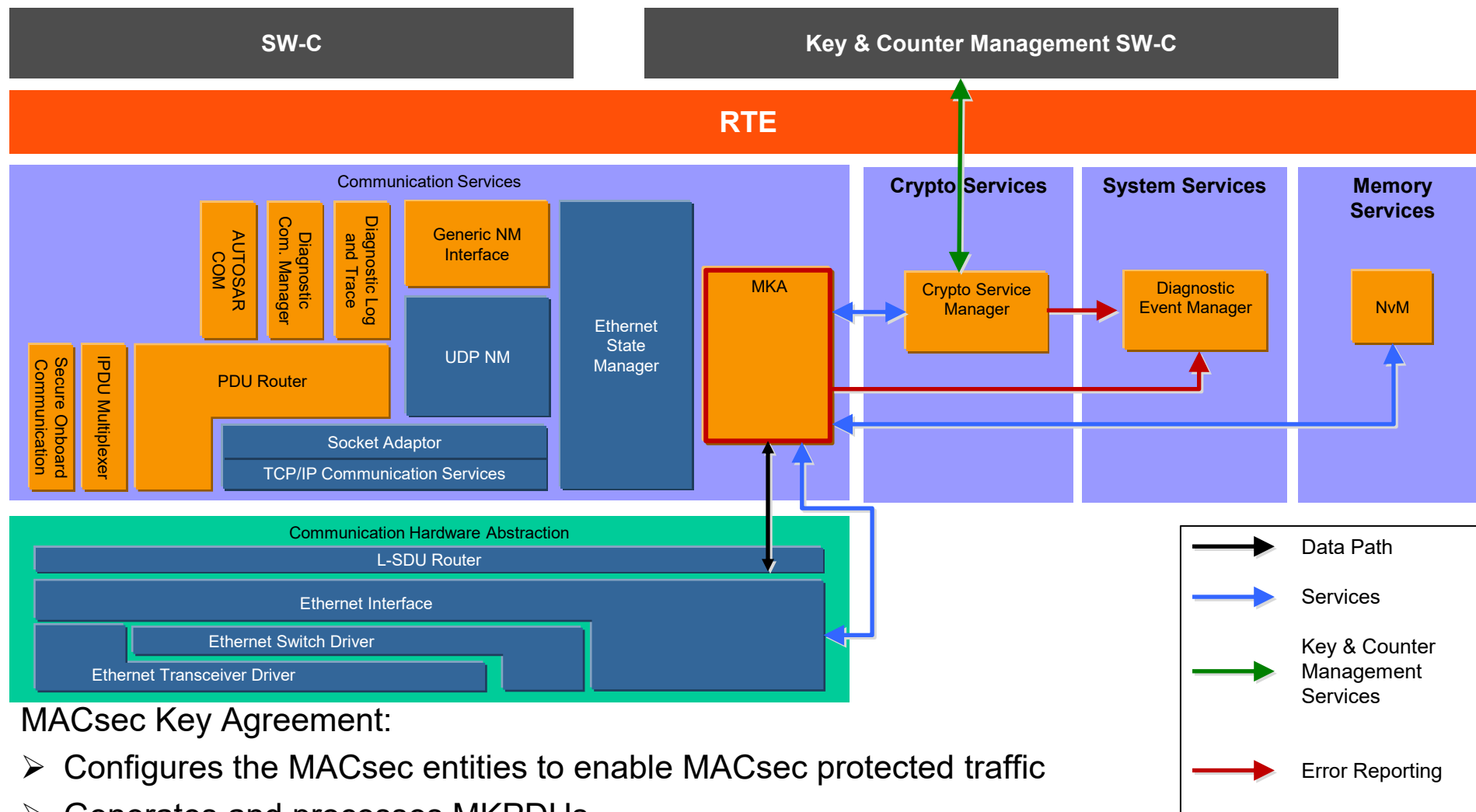
Integration and Runtime Aspects – Secure Onboard Communication

与其他服务的集成



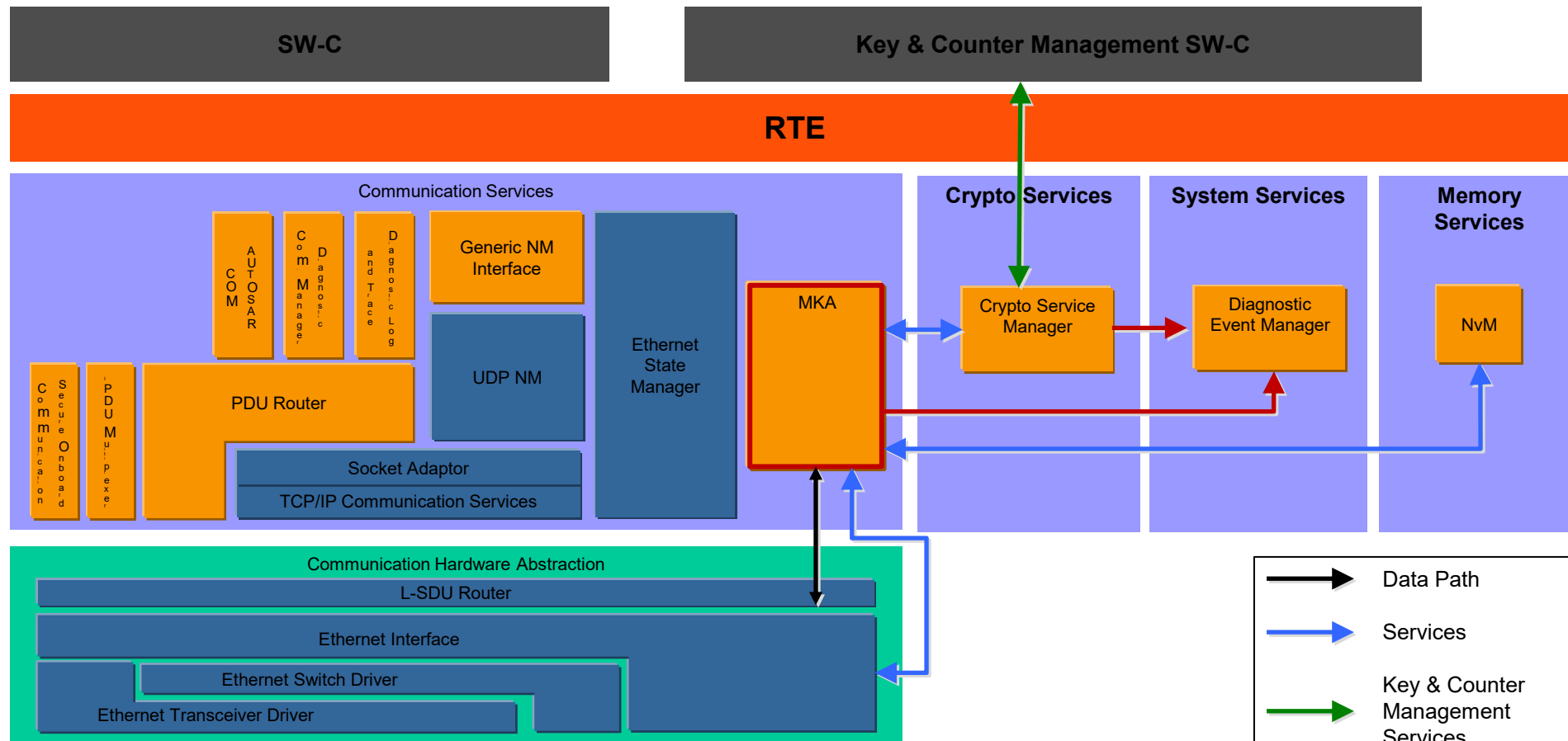
Integration and Runtime Aspects – MACsec Key Agreement

Integration with other services



Integration and Runtime Aspects – MACsec Key Agreement

与其他服务的集成



MACsec Key Agreement:

- Configures the MACsec entities to enable MACsec protected traffic
- Generates and processes MKPDUs

使用加密服务生成和验证MKPDU的完整性校验值 (ICV)

Table of contents

1. Architecture
2. Configuration
3. **Integration and Runtime Aspects**
 1. Mapping of Runnables
 2. Partitioning
 3. Scheduling
 4. Mode Management
 5. Error Handling, Reporting and Diagnostic
 6. Measurement and Calibration
 7. Security
 8. **Energy Management**
 9. Global Time Synchronization

Table of contents

1. 架构 2. 配置

3. 集成与运行时方面

1. 可运行实体的映射2. 分区管理3. 调度机制4.
模式管理5. 错误处理、报告与诊断6. 测量与
标定7. 安全机制8. 能源管理9. 全局时间同步

Energy Management Introduction

The goal of efficient energy management in AUTOSAR is to provide mechanisms for power saving, especially while **bus communication is active** (e.g. charging or clamp 15 active). AUTOSAR R3.2 and R4.0.3 support only Partial Networking.

Partial Networking

- Allows for turning off network communication across multiple ECUs in case their provided **functions** are not required under certain conditions. Other ECUs can continue to communicate on the same bus channel.
- Uses NM messages to communicate the request/release information of a partial network cluster between the participating ECUs.

ECU Degradation

- Allows to switch of peripherals.

AUTOSAR中高效能源管理的目标是提供节能机制，尤其是在总线通信处于活动状态时（例如充电或15号端子激活）。AUTOSAR R3.2和R4.0.3仅支持部分网络功能。

部分网络

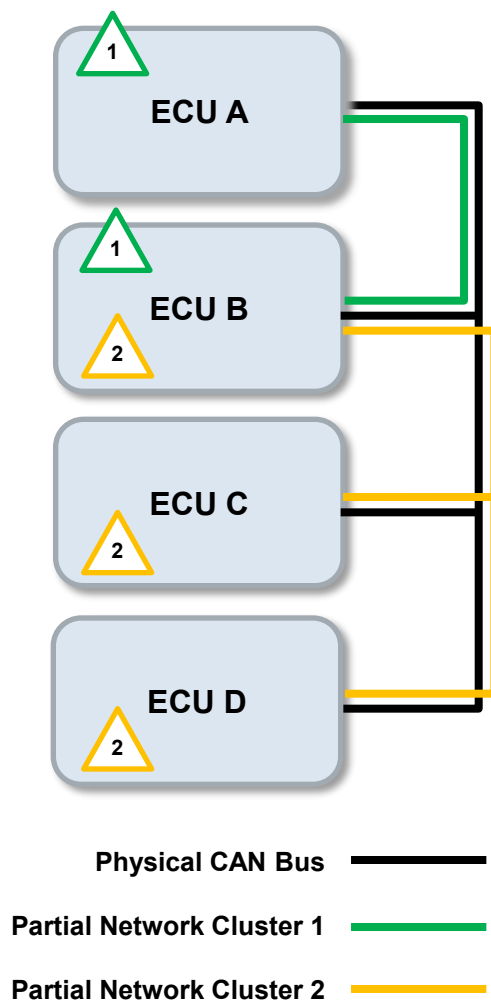
允许在特定条件下关闭多个ECU之间的网络通信，当它们提供的功能不需要时。其他ECU可以继续在同一总线通道上进行通信。使用NM消息在参与ECU之间传递部分网络集群的请求/释放信息。

ECU退化

允许切换外围设备。

Energy Management – Partial Networking

Example scenario of a partial network going to sleep



Initial situation:

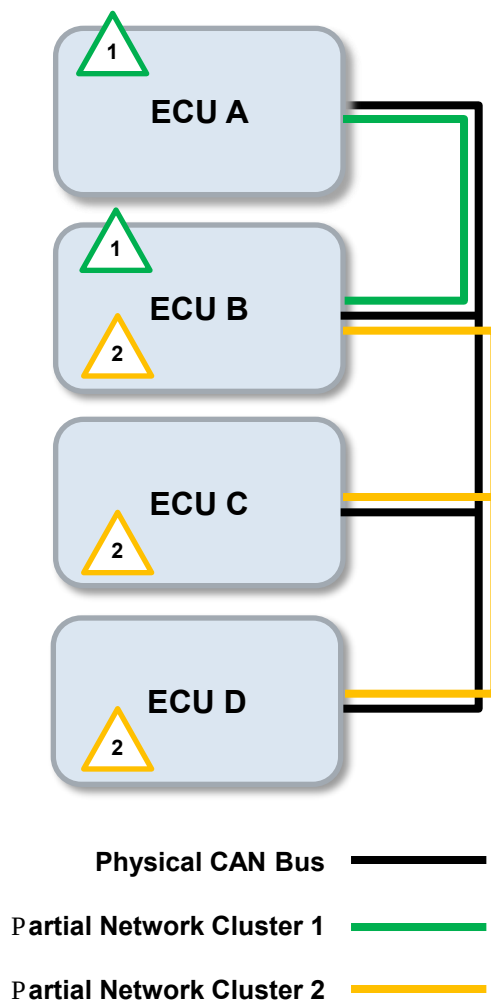
- ECUs “A” and “B” are members of Partial Network Cluster (PNC) 1. ECUs “B”, “C” and “D” are members of PNC 2.
- All functions of the ECUs are organized either in PNC 1 or PNC 2.
- Both PNCs are active.
- PNC 2 is only requested by ECU “C”.
- The function requiring PNC 2 on ECU “C” is terminated, therefore ECU “C” can release PNC 2.

This is what happens:

- ECU “C” stops requesting PNC 2 to be active.
- ECUs “C” and “D” are no longer participating in any PNC and can be shutdown.
- ECU “B” ceases transmission and reception of all signals associated with PNC 2.
- ECU “B” still participates in PNC 1. That means it remains awake and continues to transmit and receive all signals associated with PNC 1.
- ECU “A” is not affected at all.

Energy Management – Partial Networking

部分网络进入休眠的示例场景



初始情况:

ECU “A” 和 “B” 是部分网络集群（PNC）1的成员。

ECU “B”、“C” 和 “D” 是PNC 2的成员。

ECU的所有功能都组织在PNC 1或PNC 2中。

两个PNC均处于活动状态。

PNC 2 仅由 ECU “C” 请求。

需要ECU “C” 上PNC 2的功能已终止，因此ECU “C” 可以释放PNC 2。

情况是这样的:

ECU “C” 停止请求PNC 2激活。

ECU “C” 和 “D” 不再参与任何PNC，可以关闭。

ECU “B” 停止发送和接收与PNC 2相关的所有信号。

ECU “B” 仍参与PNC 1。这意味着它保持唤醒状态，并继续发送和接收与PNC 1相关的所有信号。

ECU “A” 完全不受影响。

Energy Management – Partial Networking

Conceptual terms

- A significant part of energy management is about mode handling. For the terms
 - **Vehicle Mode**,
 - **Application Mode** and
 - **Basic Software Mode**see chapter 3.4 of this document.
- **Virtual Function Cluster (VFC)**: groups the communication on port level between SW-components that are required to realize one or more vehicle functions.
This is the logical view and allows for a reusable bus/ECU independent design.
- **VFC-Controller**: Special SW-component that decides if the functions of a VFC are required at a given time and requests or releases communication accordingly.
- **Partial Network Cluster (PNC)**: is a group of system signals necessary to support one or more vehicle functions that are distributed across multiple ECUs in the vehicle network.
This represents the system view of mapping a group of buses to one ore more VFCs.

能源管理的一个重要部分是关于模式处理。对于术语

- 车辆模式、■应用模式和
- 基础软件模式

请参阅本文档的第3.4章。

虚拟功能集群（VFC）：将实现一个或多个车辆功能所需的软件组件之间在端口层面的通信进行分组。这是逻辑视图，支持可复用且独立于总线/ECU的设计。

VFC控制器：一种特殊的软件组件，用于判断在特定时刻是否需要VFC的功能，并据此请求或释放通信。

部分网络集群（PNC）：是一组系统信号，这些信号对于支持一个或多个车辆功能是必需的，并且分布在车辆网络的多个ECU中。这代表了将一组总线映射到一个或多个VFC的系统视图。

Energy Management – Partial Networking Restrictions

- Partial Networking (PN) is currently supported on CAN and FlexRay buses.
- LIN and CAN slave buses (i.e. CAN buses without network management) can be activated* using PN but no wake-up or communication of NM messages (including a PNC bit vector) are supported on those buses
- To wake-up a PN ECU, a special transceiver HW is required as specified in ISO 11898-5.
 - The standard wake-up without special transceiver HW known from previous AUTOSAR releases is still supported.
- A VFC can be mapped to any number of PNCs (including zero)
 - The concept of PN considers a VFC with only ECU-internal communication by mapping it to the internal channel type in ComM as there is no bus communication and no physical PNC
- Restrictions for CAN
 - J1939 and PN exclude each other, due to address claiming and J1939 start-up behaviour
 - J1939 need to register first their address in the network before they are allowed to start communication after a wake-up.
 - A J1939 bus not using address claiming can however be activated using PN as a CAN slave bus as described above
- Restrictions on FlexRay
 - FlexRay is only supported for requesting and releasing PNCs.
 - FlexRay nodes cannot be shut down since there is no HW available which supports PN.

* All nodes connected to the slave buses are always activated. It is not possible only to activate a subset of the nodes.

部分网络（PN）目前支持CAN和FlexRay总线。LIN和CAN从属总线（即无网络管理的CAN总线）可通过PN激活*，但这些总线不支持唤醒或NM消息（包括PNC位向量）的通信。要唤醒PN ECU，需要符合ISO 11898-5规定的特殊收发器硬件。■ 先前AUTOSAR版本中已知的无特殊收发器硬件的标准唤醒方式仍受支持。一个VFC可映射到任意数量（包括零个）的PNC ■ PN概念将通过VFC映射至ComM中的内部通道类型来考虑仅包含ECU内部通信的VFC，因为其不涉及总线通信且无物理PNC。

CAN 的限制条件

- J1939与PN相互排斥，这是由于地址声明和J1939的启动行为所致。■ J1939在唤醒后需先在网络中注册其地址，才能开始通信。■ 然而，如上所述，不使用地址声明的J1939总线可通过PN作为CAN从属总线来激活。

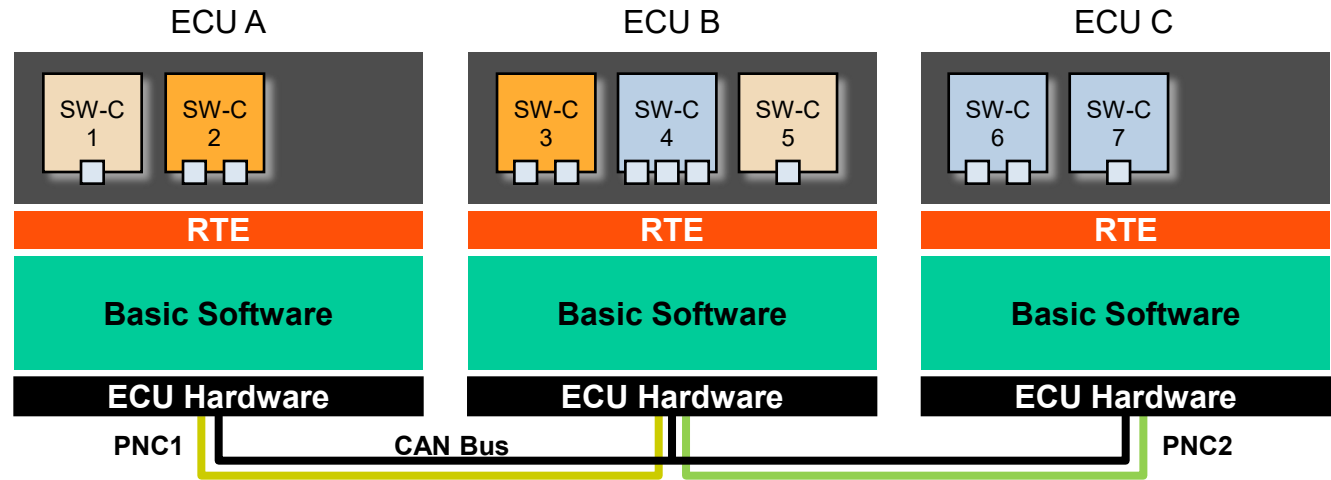
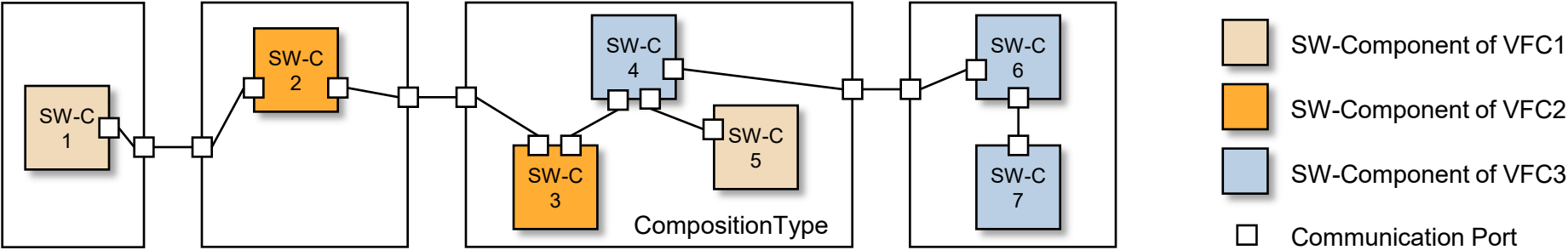
FlexRay的限制条件

- FlexRay仅支持请求和释放PNC。■ 由于没有支持PN的可用硬件，FlexRay节点无法被关闭。

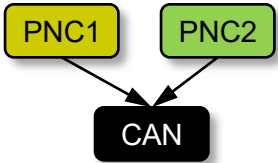
* 连接到从属总线的所有节点始终处于激活状态。不可能仅激活节点的子集。

Energy Management – Partial Networking

Mapping of Virtual Function Cluster to Partial Network Cluster

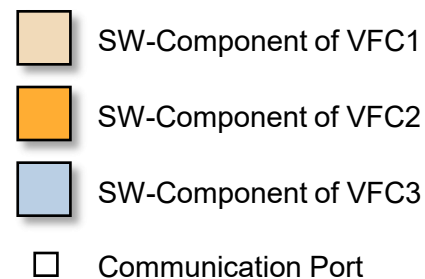
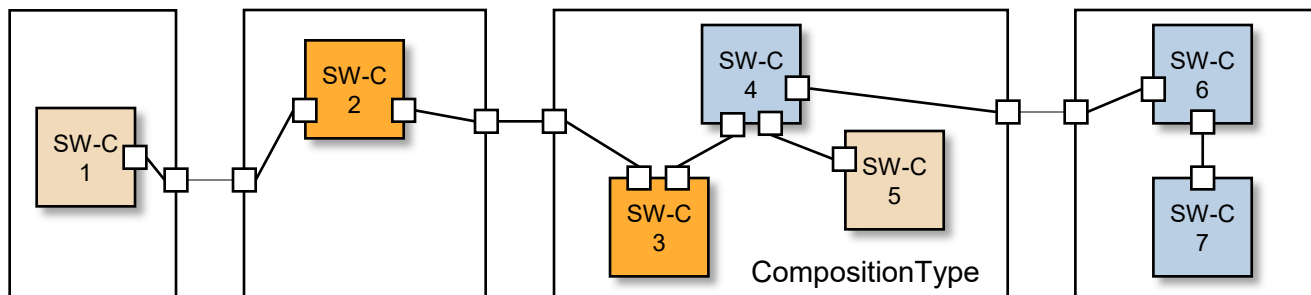


- Here both Partial Networks map to one CAN bus.
- One Partial Network can also span more than one bus.

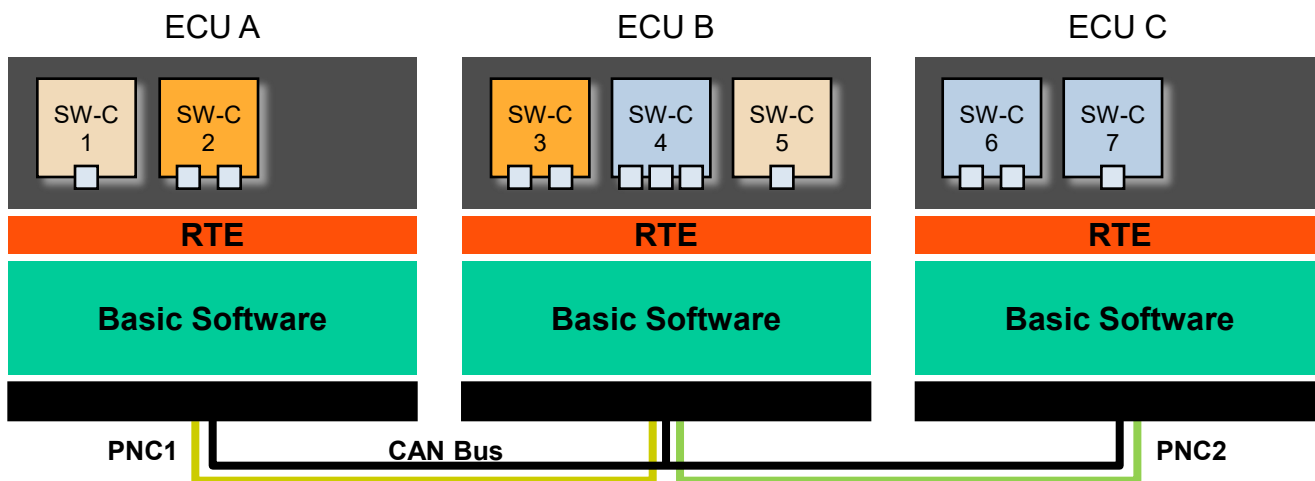
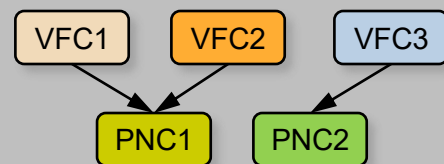


Energy Management – Partial Networking

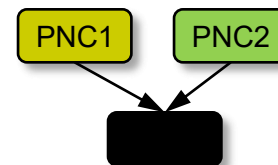
虚拟函数集群到部分网络集群的映射



Mapping of VFC on PNC

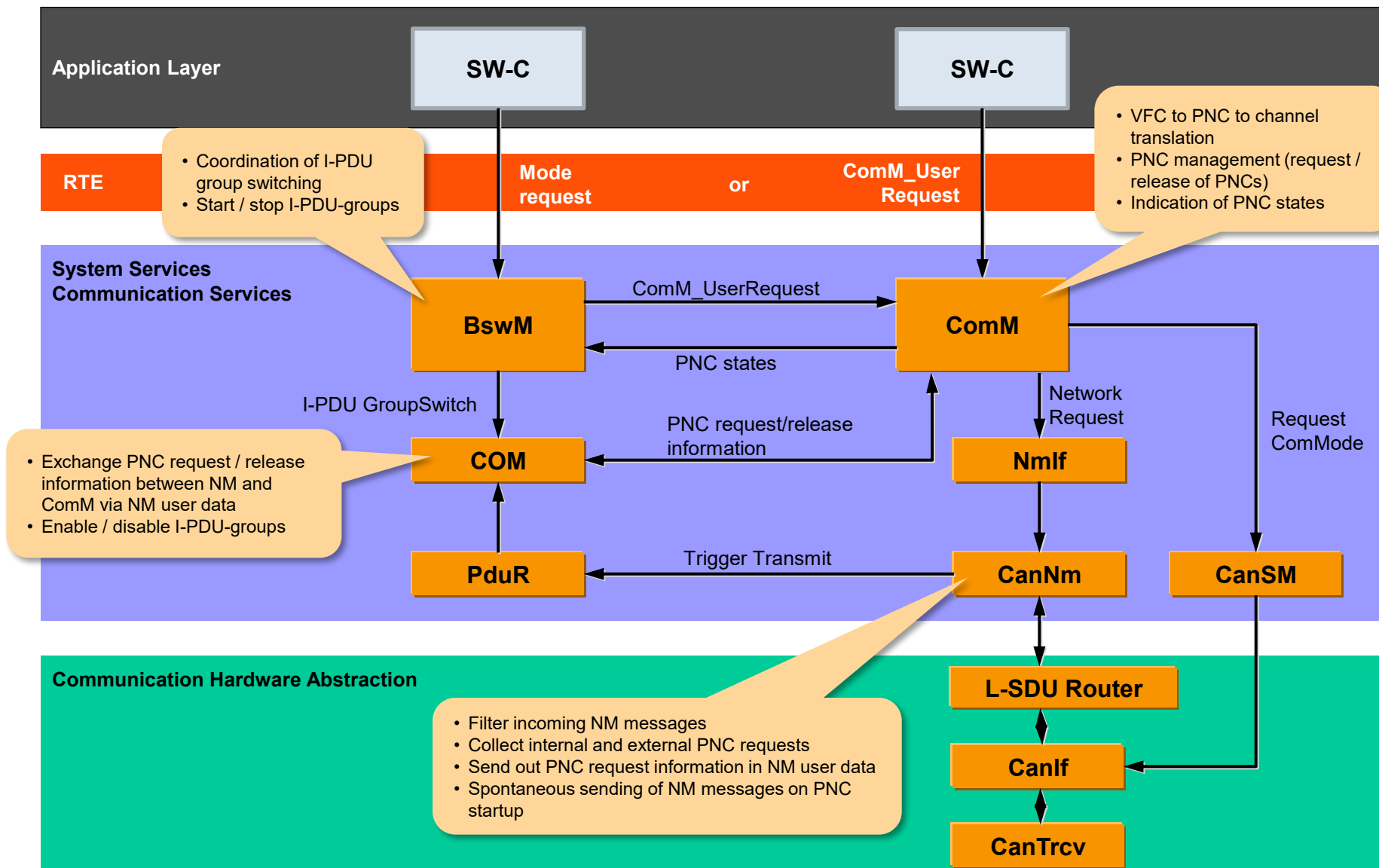


- Here both Partial Networks map to one CAN bus.
- One Partial Network can also span more than one bus.



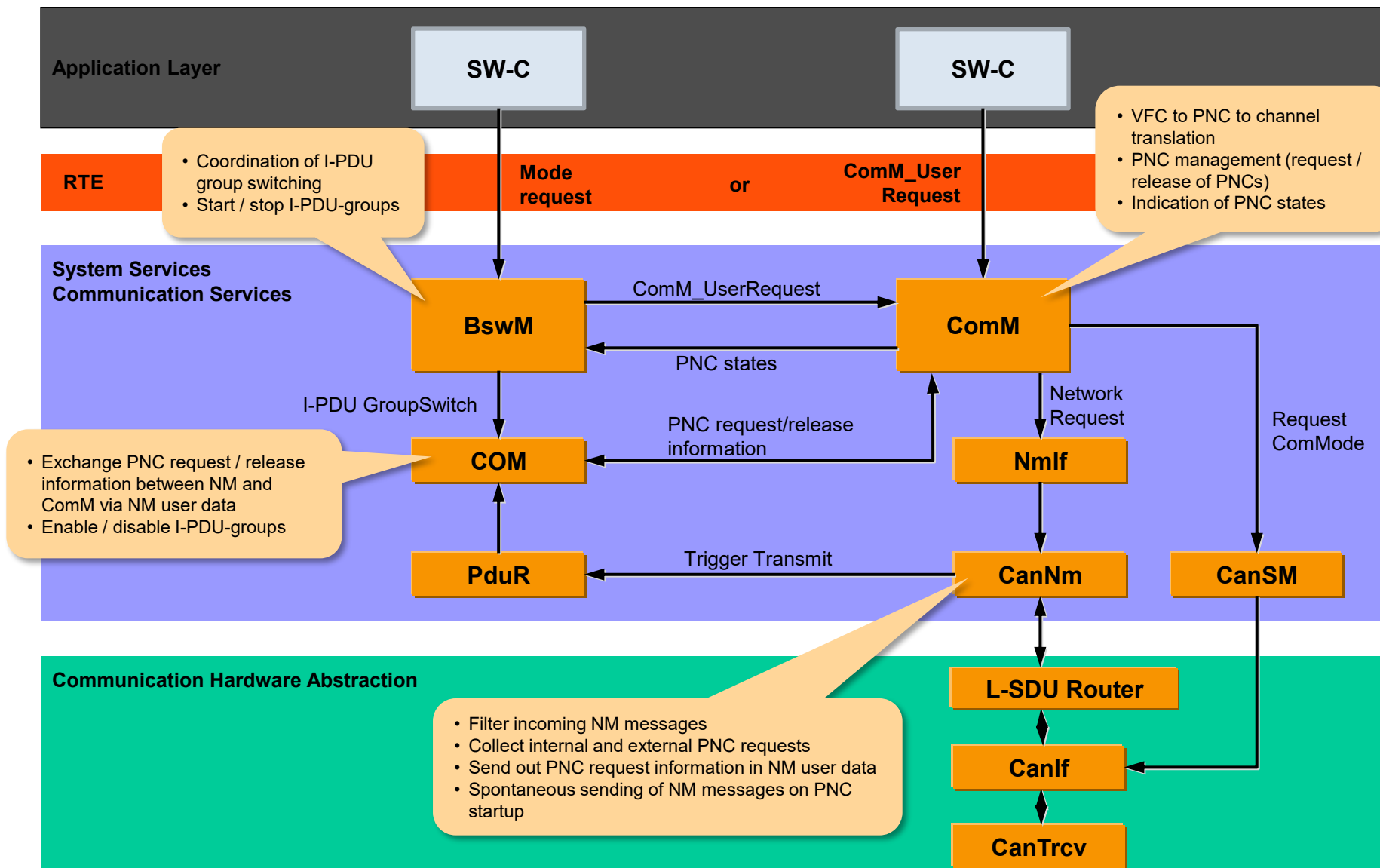
Energy Management – Partial Networking

Involved modules – Solution for CAN



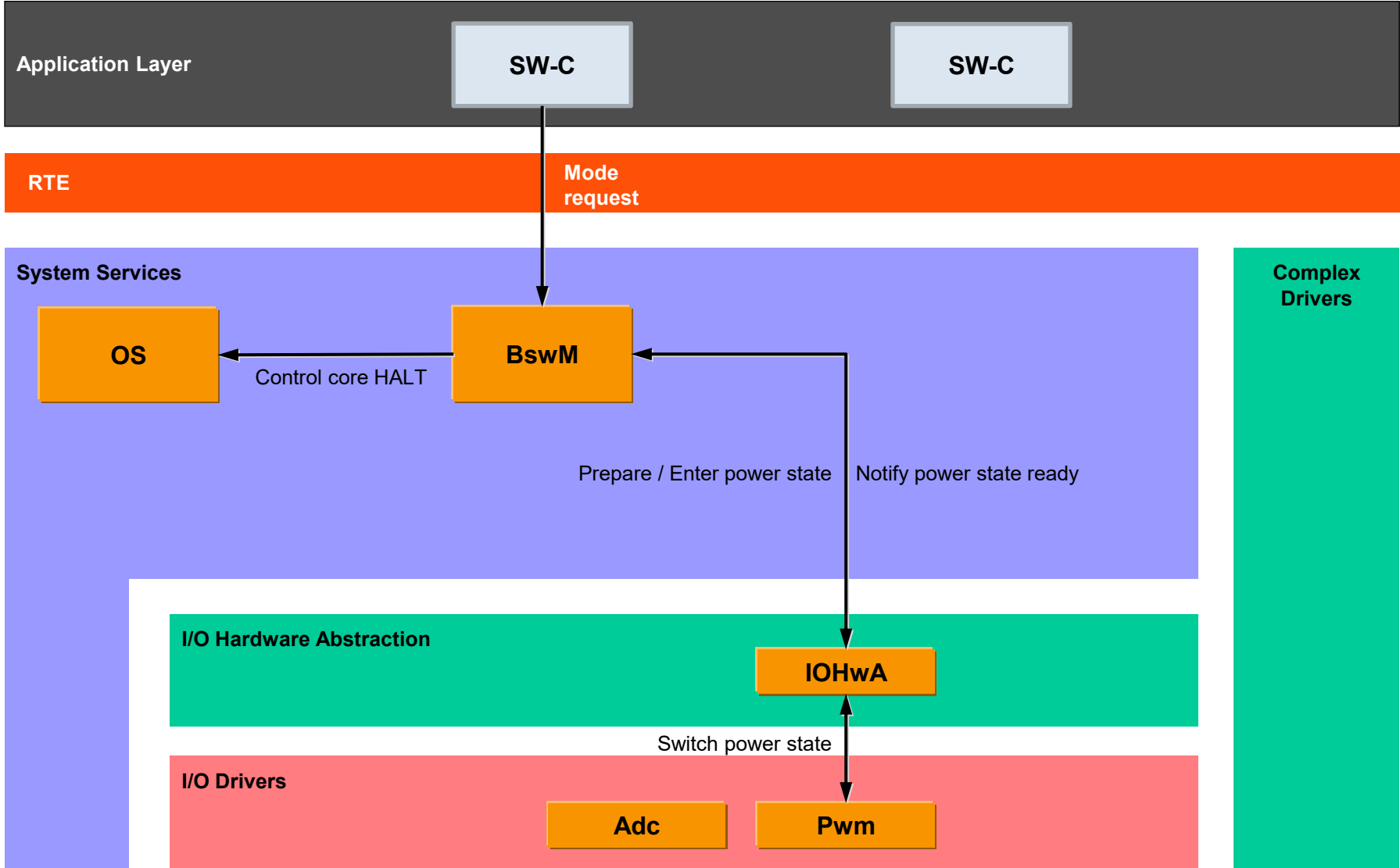
Energy Management – Partial Networking

涉及的模块 – CAN解决方案



Energy Management – ECU Degradation

Involved modules – Solution for I/O Drivers





Energy Management – ECU Degradation Restrictions

- ECU Degradation is currently supported only on MCAL drivers Pwm and Adc.
- Core HALT and ECU sleep are considered mutually exclusive modes.
- Clock modifications as a means of reducing power consumption are not in the scope of the concept (but still remain available as specific MCU driver configurations).

Energy Management – ECU Degradation

限制条件

ECU Degradation 目前仅在 MCAL 驱动 Pwm 和 Adc 上受支持。 Core HALT 与 ECU 睡眠被视为互斥模式。 通过修改时钟来降低功耗不在该概念的范围內（但仍可通过特定的 MCU 驱动配置实现）。

Table of contents

1. Architecture
2. Configuration
3. **Integration and Runtime Aspects**
 1. Mapping of Runnables
 2. Partitioning
 3. Scheduling
 4. Mode Management
 5. Error Handling, Reporting and Diagnostic
 6. Measurement and Calibration
 7. Security
 8. Energy Management
 9. **Global Time Synchronization**

Table of contents

1. 架构 2. 配置

3. 集成与运行时方面

1. 可运行实体的映射2. 分区管理3. 调度机制4.
模式管理5. 错误处理、报告与诊断6. 测量与
标定7. 安全机制8. 能源管理9. 全局时间同步

Integration and Runtime Aspects – Global Time Synchronization

Global Time Synchronization provides synchronized time base(s) over multiple in-vehicle networks.

StbM provides the following **features**:

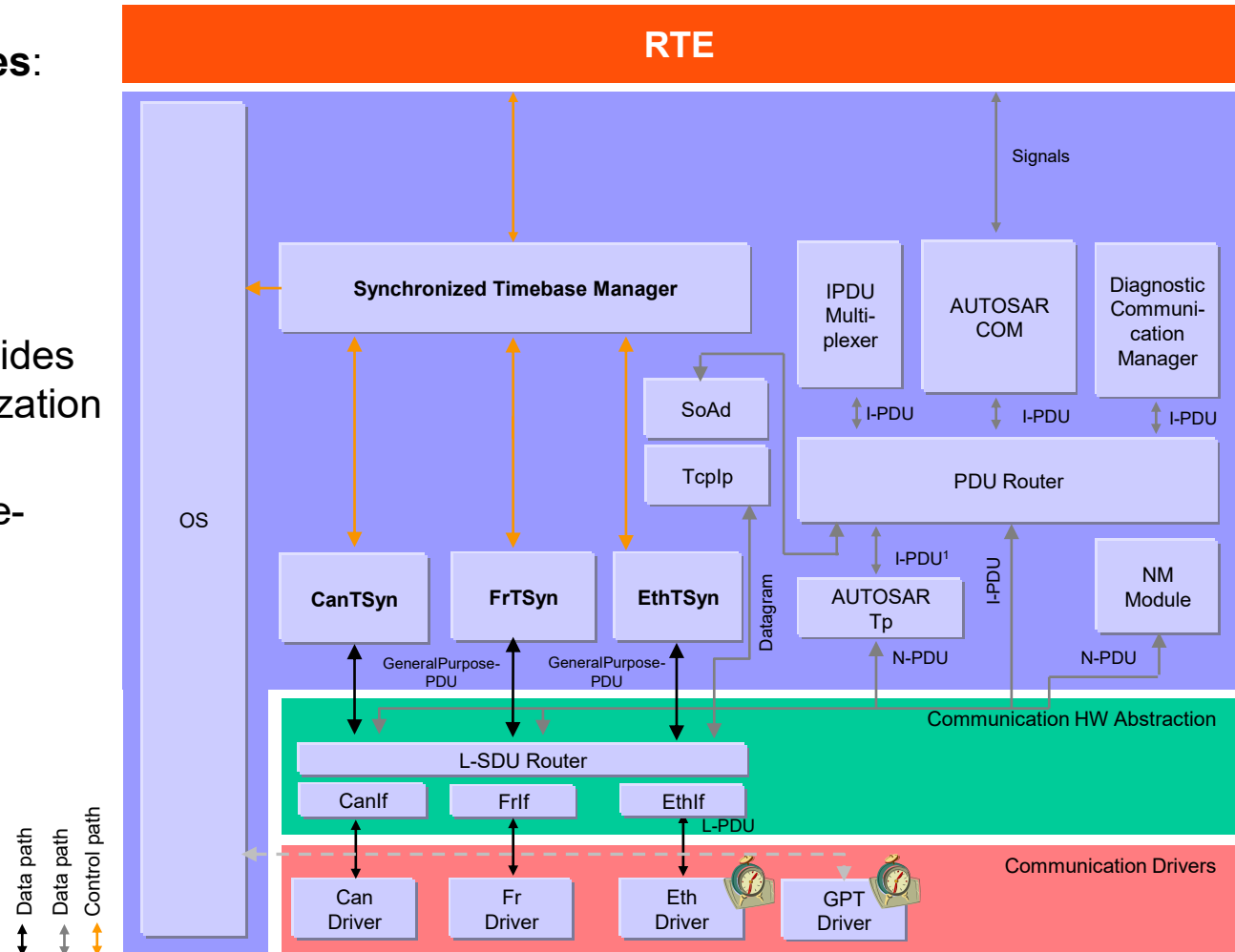
- Time provision
- Time base status
- Time gateway

CanTSyn / FrTSyn / EthTSyn provides the network-specific time synchronization protocol.

EthTSyn provides additionally a rate-correction and latency calculation.

Use-case examples:

- Sensor data fusion
- Cross-ECU logging



Integration and Runtime Aspects – Global Time Synchronization

全局时间同步在多个车载网络间提供同步的时间基准。

StbM提供以下功能：

时间规定

时间基准状态

时间网关

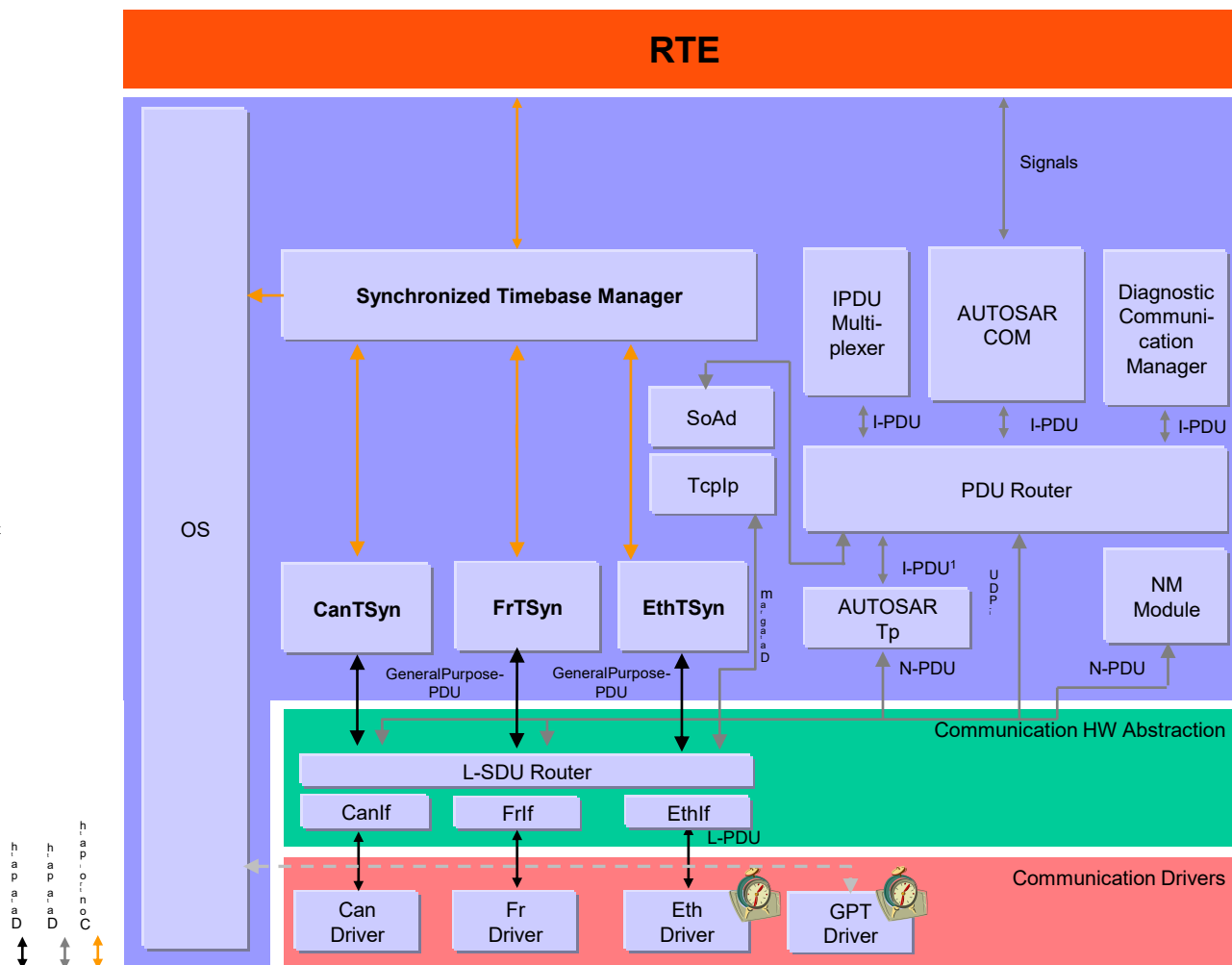
CanTSyn / FrTSyn / EthTSyn 提供网络特定的时间同步协议。

EthTSyn 还额外提供了速率校正和延迟计算功能。

用例示例：

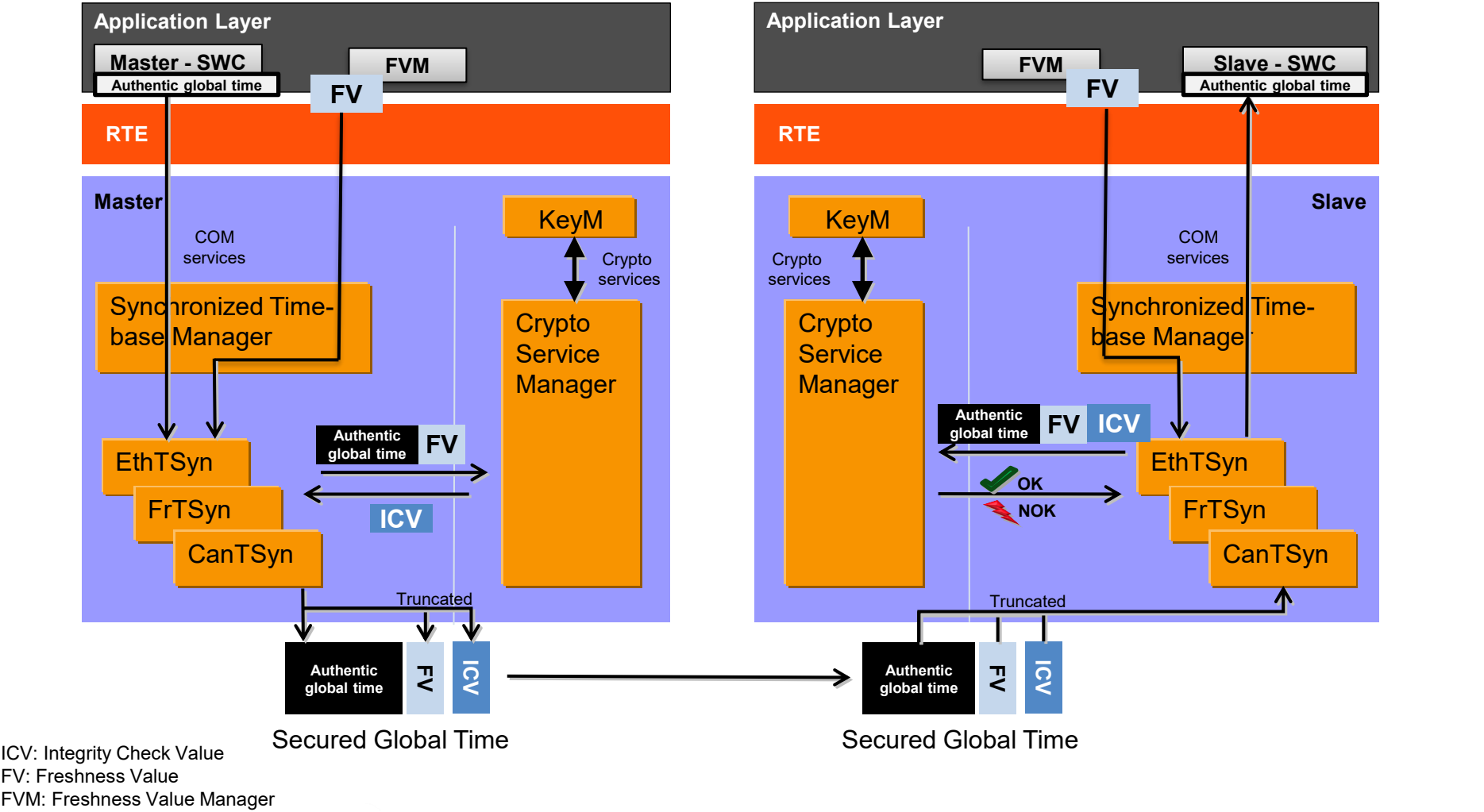
传感器数据融合

跨ECU日志记录



Integration and Runtime Aspects – Secure Global Time Synchronization

Secure Global Time Synchronization ensures integrity and authenticity of synchronized time base(s) over in-vehicle networks.



Integration and Runtime Aspects – Secure Global Time Synchronization

安全全局时间同步确保车载网络内同步时间基准的完整性和真实性。

